Destiny Church Tees Valley — Complete Repository Documentation

Version: 1.0.18
Last Updated: October 10, 2026
Repository: Square Media Group — destinychurch

This document provides a comprehensive explanation of every major component, line of code purpose, architecture decisions, and how the system works from end-to-end.

Table of Contents

1. Project Overview
2. Architecture & Philosophy
3. Technology Stack
4. Directory Structure
5. Database Schema
6. Core Application Layers
7. Routing & Pages
8. Components
9. API Routes & Server Actions
10. Content Blocks
11. Libraries & Utilities
12. Authentication & Authorization
13. Configuration
14. Deployment & Performance

Project Overview

What It Is

Destiny Church Tees Valley is a **full-stack church website platform** built by Square Media Group. It serves as the digital presence for a multi-cultural church in Stockton-on-Tees, UK, and handles:

- **Public Pages:** About, beliefs, events, contact, giving, venue hire
- **Ministry Pages:** Kids, youth, young adults, missions, connect groups
- **Media:** Sermon archive with 50+ videos, series/speaker filtering, AI-powered search
- **Member Engagement:** Prayer requests, volunteer signup, venue enquiries, connection forms
- **Admin Dashboard:** Protected area for staff to manage sermons, pages, redirects, banners, HR, jobs, training
- **Infrastructure:** Email, analytics, webhooks, caching, rate limiting

Key Visitors

- **Church members:** Watch sermons, find events, volunteer, connect groups, training
- **First-time visitors:** Plan a visit, explore beliefs, understand what to expect
- **HR/Admin staff:** Manage jobs, staff directory, leave requests, documents
- **Job applicants:** Browse positions, submit applications
- **Training participants:** Access course materials and learning paths

Non-Goals

- Real-time chat or messaging
- Video hosting (uses YouTube)
- Full ecommerce (giving is simplified)
- Social network features

Architecture & Philosophy

Key Design Principles

1. **Server-Driven:** Use server-side rendering and server actions wherever possible; minimize client-side complexity
2. **Security First:** Row-level security (RLS) on all sensitive data; rate limiting on public APIs; Supabase service role for backend operations
3. **Edge Caching:** Long TTLs on static assets; revalidate on-demand for dynamic content

4. **Mobile First:** Responsive Tailwind CSS; tested on real devices
5. **Accessibility:** Semantic HTML, ARIA labels, keyboard navigation
6. **AI-Ready:** Structured data for future AI features (smart search, content generation)
7. **No Vendor Lock-In:** Open-source stack (Next.js, React, Tailwind, Supabase PostgreSQL)

Data Flow

```
User (Browser/Phone)
  ↓
Next.js App Router (Server/Client)
  ↓
API Routes (Node.js Serverless)
  ↓
Supabase PostgreSQL
  ↓
Responses: JSON, Server-Side HTML, Redirects, Cached Assets
```

Authentication Layers

- **Public Pages:** No auth required; cached at edge
- **Member Features:** Supabase Auth (email/password); tokens in secure cookies
- **Admin Dashboard:** Supabase Auth required; additional role checks via proxy
- **API Endpoints:** Service role key (server-to-database); user tokens (client-to-server)

Technology Stack

Layer	Technology	Purpose
Framework	Next.js 16 (App Router)	Full-stack React with server-side rendering
Language	TypeScript	Type-safe code
Styling	Tailwind CSS v4	Utility-first CSS framework
Frontend	React 19	Component library and state management
Database	Supabase (PostgreSQL)	Relational database with RLS and real-time
Auth	Supabase Auth	User sessions and JWT tokens
Email	Resend	Transactional email (password resets, confirmations)
Storage	Supabase Storage	File uploads (HR documents, media, images)
Video	YouTube API v3	Sermon hosting and metadata
Podcasts	Buzzsprout	Podcast RSS and metadata
AI	OpenAI (`gpt-4.1-mini`)	Smart Search tool-calling chat (products, weather, directions, web search, page extraction)
Web Search	Tavily (`search` + `extract` APIs)	Smart Search's `search_web` / `extract_page` tools — live facts, page content
Payments	Stripe (Payment Element + Express Checkout)	`/shop` checkout — cards, Apple Pay, Google Pay, Link
Analytics	Vercel Analytics + SpeedInsights	Performance and visitor tracking
Deployment	Vercel	Edge functions, serverless, CDN
Testing	Playwright	E2E browser testing
Media Processing	Sharp	Uploaded images (posts, shop products, shop hero) resized/converted to WebP server-side; `ffmpeg-static` bundled but unused
Search	fuse.js + rbush	Client-side fuzzy search and spatial indexing
Icons	Phosphor + Material Symbols	Icon sets
Rich Text	TipTap (Prosemirror)	Sermon markdown, content editing
State	Zustand	Global client state (minimal usage)
Motion	Motion	Animation library
QR Codes	qrcode.react	QR code generation

Directory Structure

```
destinychurch/
├── app/                          # Next.js App Router routes
│   ├── layout.tsx                # Root layout (header, footer, providers)
│   ├── page.tsx                  # Home page
│   ├── globals.css               # Tailwind + custom CSS
│   ├── robots.ts                 # robots.txt generation
│   │                              # NOTE: there is no `(public-pages)` route group — every
│   │                              # page below is a flat top-level folder directly under app/
│   ├── about/                    # About page
│   ├── accessibility/            # Reduced-motion / glass-FX preferences (client component)
│   ├── administration/           # Stale-bookmark redirect → /admin
│   ├── auth/                     # callback/ + confirm/ — Supabase OAuth & email-OTP handlers
│   ├── baptism/                  # Baptism sign-up
│   ├── beliefs/                  # Beliefs page
│   ├── bible-course/             # The Bible Course (Bible Society) info page
│   ├── cap-money/                # CAP Money Course (Christians Against Poverty) info page
│   ├── child-dedication/         # Child dedication request
│   ├── connect/                  # Connect groups page
│   ├── data-gdpr/                # Data & GDPR policy
│   ├── dckids/                   # Destiny Kids Camp 2026 campaign page
│   ├── twelvetwo/                # Destiny 12:2 recovery course info page
│   ├── give/                     # Giving/donations page
│   ├── help/                     # Help centre / FAQ
│   ├── kids/                     # Kids ministry
│   ├── links/                    # "Next Steps" link-in-bio style page
│   ├── login/                    # Staff sign-in
│   ├── youth/                    # Youth ministry
│   ├── young-adults/             # Young adults ministry
│   ├── missions/                 # Missions & outreach
│   ├── privacy/                  # Privacy policy
│   ├── governance/              # Charity & company registration / transparency page
│   ├── safeguarding/             # Safeguarding policy
│   ├── serve/                    # Volunteer opportunities (overview)
│   ├── sermons/                  # Sermon archive (+ [id] detail)
│   ├── terms/                    # Terms of use
│   ├── training/                 # /training resource library (category → subgroup → post)
│   ├── contact/                  # Contact form
│   ├── design-request/           # [token]/ requester tracker (public, share-token-scoped); the request form itself lives at /portal/design/r
│   ├── visit/                    # Plan a visit
│   ├── new-here/                 # First-time visitor guide
│   ├── hire/                     # Venue hire enquiries
│   ├── connect-card/             # Prayer requests & connections
│   ├── alpha/                    # Alpha course info
│   ├── volunteer/                # Volunteer sign-up form
│   ├── whats-on/                 # Events listing + per-event pages ([slug])
│   ├── [slug]/                   # Dynamic catchall page (posts table)
│   ├── admin/                    # Protected admin dashboard
│   │   ├── forgot-password/      # Password recovery request
│   │   ├── reset-password/       # Password reset form
│   │   ├── layout.tsx            # Admin layout (sidebar)
│   │   ├── page.tsx              # Admin home (dashboard)
│   │   ├── banner/               # Manage site banners
│   │   ├── popup/                # Manage pop-ups
│   │   ├── redirects/            # Manage URL redirects
│   │   ├── cache/                # Cache invalidation tools
│   │   ├── posts/                # Standalone content pages
│   │   ├── training/             # Training/courses management
│   │   ├── alpha/                # Manage Alpha course events
│   │   ├── bible-course/         # Manage The Bible Course events
│   │   ├── cap-money/            # Manage CAP Money Course events
│   │   ├── recovery/             # Manage Recovery course events
│   │   ├── featured-course/      # Choose the What's On featured course
│   │   ├── store/                # Shop admin (products, orders, hero)
│   │   ├── hr/                   # HR admin (staff, leave, jobs, docs, reviews, checklists) — hr_admin
│   │   └── onboarding/           # Super-admin preview of each role's onboarding tour
│   ├── portal/                   # Staff self-service — dashboard, profile, team, reviews, leave, documents (linked hr_staff)
│   │   ├── layout.tsx            # Portal shell (separate auth boundary from /admin)
│   │   ├── page.tsx              # Profile + leave balance overview
│   │   ├── leave/page.tsx        # Request/withdraw own leave
│   │   ├── documents/page.tsx    # Download own + org-wide documents
│   │   └── design/page.tsx       # Own design requests (matched by staff link and by email)
│   ├── jobs/                     # Job listing & application
│   │   ├── page.tsx              # Job list
│   │   ├── [slug]/page.tsx       # Job detail
│   │   ├── ApplyForm.tsx         # Job application form
│   │   └── actions.ts            # Server actions for applying
│   ├── api/                      # API routes (serverless functions) — no `public/` subfolder
│   │   ├── admin/                # Admin API endpoints (gated by middleware.ts)
│   │   │   ├── logout/           # End admin session
│   │   │   ├── redirects/        # CRUD redirects
│   │   │   ├── popup/            # CRUD pop-ups
│   │   │   ├── revalidate/       # ISR cache invalidation
│   │   │   ├── onboarding/       # Per-admin tour progress
│   │   │   ├── posts/, training/, alpha-events/, featured-course/, hr/, store/, shop-hero/
```

```
│   │   │   │                       #   hr/ includes checklists/ + checklist-templates/
│   │   │   └── ...
│   │   ├── portal/                 # Staff self-service API — me/, reviews/, checklists/, team/, leave/, documents/, design/ (linked hr_staff)
│   │   ├── design-request/         # Public, share-token-scoped: [token]/ + deliverable downloads/confirm
│   │   ├── cron/                   # Vercel Cron — hr-review-reminders/, design-deliverables-purge/, … (Bearer CRON_SECRET)
│   │   ├── chat/                   # POST /api/chat — Smart Search tool-calling chat
│   │   ├── youtube/                 # videos/, thumbnail/[id]/, status/, live/
│   │   ├── alpha-ask/, alpha-events/ # Public Alpha info endpoints
│   │   ├── store/                  # checkout/, checkout/bypass/ (public storefront)
│   │   ├── training/                # unlock/, posts/[id]/timer/
│   │   ├── health/                  # smart-search/ health check
│   │   └── webhooks/                # stripe/ only — no GitHub/Vercel webhook route
│   ├── [slug]/                     # Dynamic catchall (posts table)
│
├── components/                     # React components (shared across pages)
│   ├── ChurchHeader.tsx            # Site header with nav
│   ├── ChurchFooter.tsx            # Site footer
│   ├── FooterLinkGroup.tsx         # Footer link column (accordion on mobile)
│   ├── MapsLink.tsx                # Address link that opens the device's map app
│   ├── Providers.tsx               # Client context providers
│   ├── CookieBanner.tsx            # GDPR cookie consent
│   ├── AnalyticsGate.tsx           # Conditional analytics loading
│   ├── SiteBanner.tsx              # Announcement banner (from DB); hidden on legal pages (lib/legalPages.ts)
│   ├── SitePopup.tsx               # Modal pop-up (from DB)
│   ├── FloatingSmartSearch.tsx     # The floating AI Smart Search widget
│   ├── smartSearch/                # Smart Search result cards (products, weather, maps, web)
│   ├── LiveBanner.tsx              # "WE ARE LIVE" banner bar
│   ├── GlassBloomTracker.tsx       # Glass-effect performance tracking
│   ├── FooterGate.tsx / PerformanceGate.tsx / BannerSpacer.tsx  # Layout/perf gating helpers
│   ├── admin/                      # Admin-specific components
│   │   ├── AdminSidebar.tsx        # Admin nav menu (desktop sidebar + mobile top bar)
│   │   ├── AdminTabBar.tsx         # Mobile bottom tab bar (glass, scrolling, group sheets)
│   │   ├── AdminHeader.tsx         # Admin shell header (breadcrumb + "View live site")
│   │   ├── RichTextEditor.tsx      # Shared TipTap editor — posts, training posts, HR jobs, shop products
│   │   ├── training/               # Course management UI (uses RichTextEditor)
│   │   ├── hr/                     # HR management UI (unlinked, in progress)
│   │   ├── posts/                  # Content editor (uses RichTextEditor)
│   │   └── ...
│   ├── shop/                       # Storefront components (ProductCard, ShopProductGrid, ShopHero, cart, checkout)
│   ├── connect-card/               # Prayer form components
│   ├── kids/                       # Kids ministry components
│   ├── home/                       # Home page components
│   ├── visit/                      # Visit page components
│   ├── serve/                      # Volunteer components
│   ├── new-here/                   # Onboarding components
│   ├── alpha/                      # Alpha course components
│   ├── AnimateIn.tsx               # Scroll-triggered animations
│   ├── ChurchSuiteEmbed.tsx        # ChurchSuite integration (events)
│   ├── ui/EmbedLoadingOverlay.tsx # Spinner + escalating copy over a loading iframe
│   └── ...
│
├── lib/                            # Utility functions and helpers
│   ├── supabase.ts                 # Supabase admin client (server-only)
│   ├── supabase-browser.ts         # Supabase client (browser)
│   ├── podcast.ts                  # Buzzsprout RSS feed (sermon audio)
│   ├── sermonPairing.ts            # Pairs videos with podcast episodes — the featured card + every confident archive match
│   ├── sermonTitle.ts              # Parses "Title | Speaker | Destiny Church LIVE" YouTube titles
│   ├── sermonSearch.ts             # Shared Fuse.js sermon search — the /sermons box + find_sermons
│   ├── buzzsprout.server.ts        # Publishes sermon audio to Buzzsprout (admin upload flow)
│   ├── youtube.ts                  # YouTube API client
│   ├── smartSearch.ts              # AI search logic (parseAnswer, fallbacks)
│   ├── smartSearch/tools.ts        # Smart Search tool-calling tools (products, sermons, weather, maps, web)
│   ├── embedLoading.ts             # Stage timings for the embed loading overlay
│   ├── pageContent.ts              # Dynamic page editing
│   ├── posts.ts                    # Post types + hero styles (postTemplates.ts: starter layouts; postPreview.server.ts: signed draft previews)
│   ├── training.ts                 # Training courses
│   ├── jobs.ts / jobs.server.ts    # Job listing & applications
│   ├── hr.ts                       # HR staff operations, types, leave/review label maps
│   ├── hrEmail.ts                  # HR notification emails (leave request/decision, review digest) via Resend
│   ├── staffPortalAuth.ts          # /portal + /api/portal auth boundary (linked hr_staff row, not admin_roles)
│   ├── designTickets.ts            # Design queue vocabulary + the canTransition state machine (client-safe)
│   ├── designTickets.server.ts     # applyTransition (the only writer of status), requester identity, requesterView
│   ├── designEmail.ts              # Design ticket notifications via Resend
│   ├── emailCard.ts                # The shared transactional email card (HR + design)
│   ├── playbook.server.ts          # Playbook DAM client — boards, two-step upload, signed display URLs
│   ├── shop.ts / shop.server.ts    # Shop types, price helpers, published-product fetchers
│   ├── stripe.ts                   # Stripe client singleton
│   ├── cart-store.ts               # Zustand basket store (localStorage-persisted)
│   ├── checkout.server.ts          # Shared order/pricing logic (checkout, webhook, test bypass)
│   ├── courses.ts                  # Alpha/Recovery/Bible Course/CAP course definitions
│   ├── courseEvents.ts             # alpha_events type registry — banner surfaces + admin pages
│   ├── cn.ts                       # className joiner (no Tailwind conflict resolution)
│   ├── openaiClient.ts             # OpenAI client + SMART_SEARCH_MODEL constant
│   ├── siteKnowledge.ts            # AI search knowledge base
│   ├── serviceStatus.ts            # Smart Search health / kill-switch state (service_status table)
│   ├── smartSearchAlertEmail.ts    # Resend email on Smart Search down/recovered transitions
│   ├── rateLimit.ts                # Rate limiting
│   ├── loginRateLimit.ts           # Login attempt limiting
```

```
│   ├── podcast.ts                # Podcast metadata
│   ├── accessRequestEmail.ts     # Email templates
│   ├── passwordResetEmail.ts     # Email templates
│   ├── staffLogins.ts            # Create/link/delete a staff record's mandatory backend login (new login or an existing admin's)
│   ├── sermonPlayerContext.tsx   # Sermon player state
│   ├── sermonSearchContext.tsx   # Sermon search state
│   ├── cookieConsent.tsx         # Cookie preferences
│   ├── alphaSession.ts           # Alpha course sessions
│   ├── trainingAccess.ts         # Training permissions
│   ├── ai/                       # Only media-types.ts remains — the AI page-generation
│   │   └── media-types.ts        # feature (llm-client, code-generator, code-validator,
│   │                              # git-automation, page-audit-email) was removed in 7aa899d
│   │                              # alongside the old "Destiny AI" page. OpenAI is now used
│   │                              # only for Smart Search (lib/openaiClient.ts, lib/smartSearch/)
│   ├── reserved-slugs.ts         # Protected URL paths
│   └── ...
│
├── supabase/                     # Supabase configuration
│   └── migrations/               # Database schema migrations (49 files) — selected highlights:
│       ├── 001_redirects.sql     # URL redirect table
│       ├── 002_hidden_videos.sql  # Hidden sermon videos (feature since removed)
│       ├── 003_content.sql       # Site banner & page content
│       ├── 004_banner_type.sql   # Banner types (sitewide, alpha, etc.)
│       ├── 006_hire_enquiries.sql # Venue hire form submissions
│       ├── 007_alpha_events.sql  # Alpha course events
│       ├── 008_alpha_events_online.sql # Online/hybrid Alpha support
│       ├── 009_alpha_events_frequency.sql # Event recurrence
│       ├── 010_alpha_events_recovery_type.sql # Recovery program
│       ├── 20260329175837_add_subject_to_contact_messages.sql # Contact form subject field
│       ├── 20260502_create_site_popup.sql # Modal pop-ups
│       ├── 20260507_create_builder_media_bucket.sql # AI page builder media (builder since removed)
│       ├── 20260514_studio_v2_schema.sql # Page builder schema (removed by 20260711_06_remove_page_builder.sql)
│       ├── 20260531_hr.sql       # HR staff, leave, reviews, documents
│       ├── 20260606_jobs.sql     # Job listings & applications
│       ├── 20260608_service_status.sql # Feature flags
│       ├── 20260614_staff_logins.sql # Staff login audit trail
│       ├── 20260616_training.sql, 20260616_0{2,3,4}_*.sql # Training categories/subgroups/folders/posts
│       ├── 20260618_posts.sql    # Standalone posts (`/[slug]` catch-all)
│       ├── 20260708_shop.sql     # Shop: products, variants, orders, items
│       ├── 20260708_shop_product_fit.sql # products.fit (male/female/unisex/kids)
│       ├── 20260710_shop_hero.sql # Editable auto-rotating /shop hero slides
│       ├── 20260711_02..07_*.sql  # RLS/security-advisor fixes; shop product_type; page builder + sermon hiding removed
│       ├── 20260711_rls_harden_base_tables.sql # RLS hardening pass across base tables
│       ├── 20260712_alpha_events_bible_course_type.sql # The Bible Course
│       ├── 20260712_02_featured_course.sql # Featured course (What's On)
│       ├── 20260728_featured_event.sql   # Featured ChurchSuite event + its popup
│       ├── 20260807_alpha_events_cap_type.sql # CAP Money Course
│       ├── 20260821_admin_onboarding.sql # Per-admin onboarding/tour progress (admin_onboarding)
│       ├── 20260822_hr_admin_role.sql  # `hr_admin` access level on admin_roles
│       ├── 20260824_hr_review_reminders.sql # hr_reviews.reminder_sent_at for the daily digest
│       ├── 20260825_hr_checklists.sql  # Onboarding/offboarding checklists (hr_checklist_* tables)
│       ├── 20260826_audit_log.sql       # Admin audit log + weekly AI reports (audit_log, audit_reports)
│       ├── 20260827_engagement_events.sql # Click analytics table + rollup/anonymise RPCs (storage layer)
│       ├── 20260828_ip_reputation.sql   # VPN/Tor/datacenter/Private-Relay tags on engagement_events
│       ├── 20260828_02_ip_reputation_advisor_fixes.sql # Advisor cleanup: revoke anon/authenticated
│       │                              # EXECUTE on the three engagement RPCs (see §24b), add
│       │                              # search_path to the ip_category trigger, relocate btree_gist
│       ├── 20260902_remove_media_boards.sql # Drops the media_boards/media_photos tables (Media Boards feature removed)
│       ├── 20260912_01_sermon_admin_role.sql # `sermon_admin` access level on admin_roles (/admin/sermons)
│       ├── 20260912_02_speaker_overrides.sql # speaker_overrides table — AI/human speaker corrections for the sermon archive
│       ├── 20260920_01_sermon_series.sql  # sermon_series table — playlist ids curated as sermon series
│       ├── 20260926_01_destiny_one.sql    # Destiny One messaging (d1_* tables, safeguarding triggers,
│       │                              # Realtime policy) + admin_roles.safeguarding_admin — see §29
│       ├── 20260928_04_destiny_one_role_tags.sql # Destiny One part 8: roles become admin / cg_leader /
│       │                              # senior_leader (identical powers, different chat tag)
│       ├── 20260927_04_destiny_one_invite_members.sql # Destiny One part 5: staff invites create the
│       │                              # member up front (groups before sign-in); d1_sign_in_status
│       ├── 20261004_01_destiny_one_message_encryption.sql # Destiny One part 10: message text
│       │                              # encrypted at rest; blind search index (d1_message_terms)
│       ├── 20261004_02_destiny_one_message_encryption_required.sql # part 10b: refuse plaintext
│       ├── 20261006_01_destiny_one_message_edits.sql # part 11 (v0.8): edit your own message for 15
│       │                              # minutes; old text kept in d1_message_edits for review
│       ├── 20261006_02_destiny_one_mentions.sql # part 12 (v0.8): @mentions (d1_messages.mentions,
│       │                              # d1_valid_mentions; d1_post_message gains p_mentions)
│       ├── 20261006_03_destiny_one_pins.sql # part 13 (v0.8): pinned messages (d1_pins, up to 3)
│       ├── 20261006_04_destiny_one_read_receipts.sql # part 14 (v0.8): "Seen by" (d1_members.read_receipts,
│       │                              # d1_read_receipts)
│       ├── 20261006_05_destiny_one_link_previews.sql # part 15 (v0.8): sealed link previews
│       ├── 20261006_06_destiny_one_forwarding.sql # part 16 (v0.8): forwarded_from; d1_post_message gains
│       │                              # p_forwarded_from (refuses messages you can't see)
│       ├── 20261006_07_destiny_one_voice_notes.sql # part 17 (v0.8): audio/mp4 attachments + duration_ms
│       ├── 20261009_02_destiny_one_destinyai.sql # part 18 (v0.10): DestinyAI member + 'assistant' chats
│       │                              # (apply after scripts/destiny-one/encrypt-messages.ts)
│       ├── 20260927_03_destiny_one_message_search.sql # Destiny One part 4: message search
│       ├── 20260927_02_destiny_one_leader_invites.sql # Destiny One part 3: leader invites
│       │                              # (needs_approval → access request; groups joined on approval)
│       ├── 20260927_01_destiny_one_admin.sql # Destiny One part 2: staff verification, d1_invites,
│       │                              # d1_settings, d1_admin_* functions, destiny_one_admin role
```

```
│
├── utils/                        # Utility modules
│   ├── supabase/                 # Supabase client factories
│   │   ├── service.ts            # Service role client
│   │   └── ...
│   └── ...
│
├── contexts/                     # React context definitions
├── docs/                         # Additional documentation, incl. mobile-app-scope.md (+ .pdf export)
│   └── content/                  # Source copy for policy/partner text — the live pages render
│                                  # an edited subset, so these are the fuller source. See its README.
├── apps/                         # Standalone companion apps that live in this repo but ship
│   ├── destiny-one/              # Destiny One — the members' messaging app (Expo SDK 58, React
│   │                              # Native, iOS + Android, Liquid Glass via expo-glass-effect).
│   │                              # Its own npm project (NOT a root workspace, so the website build
│   │                              # never installs React Native); imports @destiny/shared through a
│   │                              # file: link + metro.config.js. Backend: /api/app/v1/one/*.
│   │                              # Every screen built (design variants 1B + 1F). See "Destiny One" below.
│   ├── d1-admin/                 # D1 Admin — Destiny One on the Mac for leaders and admins (Tauri 2 +
│   │                              # React/Vite). Chat plus the admin queues (reports, access requests,
│   │                              # paused groups, people, invites, communities, feedback, audit log,
│   │                              # app settings), menu bar popover, notifications, Touch ID, session
│   │                              # in the Keychain. Own npm project; Rust shell in src-tauri/. Uses the
│   │                              # same APIs as the phone app and the web admin. See "D1 Admin" below.
│   ├── talent-buzzer/            # Talent show judge buzzers — one app on both machines: hub (LED) / Avolites
│   │                              # agent (Titan PC). Android tablets press -> Titan Web API cue +
│   │                              # X / golden takeover + sound on the LED screen. LAN only, own
│   │                              # package.json (dep: ws). See "Talent Show Buzzers" below.
│   └── live-caption/             # separately from the website. Currently:
│                                  # Live Caption — a macOS app (SwiftUI, Swift 6, XcodeGen) that
│                                  # captions live audio in real time with a local whisper.cpp model
│                                  # (Metal) for Destiny's AVL setup, displaying on a connected screen
│                                  # and/or publishing an NDI source. Audio never leaves the machine.
│                                  # whisper.cpp is a git submodule under Vendor/; the NDI SDK is
│                                  # license-gated and not committed. Build via `make` (see its README).
├── mobile/                       # Native SwiftUI iOS app (Swift 6, strict concurrency). Five tabs —
│                                  # Home/Sermons/Events/Give/More. XcodeGen project (project.yml is
│                                  # the source of truth; the .xcodeproj is generated, not committed).
│                                  # Talks only to the /api/app/v1/* BFF — no @destiny/shared import,
│                                  # the BFF does all normalising. Build via `make` (see mobile/Makefile).
│                                  # Asset catalog is generated from brand tokens: `npm run gen:ios-assets`.
├── packages/                     # npm workspaces (root package.json `workspaces: ["packages/*"]`)
│   └── shared/                   # @destiny/shared — framework-agnostic types/logic shared by the web
│                                  # app and the app BFF (the Swift app can't import TS). Ships raw TS
│                                  # (Next transpiles it via `transpilePackages`). Modules under src/:
│                                  # churchsuite/{events,dates,series,sanitize,ics}, design/tokens.ts,
│                                  # and destinyOne/{types,policy,client} (Destiny One wire types,
│                                  # safeguarding rules and the typed API client)
│                                  # (canonical DC brand palette/typography matching app/globals.css).
├── tests/                        # Playwright E2E specs (contact, cookies, give,
│                                  # navigation, sermons) — run via `npx playwright test`
├── public/                       # Static files
│   ├── img/                      # Church logos, backgrounds
│   ├── og/                       # Open Graph images (social share)
│   ├── fonts/                    # Custom fonts
│   └── ...
│
├── .github/                      # GitHub workflows
│   └── workflows/                # ci.yml (every PR), playwright.yml (manual e2e), AI content jobs
│
├── .claude/                      # Claude Code settings
├── .vscode/                      # VS Code workspace settings
├── next.config.ts                # Next.js configuration
├── playwright.config.ts          # E2E test configuration
├── eslint.config.mjs             # ESLint rules (React Compiler hints from react-hooks 7.1 are warnings, not errors)
├── postcss.config.mjs            # PostCSS plugins
├── tsconfig.json                 # TypeScript compiler options
├── package.json                  # Node dependencies and scripts
├── package-lock.json             # Locked versions
├── .gitignore                    # Git exclusions
├── CLAUDE.md                     # Project instructions
├── README.md                     # Public project README
└── REPOSITORY_DOCUMENTATION.md   # This file
```

Database Schema

Overview

The database uses Supabase (managed PostgreSQL) with Row-Level Security (RLS) for sensitive data. Public tables are readable via the anonymous key; member-only and admin tables use the service role key (server-to-database communication).

Tables

1. redirects

Purpose: Admin-managed short/vanity URLs (`destinytees.uk/<slug>` → anywhere), separate from the three hardcoded, build-time redirects in `next.config.ts` (e.g. `/prayer-request` → `/connect-card`).

```
CREATE TABLE redirects (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  slug text UNIQUE NOT NULL,            -- /short-path
  target_url text NOT NULL,             -- https://example.com or /absolute/path
  label text,                           -- Display name — shown as a pill on
                                         -- /admin/redirects and used as a Fuse.js
                                         -- search key; not unused
  active boolean DEFAULT true,          -- Toggle visibility
  created_at timestamptz DEFAULT now()
);

-- RLS, current: one policy — "Public can read active redirects" (select,
-- active = true). There is deliberately NO public/anon write policy: an
-- earlier "service role has full access" policy allowed anon to INSERT and
-- turn any slug into an open redirect, and was removed in
-- 20260711_02_fix_public_write_policies.sql. All writes go through
-- createServiceClient() (RLS-bypassing), never through a table policy.
```

Used by: - `app/[slug]/page.tsx` — resolved at request time (`force-dynamic`), as the fallback once a same-slug `posts` row doesn't match. Not build-time, and not `next.config.ts` — that file only holds three unrelated hardcoded redirects. - `app/admin/redirects/page.tsx` + `app/api/admin/redirects/*` — CRUD, plus a 30-day click count (from `engagement_events` , see §24) and a downloadable QR code per row (`components/admin/redirects/RedirectQrModal.tsx`) encoding `?s=qr` so printed scans are distinguishable from links clicked online.

---

3. site_banner

Purpose: Sitewide announcement banners (top of every page)

```
CREATE TABLE site_banner (
  id uuid PRIMARY KEY,
  message text NOT NULL,                -- Banner text (HTML allowed)
  active boolean DEFAULT false,         -- Toggle on/off
  type text,                            -- 'sitewide', 'alpha', 'youth_alpha', 'recovery', 'bible_course'
  link text,                            -- Optional URL for banner
  link_text text,                       -- CTA button text
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz
);

-- Types reference alpha_events for dynamic data (e.g., next Alpha start date)
-- RLS: Service role only
```

Used By: - `app/layout.tsx` fetches active banner on every request - Admin dashboard to edit banners

---

4. page_content

Purpose: Key-value store for editable site content

```
CREATE TABLE page_content (
  page text NOT NULL,                   -- 'give', 'general', etc.
  key text NOT NULL,                    -- 'account_name', 'service_time'
  value text DEFAULT '',                -- Content value
  updated_at timestamptz DEFAULT now(),
  PRIMARY KEY (page, key)
);

-- Example rows:
-- ('give', 'account_name', 'Destiny Church Tees Valley')
-- ('give', 'sort_code', '08-92-99')
-- ('general', 'service_time', '11am')
-- RLS: Service role only
```

Used By: - `lib/pageContent.ts` to fetch dynamic content - Admin dashboard to edit giving details, service times, etc.

---

5. contact_messages

Purpose: Messages from the public contact form

```
CREATE TABLE contact_messages (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  email text NOT NULL,
  subject text,                        -- Added later (migration 20260329)
  message text NOT NULL,
  created_at timestamptz DEFAULT now()
);

-- RLS: Public insert (form submission); service role read
```

Used By: - `app/contact/actions.ts` (server action) inserts submissions - Admin dashboard to view inquiries

Accessibility subject: Selecting "Accessibility" in `ContactForm` reveals a checkbox group (step-free access, BSL, hearing loop, large print) plus a free-text "anything else" field. There's no dedicated column for these — `submitContactForm` folds the selected items into the `message` body as a labelled "Accessibility requirements:" block before insert/email, so no migration was needed.

---

6. hire_enquiries

Purpose: Venue hire request submissions

```
CREATE TABLE hire_enquiries (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  email text NOT NULL,
  phone text NOT NULL,
  date_needed date NOT NULL,
  event_description text NOT NULL,
  approx_guests integer,
  created_at timestamptz DEFAULT now()
);

-- RLS: Public insert; service role read
```

Used By: - `app/hire/actions.ts` (server action) inserts submissions - Admin dashboard to view inquiries

---

7. alpha_events

Purpose: Alpha course event scheduling and details

```
CREATE TABLE alpha_events (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  type text NOT NULL,                  -- 'alpha', 'youth_alpha', 'recovery', 'bible_course'
  active boolean DEFAULT true,
  start_date date NOT NULL,
  signup_url text,                     -- External booking link
  format text,                         -- 'in_person', 'online', 'hybrid'
  location text,                       -- 'Destiny Centre', etc.
  meeting_platform text,               -- 'Zoom', 'Teams', etc. (if online)
  frequency text DEFAULT 'weekly',     -- 'weekly', 'fortnightly'
  custom_interval_days integer,        -- Override frequency
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz
);

-- RLS: Public read; service role write
```

Used By: - `app/layout.tsx` fetches to populate site-wide banner - `/alpha`, `/twelvetwo`, `/bible-course` and `/cap-money` pages display their next upcoming event - Admin (`/admin/alpha`, `/admin/recovery`, `/admin/bible-course`, `/admin/cap-money`) to manage event dates and URLs

> The `bible_course` and `cap` types are shared infrastructure for The Bible Course (Bible Society) and the CAP Money Course (Christians Against Poverty) — they reuse this table and the `/api/admin/alpha-events` routes rather than adding new ones. Added by migrations `20260712_alpha_events_bible_course_type.sql` and `20260807_alpha_events_cap_type.sql`.
>
> The type list lives in **`lib/courseEvents.ts`** (`COURSE_EVENT_TYPES`) and drives every banner surface *and* all four admin pages. Adding a type there without the matching `alpha_events_type_check` migration means inserts fail with a check-constraint error.
>
> **Adding a course is now three steps:** the migration, a `COURSE_EVENT_META` entry, and a `COURSE_ADMIN_PAGES` entry plus a four-line route file. See *Course admin pages* under Components. Its public landing page reuses `CourseEventCard` and `useCourseEvents` (see *Course pages* under Components) for the "when and where" card and the event fetch, but is otherwise its own page — see that section for why it isn't a fifth thin wrapper.

7b. featured_course

Purpose: Singleton setting for which course headlines the What's On "Courses" section.

```
CREATE TABLE featured_course (
  id integer PRIMARY KEY DEFAULT 1,        -- always 1 (singleton)
  course_id text NOT NULL DEFAULT 'bible_course'
    CHECK (course_id IN ('bible_course','cap','alpha','recovery')),
  updated_at timestamptz DEFAULT now()
);
-- RLS: enabled, deny-all ("service only"); server routes use the service key.
```

The four courses are defined in `lib/courses.ts` (each with a grid-card and a full-width "featured" config). The featured course renders as the wide banner; the other three render as cards, so all four are always visible. Chosen at `/admin/featured-course`.

Used By: - `app/whats-on/page.tsx` reads it and passes `featuredId` to `CoursesSection` - `app/api/admin/featured-course` (GET/PUT) — PUT revalidates `/whats-on` - Migration `20260712_02_featured_course.sql`

7b. featured_event

Purpose: The one ChurchSuite event promoted across the site — banner on What's On, first card on the homepage, and a site-wide popup.

Events are **not stored in this database** (they come live from the ChurchSuite calendar feed), so this holds a *pointer* to a feed event plus the copy that overrides it. `lib/events.server.ts` merges the overrides over live feed data at render time; anything left blank falls back to ChurchSuite.

```
CREATE TABLE featured_event (
  id integer PRIMARY KEY DEFAULT 1,          -- singleton, CHECK (id = 1)
  -- target
  event_identifier text,                     -- ChurchSuite occurrence id — the lookup key
  event_sequence   integer,                  -- series key (null for one-offs)
  event_id         integer,
  event_name       text,                     -- snapshots, for admin display and
  event_slug       text,                     --   the feed-outage fallback
  event_ends_at    timestamptz,
  active        boolean NOT NULL DEFAULT false,
  promote_from  timestamptz,                 -- optional scheduling window
  promote_until timestamptz,
  -- hero overrides (all nullable → fall back to the feed)
  headline text, blurb text, image_url text, image_path text,
  cta_text text, cta_link text,
  -- event popup
  popup_active boolean NOT NULL DEFAULT false,
  popup_title text, popup_body text,
  popup_image_url text, popup_image_path text,
  popup_cta_text text, popup_cta_link text,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);
-- RLS: enabled, deny-all ("service only"); server routes use the service key.
```

Why the hero and popup share one row rather than living in two tables: the popup must always advertise the same event as the hero. Two tables means either a duplicated `event_identifier` that drifts, or a foreign key that reduces to one row anyway — and auto-expiry plus the promote window have to apply identically to both surfaces. Two admin pages write to the one row, which is why `/api/admin/featured-event/popup` issues a **partial** update of `popup_*` only: a full-row upsert from that page would blank every hero override.

Constraints worth knowing: `active` requires an `event_identifier`; `popup_active` requires `active` (the event-popup admin page catches this and explains it rather than surfacing a raw Postgres error); `promote_until > promote_from`.

Expiry is computed in code, not SQL. The feed is forward-only, so a finished event simply stops appearing and the promotion ends. `event_ends_at` exists for the outage case: `fetchChurchSuiteEvents` returns `[]` on *any* error, which would otherwise silently un-feature the event for five minutes — so when the feed is empty but the stored copy is complete and the event has not ended, the hero renders from the snapshot.

Images reuse the existing public `popup-images` bucket with a `featured-event-` / `event-popup-` filename prefix, so there is no new bucket or storage policy.

Used By: - `lib/events.server.ts` — `getFeaturedEvent()`, `getActiveEventPopup()` - `components/events/FeaturedEventHero.tsx` (What's On), `components/home/WhatsOnSection.tsx` (carousel pin) - `components/events/EventPopup.tsx`, wired in `app/layout.tsx` - `app/api/admin/featured-event` (GET/PUT, revalidates `/whats-on` and `/`) and `.../popup` (PUT) - Migration `20260728_featured_event.sql`

---

8. site_popup

Purpose: Modal pop-ups that appear once per session/visitor

```
CREATE TABLE site_popup (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  active boolean DEFAULT false,
  title text,
  body text,                          -- Supports markdown/HTML
  cta_text text,                      -- Button label
  cta_link text,                      -- Button URL
  image_url text,                     -- Popup image
  show_once boolean DEFAULT true,     -- Show only once per browser session
  updated_at timestamptz DEFAULT now()
);

-- RLS: Service role only
```

Used By: - `app/layout.tsx` fetches active pop-up - Displayed by `SitePopup.tsx` component - Admin dashboard to manage pop-ups

---

9. nfc_tiles

Purpose: The admin-managed tiles on `/nfc`, the "digital back of seats" page

```
CREATE TABLE nfc_tiles (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  active boolean NOT NULL DEFAULT true,
  sort_order integer NOT NULL DEFAULT 0,
  title text NOT NULL,                -- ≤ 60 chars (DB check)
  subtitle text,                      -- ≤ 90 chars, the line under the title on the card
  icon text NOT NULL DEFAULT 'star',  -- Material Symbols Rounded ligature
  mode text NOT NULL DEFAULT 'info'   -- 'embed' = ChurchSuite form, 'info' = copy + CTA,
    CHECK (mode IN ('embed','info','event')),  -- 'event' = a ChurchSuite event's signup
  embed_url text,                     -- Required when mode = 'embed'; the signup URL when 'event'
  embed_size text NOT NULL DEFAULT 'md' CHECK (embed_size IN ('md','lg')),
  body text,                          -- ≤ 600 chars, mode = 'info'
  image_url text, image_path text,    -- Shared `popup-images` bucket, `nfc-` prefix
  cta_text text, cta_link text,       -- Link through to a full page on the site
  -- mode = 'event': a pointer into the ChurchSuite feed plus snapshots
  event_identifier text,              -- Occurrence identifier — the feed lookup key
  event_sequence integer,             -- Series key (null for one-offs); durable fallback lookup
  event_slug text,                    -- Resolved /whats-on slug
  event_name text,                    -- For the admin list without hitting the feed
  event_ends_at timestamptz,          -- End of the last occurrence — drives auto-expiry
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

-- CHECK (mode <> 'event' OR (event_identifier IS NOT NULL AND embed_url IS NOT NULL))
-- RLS: public read of active rows; writes service-role only
```

Why the two fixtures aren't rows: Connect Card and Giving are hardcoded as `PINNED_TILES` in `lib/nfcTiles.ts` and always render first. They have to survive an empty table, an unrun migration, or a Supabase blip ten minutes into a service — and a row with a delete button next to it is a row that eventually gets deleted. `/admin/nfc` shows them as read-only "Always shown" entries so it's obvious why they can't be edited there.

Why event tiles reuse `embed_url`: an event tile *is* an embed tile once resolved — the popup frames the event's ChurchSuite signup form the same way the Connect Card tile frames its form. So the resolved signup URL lands in `embed_url` rather than a column of its own: the renderer needs one widened condition instead of a second code path, and the row stays renderable when the feed is unreachable. `event_ends_at` is what hides the tile once the event has run; the row itself stays, so `/admin/nfc` can show an "Ended" pill and offer a repoint instead of leaving a mystery gap.

Used By: - `lib/nfcTiles.server.ts` → `getNfcTiles()`, read by `app/nfc/page.tsx` - `app/api/admin/nfc/route.ts` + `app/admin/nfc/page.tsx` for CRUD - `app/api/admin/events/route.ts` supplies the event picker (shared with `/admin/featured-event`)

9a. link_pages / link_blocks / link_form_submissions

Purpose: The Linktree-style pages at `/links` (slug `main`) and `/links/<slug>`, built in `/admin/links`. Migration: `supabase/migrations/20260922_01_link_pages.sql`.

```
CREATE TABLE link_pages (
  id uuid PRIMARY KEY, slug text UNIQUE,          -- 'main' renders at /links itself
  title text, bio text, avatar_url text,
  theme jsonb DEFAULT '{}',                       -- parsed by ThemeSchema; '{}' = Destiny Light
  socials jsonb DEFAULT '[]',                     -- [{ platform, url }]
  socials_position text,                          -- 'top' | 'bottom'
  seo_title text, seo_description text, og_image_url text,
  noindex boolean, published boolean,
  created_at timestamptz, updated_at timestamptz
);
CREATE TABLE link_blocks (
  id uuid PRIMARY KEY,                            -- generated in the browser, stable across saves
  page_id uuid REFERENCES link_pages ON DELETE CASCADE,
  sort_order int, active boolean,
  type text,                                      -- link|header|text|image|divider|event|embed|form
  data jsonb,                                     -- per-type shape, zod-checked (lib/linkPages/types.ts)
  starts_at timestamptz, ends_at timestamptz      -- optional schedule
);
CREATE TABLE link_form_submissions (
  id uuid PRIMARY KEY, page_id uuid ON DELETE CASCADE,
  block_id uuid ON DELETE SET NULL, block_label text,  -- responses outlive the form block
  email text, data jsonb,                         -- [{ id, label, value }] in field order
  created_at timestamptz
);
```

Why `data` is jsonb rather than columns: eight block types with mostly disjoint settings. The shape is enforced where it matters — `LinkBlockSchema` parses every block in the admin API before anything is written, and the renderer safeParses on the way out and skips anything that fails, so a bad row is a missing block rather than a broken page. It's the same split post content blocks use.

Why a save RPC: `link_page_save(page_id, page, blocks)` updates the page, deletes removed blocks and upserts the rest (array position = sort order) in one transaction. The editor saves a page as a unit; doing that as separate PostgREST calls could leave a page half-saved. The upsert's `WHERE page_id = …` stops a request from moving another page's block by naming its id. Execute is granted to `service_role` only.

Why the main page is seeded: the migration creates `main` with the six Next Steps that `/links` used to hardcode, so the switch-over changes the look and loses nothing. Those six also stay in `lib/linksSteps.ts` as the fallback `/links` renders if the database can't answer.

RLS: public select on published pages and on active blocks of published pages; submissions are service-only (deny-all), since they hold personal data. All writes go through `/api/admin/links/*`.

Used By: - `lib/linkPages/linkPages.server.ts` → `getLinkPage()`, read by `components/links/LinkPageRoute.tsx` - `app/api/admin/links/*` + `components/admin/links/*` for the editor - `app/api/links/submit/route.ts` writes submissions; `app/api/track/route.ts` validates clicks

---

10. hr_staff

Purpose: Employee/volunteer directory

```
CREATE TABLE hr_staff (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  first_name text NOT NULL,
  last_name text NOT NULL,
  email text,
  phone text,
  job_title text,
  department text,
  employment_type text NOT NULL DEFAULT 'full_time'
    CHECK (employment_type IN ('full_time','part_time','volunteer','contractor')),
  status text NOT NULL DEFAULT 'active'
    CHECK (status IN ('active','on_leave','left')),
  start_date date,
  end_date date,
  annual_leave_entitlement numeric DEFAULT 0,
  notes text,
  -- Public URL into the `staff-avatars` storage bucket. Set from /portal's
  -- account settings (POST /api/portal/me/avatar), never from the admin HR
  -- forms — a staff member's own picture, self-managed.
  avatar_url text,
  -- The staff record's backend login. Mandatory from creation onward:
  -- unique (no two staff share a login) and NOT NULL (there is no "no login"
  -- or "revoke access" state any more). References auth.users on delete set
  -- null so deleting the auth user never deletes the HR record.
  auth_user_id uuid NOT NULL UNIQUE REFERENCES auth.users (id) ON DELETE SET NULL,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

-- Trigger: hr_staff_updated_at auto-updates `updated_at` on every change
-- RLS: Service role only (accessed via proxy requiring auth)
```

Backend access is mandatory. Every staff record is linked to exactly one Supabase Auth login — the old optional `grant_access` toggle and the "no login" state are gone (`auth_user_id` is `unique` + `not null`, per migrations `20260902182649` / `20260902182651`). The staff form (`lib/staffLogins.ts`, `/api/admin/hr/staff/available-admins`) can either **create a brand-new staff-only login** or **link an existing admin's login**, so an admin added as staff reuses their identity rather than getting a duplicate one. Deleting a staff record that shares an admin identity leaves that admin's auth user and

`admin_roles` row intact — only a staff-only login is torn down with the record. The linked login is what powers `/portal`: a person is "staff" for the portal by having a linked `hr_staff` row, not by an `admin_roles` flag.

Used By: - HR admin to manage staff directory - `/admin/hr` dashboard - `/portal` + `/api/portal/*` — staff self-service, gated by the linked login (`lib/staffPortalAuth.ts`)

10b. admin_onboarding

Purpose: Per-admin onboarding progress for `/admin` — one row per admin, read/written only by `app/api/admin/onboarding`

```
CREATE TABLE admin_onboarding (
  auth_user_id uuid PRIMARY KEY REFERENCES auth.users (id) ON DELETE CASCADE,
  completed_steps text[] NOT NULL DEFAULT '{}',  -- TourSection ids from lib/adminOnboarding.ts
  gate_seen boolean NOT NULL DEFAULT false,      -- welcome modal answered (started or skipped)
  dismissed boolean NOT NULL DEFAULT false,      -- checklist hidden by the user
  updated_at timestamptz NOT NULL DEFAULT now()
);

-- RLS: Service role only (deny-all "service only" policy, mirroring admin_roles)
```

Created lazily by the API's upsert — there is no backfill, and super admins simply never grow a row since `toursFor()` returns nothing for them. See `lib/adminOnboarding.ts`.

Used By: - `app/api/admin/onboarding` — the only reader/writer, keyed off the caller's own cookie session - `lib/onboardingProgress.ts` — the client cache read by the welcome gate, the checklist and the running tour

> **Not yet applied.** This migration (`supabase/migrations/20260821_admin_onboarding.sql`) has not been run against the project — `CLAUDE.local.md` has no Supabase PAT to apply it with. Until it is, `GET /api/admin/onboarding` 500s and the client fails open (gate already seen, checklist already dismissed) rather than blocking anyone. Apply the migration and run `get_advisors` once a PAT is added.

10c. admin_roles

Purpose: Access levels for `/admin` — eleven independent booleans per admin login, checked by `middleware.ts` on every `/admin/*` and `/api/admin/*` request

```
CREATE TABLE admin_roles (
  auth_user_id uuid PRIMARY KEY REFERENCES auth.users (id) ON DELETE CASCADE,
  email text,
  training_admin boolean NOT NULL DEFAULT false,
  event_admin boolean NOT NULL DEFAULT false,
  store_admin boolean NOT NULL DEFAULT false,
  site_admin boolean NOT NULL DEFAULT false,
  hr_admin boolean NOT NULL DEFAULT false,  -- /admin/hr
  design_admin boolean NOT NULL DEFAULT false,  -- /admin/design — the design ticket queue
  sermon_admin boolean NOT NULL DEFAULT false,  -- /admin/sermons — publishing audio to Buzzsprout
  safeguarding_admin boolean NOT NULL DEFAULT false,  -- /admin/destiny-one/safeguarding — reports, paused groups, audited transcripts
  destiny_one_admin boolean NOT NULL DEFAULT false,   -- /admin/destiny-one — invites, approvals, members, communities (no messages)
  super_admin boolean NOT NULL DEFAULT false,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);

-- RLS: Service role only (deny-all "service only" policy, same as every other table)
```

Distinct from the legacy, unused `admin_users` table in `supabase/schema.sql` (username/password_hash — predates the Supabase Auth migration; not read by any app code). Managed by Super Admins at `/admin/users`. See Authorization Layers for the role → route mapping and `lib/adminRoles.ts` for the enforcement logic.

Used By: - `middleware.ts` — role check on every admin request - `/admin/users` + `app/api/admin/users/**` — role management UI - `app/api/admin/me` — the caller's own email + role flags, shared by the sidebar, header and ⌘K palette via `lib/useAdminSession.ts`; `app/api/admin/me/roles` remains for existing callers - `app/api/admin/dashboard` — the admin home's summary numbers in one request (counts done in the database, not by downloading whole tables). Open to any admin; each section is only queried if the caller's roles (from the `x-dc-actor-roles` header middleware sets) cover it - Performance: `vercel.json` pins functions to `lhr1` (next to Supabase `eu-west-1`), and `middleware.ts` caches each user's roles for 15s per instance, so a role change can take up to 15s to take effect - `app/api/admin/search` — cross-section record search for the ⌘K palette. Open to any signed-in admin, but it re-reads the caller's roles and only queries the sections that role can open

---

11. hr_leave_requests

Purpose: Time-off and leave requests

```
CREATE TABLE hr_leave_requests (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  staff_id uuid NOT NULL REFERENCES hr_staff(id) ON DELETE CASCADE,
  type text NOT NULL DEFAULT 'holiday'
    CHECK (type IN ('holiday','sick','other')),
  start_date date NOT NULL,
  end_date date NOT NULL,
  days numeric DEFAULT 0,              -- Calculated number of working days
  reason text,
  status text NOT NULL DEFAULT 'pending'
    CHECK (status IN ('pending','approved','rejected')),
  reviewed_by text,                    -- Name of approver
  reviewed_at timestamptz,
  created_at timestamptz DEFAULT now()
);

-- RLS: Service role only
-- Indexes: staff_id, status
```

Used By: - HR staff to request/approve leave - Admin dashboard for leave tracking

---

12. hr_documents

Purpose: HR document library (contracts, policies, payslips)

```
CREATE TABLE hr_documents (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  staff_id uuid REFERENCES hr_staff(id) ON DELETE CASCADE,  -- NULL = org-wide
  title text NOT NULL,
  category text NOT NULL DEFAULT 'other'
    CHECK (category IN ('contract','policy','payslip','other')),
  file_path text NOT NULL,             -- Supabase Storage path
  file_name text NOT NULL,
  mime_type text,
  size_bytes bigint,
  uploaded_by text,
  created_at timestamptz DEFAULT now()
);

-- RLS: Service role only
-- Indexes: staff_id
-- Files stored in 'hr-documents' private bucket (signed URLs only)
```

Used By: - Admin to upload and organize HR documents - Staff to download their documents via signed URLs

---

13. hr_reviews

Purpose: Appraisals and 1-to-1 meeting records

```
CREATE TABLE hr_reviews (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  staff_id uuid NOT NULL REFERENCES hr_staff(id) ON DELETE CASCADE,
  review_date date NOT NULL,
  type text NOT NULL DEFAULT 'one_to_one'
    CHECK (type IN ('appraisal','one_to_one')),
  reviewer text,                       -- Legacy free-text reviewer name; kept for old rows and non-admin reviewers
  reviewer_auth_user_id uuid           -- 20260922: the auth user who owns this review (references auth.users)
    REFERENCES auth.users(id),
  summary text,                        -- Notes/outcomes
  next_review_date date,               -- Scheduled follow-up
  reminder_sent_at timestamptz,        -- 20260824: set once the digest has flagged this review, so it isn't re-sent daily
  created_at timestamptz DEFAULT now()
);

-- RLS: service role full access, PLUS a "reviewer can read own reviews" SELECT
--      policy (reviewer_auth_user_id = auth.uid()) — the first non-deny-all
--      policy in this codebase (20260922_hr_review_reviewer_scoping.sql)
-- Indexes: staff_id; reviewer_auth_user_id; a partial index on next_review_date WHERE reminder_sent_at IS NULL
```

Reviewer scoping (20260922). `reviewer_auth_user_id` links a review to the admin who actually ran it, so a reviewer can self-serve "my reviews" without a service-role fetch of the whole table. The admin UI's reviewer field is now a picker sourced from the existing `available-admins` endpoint (writing both the free-text `reviewer` label and `reviewer_auth_user_id`) rather than free text. `GET /api/admin/hr/reviews?mine=1` filters to the caller's own reviews, enforced server-side from the same actor headers `lib/audit.server.ts` reads (`AUDIT_ACTOR_HEADERS`) — no actor on the request yields an empty list, never everyone's reviews. The select-own RLS policy is defense-in-depth for any future user-context client; today's admin UI still goes through the service role and filters explicitly in the route.

Used By: - Admin to log reviews - HR dashboard to track review schedule - `GET /api/admin/hr/reviews?mine=1` — the reviewer's own reviews (self-service) - `GET /api/cron/hr-review-reminders` — daily digest of reviews due within 14 days (see API Routes)

13b. hr_checklist_templates / hr_checklist_template_items / hr_checklist_items

Purpose: New-hire onboarding and leaver offboarding paperwork (contract, DBS/safeguarding checks, induction, equipment return) — **distinct from and unrelated to** the admin-dashboard product tour (`admin_onboarding` , `lib/demoMode.ts`), which is a UI walkthrough. The `hr_checklist_*` naming keeps that separation explicit. Migration: `supabase/migrations/20260825_hr_checklists.sql` .

```
CREATE TABLE hr_checklist_templates (       -- reusable named lists
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  kind text NOT NULL CHECK (kind IN ('onboarding','offboarding')),
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()   -- hr_set_updated_at() trigger
);

CREATE TABLE hr_checklist_template_items (  -- the items on a template
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  template_id uuid NOT NULL REFERENCES hr_checklist_templates(id) ON DELETE CASCADE,
  label text NOT NULL,
  sort_order integer NOT NULL DEFAULT 0,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE hr_checklist_items (           -- a staff member's own live checklist
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  staff_id uuid NOT NULL REFERENCES hr_staff(id) ON DELETE CASCADE,
  kind text NOT NULL CHECK (kind IN ('onboarding','offboarding')),
  label text NOT NULL,
  is_done boolean NOT NULL DEFAULT false,
  done_at timestamptz,
  due_date date,
  sort_order integer NOT NULL DEFAULT 0,
  notes text,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()   -- hr_set_updated_at() trigger
);

-- RLS: all three service-role only ("service only" deny-all policy)
-- Indexes: template_items(template_id); items(staff_id) and items(staff_id, kind)
```

Semantics: *starting* a checklist on a staff member **copies** the template's items into that person's own `hr_checklist_items` rows. Editing the template afterward never retroactively changes an already-started checklist, and HR can add one-off items to an individual's list freely.

Used By: - `/admin/hr/checklists` — template management (HR Admin) - `/admin/hr/staff/[id]` — a staff member's live checklists - `components/admin/hr/ChecklistUI.tsx` , `app/api/admin/hr/checklists` , `app/api/admin/hr/checklist-templates`

---

14. jobs

Purpose: Job listings and applications

```
CREATE TABLE jobs (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  title text NOT NULL,                -- Job title
  slug text UNIQUE NOT NULL,          -- URL slug
  description text NOT NULL,          -- Full job description (markdown)
  department text,                    -- 'Worship', 'Admin', etc.
  employment_type text,               -- 'full_time', 'part_time'
  salary_range text,                  -- Free text (e.g., "£25,000-£30,000")
  closing_date date,                  -- Application deadline
  active boolean DEFAULT true,        -- Hide expired jobs
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

-- RLS: Public read (active only); service role write
-- Indexes: slug, active
```

Used By: - `/jobs` page to list open positions - `/jobs/[slug]` for job detail pages - Admin to create/edit job postings

---

15. job_applications

Purpose: Submitted job applications

```
CREATE TABLE job_applications (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  job_id uuid NOT NULL REFERENCES jobs(id) ON DELETE CASCADE,
  name text NOT NULL,
  email text NOT NULL,
  phone text NOT NULL,
  cv_url text,                        -- Uploaded CV (signed URL)
  cover_letter text,                  -- Freeform text
  status text DEFAULT 'pending'
    CHECK (status IN ('pending','reviewing','rejected','offered')),
  created_at timestamptz DEFAULT now()
);

-- RLS: Public insert (application form); service role read
-- Indexes: job_id, status
```

Used By: - Job application form (`app/jobs/ApplyForm.tsx`) - Admin dashboard to review applications

---

16. service_status

Purpose: Runtime health / kill-switch state for backend services (currently only `smart_search`). The self-healing health check (`GET /api/health/smart-search`) writes this row; the site reads `enabled` to decide whether to render Smart Search.

```
CREATE TABLE service_status (
  service               text PRIMARY KEY,   -- 'smart_search'
  enabled               boolean NOT NULL DEFAULT true,
  reason                text,               -- why it was last disabled (OpenAI error, etc.)
  last_check_at         timestamptz,
  next_check_at         timestamptz,        -- daily when healthy, hourly while down
  consecutive_failures  int NOT NULL DEFAULT 0,
  updated_at            timestamptz NOT NULL DEFAULT now()
);

-- RLS enabled with NO policies: only the service-role client (which bypasses
-- RLS) touches this table; anon/authenticated get no access.
```

Migration: `supabase/migrations/20260608_service_status.sql` .

Used By: - `lib/serviceStatus.ts` — `getSmartSearchStatus()` / `isSmartSearchEnabled()` (reads **fail open**: a status-store blip never disables the feature) and `setSmartSearchStatus()` - `app/layout.tsx` — reads `isSmartSearchEnabled()` to decide whether to mount `FloatingSmartSearch` - `GET /api/health/smart-search` — the cron health check that flips `enabled` and schedules the next check

---

17. posts

Purpose: Generic published pages served by the `/[slug]` catch-all route (freeform site pages outside the main nav)

```
CREATE TABLE posts (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  title text NOT NULL,
  slug text UNIQUE NOT NULL,
  body text,
  is_published boolean NOT NULL DEFAULT false,
  -- Page settings (20260927_04_posts_page_settings.sql)
  hero_style text NOT NULL DEFAULT 'plain',   -- plain | image | banner (CHECK)
  hero_image_url text,
  subtitle text,
  description text,                           -- meta description + social card
  og_image_url text,                          -- share image; falls back to hero_image_url
  show_rails boolean NOT NULL DEFAULT true,    -- promo rails beside the page at 1600px+
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

-- RLS: Service role only; public read happens server-side via lib/posts.server.ts
```

Why presets, not styling: the page settings are deliberately page-level choices (how the page opens, what it says when shared, whether the rails show) rather than per-element colours, spacing or layout. That's what keeps the Posts editor a page builder that non-designers can use without producing off-brand pages. Columns/sections, per-block styling, revisions and scheduling were considered and left out on purpose. `lib/posts-fields.ts` validates these fields for both the POST and PATCH routes.

Used By: - `lib/posts.server.ts` (`getPostForView`) for the public `/[slug]` catch-all - `app/api/admin/posts` CRUD, admin dashboard to write pages

---

17b. training_categories / training_subgroups / training_folders / training_posts

Purpose: The `/training` member resource library — a category → sub-group (optionally password-protected) → folder → post hierarchy.

```
CREATE TABLE training_categories (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL, slug text UNIQUE NOT NULL, description text, icon text,
  sort_order integer NOT NULL DEFAULT 0,
  is_published boolean NOT NULL DEFAULT true,
  created_at timestamptz DEFAULT now(), updated_at timestamptz DEFAULT now()
);

CREATE TABLE training_subgroups (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  category_id uuid NOT NULL REFERENCES training_categories(id),
  name text NOT NULL, slug text NOT NULL, description text,
  password_hash text,                  -- NULL = no password gate; never returned to clients, only has_password
  sort_order integer NOT NULL DEFAULT 0,
  is_published boolean NOT NULL DEFAULT true,
  created_at timestamptz DEFAULT now(), updated_at timestamptz DEFAULT now()
);

CREATE TABLE training_folders (
  id uuid PRIMARY KEY DEFAULT extensions.uuid_generate_v4(),
  subgroup_id uuid NOT NULL REFERENCES training_subgroups(id),
  name text NOT NULL, sort_order integer NOT NULL DEFAULT 0,
  created_at timestamptz DEFAULT now(), updated_at timestamptz DEFAULT now()
);

CREATE TABLE training_posts (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  subgroup_id uuid NOT NULL REFERENCES training_subgroups(id),
  folder_id uuid REFERENCES training_folders(id),   -- NULL = ungrouped within the subgroup
  title text NOT NULL, slug text NOT NULL, summary text, body text,
  min_read_seconds integer NOT NULL DEFAULT 0,
  sort_order integer NOT NULL DEFAULT 0,
  is_published boolean NOT NULL DEFAULT false,
  created_at timestamptz DEFAULT now(), updated_at timestamptz DEFAULT now()
);

-- RLS: Service role only. Public read of published rows happens server-side
-- via lib/training.server.ts; password_hash is never selected into client-facing shapes.
```

Used By: - `lib/training.server.ts` (`getTrainingTree`) for the public `/training` tree - `app/api/training/unlock` to verify a sub-group password - `app/api/training/posts/[id]/timer` to enforce `min_read_seconds` before a post can be marked complete (HMAC-signed read timer; see Training API) - `app/api/admin/training/{categories,subgroups,folders,posts}` CRUD, admin dashboard

---

Row-Level Security (RLS) Strategy

18. **products / product_variants / orders / order_items** (Shop)

Purpose: The Stripe-powered store (`/shop`), replacing the old WooCommerce site. Physical apparel with size + colour variants and per-variant stock; collection-only fulfilment. Prices are integer **pennies** (GBP). Migration: `supabase/migrations/20260708_shop.sql` , extended by `20260708_shop_product_fit.sql` (`fit`) and `20260711_05_shop_product_type.sql` (`product_type`).

```
CREATE TABLE products (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  slug text UNIQUE NOT NULL,
  name text NOT NULL,
  description text,
  base_price_pennies integer NOT NULL DEFAULT 0,
  category text,
  fit text NOT NULL DEFAULT 'unisex'          -- 'male'|'female'|'unisex'|'kids' — drives size chart + storefront label
    CHECK (fit IN ('male','female','unisex','kids')),
  product_type text NOT NULL DEFAULT 'clothing'  -- 'clothing'|'books'|'other' — colour/size matrix vs. format vs. free-form
    CHECK (product_type IN ('clothing','books','other')),
  images jsonb NOT NULL DEFAULT '[]',   -- [{ url, path, alt }]
  is_published boolean NOT NULL DEFAULT false,
  is_featured boolean NOT NULL DEFAULT false,
  sort_order integer NOT NULL DEFAULT 0,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

CREATE TABLE product_variants (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id uuid REFERENCES products(id) ON DELETE CASCADE,
  size text, color text, color_hex text, sku text,
  price_pennies integer,                -- null → inherit product base price
  stock integer NOT NULL DEFAULT 0,
  is_active boolean NOT NULL DEFAULT true,
  sort_order integer NOT NULL DEFAULT 0,
  UNIQUE (product_id, size, color)
);

CREATE TABLE orders (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  order_number text UNIQUE NOT NULL,    -- human-friendly, e.g. DT-7F3K9Q2X
  stripe_payment_intent_id text UNIQUE,
  customer_name text NOT NULL, customer_email text NOT NULL, customer_phone text,
  notes text,
  status text NOT NULL DEFAULT 'pending',  -- pending|paid|fulfilled|cancelled|refunded
  fulfillment_method text NOT NULL DEFAULT 'collection',
  subtotal_pennies integer, total_pennies integer, currency text DEFAULT 'gbp',
  paid_at timestamptz, created_at timestamptz DEFAULT now(), updated_at timestamptz DEFAULT now()
);

CREATE TABLE order_items (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id uuid REFERENCES orders(id) ON DELETE CASCADE,
  product_id uuid, variant_id uuid,     -- snapshot fields survive product deletion
  product_name text NOT NULL, size text, color text,
  unit_price_pennies integer NOT NULL, quantity integer NOT NULL DEFAULT 1
);
-- RLS: single "service only" policy on all four (like jobs). Public storefront
-- reads via lib/shop.server.ts; checkout/webhook write via the service role.
-- Storage: public `product-images` bucket for product photos (WebP).
```

> **Storage writes are service-role only** (migration `20261017_01_bug_fixes`). `builder-media`, `post-media`, `product-images` and `shop-hero-images` used to let any `authenticated` user insert/update/delete, on the old assumption that "authenticated = staff". Destiny One sign-in creates Auth users for members, so those policies were dropped; every upload already goes through an admin API route using the service client. Public read policies are unchanged.

Used By: `/shop` storefront, `/admin/store` CRUD, `POST /api/store/checkout`, `POST /api/webhooks/stripe`. `description` is HTML written via the shared `RichTextEditor` (same component used for posts/training) — the admin editor no longer has a markdown Write/Preview toggle, and the public product page renders it with `dangerouslySetInnerHTML` (no markdown fallback).

---

19. **shop_hero_slides** (Shop hero)

Purpose: Dynamic, admin-editable hero at the top of `/shop`. Multiple active slides auto-rotate (crossfade) on the storefront; with no active slides the shop shows its static "The Destiny Store" masthead. Migration: `supabase/migrations/20260710_shop_hero.sql`.

```
CREATE TABLE shop_hero_slides (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  active boolean NOT NULL DEFAULT true,
  heading text, subheading text,
  cta_text text, cta_link text,
  image_url text, image_path text,      -- storage path in shop-hero-images bucket
  sort_order integer NOT NULL DEFAULT 0,
  created_at timestamptz DEFAULT now(), updated_at timestamptz DEFAULT now()
);
-- RLS: single "service only" policy (like the shop tables). Public storefront
-- reads active slides via lib/shop.server.ts → getActiveShopHeroSlides().
-- Storage: public `shop-hero-images` bucket for hero backgrounds (WebP).
```

Used By: `/shop` storefront (`components/shop/ShopHero.tsx`), `/admin/store/hero` CRUD.

20. Base-schema & legacy tables (`supabase/schema.sql`)

`supabase/schema.sql` defines the original **base-schema** tables that predate the migration-per-feature workflow. They still exist in the live database (and are recreated by a fresh rebuild), but are **not read by the current Next.js app** — the live site serves sermons directly from the YouTube Data API (`lib/youtube.ts`), so these DB tables are effectively legacy/service-only and are populated (if at all) by external tooling rather than the web app:

Table	Shape (key columns)	Status
`sermons`	`id` , `title` , `date` , `podcast_*` , `youtube_video_id` , `summary` , `summary_points` (jsonb), `transcript`	Legacy sermon catalogue — app now reads from YouTube API, not this table
`sermon_transcripts`	`sermon_id` (→ sermons), `segments` (jsonb), `status`	Legacy per-sermon transcript segments
`sermon_link_suggestions`	`podcast_guid` , `youtube_video_id` , `status`	Legacy podcast↔YouTube link matching
`ai_reports`	`sermon_id` , `issue_type` , `name` , `email` , `description`	Legacy "report an issue" submissions
`auth_users`	`email` (unique), `last_login`	Legacy auth bookkeeping (current auth is Supabase Auth)
`admin_users`	`username` , `password_hash`	Legacy admin credentials (current admin auth is Supabase Auth)

All base-schema tables have RLS enabled with a deny-all `"service only"` policy (see the `do $$ … $$` block at the foot of `schema.sql` , codified for rebuilds by `supabase/migrations/20260711_rls_harden_base_tables.sql`). The service/secret key bypasses RLS; the public anon key gets nothing.

Orphaned Studio tables: `studio_assets` and `studio_components` were created by `supabase/migrations/20260514_studio_v2_schema.sql` for an experimental page/Studio builder. The builder was later removed, but `20260711_06_remove_page_builder.sql` only drops `builder_pages` / `builder_templates` / `builder_media` — the two `studio_*` tables were **never dropped**, so they still exist in the database (RLS service-only) as dead schema. They are unused by the app.

All tables have RLS enabled. Access rules:

Table	Public Read	Authenticated Read	Service Role	Purpose
redirects	Yes	-	Yes	Public navigation
site_banner	Yes	-	Yes	Show on every page
page_content	-	-	Yes	Protect sensitive values
contact_messages	-	-	Yes	Protect submissions
hire_enquiries	-	-	Yes	Protect submissions
alpha_events	Yes	-	Yes	Public event dates
featured_course	-	-	Yes	Featured course setting (read via server component)
site_popup	-	-	Yes	Protect pop-up content
nfc_tiles	Yes	-	Yes	Public tiles on /nfc (read via server component)
link_pages / link_blocks	Yes	-	Yes	Published pages and their active blocks (read via server component)
link_form_submissions	-	-	Yes	Form responses — personal data, deny-all "service only"
hr_* (staff, leave, reviews, docs)	-	-	Yes	Sensitive HR data
jobs	Yes	-	Yes	Public listings
job_applications	-	-	Yes	Protect applications
service_status	-	-	Yes	Service health / Smart Search kill-switch (service-role only)
posts	-	-	Yes	Freeform pages (public read via server components)
training_categories / training_subgroups / training_folders / training_posts	-	-	Yes	/training resource library (public read via server components; sub-group passwords hashed)
products / product_variants	-	-	Yes	Shop catalogue (public read via server components)
orders / order_items	-	-	Yes	Store orders (written by Stripe webhook)
shop_hero_slides	-	-	Yes	Editable /shop hero (public read via server components)
sermons / sermon_transcripts / sermon_link_suggestions / ai_reports / auth_users / admin_users	-	-	Yes	Base-schema legacy tables (deny-all "service only"; not read by the app)
studio_assets / studio_components	-	-	Yes	Orphaned Studio-builder tables (never dropped; unused)
engagement_events	-	-	Yes	Click analytics across shortlinks / nfc / links (deny-all "service only"; read via `security definer` rollup RPCs)
ip_reputation_ranges	-	-	Yes	VPN/Tor/datacenter/Apple-Private-Relay CIDR ranges (deny-all "service only"; read only by the `before insert` trigger on `engagement_events`)

23. audit_log / audit_reports

Purpose: One row for every change an admin makes, and the weekly AI report written over them. This is the record behind `/admin/audit` — "who added that to the store", "who gave them Super Admin", "what did we delete last month".

```
create table audit_log (
  id           bigint generated always as identity primary key,
  created_at   timestamptz not null default now(),

  actor_id     uuid,                        -- deliberately NOT an FK (see below)
  actor_email  text,
  actor_roles  text[] not null default '{}',  -- what they held *at the time*

  action       text not null,               -- create | update | delete | upload |
                                            -- approve | reject | moderate |
                                            -- revalidate | login | logout
  section      text not null,               -- store | hr | posts | training | …
  entity       text not null,               -- "product", "leave request"
  entity_id    text,
  entity_label text,                        -- "Faith Hoodie", captured at the time

  summary      text not null,               -- one plain-English sentence
  changes      jsonb,                       -- { field: { from, to } }
  metadata     jsonb,                       -- anything that isn't a field change

  method       text,  path text,  ip text,  user_agent text
);

create table audit_reports (
  id           uuid primary key default gen_random_uuid(),
  period_start timestamptz not null,
  period_end   timestamptz not null,
  headline     text,
  body         text not null,               -- markdown, written by the model
  stats        jsonb not null default '{}',   -- counted in code, not by the model
  entry_count  integer not null default 0,
  emailed_to   text[] not null default '{}',
  created_at   timestamptz not null default now()
);
```

Why a table rather than server logs. The question people ask names a *thing* ("who added the Faith Hoodie"), and answering it needs the actor as a person, the entity by its name, and the fields that moved. None of that survives in a request log, and Supabase's own logs are short-retention anyway.

No foreign key on `actor_id`. Removing someone's admin login is itself one of the things this table exists to record; `on delete cascade` / `set null` would erase their history at exactly the moment it matters. The email is denormalised for the same reason, and the roles are a snapshot so "they were a Store Admin when they did this" stays true after their access changes.

Every entry is a sentence first. `summary` is what the search box matches and the only thing the AI reads; the structured columns exist for filtering and for the detail view. Call sites write that sentence by hand — see the entity/label conventions in `lib/audit.server.ts`.

Secrets never get in. Everything routes through `sanitiseValue()` in `lib/audit.ts`, which redacts any field whose name contains password/token/secret (so `password_hash` is caught as well as `password`), clips long values, and summarises big nested arrays. HR call sites additionally pass `redactFields` for free text like leave reasons and one-to-one notes: *that* they changed is recorded, what they say is not — the same instinct that keeps HR out of the ⌘K search.

Append-only in practice. Nothing in the app updates or deletes a row except the retention purge at the end of the weekly cron (`AUDIT_RETENTION_DAYS`, default 365).

Used By: - `lib/audit.server.ts` — `recordAudit()`, the only writer - `app/api/admin/audit/*` — list, ask, reports (Super Admin only) - `app/api/cron/audit-weekly-report` — writes `audit_reports`, runs the `audit_log` purge and `purge_old_notifications()` - `app/admin/audit/page.tsx` — the console

---

24. `engagement_events` (click analytics — storage layer)

Purpose: One row every time someone follows something the church published, across three surfaces that are really the same act arriving by three routes: a shortlink redirect (`destinytees.uk/<slug>`), a tap on a `/nfc` seat-back tile, and a press on a `/links` Next Steps card. It answers the question print has never been able to answer — "did the Alpha flyer work?" — because a redirect renders no HTML, so Vercel Analytics never sees the hop.

> **Status: live.** Every consumer named below now exists: `app/[slug]/page.tsx` records a redirect hit via `after()`, `POST /api/track` records `/nfc` and `/links`, `/admin/analytics` (Site Admin + Super Admin) reads it through `GET /api/admin/analytics`, and `/api/cron/analytics-anonymise` runs nightly at 03:00 via `vercel.json`. The table was landed a few commits ahead of the pages that read/write it so the schema was stable before anything was built against it — see the migration's own header comment for the full reasoning.

```
create table engagement_events (
  id           bigint generated always as identity primary key,
  created_at   timestamptz not null default now(),

  source       text not null,          -- 'redirect' | 'nfc' | 'links' (text, not
                                        -- an enum: a 4th surface is an app change)
  target_key   text not null,          -- the slug / nfc_tiles id / /links href
  target_label text,                   -- human name captured at click time, so a
                                        -- rename or delete doesn't rewrite history
  redirect_id  uuid references redirects(id) on delete set null,  -- NOT cascade

  -- Where from (Vercel x-vercel-ip-* headers, read in the handler)
  country text, region text, city text,
  referrer_host text, referrer_url text,
  utm_source text, utm_medium text, utm_campaign text,
  src_tag text,                         -- ?s= — qr | nfc | print | social

  -- What on (parsed from the UA in lib/botDetect.ts)
  device text, os text, browser text, user_agent text,

  is_bot boolean not null default false,-- flagged, never dropped (link-preview crawlers)

  -- Who (pseudonymous)
  ip              text,                -- nulled in place after 90 days
  ip_anonymised_at timestamptz,
  visitor_hash    text                 -- sha256(ip|ua|salt), OUTLIVES the raw IP
);
```

One table for three surfaces, aggregated in Postgres. Same act, same index strategy, one purge job, one query. Reads go through two `security definer` functions — `engagement_rollup()` returns the whole `/admin/analytics` page in one RPC (totals, daily timeseries bucketed in `Europe/London`, and top-N breakdowns by source/target/country/referrer/device/browser/src-tag), and `engagement_top()` does one dimension. `engagement_top` checks `p_column` against a **fixed allowlist** before interpolating it — the function is `security definer`, so an unchecked identifier would be an injection point reachable from the admin API. Both are `revoke`d from `public` and granted only to `service_role`. Aggregating in SQL (not the JS facet-scan the audit log uses) because here the numbers *are* the product — an approximate click count is a wrong click count.

`visitor_hash` deliberately outlives the raw IP. Uniques are counted from the hash, so `engagement_anonymise_ips()` (a nightly job that `UPDATE`s `ip` to null after `ANALYTICS_IP_RETENTION_DAYS`, default 90) ages out the only personal-data column without retroactively collapsing last quarter's visitor numbers — which counting `distinct ip` would have done. The row keeps counting; it just stops being personal data. Caveat baked into the design: on a Sunday the whole congregation shares the church WiFi's single NAT IP, so this under-counts `/nfc` uniques — taps are the honest metric there.

Bots flagged, never dropped. A shortlink posted in a WhatsApp group is fetched once per member by the preview crawler; deleting those rows would make a popular link look broken with nothing to explain why, so they are written with `is_bot = true` and the page hides them by default.

Indexes are all `(column, created_at desc)` — filter and ordering served by one scan, same shape as `audit_log` — plus a partial `where is_bot = false` index for the default bots-excluded view.

RLS: deny-all `"service only"` policy, same as `audit_log`. Note `redirects` itself is publicly readable; this log of who followed them is not.

Used by: - `lib/engagement.server.ts` — `recordEngagement()`, the only writer - `app/[slug]/page.tsx` (redirects, via `after()`) and `app/api/track/route.ts` (nfc/links beacon) - `app/api/admin/analytics/route.ts` (`engagement_rollup` RPC) + `app/admin/analytics/page.tsx` — Site Admin / Super Admin - `app/api/cron/analytics-anonymise/route.ts` — the nightly IP-anonymise job, `0 3 * * *`

`ip_category` column (added by `20260828_ip_reputation.sql`, see §24b below) — a second, independent signal from `is_bot`: not "is this a machine", but "is this connection routed through something that isn't the visitor's own network". Set once, at write time, by a `before insert` trigger, so it's correct regardless of which of the two write paths inserts the row.

24b. `ip_reputation_ranges` (VPN / Tor / datacenter / Apple Private Relay)

Purpose: The CIDR range lists behind `engagement_events.ip_category`. Four public sources, refreshed weekly:

Category	Rows (approx.)	Source
`tor`	~1,400	torproject.org's official bulk exit list
`vpn`	~11,000 CIDRs	X4BNet `lists_vpn` — known commercial VPN netblocks
`datacenter`	~43,000 CIDRs	X4BNet `lists_vpn` — cloud/hosting, "not an eyeball network"
`apple_private_relay`	~290,000 CIDRs	Apple's own published iCloud Private Relay egress ranges

```
create table ip_reputation_ranges (
  cidr       cidr not null,
  category   text not null check (category in ('tor','vpn','datacenter','apple_private_relay')),
  source     text not null,
  batch_id   uuid not null,          -- every refresh writes one batch_id, then
                                     -- deletes anything from the same source
                                     -- left over from the previous batch
  created_at timestamptz not null default now(),
  primary key (category, source, cidr)
);
-- GiST index (via the bundled btree_gist extension) for "which range contains
-- this IP" containment lookups — a btree can't do that efficiently.
```

~345,000 rows sounds large as a download; it's trivial as a Postgres table — a few MB with its index. Deliberately not collapsed to parent supernets, which would throw away Apple's published precision for no real storage or query-time saving.

License note: X4BNet's `lists_vpn` repository has no `LICENSE` file. Its README invites contribution and the list exists for exactly this kind of defensive/detection use, but nothing formalises reuse terms. Used here for internal tagging only — the ranges are never re-published or served back out.

The tagging trigger (`engagement_events_tag_ip_category()` , `before insert` on `engagement_events`) picks, in order: **Apple Private Relay wins over a coincidental VPN/datacenter match** — it's the benign, extremely common explanation (default-on for iCloud+ subscribers), and "Private Relay" is a far more useful label than "VPN" for a click that's almost certainly a real congregation member's phone. Otherwise the **most specific (smallest) matching CIDR** wins. A lookup failure falls back to `null` rather than failing the insert — same "never break the thing the visitor was doing" rule `recordEngagement()` follows.

Flag, never block. Same instinct as `is_bot` , applied to a different axis: a VPN or Tor connection with an ordinary browser UA is very likely still a real person, just a privacy-conscious one, so this never rejects a click or reclassifies `is_bot` . A datacenter IP with an ordinary UA is a stronger automation signal than the UA alone, but even that is surfaced as a tag on `/admin/analytics` 's "Connection" card, not acted on automatically.

Refresh — `app/api/cron/ip-reputation-refresh/route.ts` , weekly (`0 5 * * 1` , Monday 05:00). Same `CRON_SECRET` fail-closed pattern as every other cron here. Fetches all four lists, chunks them (~10,000 rows), and `upsert` s through the existing service-role Supabase client — no new dependency, at the cost of ~35 round trips for the largest list rather than one bulk load; irrelevant at once-a-week cadence. Each source is fetched and loaded independently, so one being temporarily unreachable doesn't stop the other three refreshing. `maxDuration = 300` .

RLS: deny-all `"service only"` , same as every other table in this feature. Only the `security definer` -free trigger function (running as the inserting role, i.e. the service role via `recordEngagement()`) and the refresh cron's service client ever touch it.

Advisor follow-up (`20260828_02_ip_reputation_advisor_fixes.sql`) — `get_advisors` found three issues after `20260828_ip_reputation.sql` shipped, all fixed in this second migration: 1. **The real one.** `revoke all on function ... from public` in the first migration did *not* revoke `anon` / `authenticated` 's EXECUTE on `engagement_rollup` , `engagement_top` and `engagement_anonymise_ips` — Supabase grants EXECUTE on a newly created function directly to those two roles at creation time, not via the `PUBLIC` pseudo-role, so revoking from `PUBLIC` is a no-op against them. This left all three callable by anyone unauthenticated via `/rest/v1/rpc/<name>` , `engagement_anonymise_ips` included — a data-mutating RPC. Fixed with an explicit `revoke execute on function ... from anon, authenticated` . 2. `engagement_events_tag_ip_category()` was missing `set search_path` , unlike every other function in this feature. 3. `btree_gist` was installed into `public` instead of this project's `extensions` schema. Relocated in place with `alter extension btree_gist set schema extensions` — not drop-and-recreate, which would have cascade-dropped `ip_reputation_ranges_cidr_idx` .

Both migrations are applied to production; `get_advisors` is clean on all of them.

---

25. **design_tickets / design_ticket_deliverables / design_ticket_events**

The design request queue (`/portal/design/request` → `/admin/design`). Someone asks the design team for a poster, a designer claims it, works it, uploads the finished file, and the requester downloads it or asks for changes.

```
-- A uuid pk so tickets aren't enumerable, plus a separate sequence for the
-- human reference: "DT-0007" is what gets said out loud and put in a subject line.
create sequence design_ticket_ref_seq start 1;

create table design_tickets (
  id                    uuid primary key default gen_random_uuid(),
  ref                   integer not null unique default nextval('design_ticket_ref_seq'),
  title                 text not null,
  brief                 text not null,
  category              text not null default 'other'
                          check (category in ('social','print','apparel','signage','web','video','other')),
  needed_by             date,
  specs                 text,
  -- Never anonymous: the public form always collects both.
  requester_name        text not null,
  requester_email       text not null,
  requester_phone       text,
  -- Set from the session, never from the form.
  requester_auth_user_id uuid,
  requester_staff_id    uuid references hr_staff (id) on delete set null,
  requester_verified    boolean not null default false,
  priority              text not null default 'normal'
                          check (priority in ('normal','fast_track')),
  status                text not null default 'open'
                          check (status in ('open','claimed','in_progress','delivered',
                                            'changes_requested','closed','cancelled')),
  revision              integer not null default 1,
  assignee_email        text,
  assignee_name         text,
  assignee_auth_user_id  uuid,
  assigned_at           timestamptz,
  share_token           text not null unique default encode(gen_random_bytes(16), 'hex'),
  playbook_board_token   text,
  delivered_at          timestamptz,
  closed_at             timestamptz,
  resolution_note       text,
  requester_ip_hash     text,          -- sha256(ip + salt), throttling only
  last_activity_at      timestamptz not null default now(),
  created_at            timestamptz not null default now(),
  updated_at            timestamptz not null default now()
);

-- Metadata only — the bytes live elsewhere, keyed by storage_kind:
--   'supabase' → a file in the private design-deliverables bucket (file_path)
--   'link'     → a Drive/Playbook share URL the designer pasted (link_url); we
--                own no bytes at all
--   'playbook' → a legacy Playbook asset (playbook_asset_token); the original
--                delivery mechanism, kept working for existing rows only
create table design_ticket_deliverables (
  id                   uuid primary key default gen_random_uuid(),
  ticket_id            uuid not null references design_tickets (id) on delete cascade,
  revision             integer not null default 1,
  storage_kind         text not null default 'playbook'
                         check (storage_kind in ('playbook','supabase','link')),
  playbook_asset_token text,               -- 'playbook' rows only
  file_path            text,               -- 'supabase' rows: object key in the bucket
  link_url             text,               -- 'link' rows: the pasted share URL
  link_provider        text check (link_provider is null
                         or link_provider in ('drive','playbook','other')),
  file_name            text not null,
  mime_type            text,
  size_bytes           bigint,
  uploaded_by_email    text,
  -- Requester-confirmed deletion: set once by the requester's own confirm
  -- action, read by the purge cron. Never set by an admin.
  confirmed_at         timestamptz,
  confirmed_by_email   text,
  created_at           timestamptz not null default now(),
  -- Exactly one storage backing is populated per row.
  constraint design_ticket_deliverables_storage_shape check (
    (storage_kind = 'playbook' and playbook_asset_token is not null and file_path is null and link_url is null) or
    (storage_kind = 'supabase' and file_path is not null and playbook_asset_token is null and link_url is null) or
    (storage_kind = 'link'     and link_url is not null and playbook_asset_token is null and file_path is null)
  )
);

-- Private Supabase Storage bucket for image/PDF deliverables (50MB cap,
-- image/* + application/pdf only). Private: reached only through server-minted
-- signed URLs, no storage.objects policies (the service role bypasses RLS),
-- matching hr-documents and every other private bucket here.
--   bucket: design-deliverables
--
-- Partial index on confirmed_at (where confirmed_at is not null) so the daily
-- purge cron sweeps only confirmed rows.

-- Status history and messages in ONE append-only table, not two.
create table design_ticket_events (
```

```
  id          uuid primary key default gen_random_uuid(),
  ticket_id   uuid not null references design_tickets (id) on delete cascade,
  kind        text not null check (kind in ('status','note','change_request')),
  actor_type  text not null default 'system'
                check (actor_type in ('requester','designer','system')),
  actor_name  text,
  actor_email text,
  from_status text,
  to_status   text,
  body        text,
  is_internal boolean not null default false,
  created_at  timestamptz not null default now()
);
```

RLS: all three are deny-all (`using (false) with check (false)`), reached only through service-role routes. `design_set_updated_at()` pins `search_path = ''` , matching `hr_set_updated_at` and `shop_set_updated_at` .

Why one events table rather than a comments table and a status-history table. It lets the admin panel and the requester's page render from a single ordered query, and it makes "changes requested" one row that is simultaneously the note and the transition — rather than a comment that has to be correlated with a status change by timestamp. `is_internal` defaults to `false` so forgetting the flag over-shares nothing the requester couldn't already see on their own ticket.

Why `revision` is on the ticket and stamped onto deliverables. A change request bumps `design_tickets.revision` ; the next upload is stamped with the new number and gets its own row (a distinct file or link). Nothing is ever overwritten, so "what did I approve in round 1" stays answerable — which is the entire point of having a revisions workflow.

Why requester-confirmed deletion. A deliverable the requester has downloaded and confirmed ("I have this, you can delete it") doesn't need to sit in storage forever. The confirm action stamps `confirmed_at` ; the daily purge cron (`/api/cron/design-deliverables-purge`) deletes the underlying file and the row once it has sat for at least 48 hours — a floor, not an exact deadline, since the cron runs once a day (real-world 48–72h). The floor gives the requester room to notice a corrupt download or grab it again on another device. `'link'` deliverables are excluded: they own no bytes of ours, so there is nothing to schedule.

Why the assignee is an email, not a foreign key. A designer is an `admin_roles` row, and `admin_roles` has no target worth pointing at from here. Same reasoning as the old `media_photos.reviewed_by` .

Used by: `app/design-request/[token]` (public tokenised tracker), `app/portal/design/request` (staff-only form), `app/admin/design/*` (queue and detail), `app/portal/design` (staff), `app/api/admin/design/**` , `app/api/design-request/**` , `app/api/portal/design` , `lib/designTickets.ts` , `lib/designTickets.server.ts` , `lib/designEmail.ts` .

---

26. `speaker_overrides` (sermon speaker corrections)

Purpose: An AI/human-corrected speaker per YouTube video id, overriding the regex parse in `lib/sermonTitle.ts` . The sermons feature is otherwise deliberately "no DB" (it reads straight from the YouTube Data API); this table is the one small, deliberate exception — same pattern as `admin_roles` . Migration: `supabase/migrations/20260912_02_speaker_overrides.sql` .

```
CREATE TABLE speaker_overrides (
  video_id text PRIMARY KEY,
  speaker text,                                  -- NULL = *confirmed* "no individual speaker"
  reviewed_by text NOT NULL DEFAULT 'ai',
  reviewed_at timestamptz NOT NULL DEFAULT now()
);

-- RLS: Service role only (deny-all "service only" policy, same as every other table)
```

Why it exists. `lib/sermonTitle.ts` parses a speaker out of the YouTube title by regex, which is right for today's well-formatted titles but produces noise on years of inconsistent older ones (`"22.03.20"` , `"FULL SERVICE"` , `"DESTINY CHURCH LIVE!"`). An AI review (`lib/speakerReview.server.ts` , triggered from `/admin/sermons`) reads each video's title + description and stores a correction here when it disagrees — without ever touching YouTube itself.

The null distinction is load-bearing. A row with `speaker = NULL` is a *confirmed* "no individual speaker" (e.g. a child-dedication service, a worship-only clip). That is different from **no row at all**, which means nobody has reviewed this video yet. So an override only ever replaces a value, never silently blanks a perfectly good regex-parsed speaker that just hasn't been looked at.

Read it through the wrapper. Every video-serving path reads through `lib/speakerOverrides.server.ts` rather than `lib/youtube.ts` directly, so a correction takes effect everywhere at once; `lib/youtube.ts` itself stays a pure YouTube API client with no Supabase dependency.

Used By: `lib/speakerOverrides.server.ts` (read wrappers), `lib/speakerReview.server.ts` (writes), `app/api/admin/sermons/review-speakers` , `components/admin/SpeakerReviewPanel.tsx` .

---

27. `sermon_series` (playlist-backed sermon series)

Purpose: A YouTube playlist id curated as a sermon "series", filterable on `/sermons` . Same "no DB" exception as `speaker_overrides` above — only the playlist id is stored; title, description and video membership are always read live from YouTube through `lib/sermonSeries.server.ts` . Migration: `supabase/migrations/20260920_01_sermon_series.sql` .

```
CREATE TABLE sermon_series (
  playlist_id text PRIMARY KEY,
  added_by text,
  sort_order integer NOT NULL DEFAULT 0,
  created_at timestamptz NOT NULL DEFAULT now()
);

-- RLS: Service role only (deny-all "service only" policy, same as every other table)
```

Why it exists. Speaker sorting/filtering was removed from `/sermons` in favour of series, sourced from YouTube playlists an admin curates in `/admin/sermons`. Rather than duplicate a playlist's title/description into Postgres (which would drift from YouTube), only the id is persisted — `lib/sermonSeries.server.ts` resolves it against the YouTube Data API (`playlists.list` for the snippet, `playlistItems.list` for membership) on every read, the same live-fetch philosophy the rest of the sermons feature uses.

Used By: `lib/sermonSeries.server.ts` (reads), `app/api/admin/sermons/series` and `app/api/admin/sermons/series/[id]` (writes), `components/admin/SeriesManager.tsx`, `app/sermons/page.tsx` → `components/sermons/SermonGrid.tsx` (the public Series filter).

---

28. **notifications / notification_reads** (admin notification center)

Purpose: The data behind the admin **notification bell** — one row per inbound event a section's admins should see (new order, new job application, new design ticket, prayer request, contact message, leave request). Migration: `supabase/migrations/20260922_02_notifications.sql`.

```
CREATE TABLE notifications (
  id           bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  created_at   timestamptz NOT NULL DEFAULT now(),
  section      text NOT NULL,               -- e.g. "store", "hr", "design"
  kind         text NOT NULL,               -- e.g. "order", "application", "ticket"
  entity_id    text,                         -- the source row's id, as text
  entity_label text,
  summary      text NOT NULL,                -- one plain sentence: "New order #1042 from Jane Doe"
  href         text NOT NULL,                -- admin deep link, e.g. /admin/store/orders?open=<id>
  roles        text[] NOT NULL,              -- AdminRole values this notification is relevant to
  metadata     jsonb
);

CREATE TABLE notification_reads (             -- per-admin read state (several admins share one notification)
  notification_id bigint NOT NULL REFERENCES notifications(id) ON DELETE CASCADE,
  auth_user_id    uuid   NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  read_at         timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (notification_id, auth_user_id)
);

-- RLS: both tables are deny-all ("service only") — the API routes always go
--      through createServiceClient(). Indexes: notifications(created_at desc),
--      GIN(roles); notification_reads(auth_user_id).
```

Read state is per-viewer, not per-notification. Several admins share the same source tables, so "read" is a fact about `(notification, viewer)`, which is why it lives in its own `notification_reads` table rather than a column on `notifications`.

Delivery uses Realtime Broadcast, not Postgres Changes. Opening Postgres Changes on `notifications` would need an authenticated SELECT policy straight on the table — exactly the class of mistake to avoid. Instead the service-role insert path calls `admin_notify_emit(topic, event, payload)` (a `SECURITY DEFINER` wrapper over `realtime.send`) to push the row onto a **private Broadcast topic** named `admin-notifications:<role>`, one per `AdminRole`. A notification for `['store_admin']` is broadcast to `admin-notifications:store_admin` **and always** `admin-notifications:super_admin` — mirroring how a Super Admin's route access bypasses every `ROUTE_RULES` entry. A `realtime.messages` SELECT policy (`admin_notifications_receive`) only lets a signed-in admin receive a topic for a role they actually hold, gated by `admin_has_role(text)` (also `SECURITY DEFINER`, because `admin_roles` is deny-all). There is no insert policy — the browser never posts to the topic, only the service key does.

Retention. `purge_old_notifications()` deletes notifications (and, via cascade, their read-receipts) older than **60 days**, called from the weekly audit cron (`/api/cron/audit-weekly-report`) alongside the existing `audit_log` purge.

Used By: `lib/notify.server.ts` (`recordNotification()`, the one writer — called alongside, never in place of, each submission's existing insert/email logic, and never throws), `lib/useNotifications.ts` (the client Realtime hook), `components/admin/AdminNotificationBell.tsx` (the bell in `AdminHeader`), `app/api/admin/notifications` + `/[id]/read` + `/read-all` (feed and read state). Call sites: new order (`lib/checkout.server.ts`), job application (`app/jobs/actions.ts`), design ticket (`app/portal/design/request/actions.ts`), contact message (`app/contact/actions.ts`), leave request (`app/api/portal/leave`).

---

29. **d1_*** (Destiny One — members' group messaging)

Purpose: The data behind Destiny One, the Expo app (`apps/destiny-one`): WhatsApp-Communities-style messaging — a **community** holds an **Announcements** group everyone is in (admins post) plus department **sub-groups**. Migration: `supabase/migrations/20260926_01_destiny_one.sql`.

Table	Holds
`d1_members`	One row per app account. `auth_user_id` (nullable — set null when the account is deleted), `first_name` / `last_name` (the member can change these themselves via `PATCH /me`; `name_edited_at` records that; `name_change_log` holds the timestamps that the 2-per-30-days limit counts) and `display_name` (always `first last` — the `d1_members_sync_names` trigger derives it from first/last, or splits it into first/last when older code such as the ChurchSuite sync, invites, erasure or staff edits write only `display_name`). Once a member edits their name, the ChurchSuite re-sync no longer overwrites it, `churchsuite_contact_id` / `churchsuite_child_id` / `churchsuite_user_id`, **`adult_on`** (the 18th birthday — the full date of birth is never stored), `status` (`pending` / `active` / `suspended` / `deleted`), `roles` (`admin`, `cg_leader`, `senior_leader` — see part 8)
`d1_consents`	Which version of `privacy` / `terms` / `chat_review_notice` a member accepted, when
`d1_communities`, `d1_community_members`	Communities and who is in them (`admin` / `member`)
`d1_groups`	`kind` (`announcements` / `group` / `assistant`, see part 18), `owner_id` (assistant chats only), `department`, `state` (`active` / `frozen` / `archived`), `freeze_kind` (`auto` / `manual`), `frozen_reason`
`d1_group_members`	Membership incl. history (`left_at` kept, so a review can see who was present when), `last_read_message_id`, `muted_until`, `archived_at` (per-person chat archive: hides + silences the chat for that member only)
`d1_messages`	`body` (sealed ciphertext, see part 10; 4000 characters before sealing), `reply_to`, `attachment_id`, `content` (jsonb: a poll, whose question and option labels are sealed, or an event snapshot taken at send time), soft-delete `deleted_at` / `deleted_by`. Immutable except the delete stamp
`d1_message_terms`	The blind search index (part 10): keyed hashes of every prefix of every word in a body, per group. No plaintext
`d1_message_edits`	Part 11: the text an edit replaced (sealed), one row per edit, for the safeguarding transcript. Deny-all RLS; removed with its message by the purge
`d1_pins`	Part 13: pinned messages per group (who pinned, when). At most 3; deny-all RLS
`d1_poll_votes`	One row per (poll message, member, option). Written only by `d1_vote()`, which enforces single choice and current membership and broadcasts `poll_vote` (with `groupId`)
`d1_reactions`, `d1_attachments`	Reactions; files in the private `d1-chat-media` bucket (images/PDF, 20 MB)
`d1_reports`, `d1_safeguarding_events`	The safeguarding queue
`d1_push_tokens`	Expo push tokens
`d1_blocks`	Who has blocked whom (part 6). Hides the blocked person's messages and notifications for the blocker only

There is no phone column anywhere, by rule (the safeguarding policy forbids phone numbers), and `tests/sql/destiny-one.sql` fails if one is ever added.

The rules are enforced in the database, not the API, so no client and no route bug can get round them: 1. **No 1:1 chats.** A group with fewer than 3 people is paused (read-only), never usable. `d1_create_group` still accepts any size (even just the creator, migration `20260929_05`; `d1_admin_create_group` likewise, `_06`): it inserts the group already `frozen` / `auto` with no safeguarding event, and it opens itself once it has 3 people including 2 adults. 2. **Only leaders create groups and communities** (any of `admin` / `cg_leader` / `senior_leader`, adults only; the three have identical powers). 3. **At least 2 verified adults in every group, always.** Evaluated at creation (paused if unmet), then by a *deferred* constraint trigger on `d1_group_members` (and on `d1_members` status/`adult_on` changes) that calls `d1_evaluate_group`: below 3 members or 2 adults → `frozen` (read-only) + a `d1_safeguarding_events` row; back above → unfrozen automatically. Leaving is never blocked (scoping doc D1: freeze + notify). "Adult" is `adult_on <= current_date`, so turning 18 needs no job; no `adult_on` = minor (fail safe). Group/community admins and leader roles must be adults. 4. **No E2EE, reviewable.** Deleted messages keep their body until the retention purge; messages can't be edited in place. 5. An account whose `auth.users.phone` is set can't be made `active`.

Safeguarding events are written into `notifications` for the `safeguarding_admin` role by a trigger and broadcast on `admin-notifications:safeguarding_admin`, so they ring the existing admin bell.

Functions (all `SECURITY DEFINER`, `EXECUTE` revoked from `anon` / `authenticated` — service role only; each takes the acting member explicitly): `d1_create_community`, `d1_add_community_members`, `d1_remove_community_member`, `d1_create_group`, `d1_add_group_members`, `d1_remove_group_member`, `d1_post_message`, `d1_delete_message`, `d1_react`, `d1_report_message`, `d1_set_manual_freeze`, `d1_erase_member` (GDPR erasure: leave everything, anonymise to "Former member"), `d1_purge_expired`, `d1_reconcile_all`, `d1_group_overview` (the chat list with unread counts in one query), plus `d1_emit` (wraps `realtime.send`). Writes emit on private Broadcast topics `d1-group:<id>` and `d1-member:<id>`; the `d1_receive` policy on `realtime.messages` (via `d1_can_receive`, the only functions granted to `authenticated`) lets only current, active members receive them.

Also adds `admin_roles.safeguarding_admin` and redefines `admin_has_role()` to know it.

Part 2 — `20260927_01_destiny_one_admin.sql`: staff verify people, not ChurchSuite. Destiny wants minimal reliance on ChurchSuite, so identity now comes from Destiny's own staff: - **`d1_members`** gains `verified_at`, `verified_by`, `verification_source` (`invite` / `admin` / `churchsuite`), `declared_adult_on`, `request_note`, `request_submitted_at`. **New rule: nobody becomes `active` without a verification record.** `adult_on` is still the one field the 2-adult rule reads; staff set it — adult without a DOB → the verification date; under-18 with a DOB → their 18th birthday; under-18 without → null. `declared_adult_on` is what a person said about themselves in an access request: shown to the reviewer, **never read by any rule**. - **`d1_invites`** — email, name, `is_adult`, optional `adult_on`, leader `roles`, `community_ids`, `status` (`pending` / `accepted` / `revoked` / `expired`), `expires_at`. One open invite per email. `d1_accept_invite(auth_user, email)` activates the member on sign-in with that email (the email one-time code proves ownership —

no invite token) and joins the listed communities. - **`d1_settings`** (one row) — `allow_access_requests` (off = invite-only), `invite_expiry_days`, `min_build_ios` / `min_build_android` (builds below see the app's "update" screen), `force_update_message`, `maintenance_message` (set = the whole app shows it instead of working). The last four are served publicly by `GET /api/app/v1/one/config`. - **Admin-path functions** with no acting member (website staff may have no app account): `d1_admin_create_community`, `_add_community_members`, `_set_community_role`, `_remove_community_member`, `_create_group`, `_add_group_members`, `_remove_group_member`, `_set_group_role`, (group creation no longer uses `d1__check_composition`; small groups are created paused). Every part-1 trigger still applies. Plus reads `d1_admin_members` (joins the sign-in email from `auth.users` in one query) and `d1_admin_groups` (live counts). - **Notifications re-routed:** an automatic pause → `destiny_one_admin` + `safeguarding_admin`; a report → `safeguarding_admin` only. - **`admin_roles.destiny_one_admin`**, and `admin_has_role()` redefined to include it.

Part 6 — `20260928_01_destiny_one_safeguarding.sql` : fixes from the production readiness audit (`apps/destiny-one/todo.md`): - **Retention keeps evidence.** `d1_purge_expired` never deletes a message (or its attachment) that is linked to a report whose status isn't `closed`, however old it is. - **Complete erasure.** `d1_erase_member` also deletes `d1_consents`, the member's `d1_blocks` rows and the `avatar_url` reference; `eraseMember()` (`lib/destinyOne/identity.server.ts`) deletes the picture file from Storage, which SQL can't reach. All three erase paths use it (self-delete, admin delete, revoking an unused invite). - **Private profile pictures.** The `d1-avatars` bucket is no longer public and its public-read policy is dropped. `d1_members.avatar_url` now holds the storage path; `toMe()` returns a 7-day signed URL (the app re-fetches `me` on every cold start). - **Blocking.** `d1_blocks` + `d1_set_block(actor, target, block)` (no self-blocks). Nobody leaves a group, so the 2-adult rule is untouched. `d1_group_overview` (preview, unread count) and `d1_search_messages` ignore blocked senders; the API filters message pages and push. Each block / unblock writes a `d1_safeguarding_events` row (new kinds `block`, `unblock`, created resolved so they don't queue or ring the bell) and emits `blocks_changed` on `d1-member:<blocker>`. - **Safeguarding takedown.** `d1_messages.deleted_by_admin` + `d1_admin_delete_message(message, admin)` : a soft delete like any other (content kept for review), live `message_deleted` event.

Applied to the live project on 2026-09-28. The same day, the migration history was backfilled with rows for parts 1 and 2 (`20260926_01`, `20260927_01`), which had been run outside it.

Part 7 — `20260928_03_destiny_one_rate_limits.sql` : rate limits every server instance shares. `d1_rate_limits` (one row per key per minute, deny-all RLS) and `d1_rate_limit(key, max)`, which counts atomically and returns whether the hit is allowed, pruning rows older than an hour now and then. `limit()` in `lib/destinyOne/http.ts` checks the in-memory limiter first (free, with escalating cooldowns), then this; if the database can't be reached the request is let through and logged.

Part 8 — `20260928_04_destiny_one_role_tags.sql` : Admin, CG Leader, Senior Leader. Replaces `group_leader` / `senior_leadership`. All three roles have the same powers (create groups and communities, search the directory); they differ only in the tag beside the person's name. Existing data is migrated (`group_leader` → `cg_leader`, `senior_leadership` → `senior_leader`), the check constraints on `d1_members.roles` and `d1_invites.roles` are replaced, and `d1_is_leader()` / `d1_is_senior()` both mean "holds any of the three". A person holding several roles shows the highest (`topRole()` in `packages/shared/src/destinyOne/policy.ts` : Admin, then Senior Leader, then CG Leader). `D1GroupMember.tag` carries it to the chat screen, which draws it via `MemberTag`; someone with no role who administers the group shows "Group admin" instead. The Profile tab shows the same label. **Deploy order:** old app builds only know the retired role names, so a leader on one loses the leader UI until they update; apply the migration when the API change ships.

Part 9 — `20260929_01_destiny_one_feedback.sql` : "Report a problem" and "Send feedback". `d1_feedback` (member, `kind` problem/idea, `body` up to 2000, optional app version / platform / OS version / phone model / crash-report id, `status` new/done; deny-all RLS). Kept in the database, not as GitHub issues, because the repository is public and many members are young people. An insert rings the Destiny One Admin bell (`d1_feedback_notify`; the notification never includes the text). `d1_erase_member` deletes a member's feedback and `d1_purge_expired` removes feedback older than the retention period. Staff read it at `/admin/destiny-one/feedback`. Applied to the live project on 2026-09-29, together with `20260922_02_notifications.sql` (the admin bell's tables), which had never been applied there. Until then, every Destiny One report and group pause would have failed at its bell notification.

Part 10 — `20261004_01_destiny_one_message_encryption.sql` (+ `_02`): message text encrypted at rest. See "Message encryption" under `lib/destinyOne/*` for the design and threat model. In the database: `d1_messages.search` (the tsvector) is dropped and search moves to `d1_message_terms` (group, term, message; deny-all RLS, service role only), matched by `d1_search_messages(actor, terms jsonb, limit)` with the same visibility rules as before. `d1_post_message` gains `p_terms text[]` and **no longer broadcasts**: `realtime.send` keeps every payload in `realtime.messages` for three days, which would have kept a readable copy, so the API broadcasts the opened message over Realtime's REST endpoint instead, which stores nothing (the stored copies are deleted). Length checks are widened for ciphertext (the API enforces 4000 / 1000). `d1_messages_immutable` allows exactly one change: a plaintext body to its sealed form, for the backfill. `_02` then adds `d1_messages_body_sealed`, `d1_messages_poll_sealed` and `d1_reports_reason_sealed`, so the database refuses plaintext from then on (and the backfill exception can never fire again).

Part 11 — `20261006_01_destiny_one_message_edits.sql` : editing (D1 v0.8). `d1_messages.edited_at` and `d1_message_edits` (message, group, the replaced body still sealed, when). `d1_edit_message(actor, message, body, terms)` only lets the sender edit a message that has text and isn't deleted, within 15 minutes of sending, at most 10 times, while they're a current member of an active group. It saves the old text first, then swaps the body (still sealed: `_02` 's check applies) and replaces the search terms. `d1_messages_immutable` lets the body and `edited_at` change only while `d1.editing` is set, which `d1_edit_message` does for its own transaction (`set_config(..., true)`), so a direct UPDATE is still refused. Members see only the latest text, marked "Edited"; the safeguarding transcript shows every version.

Part 12 — `20261006_02_destiny_one_mentions.sql` : @mentions. `d1_messages.mentions uuid[]` (up to 50) holds who a message "@names". The text stays sealed; the ids are plain because the server needs them to notify people, and they say no more than the member list does. `d1_valid_mentions(group, actor, ids)` keeps only current members of the group, never the sender, so a mention can't reach anyone outside it. `d1_post_message` gains `p_mentions` (the 7-argument version is dropped), `d1_edit_message` gains `p_mentions` (null leaves them alone), and the immutability trigger lets `mentions` change only during an edit. Push (`lib/destinyOne/push.server.ts`): a mentioned member gets "Sender mentioned you: ...", even if they muted the group; blocking still wins. The app parses "@Name" with `findMentions` / `mentionSegments` / `mentionQuery` / `mentionSuggestions` in `@destiny/shared` (longest name first, never inside an email address); the composer suggests members as you type "@", and bubbles draw mentions in bold, highlighting your own name.

Part 13 — `20261006_03_destiny_one_pins.sql` : pinned messages. `d1_pins (group, message, pinned_by, pinned_at)` . `d1_pin_message(actor, message, pin)` is for people who manage the group (`d1_can_manage_group`) and are in it; a deleted message can't be pinned, and pinning a fourth unpins the oldest. It emits `pins_changed` (no text, so `d1_emit` is fine). `GET groups/[id]` returns `pinned` : the pinned messages the caller can see (since they joined, not deleted, not from someone they've blocked), newest pin first. In the app a glass bar under the chat header shows the newest pin; tapping it scrolls to that message (`jumpTo`) and moves on to the next, with a small indicator when there are several. Pin / Unpin is in the message menu for managers.

Part 14 — `20261006_04_destiny_one_read_receipts.sql` : "Seen by". Read state used to be private; now `d1_read_receipts(actor, message)` lists who (of the members who were there when it was sent) has read a message, from the existing `last_read_message_id` . No read time exists or is shown. Only the sender, or someone who manages the group, can ask. `d1_members.read_receipts` (default on) is reciprocal as in WhatsApp: off means you show as hidden (counted, not named) and you can't see anyone's. The API reads that column on its own (`loadReadReceipts`), not through `MEMBER_COLUMNS` , so an API deployed before the migration still works. The peek preview never moves the read marker. The draft notice in `docs/content/destiny-one-notices-draft.md` has a Read receipts paragraph to publish with it.

Part 15 — `20261006_05_destiny_one_link_previews.sql` : link previews. `d1_messages.link_preview` holds the first link's preview as sealed JSON (`D1LinkPreview` : url, title, description, siteName, https imageUrl), written only by `d1_set_link_preview` (the immutability trigger allows it only while that runs, via `d1.previewing`). The API builds it **after** sending, in `after()` (`lib/destinyOne/linkPreview.server.ts`), and broadcasts `link_preview` over the REST broadcast. So sending never waits for another website, and members' phones never contact the site to build it (only the preview picture is loaded by the phone, https only). The fetch is locked down against SSRF: http(s) on ports 80/443, at most 3 redirects, 5 s, 512 KB, HTML only, and every connection must reach a public address. The check runs inside the connection's own DNS lookup, so there's no rebinding gap, and bare IP links, which skip DNS, are checked before connecting (`isPublicAddress` refuses loopback, private, link-local/cloud-metadata, CGNAT, multicast and their IPv6 forms). Parsing (`firstUrl` , `parsePreview`) is in `lib/destinyOne/linkPreview.ts` , unit-tested. An edit that changes the link rebuilds or clears the preview. In the app, links in message text are underlined and open in the in-app browser, and the preview shows as a card under the text.

Part 16 — `20261006_06_destiny_one_forwarding.sql` : forwarding. `d1_messages.forwarded_from` links a copy to its original (shown as "forwarded from message #..." in the safeguarding transcript). Members only see "Forwarded"; the original author isn't carried across. `d1_post_message` gains `p_forwarded_from` (the 8-argument version is dropped) and refuses it unless the forwarder is in the original's group now, was when it was sent, and it isn't deleted. `POST messages/[id]/forward { groupIds }` (up to 5, 20 a minute, never polls) opens the text and seals it again for each target group, copies any photo or PDF into that group's own folder with its own `d1_attachments` row, then posts, broadcasts and pushes like any message.

Part 17 — `20261006_07_destiny_one_voice_notes.sql` : voice notes. `audio/mp4` joins the `d1_attachments` mime check and the `d1-chat-media` bucket's allowed types, and `duration_ms` (up to 5 minutes) is kept so the app can show a voice note's length before it downloads. A voice note is an ordinary attachment: kept, deleted, purged and shown in the safeguarding transcript like a photo, and pushed as "Sender: Voice message". In the app (`src/components/VoiceNote.tsx`) the microphone sits next to the text field when the box is empty. It works like WhatsApp: hold to record and let go to send, slide left to cancel, slide up to lock it hands-free (then the bin and Send appear); a quick tap just shows a hint. The gesture is a `PanResponder` in `Composer.tsx` ; the field shows a recording readout (time, "Slide to cancel") and it stops by itself at 5 minutes, recording mono AAC at 64 kbps (`expo-audio`). The camera is no longer in the text field; it is under the + sheet. Under a second is dropped. In a bubble, nothing loads until Play, and only one voice note plays at a time. Voice notes can't be searched (there's no text).

Part 18 — `20261009_02_destiny_one_destinyai.sql` : DestinyAI (v0.10). The website's Smart Search, as a chat member. One `d1_members` row has `is_assistant` (fixed id `DESTINY_AI_ID` in `@destiny/shared` , no auth user, no ChurchSuite record; hidden from the directory and `d1_admin_members`). Two ways in: - **Its one-to-one chat**, the only one-to-one chat in Destiny One: a `d1_groups` row of kind `assistant` , outside every community (`community_id` null, `owner_id` set; a check constraint ties the two together, one per member), holding just its owner and DestinyAI. `d1_assistant_group(member)` makes it on first use (and puts them back if they'd left). Being a group, it gets the chat screen, encryption, Realtime, push, read state and reporting for free. The 3-members/2-adults rule doesn't apply (`d1_evaluate_group` skips it); nobody can join it or manage it, not even senior leaders (`d1_group_members_guard` , `d1_can_manage_group`); `d1_admin_groups` leaves it out. A reported DestinyAI message still reaches the safeguarding queue and transcript like any other. - **"@DestinyAI" in a group.** DestinyAI is never a member and can't read the group. The API hands it the asking message, plus the message that one replies to and that one's reply chain (up to 6, only what the asker can see: since they joined, not deleted, not from someone they blocked), and nothing else. So "reply to a message, then tag @DestinyAI" is how to give it context. Its answer is posted as a reply to the question (`d1_messages_guard` lets DestinyAI post in a group only as a reply, never with an attachment), mentioning the asker so they're notified even if the group is muted. `d1_post_assistant_message` (service_role only) is the only way to post as DestinyAI. The answering itself is `lib/destinyOne/assistant.server.ts` , run from `after()` in `POST groups/[id]/messages` : the same model and tools as Smart Search (`lib/smartSearch/tools.ts` , `CHURCH_FACTS`), plus `find_events` (the live ChurchSuite calendar, via `upcomingEventSeries()` in `lib/destinyOne/events.server.ts`) and `share_event` (attaches one event card to the answer). It broadcasts "DestinyAI is typing..." while it works, is limited to 6 questions a minute per member, has a safeguarding section in its prompt (999, Childline, Samaritans, the Safeguarding Lead), and says sorry in the chat rather than going quiet if anything fails. In its own chat it re-reads the last 16 messages for follow-ups, and only reads text (the composer is text-only there).

Live answers (streaming). While DestinyAI works, `liveUpdates()` in `assistant.server.ts` sends `assistant_status` (what it's doing: "Checking the calendar", "Searching the web", or "Thinking") and `assistant_delta` (the whole text so far, throttled to every 150 ms) over the REST broadcast, so nothing is stored. Each model turn streams (`streamTurn`). The app shows them through `state/assistantStream.ts` as a bubble at the bottom of the chat (`components/AssistantStreaming.tsx` : animated dots, then the text with a blinking cursor). The finished message (the normal `message` event from DestinyAI) clears the bubble. A bubble with no update for 60 s is dropped, in case that message was lost. `liveUpdates().done()` waits for in-flight updates before posting, so a stale update can't land after the answer.

Scope, beta, sermons (0.11.x). DestinyAI answers only about Destiny (its services, events, groups, beliefs, giving, shop, sermons, charity and company records, visiting). Everything else gets one line saying so, and no tool call; safeguarding replies still apply. Its label says **beta** (`BetaTag` in `components/ui.tsx`). `share_sermon` attaches one sermon from the YouTube archive (`getFullSermonArchive` , checked by id) as a `D1SermonContent` (`kind: "sermon"`). `SermonCard` (in `MessageBubble.tsx`) renders the video through `SermonPlayer` (`components/SermonPlayer.tsx`), which plays it inline with `react-native-webview` on `youtube-nocookie.com` when the build has that native module, and nothing loads until it's tapped. Adding `react-native-webview` is a native change, so a build made before it was added (anything before 0.11.2) doesn't have the module. Rather than crash on those builds, `SermonPlayer` checks for the module with `TurboModuleRegistry.get("RNCWebViewModule")` (returns `null` instead of throwing the way `getEnforcing`

does) and wraps the player in an error boundary; on an older build — or if the player fails to render — the card falls back to the sermon thumbnail with a play button that opens the video in the in-app browser, instead of the throw taking the whole chat list down (fixed in 0.11.3, #120).

Tested by: `scripts/test-sql.sh` (`npm run test:sql` , and the "Database rules" CI job) — applies Supabase stubs + every Destiny One migration (parts 1–9, plus the profile-picture, min-build and group-icon migrations) to a throwaway local Postgres and runs `tests/sql/destiny-one.sql` . A second suite, `destiny-one-encryption` , adds part 10 and runs `tests/sql/destiny-one-encryption.sql` (plaintext refused, search on terms, no realtime copy, privileges).

Timed restrictions and escalation (`20261011_01_destiny_one_restrictions_escalation.sql` , used by the D1 Admin desktop app): - `d1_members.posting_restricted_until` — while `now()` is before it, the member can read but not post or react. Enforced by `d1_enforce_posting_restriction()` as BEFORE INSERT triggers on `d1_messages` and `d1_reactions` , so every write path (posting, forwarding, voice notes) is covered; raises a P0001 the API shows as written. Lifts itself (no job). Unlike suspension, a restricted member still counts towards the group rules, so restricting never pauses a group. - `d1_reports.escalated_at` / `escalated_by` — the report went to the Designated Safeguarding Lead. Status stays `reviewing` (the web admin's set is unchanged). - Tested by the `destiny-one-restrictions` SQL suite (`tests/sql/destiny-one-restrictions.sql`).

Used By: `lib/destinyOne/*` , `app/api/app/v1/one/**` , `app/api/admin/destiny-one/**` , `app/api/cron/destiny-one-{sync,purge}` , `apps/d1-admin` .

30. song_request_settings / song_request_spotify / song_requests (Event song requests)

Purpose: `/song-requests` . Migration: `supabase/migrations/20260930_01_song_requests.sql` . All three are RLS-on with a deny-all policy; every read/write goes through API routes using the service client.

Table	Holds
`song_request_settings`	One row: `open` , `event_name` , `max_queue` , `per_device_limit`
`song_request_spotify`	One row: the church account's `refresh_token` (never sent to a client), `account_name`
`song_requests`	`spotify_track_id` , `title` , `artist` , `artwork_url` , `status` (`queued` / `played` / `removed`), `device_hash` , `ip_hash` (hashed, not raw). Unique index: a track can be queued once at a time

---

Key Point: Member-facing features use API proxy routes that enforce authentication in application code, then access the database with the service role key — this gives finer control and better error messages. Two authenticated SELECT policies now exist as **defense-in-depth** alongside that model (they are not the primary boundary): `hr_reviews` ' "reviewer can read own reviews" (`reviewer_auth_user_id = auth.uid()`) and the `realtime.messages` `admin_notifications_receive` policy for the notification Broadcast topics, and `d1_receive` for Destiny One's `d1-group:*` / `d1-member:*` topics. Every base data table is still deny-all to the anon/authenticated roles.

Core Application Layers

Layer 1: Root Layout (`app/layout.tsx`)

The root layout wraps every page in the application. It:

1. **Defines metadata & SEO** — Title templates, description, Open Graph images for social share
2. **Fetches server data** — Banner, pop-up, feature flags (called on every request)
3. **Renders custom SVG filter** — A sophisticated glass refraction effect used for visual polish
4. **Wraps in providers** — Context providers for auth, theme, analytics
5. **Loads fonts** — Roboto, Anton, Playfair Display from Google Fonts
6. **Sets up structure** — Header, main content area, footer

Key Code:

```
// Fetch active banner (shown at top of site)
async function getActiveBanner() {
  const supabase = createServiceClient();
  const { data: rows } = await supabase
    .from("site_banner")
    .select("active, message, type, link, link_text")
    .eq("active", true);

  // If multiple banners, prioritize sitewide > event banners
  const sitewide = banners.find((b) => b.type === "sitewide");
  if (sitewide) return sitewide;

  // Otherwise return first active event banner (Alpha, Youth Alpha, Recovery)
  // ...
}

// Fetch active pop-up (modal overlay)
async function getActivePopup() {
  const supabase = createServiceClient();
  const { data } = await supabase
    .from("site_popup")
    .select("active, title, body, cta_text, cta_link, image_url, show_once")
    .eq("active", true)
    .limit(1)
    .maybeSingle();  // Returns null if no results
  return data ?? null;
}

// Render root HTML (simplified — the real tree also renders LiveBanner,
// GlassBloomTracker, FooterGate/PerformanceGate wrappers, and passes a
// server-seeded `live` status into Providers; see components/ for each)
export default async function RootLayout({ children }) {
  const banner = await getActiveBanner();
  const popup = await getActivePopup();
  const smartSearchEnabled = await isSmartSearchEnabled();

  return (
    <html>
      <body className="...">
        <svg aria-hidden>
          {/* Glass refraction filter (SVG filters for visual effects) */}
        </svg>

        <Providers banner={banner}>
          <CookieBanner />
          <div className="flex flex-col">
            <ChurchHeader />
            <main>{children}</main>
            <ChurchFooter />
          </div>
          <AnalyticsGate />        {/* Conditionally load Vercel Analytics */}
          <SitePopup popup={popup} />
          <FloatingSmartSearch searchEnabled={smartSearchEnabled} />  {/* AI Smart Search widget */}
        </Providers>

        <SpeedInsights />             {/* Vercel performance monitoring */}
      </body>
    </html>
  );
}
```

SVG Glass Refraction Filter Explanation:

The `<filter id="glass-refract">` is a sophisticated visual effect:

1. **feGaussianBlur** — Blur the alpha channel to create a soft edge ramp
2. **feOffset** — Offset the blurred version left/right and up/down to detect edges
3. **feComposite** — Subtract offsets to create signed X/Y displacement vectors
4. **feDisplacementMap** — Apply the displacement at three scales (42/48/54) to separate color channels slightly
5. **feBlend** — Recombine RGB channels; the offset makes colors separate at edges (chromatic dispersion)

Result: Elements with `.glass-refract` backdrop-filter get a thick glass effect with color separation at edges.

Layer 2: Providers (`components/Providers.tsx`)

Wraps children with React context providers:

- **Supabase Client Provider** — Browser auth token management
- **Theme Provider** — Dark/light mode
- **Query Client** (if using) — API caching
- **Toast Provider** (`components/ToastProvider.tsx`) — Passive notifications via `useToast()` (`success` / `error` / `info`)
- **Dialog Provider** (`components/DialogProvider.tsx`) — Styled, promise-based modal dialogs via `useDialog()`

Dialogs & notifications — no native browser dialogs

Native `alert()`, `confirm()`, and `window.prompt()` are **banned** in this codebase — they look untrustworthy, can't be styled, and freeze the tab. Use the in-app equivalents instead:

- **Passive message** → `useToast()` from `components/ToastProvider.tsx`: `toast.error("…")`, `toast.success("…")`, `toast.info("…")`.
- **Yes/No decision** → `useDialog().confirm(opts)` → `Promise<boolean>`. Renders a centered, blocking styled modal. Options: `{ title?, message, confirmLabel?, cancelLabel?, tone? }` where `tone: "danger"` gives a red confirm button for destructive actions. Pattern: `if (!(await confirm({ message: "Delete this?", tone: "danger", confirmLabel: "Delete" }))) return;` (the enclosing handler must be `async`).
- **Free-text input** → `useDialog().prompt(opts)` → `Promise<string | null>` (`null` = cancelled, matching native `prompt`). Options: `{ title?, message?, placeholder?, defaultValue?, confirmLabel? }`.

Both dialog helpers reuse the visual conventions from `components/admin/hr/HrUI.tsx` (backdrop + rounded card + shared button/input classes). Escape and backdrop-click cancel.

Layer 3: Header & Navigation (`components/ChurchHeader.tsx`)

Rendered on the public site (client component). It self-suppresses (returns `null`) on the chrome-free shells — `/admin`, `/nfc`, `/links` (and `/links/*`), and `/portal` — so those areas render without site nav. `/links` is matched by `isLinksPagePath()` (`lib/linkPages/paths.ts`) so `/linkshare`-style paths aren't caught; the same helper gates the footer, banners, popups and Smart Search.

- **Logo** — Clickable link to home
- **Navigation menu** — Top-level links plus hover **dropdowns** ("About", "What's on") that fade in as white rounded cards with a staggered per-item reveal
- **Mobile menu** — Animated hamburger (bars morph into an X) opens a **full-screen brand-colour overlay** with drill-down submenus and a top-down reveal (see `ChurchHeader.tsx` below); no left-hand drawer. Top-level items and a submenu ("What's on"/"About") sit side by side in a double-width row that slides horizontally (`translateX`) between the two, rather than an instant swap — both panels stay mounted so the transition is a real slide in both directions, and the submenu panel keeps rendering the last-active submenu's items while sliding back out so it never flashes empty.
- **Scroll morph** — The header pill reshapes as you scroll (progress ramps over `MORPH_DISTANCE`)
- **Auth indicator** — Login/logout buttons

Site search is **not** in the header — it lives in the floating `FloatingSmartSearch` mark (Smart Search); see that component below.

Layer 4: Main Content (Route-Specific Components)

Each page route (`app/*/page.tsx`) renders page-specific content. Examples:

`/app/sermons/page.tsx` — Sermons

Video-first. Structure, top to bottom: photo hero with podcast-platform chips → **featured message** → **video archive grid** (`components/sermons/SermonGrid.tsx`) → "Every sermon, on the big screen" YouTube band → `WorshipWithUsSection`. It uses the same light bands, container widths and card shell as every other content page — it was hard-coded dark until August 2026, which made it look like a different site, and there is no dark mode here to opt into.

The featured card (`components/sermons/FeaturedSermon.tsx`) opens on **Watch** — a privacy-enhanced `youtube-nocookie` embed behind `VideoConsentGate` — with a **Listen** tab for the same message's podcast audio, when the latest video pairs to an episode via `lib/sermonPairing.ts` (video from `lib/youtube.ts`, audio from the Buzzsprout RSS feed in `lib/podcast.ts`). Pairing tries, in order: an exact match via a YouTube video id embedded on the Buzzsprout episode (see the admin upload flow below), then a heuristic scoring publish-date proximity, title-word overlap and a matching speaker name — falling back to the newest episode rather than disabling the toggle. Only the active pane is mounted; unmounting the iframe is what stops video playback on switch. The speaker's name (see below) is promoted to its own kicker line above the title, sourced from whichever pane is active. The Watch/Listen toggle itself (`components/sermons/podcast/ModeSwitch.tsx`) and the audio pane (`.../podcast/ListenPane.tsx`) were extracted out of `FeaturedSermon.tsx` in September 2026 so the sermon detail page can reuse the exact same switch — nothing is duplicated between the two surfaces.

Speaker parsing (`lib/sermonTitle.ts`, added September 2026) — the channel's videos are titled `"Sermon Title | Speaker Name | Destiny Church LIVE"` in convention, but real titles vary: `"|"` and `"||"` are used interchangeably, most speaker segments carry no honorific, multiple guest speakers are joined by `"&"` / `","` and kept as one string, and a few titles skip the pipe entirely in favour of `" - Ps Someone"` (gated on an honorific so a title that legitimately contains `" - "` isn't misparsed). `parseYouTubeTitle()` handles the YouTube side; `lib/podcast.ts`'s `splitTitle` handles Buzzsprout's own (slightly different) convention and shares only the `" - "` fallback via `splitOnDash()`, since a Buzzsprout episode title is often the exact same raw text as its YouTube counterpart. `normalizeSpeakerName()` (honorifics stripped, case/whitespace collapsed) is what lets "Ps John Smith" and "Pastor John Smith" collapse to one filter chip or one pairing match. See `tests/unit/sermon-title.spec.ts`, whose fixtures are titles pulled directly from the channel's real RSS feed rather than invented.

"Latest sermon" — `getLatestVideo()` delegates to `getUploadedVideos()` (below) and returns its first result, so the featured card can never disagree with the top of its own archive grid. It previously ran an independent `search.list?eventType=completed` query that could, in principle, answer differently than the playlist-backed grid.

Archive (`components/sermons/SermonGrid.tsx` + `SermonCard.tsx`) — the full back-catalogue as a searchable, filterable video grid. `lib/youtube.ts`'s `getUploadedVideos()` pages the channel's **uploads playlist** (id = `"UU" + CHANNEL_ID.slice(2)`, no extra API call to look up) via `playlistItems.list` — 1 unit per 50 videos, vs. `search.list`'s ~100 — then batches `videos.list` (adding `liveStreamingDetails` to `part=`) to filter out anything that isn't an actual broadcast (shorts, trailers). `getFullSermonArchive()` (added September 2026) pages this to exhaustion so the grid can filter/sort across the *entire* archive rather than one page at a time — the whole thing is fetched once server-side, cached via each page's own `next: { revalidate: 3600 }`, and handed to `SermonGrid` as a single in-memory array. The grid itself is a pure client component: it derives speaker checkboxes and month chips from

that array, ranks free-text search via `lib/sermonSearch.ts`'s `searchSermons()` (Fuse.js — the same shared function the sitewide Smart Search chat's `find_sermons` tool uses, see below), and reveals more results 24 at a time with no further network calls. This replaced the September-2026 infinite-scroll design (`app/api/sermons/more/route.ts`, now deleted) once filtering by speaker required knowing every video up front. The legacy `getAllVideos()` (`search.list`-based) is still in `lib/youtube.ts`, used only by `/api/youtube/videos` and the mobile app's sermons endpoint — do not wire anything new to it.

The same `FilterPanel` is rendered twice: as a **persistent sidebar** at desktop widths and as the original **dropdown** on mobile, where a standing sidebar doesn't fit. Speaker filtering/sorting was removed in September 2026 (a sermon's speaker still shows on its card and title — it just isn't a facet any more) in favour of a **Series** filter and a standalone **"Guest speakers only"** toggle (and a per-card badge).

Series (added September 2026, single-select like Month) is backed by admin-curated YouTube playlists — `lib/sermonSeries.server.ts`'s `getSermonSeriesList()` reads the `sermon_series` table (Database Schema §27, playlist ids only) and resolves each against the YouTube Data API for its title/description (`lib/youtube.ts`'s new `getPlaylistSnippet()`) and video membership (`getPlaylistVideoIds()`), server-side in `app/sermons/page.tsx`. A playlist that fails to resolve (deleted/private) is silently dropped — fail-open, same posture as guest speakers below — and the whole Series section hides itself when nothing is configured. Series are managed from `/admin/sermons` (`components/admin/SeriesManager.tsx`): an admin pastes a playlist URL or bare id, the API validates it exists via `getPlaylistSnippet` before inserting, and removing a series only ever deletes the pointer row — the YouTube playlist itself is untouched.

Guest status is *not* inferred from parsed speaker names — it's backed by the channel's curated "Guest Speakers" YouTube playlist: `lib/youtube.ts`'s `getGuestSpeakerVideoIds()` (over the shared `getPlaylistVideoIds()`) returns the set of video ids in `GUEST_SPEAKERS_PLAYLIST_ID`, fetched once server-side and matched against the archive (45 of 288 sermons are guest-preached as of that build). The fetch fails open to an empty set, so a YouTube outage just hides the badge rather than breaking the page.

Listen to any sermon — every archive card whose video has a *confident* audio pairing gets a **Listen** button, not just the featured card. `lib/sermonPairing.ts`'s `pairArchiveWithEpisodes()` builds the id→episode map for the grid; unlike `pairAudioForVideo()`'s single-video "always guess something" fallback on the featured card, it keeps only confident matches, or every one of a few hundred cards would be tagged with a guessed pairing. `components/sermons/PodcastPlayerProvider.tsx` was moved out of the `/sermons` route tree and into the **root layout** (`app/layout.tsx`, which already had a player-height CSS variable wired up waiting for it) so audio survives client-side navigation anywhere on the site — starting a message on a grid card keeps it playing, uninterrupted, into that sermon's detail page.

Publishing audio (`/admin/sermons`, `sermon_admin` role, added September 2026) — an admin form that posts sermon audio straight to Buzzsprout via `lib/buzzsprout.server.ts`'s `createBuzzsproutEpisode()` (`BUZZSPROUT_API_TOKEN` + `BUZZSPROUT_PODCAST_ID` env vars). The video still goes to YouTube the normal way (YouTube Studio) — this only ever creates the audio episode, so no new Supabase table or storage bucket is involved. When the admin also pastes the matching YouTube video id, it's embedded as an HTML comment (`<!--yt:VIDEO_ID-->`) in the episode's description — the only Buzzsprout field confirmed to survive to the public RSS feed `lib/podcast.ts` reads, since `tags` has no documented guarantee of reaching public RSS output. `extractYouTubeIdHint()` reads it back out and `stripHtml()` strips it before it's ever shown as show notes; `pairAudioForVideo()` treats a match as deterministic, skipping the heuristic entirely.

Manual speaker correction (`components/admin/SpeakerEditor.tsx`, added September 2026) — the hand-edit counterpart to the AI review above. Searches the full archive (`useAdminList`, same fuzzy search every other admin list uses) and lets an admin type a speaker directly per video, showing whether the current value is AI-set, manually-set or unreviewed. Both paths write the same `speaker_overrides` row (`reviewed_by: "admin"` vs `"ai"`); a **Revert** button — enabled only when an override row exists — deletes it outright rather than setting it to blank, falling back to `lib/sermonTitle.ts`'s regex-parsed name (distinct from explicitly saving an empty speaker, which records a *confirmed* "no individual speaker", same null-vs-no-row rule as the AI path). `POST` / `DELETE /api/admin/sermons/speaker` are the two routes behind it. Title and description are never editable anywhere in this admin section — both always follow YouTube.

AI-recommended sermons — `find_sermons`, a tool on the sitewide Smart Search chat (`lib/smartSearch/tools.ts`, same pattern as its `find_products` tool), fuzzy-searches the full archive and answers with sermon cards in the conversation from anywhere on the site. `SermonGrid`'s own search box uses the same `lib/sermonSearch.ts` ranking function directly (no network route — the full archive is already in the browser), deliberately without an LLM call: higher-traffic public surface, no conversational context to extract filters from beyond the query text itself.

`/app/[slug]/page.tsx` — Dynamic Catchall

- Looks up `slug` via `getPostForView()` (`lib/posts.server.ts`) against the `posts` table — there is no separate `dynamic_pages` table. Renders the post's hero (`PostHero`, from `hero_style`), then its `body` through `RichContent` (upgrades embedded content blocks into real components; falls back to the equivalent of raw HTML for a post with none), with the promo rails from `components/posts/PostRails.tsx` unless the post turns them off. `generateMetadata` emits the post's description and Open Graph image.
- **Draft previews:** `?preview=<token>` shows an unpublished post when the token is a valid signature for that post's id (`lib/postPreview.server.ts` — `<expiry>.<hmac>`, one hour, signed with the Supabase secret key like the training unlock cookies). Tokens come from `GET /api/admin/posts/[id]/preview`. The page shows a "Preview" bar and is `noindex`. Without a valid token a draft 404s exactly as before.
- If no post matches, falls back to an active `redirects` row for the same slug (see Database Schema §1) and `redirect()`s there. A hit is recorded via `after()` — read `headers()` during render (an `after()` callback in a Server Component can't call it), close over the values, and register the write before `redirect()` throws; see `lib/engagement.server.ts` and Database Schema §24. `force-dynamic`, so this and the redirect lookup are never cached.
- Falls back to 404 (`notFound()`) if neither a post nor a redirect matches.

Layer 5: Footer (`components/ChurchFooter.tsx`)

Displayed on every page:

- **Contact info** — Address (tappable — opens the device's map app via `MapsLink`), phone, email
- **Social links** — YouTube, Facebook, Instagram
- **Quick links** — Common pages

- **Copyright** — Auto-updates year
- **Accessibility statement**

Routing & Pages

Development-only Routes

- `/dev/blocks` — Gallery of every registered content block, rendered through the real serialise → parse → render pipeline rather than by importing the components directly, so it exercises the same path a live post does. Useful when building a block: no need to create a draft post to look at it.
- `/dev/blocks/editor` — Harness that mounts the real editor with the real Blocks sidebar and inspector, with no auth, no database and no save. It exists because the real editor lives behind `/login` and a post record, which makes the editor UI hard to exercise otherwise. Shows the stored HTML and a live public render beneath the editor, so serialisation problems are visible as you type, and exposes the TipTap instance as `window.editor` for console work.

Both call `notFound()` when `NODE_ENV === "production"`. They mount admin UI without an auth check, so they must never be reachable on the live site.

Public Pages (No Auth Required)

Route	File	Purpose
`/`	`app/page.tsx`	Home page — hero, fe
`/about`	`app/about/page.tsx`	About church, team, vi
`/beliefs`	`app/beliefs/page.tsx`	Statement of faith, doc
`/sermons`	`app/sermons/page.tsx`	Latest message as vid series/month filters, a search over the full arc
`/sermons/[id]`	`app/sermons/[id]/page.tsx`	Individual sermon — a (`components/sermon` same `ModeSwitch` / `L` when a confident audi YouTube embed; plus Title/meta rows stay se player area switches
`/contact`	`app/contact/page.tsx`	Contact form, address,
`/portal/design/request`	`app/portal/design/request/page.tsx`	Ask the design team fo the `/portal` middlew `hr_staff` row. Name record; every request i
`/design-request/[token]`	`app/design-request/[token]/page.tsx`	The requester's own tr than a login — status, t revision's files, and bu `robots: noindex`
`/give`	`app/give/page.tsx`	Giving info — bank det
`/shop`	`app/shop/page.tsx`	Store front — publishe chips (`ShopProductG`
`/shop/[slug]`	`app/shop/[slug]/page.tsx`	Product detail — galler basket
`/shop/cart`	`app/shop/cart/page.tsx`	Basket (client, zustand
`/shop/checkout`	`app/shop/checkout/page.tsx`	Contact details + embe
`/shop/checkout/success`	`app/shop/checkout/success/page.tsx`	Order confirmation; cle
Also served at `shop.destinytees.uk/*`	`next.config.ts` `rewrites()`	Legacy subdomain hos
`/visit`	`app/visit/page.tsx`	First-time visitor guide
`/new-here`	`app/new-here/page.tsx`	Onboarding for new m
`/hire`	`app/hire/page.tsx`	Venue hire enquiry for
`/serve`	`app/serve/page.tsx`	Volunteer opportunitie
`/connect`	`app/connect/page.tsx`	Connect groups (small
`/kids`	`app/kids/page.tsx`	Kids ministry (ages 0-
`/youth`	`app/youth/page.tsx`	Youth ministry (ages 11
`/young-adults`	`app/young-adults/page.tsx`	Young adults (18-30s)
`/missions`	`app/missions/page.tsx`	Mission partners, outre
`/alpha`	`app/alpha/page.tsx`	Alpha course info, nex
`/bible-course`	`app/bible-course/page.tsx`	The Bible Course (Bibl
`/cap-money`	`app/cap-money/page.tsx`	CAP Money Course (C event. CTAs fall back t scheduled
`/song-requests`	`app/song-requests/page.tsx`	Guests search Spotify shows now playing an `components/song-re`
`/whats-on`	`app/whats-on/page.tsx`	Events listing — featur events grouped by mo

Route	File	Purpose
`/whats-on/[slug]`	`app/whats-on/[slug]/page.tsx`	On-site event page — upcoming sessions, sa and .ics
`/connect-card`	`app/connect-card/page.tsx`	Prayer requests, conne
`/jobs`	`app/jobs/page.tsx`	Job listings
`/jobs/[slug]`	`app/jobs/[slug]/page.tsx`	Job detail page
`/training`	`app/training/page.tsx`	`/training` resource (optional password) →
`/training/[categorySlug]`	`app/training/[categorySlug]/page.tsx`	A training category —
`/training/[categorySlug]/[subgroupSlug]`	`app/training/[categorySlug]/[subgroupSlug]/page.tsx`	A subgroup's post list password)
`/training/[categorySlug]/[subgroupSlug]/[postSlug]`	`app/training/[categorySlug]/[subgroupSlug]/[postSlug]/page.tsx`	A single training post – tracking and timed mo
`/baptism`	`app/baptism/page.tsx`	Baptism sign-up
`/child-dedication`	`app/child-dedication/page.tsx`	Child dedication reque
`/volunteer`	`app/volunteer/page.tsx`	Volunteer sign-up form
`/help`	`app/help/page.tsx`	Help centre / FAQ
`/links`	`app/links/page.tsx`	The main Linktree-styl built in `/admin/links` Rendered by `compone` `LinkPageView`; falls Steps in `lib/linksSt` answer, so it is never b
`/links/[slug]`	`app/links/[slug]/page.tsx`	Any other links page (y Unpublished or unknov a 404 (it would duplica from search" are `noir`
`/nfc`	`app/nfc/page.tsx`	"Digital back of seats" seat opens during a se site popup or smart se and Giving are hardcoc from `nfc_tiles`, incl the live ChurchSuite fe event has run
`/twelvetwo`	`app/twelvetwo/page.tsx`	Destiny 12:2 recovery
`/dckids`	`app/dckids/page.tsx`	Destiny Kids Camp 20
`/accessibility`	`app/accessibility/page.tsx`	Accessibility statemen review date, feedback glass-FX preferences t (`app/accessibility` client component)
`/privacy`	`app/privacy/page.tsx`	Privacy policy
`/terms`	`app/terms/page.tsx`	Terms of use
`/safeguarding`	`app/safeguarding/page.tsx`	Safeguarding policy
`/data-gdpr`	`app/data-gdpr/page.tsx`	Data & GDPR policy
`/governance`	`app/governance/page.tsx`	Transparency page: ch trustees/directors, cha history and filings. Dat Commission and Comp `lib/governance.ser` (`revalidate = 6048` snapshot per-regulato down
`/administration`	`app/administration/page.tsx`	Stale-bookmark redire
`/auth/callback`, `/auth/confirm`	`app/auth/callback/route.ts`, `app/auth/confirm/route.ts`	Supabase OAuth callb handlers

Route	File	Purpose
`/[slug]`	`app/[slug]/page.tsx`	Dynamic catchall — lo `posts` table. At ≥160 three-track grid with a `posts/PostRails.ts` unchanged at every wi

Admin Pages (Auth Required, Checked in `middleware.ts`)

All admin/staff features live under a single `/admin` prefix with one login at `/login` . Each section requires a specific access-level role (see Authorization Layers); Super Admins get everything.

Route	File	Purpose
`/login`	`app/login/page.tsx`	Staff sign-in. On success (or on revisiting wh already signed in), shows a "choose a systen screen — Admin (`/admin`) and Portal (`/portal`) cards, greyed out for whichever account can't open (`getSystemAccess` in `lib/staffPortalAuth.ts`)
`/admin/forgot-password`	`app/admin/forgot-password/page.tsx`	Password reset request
`/admin/reset-password`	`app/admin/reset-password/page.tsx`	Password reset form. A password Supabase refuses (found in a data breach, or too weak) an explanation panel from `passwordRejection()` instead of a raw er
`/admin`	`app/admin/page.tsx`	Admin dashboard home
`/admin/banner`	`app/admin/banner/page.tsx`	Manage site banners
`/admin/popup`	`app/admin/popup/page.tsx`	Manage pop-ups
`/admin/redirects`	`app/admin/redirects/page.tsx`	Manage URL redirects — click counts and a p link QR code
`/admin/analytics`	`app/admin/analytics/page.tsx`	Which links, QR codes and tiles people actua use, plus whole-site traffic. Three tabs: Short links, In person (`/nfc` + `/links`), Whole s (Vercel Web Analytics)
`/admin/cache`	`app/admin/cache/page.tsx`	Invalidate ISR cache
`/admin/posts`	`app/admin/posts/page.tsx`	Standalone content pages
`/admin/training`	`app/admin/training/page.tsx`	Training top level — manage categories
`/admin/training/[categoryId]`	`app/admin/training/[categoryId]/page.tsx`	A category's subgroups — add/edit/reorder t subgroups inside one training category
`/admin/training/[categoryId]/[subgroupId]`	`app/admin/training/[categoryId]/[subgroupId]/page.tsx`	A subgroup's folders and posts — the course material editor, with fuzzy search over posts
`/admin/alpha`	`app/admin/alpha/page.tsx`	Manage Alpha **and Youth Alpha** events — wrapper over `CourseAdminPage`
`/admin/bible-course`	`app/admin/bible-course/page.tsx`	Manage The Bible Course events — wrapper `CourseAdminPage`
`/admin/cap-money`	`app/admin/cap-money/page.tsx`	Manage CAP Money Course events — wrapp over `CourseAdminPage`
`/admin/recovery`	`app/admin/recovery/page.tsx`	Manage Recovery course events — wrapper `CourseAdminPage`
`/admin/featured-course`	`app/admin/featured-course/page.tsx`	Choose the What's On featured course
`/admin/featured-event`	`app/admin/featured-event/page.tsx`	Promote one ChurchSuite event — picker plu headline/blurb/image/CTA overrides and a promote window
`/admin/event-popup`	`app/admin/event-popup/page.tsx`	Copy for the popup advertising the featured (writes `popup_*` on the same row)
`/admin/nfc`	`app/admin/nfc/page.tsx`	Tiles on the `/nfc` page — add/edit/reorder/ A ChurchSuite form embed, artwork + copy - CTA, or an event picked from the live calenda (events without a framable signup are shown disabled with the reason)
`/admin/links`	`app/admin/links/page.tsx`	Links pages list — 30-day views/clicks per p create from a theme preset or as a copy, dele (not `main`) (Event Admin)
`/admin/links/[id]`	`app/admin/links/[id]/page.tsx`	The links page editor (`components/admin/links/LinksEditor.` Blocks, Appearance, Profile, Settings, Respo and Stats tabs beside a live phone preview. `tab=` deep-links a tab (Event Admin)
`/admin/song-requests`	`app/admin/song-requests/page.tsx`	Connect the church Spotify account, open/cl requests, name the event, set limits, remove requests (Event Admin)

Route	File	Purpose
`/admin/hr`	`app/admin/hr/page.tsx`	HR dashboard (staff, leave, jobs, documents, reviews, checklists) (HR Admin)
`/admin/hr/staff`	`app/admin/hr/staff/page.tsx`	Staff directory — searchable list of every sta record, filterable by employment type and st (HR Admin)
`/admin/hr/staff/[id]`	`app/admin/hr/staff/[id]/page.tsx`	Staff record — profile, leave, reviews, docum live checklists (HR Admin)
`/admin/hr/leave`	`app/admin/hr/leave/page.tsx`	Leave queue — pending/approved requests across all staff, approve/decline with remaini balance context (HR Admin)
`/admin/hr/jobs`	`app/admin/hr/jobs/page.tsx`	Job board editor — create/edit/close listings feed the public `/jobs` page (HR Admin)
`/admin/hr/applications`	`app/admin/hr/applications/page.tsx`	Job applications — review submissions and m each through its status pipeline (HR Admin)
`/admin/hr/documents`	`app/admin/hr/documents/page.tsx`	Documents — upload/categorise staff and or wide files surfaced in `/portal` (HR Admin)
`/admin/hr/reviews`	`app/admin/hr/reviews/page.tsx`	Reviews — schedule and record staff performance/probation reviews (HR Admin)
`/admin/hr/checklists`	`app/admin/hr/checklists/page.tsx`	Onboarding/offboarding checklist templates Admin)
`/portal`	`app/portal/page.tsx`	Staff self-service dashboard — holiday balan onboarding progress, next review, open desig requests and document count as cards linkin each section. Separate auth boundary from `/admin` ; see Authorization Layers
`/portal/profile`	`app/portal/profile/page.tsx`	Own profile fields (department, employment, dates) plus account settings — profile picture email, password
`/portal/team`	`app/portal/team/page.tsx`	Directory of colleagues with a portal login — name, role, department, work email, picture. Searchable client-side; never shows phone o anything else from `hr_staff`
`/portal/reviews`	`app/portal/reviews/page.tsx`	Own onboarding checklist (tick off items) and review history (dates/type only — see `GET /api/portal/reviews` , which never sends `hr_reviews.summary`)
`/portal/leave`	`app/portal/leave/page.tsx`	Staff self-service — request and withdraw ow leave
`/portal/documents`	`app/portal/documents/page.tsx`	Staff self-service — download own + org-wi documents
`/portal/design`	`app/portal/design/page.tsx`	Staff self-service — own design requests, plu link to `/portal/design/request` to file a n one. Matched by staff link *and* by email, so requests filed before this page existed still ap
`/admin/destiny-one`	`app/admin/destiny-one/page.tsx`	Destiny One overview: requests waiting, activ members, open invites, paused groups (Dest One Admin)
`/admin/destiny-one/requests`	`app/admin/destiny-one/requests/page.tsx`	Approve access requests as adult or under 18 (optional DOB, name fix, communities), or de optional ChurchSuite lookup (Destiny One Ad
`/admin/destiny-one/invites`	`app/admin/destiny-one/invites/page.tsx`	Invite one or many people by email with age, leader role and communities; resend, revoke (Destiny One Admin)
`/admin/destiny-one/members`	`app/admin/destiny-one/members/page.tsx`	Everyone with an account; panel to fix name, leader roles, suspend, delete (Destiny One A
`/admin/destiny-one/communities` (+ `/[id]`)	`app/admin/destiny-one/communities/**`	Communities, their Announcements and department groups with live people/adult cou and pause reasons; add/remove people, role create/archive groups with a live rule check. message content (Destiny One Admin)

Route	File	Purpose
`/admin/destiny-one/safeguarding`	`app/admin/destiny-one/safeguarding/page.tsx`	Queue, reports, all groups, manual pause, an reason-gated, audited transcript viewer (Safeguarding Admin)
`/admin/destiny-one/feedback`	`app/admin/destiny-one/feedback/page.tsx`	"Report a problem" and "Send feedback" fro app: New / Done / All, the text, who sent it, p and app version, crash-report id; mark done reopen (Destiny One Admin)
`/admin/destiny-one/settings`	`app/admin/destiny-one/settings/page.tsx`	Invite-only vs open to requests, invite expiry, minimum app builds per platform + update message, maintenance switch; retention and ChurchSuite status read-only (Destiny One A
`/admin/sermons`	`app/admin/sermons/page.tsx`	Publish sermon audio to Buzzsprout (video k going to YouTube separately); add/remove YouTube playlists as sermon series; run the A speaker review or manually search-and-corr any sermon's speaker; a read-only recent- episodes list showing pairing status (Sermon Admin)
`/admin/design`	`app/admin/design/page.tsx`	Design ticket queue — search, status/priority filters, inline Claim. Defaults to "Needs some rather than everything (Design Admin)
`/admin/design/[id]`	`app/admin/design/[id]/page.tsx`	Ticket detail — brief, requester, the thread, th deliverable uploader, and only the transition buttons `canTransition` allows from here (Design Admin)
`/admin/store`	`app/admin/store/page.tsx`	Store — product list
`/admin/store/products/new`	`app/admin/store/products/new/page.tsx`	Create a product (name → editor)
`/admin/store/products/[id]`	`app/admin/store/products/[id]/page.tsx`	Product editor — details, photos, size/colour variants, stock
`/admin/store/hero`	`app/admin/store/hero/page.tsx`	Shop hero slides — add/edit/reorder rotating
`/admin/store/orders`	`app/admin/store/orders/page.tsx`	Orders list
`/admin/store/orders/[id]`	`app/admin/store/orders/[id]/page.tsx`	Order detail — mark fulfilled/cancelled/refun
`/admin/users`	`app/admin/users/page.tsx`	Manage admin logins and their access-level (Super Admin only)
`/admin/onboarding`	`app/admin/onboarding/page.tsx`	What each access level is taught on first sign and the admin previewed from their side (Sup Admin only)
`/admin/audit`	`app/admin/audit/page.tsx`	Audit log — everything anyone does in the ac Ask it in plain English ("who added the Faith Hoodie to the store?") or read it: search, filte person/area/kind/date, open any entry for th field-by-field before and after. Second tab ho the weekly AI reports (Super Admin only)

Admin navigation, search and keyboard shortcuts

Everywhere you can go inside `/admin` is declared once, in `lib/adminNav.ts`. The sidebar, the header breadcrumbs, the dashboard's "Manage" grid, the mobile tab bar and the ⌘K palette all render from that one list, so adding a section means editing one file.

On a phone the sidebar becomes a tab bar. Below `md` the desktop `<aside>` is replaced by `components/admin/AdminTabBar.tsx`, pinned to the bottom of the viewport — the shape phones actually use, and one tap per section instead of the hamburger drawer's two or three. A tab is a *group*, not a page: a super admin can see around forty pages but only eleven groups, so `tabsFor(roles)` projects `ADMIN_GROUPS` down to one tab per top-level entry. Ungrouped items (Dashboard, Posts, Training) are their own tabs, a group left with one visible item collapses to a plain link, and a group with several opens a bottom `Sheet` listing its pages with their descriptions.

The row scrolls when it has to and centres itself when it doesn't — a store admin's two tabs sit in the middle, a super admin's eleven overflow and swipe. That is `justify-content: safe center` on `.admin-tabbar-scroll` in `app/globals.css`, not a Tailwind utility: plain `center` would centre the overflow as well, putting the first tabs off the left edge where scrolling can never reach them. The bar auto-scrolls the active tab into view on every route change, so arriving from a breadcrumb or the ⌘K palette still shows you where you are.

The mobile top bar in `AdminSidebar.tsx` keeps the logo, ⌘K, the theme toggle and an account button — the last opens a `Sheet` with your email, "View site" and Sign out, which used to live at the foot of the drawer and would otherwise have nowhere to go. It also renders `breadcrumbsFor(pathname)`; below `md` `AdminHeader` is hidden, so before this there was nothing on a phone saying which page you were on.

Route transitions. `app/admin/template.tsx` wraps admin pages in React's `<ViewTransition name="admin-page">` (stable in React 19.3; Next 16.4 needs no config flag — the old `experimental.viewTransition` option was removed). Pages cross-fade instead of snapping, and the tab bar's active pill — the

one element named `admin-tab-pill` in the document at a time — morphs from the old tab's box to the new one in the same frame, which is what makes switching tabs read as glass moving rather than two highlights blinking. The keyframes and the reduced-motion opt-out are at the foot of `app/globals.css`; the flag is app-wide but nothing outside `/admin` renders a `ViewTransition`, so the public site navigates exactly as it did.

Finding things. Every list page has the same toolbar — fuzzy search on the left, status filter chips with live counts beside it, a "showing x of y" count on the right — built on `lib/useAdminList.ts`. Search, filters and sort are mirrored into the query string, so a filtered list is a link you can send someone and Back behaves. `/` jumps to the search box.

⌘K palette (`components/admin/AdminCommandPalette.tsx`) searches two things at once: admin pages and quick actions locally from the registry, and real records — posts, products, orders, redirects, training, course dates, users — from `GET /api/admin/search`. Opening it with an empty box lists your recently visited sections. Records are role-filtered server-side, so a Store Admin's results contain orders and products and nothing else.

Shortcut	Does
`⌘K` / `Ctrl-K`	Open the palette
`?`	Keyboard shortcut help
`/`	Focus the current list's search box
`g` then `d` / `p` / `t` / `s` / `o` / `r` / `b` / `u`	Dashboard, Posts, Training, Store, Orders, Redirects, Banner, Users

HR is in the sidebar and dashboard grid like any other section, but it is deliberately excluded from the ⌘K palette's record search — staff records, leave reasons and applicant CVs shouldn't surface in a fuzzy search box, regardless of who's searching. See `app/api/admin/search/route.ts`.

Components

Global Components

`ui/Button.tsx`

Client-safe, no directive. The site's one button. Before it existed there was no UI layer at all: every button was a hand-written Tailwind string, and a survey found ~60 variations of the orange pill alone — same intent, drifting padding and shadow.

```
<Button href="/contact" size="lg">Find out more</Button>
<Button variant="onDark" onClick={openSignup}>Sign up</Button>
```

Prop	Values	Notes
`variant`	`primary` · `secondary` · `outline` · `accentOutline` · `onDark` · `glass`	`accentOutline` is the orange-outline-fills-on-hover secondary CTA (previously pasted separately in three files); `onDark` / `glass` are for hero photography
`shape`	`pill` (default) · `soft` · `card`	`soft` is `rounded-xl`, the `/admin` chrome; `card` is the larger icon+label CTAs (Give Online, Text to Give)
`size`	`xs` · `sm` · `md` · `lg` · `xl` · `cta` · `icon`	`sm` is the `/admin` default; `icon` is 44px square for icon-only buttons
`href`	string	Renders `next/link`; `http(s):` / `mailto:` / `tel:` get an external anchor
`fullWidth`	boolean	
`loading`	boolean	Shows a spinner, sets `aria-busy`, disables the button; label stays in place

Every size is a padding cluster that already existed in the codebase (`px-6 py-3` appeared 14×, `px-7 py-3` 13×, `px-6 py-2.5` 13×), so adopting it does not shift anything by a few pixels. It also adds focus-visible rings and disabled states, which most of the hand-written buttons lacked.

> **Do not override padding through `className`.** `lib/cn.ts` is a plain join with no Tailwind conflict resolution (no `tailwind-merge`), so a `px-7` beating a size's `px-6` depends on stylesheet order rather than anything guaranteed. Add a size instead.
>
> Genuinely one-off buttons should stay plain `<button>` elements — this is for the repeated cases. Migration is opportunistic; most call sites are still hand-written strings.

`ui/Container.tsx` / `ui/Section.tsx`

Server-safe, no directive. The horizontal frame (`Container`, three widths — `prose` `max-w-3xl`, `content` `max-w-5xl`, `wide` `max-w-7xl` default) and a page section (`Section` — tinted band via `tone` reusing the block system's `TONE_SURFACE`, plus a `dark` tone `Section` adds itself since the block system has no equivalent; padding via `padding`: `sm` `py-12 sm:py-16`, `md` `py-16 sm:py-20` default, `lg` `py-20 sm:py-28`). Replace the `mx-auto max-w-* px-4 lg:px-8` / `py-*` pairs that were hand-written at ~67 and ~130+ call sites respectively.

`ui/SectionHeading.tsx` / `ui/Eyebrow.tsx`

Server-safe, no directive. `Eyebrow` is the exact `text-xs font-bold uppercase tracking-widest text-destiny-orange` string, pulled out of `components/blocks/tokens.ts`'s `EYEBROW` constant so it's reachable without importing the block system. `SectionHeading` is eyebrow + heading + lead + an optional action link, one ramp replacing six per-section variants. Both carry the `FONT_ROBOTO` inline escape hatch documented under Content Blocks below — `h1,h2,h3` are unlayered Arial in `globals.css`.

`ui/Card.tsx`

Server-safe, no directive. The card shell (`CARD_SHELL` / `CARD_HOVER` from `components/blocks/tokens.ts` , promoted). Pass `href` to make the **whole card** the link — motivated by `GetInvolvedSection` , whose image scaled on `group-hover` across the entire card while only a small pill at the bottom was actually clickable. `interactive` gets the hover lift without a link, for cards whose action lives inside them.

`ui/Icon.tsx`

Server-safe, no directive. Wraps a Material Symbols ligature span. Decorative (`aria-hidden`) by default — pass `label` only when the icon IS the control or carries meaning no adjacent text repeats. Exists because the icon font renders by literal ligature text (`play_arrow` , `volunteer_activism` , …), which a screen reader reads verbatim on any span not marked `aria-hidden` ; roughly 500 of ~570 such spans site-wide have been converted. The icon font is **subset** to the names in `lib/iconNames.ts` (see Libraries & Utilities) — any name outside that list renders as its literal text.

`ui/PageHero.tsx` / `ui/MediaBanner.tsx`

Server-safe, no directive. `PageHero` (generalised from `components/ministry/ MinistryHero.tsx`) is the inner-page hero: sharp photo via `next/image priority` , two scrim layers, Roboto semibold heading, optional `chips` and `actions` . `MediaBanner` is the inset blurred-photo CTA banner (`WorshipWithUsSection` , `ConnectGroupsBanner` , …) — copy left, stacked buttons right. Both replace the CSS- `background-image` / independently-pasted blur+scrim idiom that had accumulated across ~8–11 files.

`ui/Field.tsx` (`Field` , `TextareaField` , `SelectField`)

Client component. The one input shape: generates an id, wires `label` / `aria-describedby` / `aria-invalid` , and renders a hint or an error. Replaces three competing `inputClass` constants plus a fourth written inline in `ContactForm` .

`ui/Badge.tsx` / `ui/Disclosure.tsx`

Server-safe, no directive. `Badge` is a status/category pill with semantic tones (`success` / `warning` / `danger` / `info` map onto the `--color-*-bg` / `-fg` pairs in `globals.css` , plus `neutral` / `accent` / `onDark`). `Disclosure` is a `<details>` / `<summary>` expand-collapse row with no client JS — used for `/visit` 's FAQ, which used to be seven permanently-open `<div>` s despite shipping `FAQPage` JSON-LD.

`/dev/ui` is the gallery for all of the above — every variant of every primitive on one page, the `components/ui/` sibling of `/dev/blocks` . 404s in production.

`ui/BackgroundVideo.tsx`

Client component. A decorative autoplaying hero video that honours the site's reduced-motion preference (`AccessibilityContext` , which mirrors the OS setting) by simply not rendering when it's on, rather than looping regardless — used on `/serve` and `/alpha` .

`ui/VideoPlayer.tsx`

Client component. A content video with the site's own controls (big play button, orange scrubber, play/pause, mute, time, fullscreen that keeps the overlay; keys: space/k, arrows ±5s, m, f) and optional **checkpoints** that pause playback and offer choices:

```
<VideoPlayer
  src={url}
  label="What to expect at Destiny Church"
  checkpoints={[{ at: 6, options: [{ label: "Continue" }, { label: "Jump to 0:10", seekTo: 10 }] }]}
/>
```

Checkpoints are polled per animation frame (not `timeupdate` , which overshoots by up to 250ms) and fire only when playback runs *through* them — scrubbing past one does not stop the video, seeking back before it re-arms it. Never autoplays. Used on `/visit` .

`ui/Modal.tsx`

Client component (`"use client"`). The shared dialog shell.

```
<Modal open={open} onClose={close} title="Team sign-in" size="sm">
  {children}
</Modal>
```

Prop	Values	Notes
`open`	`boolean`	Renders nothing when false; no portal is mounted
`onClose`	`() => void`	Fired by Escape, the close button, and the backdrop
`title`	`string`	Becomes the dialog's accessible name (`aria-labelledby`)
`description`	`string?`	Optional sub-line; wired to `aria-describedby`
`size`	`"sm" | "md" | "lg"`	`max-w-md` / `max-w-2xl` / `max-w-3xl` . Default `md`
`showClose`	`boolean`	Default true
`closeOnBackdrop`	`boolean`	Default true; turn off for forms where a stray click loses typed input
`footer`	`ReactNode?`	Pinned below the scrolling body

Portals to `document.body` at `z-[200]` , traps Tab inside the panel, returns focus to whatever opened it, and locks body scroll — restoring the *previous* `overflow` value rather than blanking it. Entrance animation is the CSS keyframes `.dc-modal-backdrop` / `.dc-modal-panel` in `globals.css` , with a `prefers-reduced-motion` block.

> Before it there were six one-off modals and only `NfcTileModal` had real dialog semantics — its own header comment notes the others have "no role, no focus management and no trap". New modals should use this; the existing six can migrate when next touched.

`ChurchHeader.tsx`

- **What:** Site navigation header
- **Props:** None (client component; holds its own open/submenu/scroll state and reads `useBannerBars()` + `usePathname()`)
- **Behavior:**
- Desktop: horizontal menu bar; "About" and "What's on" open hover **dropdowns** (`Dropdown`) rendered as white rounded cards that fade in with a staggered per-item reveal (a brief close delay keeps them from flickering shut between the trigger and the card)
- Mobile: an animated hamburger (bars morph into an X) toggles a **full-screen brand-colour overlay** (`fixed inset-0`, `bg-destiny-orange/60 backdrop-blur`, a sibling of `<header>`) with **drill-down submenus** (`mobileSubmenu` state) and a top-down reveal; scrolling auto-closes it
- The top-level list and the active submenu's list are laid out side by side in a `width: 200%` flex row that translates `-50%` / `0` (`transition: transform 0.38s cubic-bezier(0.4, 0, 0.2, 1)`) to switch between them — a real slide left/right rather than an unmount/mount swap. Both panels stay mounted; a `lastSubmenu` value (updated during render, not in an effect, guarded against re-render loops) keeps the submenu panel showing its last items while it slides back off-screen so it never goes blank mid-transition. The per-item stagger fade/translateY on entry is unchanged.
- A scroll-driven **morph** reshapes the header pill (`progress` ramps 0→1 over `MORPH_DISTANCE`)
- Active route highlighting
- Search is not in the header — it lives in `FloatingSmartSearch`
- Self-suppresses: returns `null` on `/admin`, `/nfc`, and `/portal` (each is a chrome-free shell with its own minimal layout)
- Clears its conversation (`reset()`) whenever the menu closes, so reopening it starts fresh

`ChurchFooter.tsx`

- **What:** Sitewide footer (server component — awaits `isYouTubeQuotaExceeded()` to drop the Sermons link when the YouTube quota is blown)
- **Displays:** Brand blurb + address, three link columns (Church / Connect / Legal), copyright, Report a Bug, phone
- **Layout:** 4-column grid from `md:` up; on mobile the three link columns render as accordions via `FooterLinkGroup`
- **Address:** The four `ADDRESS` lines from `lib/churchInfo.ts` are one `MapsLink` wrapping an `<address>` — the whole block is a single tap target that opens the device's map app. What it *sends* the map app omits `ADDRESS.venue` (see `lib/maps.ts`), so the text on screen and the map query deliberately differ

`MapsLink.tsx` + `lib/maps.ts`

- **What:** Client component that wraps an address in a link to the device's map app
- **Why two URLs:** There's no single "open in maps" URL. `https://www.google.com/maps/search/?api=1&query=…` is the cross-platform default (Android and desktop hand it to Google Maps or the browser); `https://maps.apple.com/?q=…` is what iOS/iPadOS/macOS want, where Google Maps often isn't installed
- **Hydration:** The server always renders the Google URL; `useHydrated()` (the `useSyncExternalStore` snapshot in `lib/useHydrated.ts`) flips the href to Apple Maps after hydration on Apple devices. Sniffing the user agent during render instead would produce a server/client markup mismatch
- **`lib/maps.ts`:** Holds `DESTINY_CENTRE_MAP_QUERY` and `googleMapsUrl()`, `appleMapsUrl()`, `isApplePlatform()`, `deviceMapsUrl()`. The address comes from `lib/churchInfo.ts`; this only decides which map app
- **Printed vs. sent:** the UI prints `ADDRESS.venue` first, the map app gets the postal address without it — see `lib/maps.ts` below
- **Gotcha:** iPadOS reports a "Macintosh" user agent, so the Apple check also tests `navigator.maxTouchPoints > 1`
- **Gotcha:** because the visible text and the map query differ, callers should pass `aria-label` built from the children rather than letting it default to the query — WCAG 2.5.3 wants the accessible name to contain the visible text

`FooterLinkGroup.tsx`

- **What:** One footer link column — a client component so it can hold open/closed state
- **Mobile:** Collapsed accordion with a chevron header (~44px tap targets), so the footer isn't ~23 stacked links
- **Desktop:** Forced open with CSS only (`md:grid-rows-[1fr]`, `md:pointer-events-none`, chevron wrapper `md:hidden`) — no JS media query, so no hydration mismatch
- **Why it matters:** Links stay in the DOM at every breakpoint (SEO/crawl-safe). Collapsed panels also get `invisible` so hidden links leave the keyboard tab order
- **Gotcha:** The chevron's `md:hidden` lives on a plain wrapper `<span>` — the global `.material-symbols-rounded` rule in `app/globals.css` overrides Tailwind display utilities applied directly to the icon

`CookieBanner.tsx` + `lib/cookieConsent.tsx`

- **What:** GDPR cookie consent, split into **three categories** rather than a single on/off:
- `essential` — always `true`, keeps the site running
- `media` — third-party embeds: ChurchSuite forms, and YouTube in the live/sermon/missions players
- `analytics` — Vercel Analytics
- **State:** `CookieConsentProvider` (a React context, hook `useCookieConsent`) holds the choice and persists it to `localStorage` under `destiny-cookie-consent`. `decided` is `true` once any choice is stored.
- **Banner behavior:**
- Returns `null` under `/portal` (the staff self-service shell is chrome-free, like `/admin`)
- Renders only after mount and only while no choice is stored (hydration-guarded so SSR/client markup match)

- Two buttons: **"Accept all cookies"** (`allowAll` → media + analytics on) and **"Necessary cookies only"** (`denyOptional` → both off). Both link out to the Privacy Policy and Terms of Use
- A finer `savePreferences({ media, analytics })` also exists for per-category control
- **Gotcha:** Because consent loads from `localStorage` in an effect, `consent` is `null` on first render. Every gated component waits for `mounted` before reading it, so nothing embeds or tracks until the stored choice (or its absence) is known.

`AnalyticsGate.tsx`

- **What:** Conditional analytics loading
- **Logic:** Renders `<Analytics />` (Vercel) only when `consent?.analytics` is `true` ; returns `null` otherwise

Media-gated embeds (`consent?.media`)

`ChurchSuiteEmbed.tsx` , `live/LivePlayer.tsx` , `sermons/SermonPlayer.tsx` and `missions/MediaEmbed.tsx` all gate their third-party `<iframe>` on `consent?.media === true` . Until media cookies are accepted they render an in-place placeholder ("Cookies required to load this form/video") with an **Accept all cookies** action wired to the same `useCookieConsent` context, so a visitor can opt in without scrolling back to the banner.

Once consent is in, `ChurchSuiteEmbed` and `MediaEmbed` cover the iframe with `ui/EmbedLoadingOverlay` until it fires `onLoad` — see below.

`ui/EmbedLoadingOverlay.tsx`

- **What:** The spinner-and-copy layer over a third-party iframe that hasn't painted yet. Used by `ChurchSuiteEmbed` (forms, light tone) and `MediaEmbed` (video, dark tone).
- **Why it escalates:** a cross-origin iframe gives us exactly one signal, `onLoad` — there is no progress to report. A single static "Loading" sitting there past a few seconds reads as *broken* rather than slow, and the visitor closes the modal. On `/give` , `/connect` and the `/nfc` tiles that is the whole conversion. So the line changes as time passes: movement is what says something is still happening.
- **Stages:** `Loading` → `Still loading` (3s) → `Taking longer than usual` (8s) → `Thanks for waiting` (14s), the last one alongside an **Open the form in a new tab** link. Timings live in `lib/embedLoading.ts` and are pinned by `tests/unit/embed-loading.spec.ts` .
- **The copy is true at the point it appears**, deliberately. Invented machinery ("Connecting to secure server", "Encrypting your details") holds attention *because* it claims things the site is not doing, and half these embeds are the giving form — being caught inventing a security step on a donation page costs more than the bounce it saves. The escalation does the work without the claim.
- **Props:** `loaded` (drives the fade), `tone` (`light` | `dark`), `fallbackHref` / `fallbackLabel` (omit to leave the escape hatch out — worth having for a form the visitor came to fill in, less so for a video).
- **A11y:** the message is a `role="status"` live region — "taking longer than usual" is the one thing a screen-reader user cannot otherwise get from an iframe that hasn't painted. The spinner is `aria-hidden` . On load the overlay fades and *then* flips to `visibility: hidden` (stepped transition in `globals.css`), which takes the stale copy out of the accessibility tree and the fallback link out of the tab order without cutting the fade short.

`SiteBanner.tsx`

- **What:** Top-of-page announcement banner
- **Data:** `site_banner` table (fetched in root layout)
- **Features:**
- Multiple banner types (sitewide, alpha events, recovery)
- Optional CTA link
- Can be dismissed by user (session storage)
- **Path suppression:** hidden on `/admin` and `/portal` (both are chrome-free staff shells with their own minimal layout); the `isAdmin` check in the component covers both prefixes

`SitePopup.tsx`

- **What:** Modal pop-up overlay
- **Data:** `site_popup` table
- **Features:**
- Shows once per session (if `show_once=true`)
- Title, body (markdown), image, CTA button
- Dismiss button
- Centered on screen
- **Suppressed on `/nfc` and `/admin/*` :** the layout is a server component and can't know the path, so both pop-ups check `usePathname()` themselves. `/nfc` is a grid of pop-ups people opened on purpose mid-service, and `/admin/*` should never interrupt staff with announcements. `EventPopup.tsx` applies the same two exclusions (plus `/whats-on` and the event's own page).

`WelcomeWidget.tsx` **— removed**

The "New here?" script (formerly merged into `FloatingSmartSearch` as its `welcome` mode) has been removed entirely — the resting-circle teaser flip, the scripted conversation tree, and `lib/welcomeChat.ts` are all gone. Smart Search now does one thing: the AI conversational search. Visitors who don't know what to ask reach the same destinations (visiting, giving, Alpha, etc.) through the site's normal nav and the "New Here?" page/link, which were never part of this widget.

`shop/ShopHero.tsx`

- **What:** Dynamic, auto-rotating hero at the top of `/shop`
- **Data:** `shop_hero_slides` table (via `getActiveShopHeroSlides()` ; passed in as a prop from the server component)
- **Features:**

- Image-backed slides (next/image `fill`) with dark gradient + Anton headline + orange CTA pill
- Auto-rotates (~6s crossfade) with 2+ slides; static with one
- Pauses on hover/focus; honours `prefers-reduced-motion` (no auto-advance)
- Keyboard-navigable dot indicators
- Falls back to the static "The Destiny Store" masthead when no slides are active (handled in `app/shop/page.tsx`)

`FloatingSmartSearch.tsx`

- **What:** the site's single floating widget — the AI conversational search. Collapsed it's a plain circle with a search mark; tapping it expands to the full pill and input.
- **Two widths, transitioned not keyframed.** `--fs-w` holds the resting width per state (circle `3.5rem` → pill `min(100vw-2rem, 28rem)`) with `transition: width`. The transition also leaves the loading glow (`<BorderBeam>` from the `border-beam` package, which owns its own wrapper element) free to animate independently, and doesn't run on first paint, which is what the transient `.morph-*` classes existed to prevent.
- **Loading/thinking indicators.** The pill's rotating rainbow border while a request is in flight is `<BorderBeam active={expanded && loading}>` (`border-beam`), wrapping the `.glass` layer in place of the old hand-rolled `.search-glow` conic-gradient CSS. The "thinking" dots in the chat thread and the input-field spinner are both `<ThinkingOrb state="searching" size={20} />` (`thinking-orbs`).
- **Both are `React.lazy` + `Suspense`, not static imports.** They're decorative only, so the code shouldn't ship in the main bundle for every visitor who never opens Smart Search. `BorderBeam`'s Suspense fallback renders the exact same `pillGlass` content (the button/input/form — everything interactive) in a plain `<div>`, so the pill is never hidden or non-functional while the chunk fetches; only the animated border is deferred. `ThinkingOrb`'s fallback is a small CSS spin-ring (`OrbFallback`) at the same size, so there's no layout shift. Confirmed via network tab: `border-beam` loads as its own chunk (not inlined into the page bundle), and `thinking-orbs` doesn't fetch at all until a chat request actually sets `loading` true.
- **`searchEnabled` prop.** Mirrors the `smart_search` service kill-switch. With the prop false the component returns `null` — there's no fallback experience without the AI, so the widget just isn't rendered.
- **Path suppression.** The widget hides itself on the chrome-free / auth-gated areas via a `usePathname()` check: `/admin`, `/training`, `/nfc`, `/links`, and `/portal` all return `null`. `/portal` is the staff self-service shell (its own minimal chrome), so the floating widget — like the site header, footer, and the cookie banner (`CookieBanner.tsx`, which also returns `null` under `/portal`) — is suppressed there.
- **Feature:** the widget runs if the `smart_search` service is enabled
- **Behavior:**
- Click button → pill expands, chat thread opens
- User types query → full history sent to `/api/chat` (OpenAI, `gpt-4.1-mini`). The endpoint is protected by a per-IP rate limit (20 requests/minute) and strict message-shape validation rather than a bot-challenge gate.
- The route uses **tool-calling** and streams **NDJSON** events (`text`, `tool_call`, `tool_result`, `done`). Prose is still parsed for the trailing `OPTION:` / `PAGE:` / `CTA:` lines (clarifying chips + a navigation CTA). Each `tool_call` event shows a short status line ("Searching the web...", "Reading the page...", etc.) while that tool runs.
- **Tools** (`lib/smartSearch/tools.ts`): `find_products` (searches published shop products via `getPublishedProducts()` + fuse.js, returns cards), `get_weather` (Open-Meteo, no key), `get_directions` (Google Maps embed), `search_web` (Tavily search, `search_depth: "advanced"`), `extract_page` (Tavily extract — reads the full content of one URL a prior `search_web` call actually returned; rejects any URL not in that request's `seenUrls` set, so the model can't be steered into fetching arbitrary pages).
- **Result cards** (`components/smartSearch/ResultCards.tsx`) render below the prose in Smart Search's glass style. The product card offers inline size/colour selection and **add-to-cart** (via `useCart()`), so a visitor can buy without leaving the conversation.
- Chat history + cards stored in component state (not persisted)
- **Optional env vars:** `NEXT_PUBLIC_GOOGLE_MAPS_EMBED_API_KEY` (directions embed — degrades to an "Open in Maps" link without it) and `TAVILY_API_KEY` (web search + page extraction — degrades to a "not configured" note). Weather and product search need no extra key.

> `VisualEditOverlay.tsx` **does not exist.** There is no in-page visual editing overlay — it was part of the removed page-builder/Studio feature (migration `20260711_06_remove_page_builder.sql`). Content is now edited through dedicated admin forms (`/admin/posts`, `/admin/store/products/[id]`, etc.), each using `RichTextEditor.tsx` for rich text where applicable.

`GlassBloomTracker.tsx`

- **What:** Performance tracking script
- **Purpose:** Tracks bloom/glass effect performance for optimization

`LiveBanner.tsx`

- **What:** "WE ARE LIVE" banner bar, styled like `SiteBanner.tsx`'s bars
- **Data:** `LiveContext` (server-seeded in root layout via `getLiveStatus()`, then polled client-side every 30s)
- **Behavior:** Renders at the top banner slot whenever the channel is live; hidden on `/admin/*` and `/portal/*`. CTA opens the broadcast on YouTube in a new tab (`youtubeWatchUrl()` in `lib/youtubeId.ts`); the old `/live` URL redirects to the channel's live tab (`next.config.ts`). The live bar **takes priority over the DB banners** — while it shows, the sitewide/alpha/recovery banners are hidden rather than stacked beneath it (`lib/useBannerBars.ts` returns `1` when live off `/live`), so there is only ever one bar to notice during a service.

`contexts/LiveContext.tsx`

- **What:** The single client-side source of live state, consumed by the banner and the homepage status line
- **Seeding:** The root layout passes `getLiveStatus()` straight in, so the first paint is already correct — polling only ever corrects it afterwards
- **Polling:** `/api/youtube/live` every 30s, plus an immediate poll on mount and on `visibilitychange` / `focus` / `online`. The mount poll matters: the server render can be a minute stale, and "we went live 40 seconds ago" is exactly when someone opens the page.

- **Grace period:** `live` does not drop to false until **two consecutive** negative polls (`OFFLINE_GRACE`), so one flaky request doesn't pull a running service off someone's screen. The streak is counted per poll — an earlier version counted it in an effect keyed on `live` , which only re-runs when the boolean flips, so it could never reach two.

Page-Specific Components

Destiny One admin (`components/admin/destinyOne/PeoplePicker.tsx`)

- `PeoplePicker` — searchable multi-select of people with an Adult / Under 18 badge each.
- `RuleCheck` — the live "N people, M adults — meets the rules / needs 3 people including 2 adults" line under the picker, so whoever builds a group sees the rule before the server enforces it.

Ministry pages (`components/ministry/*`)

The shared vocabulary for the five ministry pages — `/child-dedication` , `/kids` , `/youth` , `/young-adults` and `/connect` . Before this existed each page carried its own byte-identical copy of the hero and the feature-card grid, so the five were visually interchangeable and a change meant editing five files. All server components.

- `MinistryHero.tsx` — the page hero. Sharp photo through `next/image` `fill` + `priority` + `sizes="100vw"` (the previous heroes used a blurred CSS `background-image` , which could be neither optimised nor prioritised as the LCP element). Responsive `min-h` , centred content, and a `chips` prop rendering `glass glass-sm glass-pill` fact pills — those chips are what differentiate the five heroes, and they're where `/kids` and `/youth` carry their schedule instead of cramming it into the eyebrow. `objectPosition` frames per-page (`/kids` needs `top`).
- `SplitSection.tsx` — image-and-copy split on a 7/5 asymmetric 12-column grid rather than an even 50/50. `reverse` swaps columns via `lg:order-*` so JSX stays in reading order; `wideMedia` flips the ratio for photo-led sections; `tone="dark"` renders the shared `#363f48 → #242e37` band.
- `ImageMosaic.tsx` — 3-or-4 image offset mosaic replacing the old `col-span-2` -plus-two-squares collage. Every tile is `fill` inside a fixed aspect box with a real `sizes` , which is what removes the layout shift and oversized downloads the old fixed- `width` / `height` collages caused.
- `FeatureGrid.tsx` — the "why join / what we're about" cards. Card shell borrowed from `events/EventCard` (hairline border, two-layer shadow, hover lift); the orange-tinted icon square was dropped in favour of an oversized index numeral. One `AnimateIn` wraps the whole grid — `AnimateIn` renders a plain `<div>` , so wrapping cells individually would break the grid.
- `AgeGroupCards.tsx` — age-banded groups (`/kids` classes, `/youth` key stages), which previously had two different designs for the same job. Colours come from the `destiny-*` theme tokens, not the inline hex strings the pages used to carry.
- `FactStrip.tsx` — when/where/cost as one divided panel instead of a third identical card row.
- `FormSection.tsx` — the dark ChurchSuite band on `/child-dedication` and `/connect` . Wraps `ChurchSuiteEmbed` in a white panel; the embed keeps its own cookie-consent gate untouched.

Kids Camp (`components/kids/*`)

Section components for `/dckids` (Kids Camp). Separate from `components/ministry/*` — this page has not been brought onto the shared vocabulary yet.

- `KidsCampHero.tsx` , `KidsCampDetails.tsx` , `KidsCampTeam.tsx` (safeguarding leads), `KidsCampVideo.tsx` , `KidsCampForm.tsx` , `KidsCampFAQ.tsx` (client-side accordion)

Course pages (`app/alpha` , `app/bible-course` , `app/twelvetwo` , `app/cap-money`)

Alpha, The Bible Course, Destiny 12:2 and CAP Money each hand-rolled the same ~150-line "when and where" ticket-stub event card and the same ~35-line fetch/filter/loading boilerplate against `/api/alpha-events` — identical structure, differing only in each course's brand accent colour and which event `type` it filters for. That exact duplication (not the whole page — see below) is now shared:

- **`components/courses/CourseEventCard.tsx`** — the event card. Takes `accentColor` (each course's own brand colour — pull it from `COURSE_ADMIN_PAGES` in `lib/courseEvents.ts` rather than re-hardcoding a hex) and an optional `label` , Alpha's one real variation: it runs Alpha and Youth Alpha through the same feed, so when more than one event is showing it needs a header saying which is which.
- **`lib/useCourseEvents.ts`** — `useCourseEvents(matchesType)` for the fetch (each page passes its own type filter, e.g. `(e) => e.type === "cap"`), and `summarizeCourseSession(event)` for the "Starting/Next session <date>" line every course's `ChurchSuiteModal` subtitle shows.

This did **not** become one monolithic `CourseLanding` template covering the whole page. Past the event card, each page has genuinely different content — Alpha's video modal and Youth Alpha cross-promo, Bible Course's 8-session/4-feature grids, Destiny 12:2's Life Recovery Bible cross-sell and pastor contact card, CAP Money's stats row and debt-help CTA — and a template flexible enough to cover all of that would need an "arbitrary sections" escape hatch, which reduces duplication less than it looks like. Each page also keeps its own brand-coloured hero and CTA buttons rather than going through `components/ui/Button` — that component's variants are fixed Tailwind classes, not arbitrary hex colours, and each course's colour is a deliberate, already-coordinated brand decision (matched to `COURSE_ADMIN_PAGES` 's admin-side accent), not drift to converge.

Governance (`components/governance/*`)

Section components for `/governance` . All are server components taking plain props from `getGovernanceData()` , and each returns `null` when it has nothing verified to show — an absent section is better than an empty heading on a transparency page.

- `GovernanceHero.tsx` — eyebrow, title, lede
- `RegistrationCards.tsx` — paired charity/company cards with numbers, status, dates, registered office and outbound links to each official register
- `CharitableObjects.tsx` — charitable objects, activities and areas of operation
- `TrusteesDirectors.tsx` — trustee and director lists. Deliberately name-and-role only: both APIs also return dates of birth and correspondence addresses, which are not republished here
- `FinancialHistory.tsx` — headline income/expenditure plus per-year bars scaled against the largest figure across all years, so years stay comparable to each other

- `FilingHistory.tsx` — recent Companies House filings (linked to the official documents) and charity annual-return submission dates
- `GovernanceSourceNote.tsx` — attribution, refresh cadence, and an explicit notice naming any regulator whose data is being served from the stored snapshot rather than live

Events (`components/events/*`)

The one place event UI lives, shared by What's On, the homepage carousel and the preview routes. Before this existed the card was duplicated three times (`whats-on/EventsGrid` , `home/WhatsOnSection` , `posts/PromoRail`) and the copies had drifted.

- `EventCard.tsx` — **the** event card, in three trial variants selected by a `variant` prop: `a` (restrained: neutral surface, hairline border, orange kicker + CTA only — the default that ships live), `a-pill` (`a` plus a category chip on the artwork), `c` (full-bleed poster, copy on a `.glass` panel). Variants `a-pill` and `c` exist only for the preview routes; delete the unused bodies once a variant is chosen. Server-safe (no `"use client"` , no hooks) and — importantly — **never reads the clock**: filtering and month grouping happen server-side, or the client would hydrate a different set of events than the server rendered at midnight and DST boundaries.
- `EventCardArtwork.tsx` — artwork plus the two no-artwork fallbacks. Roughly half the feed has no image, so the fallback is a first-class state (large date numeral, or a brand panel for `c`).
- `EventsGrid.tsx` — the What's On grid. Receives month groups pre-filtered and pre-sorted from the server and does **only** the text search. Sticky month headings; keeps `id="events"` , which ChurchHeader, the footer and the sitemap all deep-link to.
- `FeaturedEventHero.tsx` — the wide banner above the grid. Takes an *already-merged* `ResolvedFeaturedEvent` , so it holds no fallback logic and renders `null` when nothing is live. Sits deliberately **outside** the `#events` section so that anchor still lands on the grid.
- `EventPopup.tsx` — the featured-event popup. `sessionStorage` (once per visit, unlike SitePopup's `localStorage` once-per-visitor), keyed on `identifier:updated_at` so changing the event or editing the copy re-shows it. Suppressed on `/whats-on` and the event's own page via `usePathname` , because the root layout is a server component and cannot know the path.
- `EventSignupButton.tsx` — the event page's primary CTA. Opens the ChurchSuite event **in a modal** (`ChurchSuiteModal` at `size="lg"`) instead of a new tab, so the whole multi-step signup — ticket picker, details form, confirmation — completes without leaving the site. ChurchSuite's own event pages frame fine and submit over AJAX. Third-party ticket URLs (Eventbrite and friends) send `X-Frame-Options: DENY` , so any non-`churchsuite.com` host keeps the old new-tab link. When the event takes signups the embed loads at `#form_event_signup` , so it opens scrolled past the artwork, dates, description and map that the page around the modal already shows. A fragment is the only lever available — the iframe is cross-origin, so its DOM is opaque to our CSS and JS, and none of ChurchSuite's URL variants (`/embed/events/…` , `/events/…/signup` , `?embedded=1`) serve a signup-only view; all return the identical full page. Events with signups closed have no such element and open at the top, which is the right fallback.

`ui/ChurchSuiteModal.tsx` / `ChurchSuiteEmbed.tsx`

The one ChurchSuite-in-a-modal treatment, used by the course pages, every event, giving, the connect card and the You Said Yes form. `size="lg"` gives a `h-[88vh] max-w-3xl` panel whose embed **fills** it (`ChurchSuiteEmbed fill`) rather than taking a fixed height: an event page is arbitrarily long, and a fixed height would mean either dead space or two nested scrollbars. The default `md` size keeps the fixed 620px box the course forms were built against. `embedTitle` names the iframe separately when the panel heading is too terse to stand alone out of context.

This replaces four hand-rolled copies (`AlphaSignupModal` , `ConnectCardCTAs` , `GiveCTA` , `YouSaidYesButton`) that were identical down to the cubic-bezier on the panel transition, and that shared three faults now fixed here once:

- The Escape listener was bound for the life of the component rather than while the modal was open, so a *closed* modal still ran its close handler on every Escape — and that handler cleared `document.body.style.overflow` , unlocking scrolling behind anything else that was holding it.
- The close timeout and the two rAFs were never cancelled, so reopening inside 350ms let the old timer tear the new modal down.
- Closing wrote `overflow = ""` instead of restoring the previous value. Scroll locking is now `lib/useScrollLock.ts` for every modal on the site.

The animated panel is a separate inner component mounted only while the modal is open. That is what makes the entrance work with no "reset on close" bookkeeping: a fresh mount always starts at `visible: false` , and the rAF is the only thing that ever writes it. The exit transition never ran in any of the originals either — the portal is gone the moment `open` goes false.

Dialog semantics (`role="dialog"` , `aria-modal` , `aria-labelledby` , focus in on open, focus back to the opener on close, Tab trap) come from `NfcTileModal` and now apply to every caller.

`PopupShell.tsx`

Shared modal chrome (backdrop, close button, image, title, body, CTA, `z-[100]`), extracted from `SitePopup` so it and `EventPopup` cannot drift visually. The *behaviour* around it stays with each caller — including the open delay: both `SitePopup` and `EventPopup` wait **7 seconds after load** (`setTimeout(..., 7000)`) before showing, so a visitor sees the page before a modal covers it. **An active event popup suppresses the site popup**: `app/layout.tsx` passes `null` to `<SitePopup>` whenever `getActiveEventPopup()` resolves, so only ever one modal shows and there is no client-side coordination to get wrong.

These two are the only self-opening surfaces on the site. `FloatingSmartSearch` shares the screen with them but needs no coordination, because it never opens itself — an earlier auto-opening version did, and carried a `lib/popupGate.ts` zustand flag that both popups checked before arming their timer. That gate was deleted along with the overlay.

Onboarding (`components/admin/onboarding/*`)

Role-based guided tours for `/admin` . See `lib/adminOnboarding.ts` for the registry and the "nothing is saved" guarantee this whole surface exists to keep.

- `OnboardingProvider.tsx` — **Client.** Mounted in `app/admin/layout.tsx` inside `AdminCommandProvider` . Owns the state machine: which section a step belongs to, navigating to a step's route, polling for its `data-tour` anchor (the page it lands on is usually still fetching), and switching demo mode on for the whole run — not just the sandbox steps, because a tour where only some buttons are safe is a tour nobody trusts. Exposes `useOnboardingTour()` (`sections` , `completed` , `running` , `start` , `startRole` , `stop`) to the rest of the tree.

- `WelcomeGate.tsx` — Full-screen modal on a person's first `/admin` visit, naming the access levels they hold and how long their tour takes. "Skip for now" is a real answer and is recorded as one — this is a gate, not a wall.
- `OnboardingChecklist.tsx` — The persistent "Get Started" launcher (bottom-right, remaining-count badge) and its expanding panel — percent bar, tickable sections, "Dismiss checklist". Reuses `<BorderBeam size="pulse-inner">` (`border-beam`) as the attention cue while anything is left to do. The collapsed launcher pill is mounted for the whole `/admin` session (not just while the panel is open), so its beam is explicitly `active={left > 0}` — it stops running once the checklist is actually done, rather than pulsing forever in the background.
- `TourSpotlight.tsx` — The overlay itself: four scrim rectangles frame the anchored element rather than a clip-path, so the real element stays fully clickable — the sandbox steps depend on that. An orange ring lights the target; the tooltip card carries Back/Next/Exit and picks whichever side of the anchor actually has room.
- `RolePreviewBar.tsx` — Sticky bar shown while a super admin is previewing another access level (from `/admin/onboarding`). States plainly that this is presentation only — middleware still knows they're a super admin.

Admin Components (`components/admin/*`)

- `AdminSidebar.tsx` — Admin navigation. A group with its own overview page (Destiny One, HR — `groupLanding()` in `lib/adminNav.ts` , the item named the same as the group) makes the group heading the link to it, with the chevron beside it as the expand/collapse control, instead of repeating the name as the first child. At `md` + the full sidebar; below `md` only a slim top bar (logo, breadcrumbs, ⌘K, theme toggle, account sheet), with navigation itself handed to `AdminTabBar`
- `AdminTabBar.tsx` — The mobile bottom tab bar: a scrolling row of Liquid Glass group tabs derived from `tabsFor()` , with a `Sheet` for multi-page groups (where a group's own landing page is listed as "Overview", highlighted only on that exact page) and a view-transitioned active pill. See "Admin navigation, search and keyboard shortcuts"
- `AdminHeader.tsx` — Sticky desktop header for the admin shell (`md` + only); shows the full breadcrumb trail from `breadcrumbsFor(pathname)` — every level above the current page a link — plus ⌘K, the notification bell (`AdminNotificationBell`), the theme toggle, shortcut help and a "View live site" button
- `AdminNotificationBell.tsx` — The notification bell: an unread badge and a dropdown of the most recent notifications for whatever roles the signed-in admin holds. Data and Realtime delivery both come from `lib/useNotifications.ts` (initial `GET /api/admin/notifications` , then a private Broadcast channel per held role); marking one or all read is optimistic and reconciled on the next refresh. See the `notifications` table in Database Schema for the delivery model.
- `RichTextEditor.tsx` — Shared TipTap rich-text editor (HTML output); used by posts, training posts, HR job descriptions, and (since `a22301b`) shop product descriptions. Optional `blocks` / `onEditor` props admit content blocks — schema and drop handling only; the blocks UI is a separate surface owned by the parent, deliberately **not** part of this toolbar. The toolbar does exactly one job: reformat the current text selection. The old "Embed YouTube video", "Embed ChurchSuite form" and "Embed HTML" buttons are now the Video, ChurchSuite form and Custom embed blocks — they inserted new objects rather than formatting a selection, so they belonged in the sidebar. `enableYouTube` and `enableHtmlEmbed` now only register the legacy `youtube` / `htmlEmbed` nodes so pages authored with those buttons keep parsing; `enableChurchSuite` is gone entirely (it only ever drove a button, since ChurchSuite embeds were stored as `htmlEmbed`).
- `blocks/*` — **Client.** Editor side of the content-block system: the TipTap node factory, node-view chrome, the Blocks sidebar, the schema-driven settings inspector and its field components. See Content Blocks.
- `ChurchSuiteEventFill.tsx` — **Client.** Collapsible "Fill from ChurchSuite event" picker embedded in the "Add Event" form of every course admin page. Fetches the same picker feed as `/admin/featured-event` (`/api/admin/events`) and, on selection, calls `onFill({ startDate, location, signupUrl, name })` to prefill those three form fields — no new table or API route. Deliberately leaves `format` / `frequency` /meeting fields untouched, since those `alpha_events` columns have no ChurchSuite equivalent.
- `CourseAdminPage.tsx` — **Client.** The single implementation behind `/admin/alpha` , `/admin/recovery` , `/admin/bible-course` and `/admin/cap-money` . See below.
- `AdminThemeToggle.tsx` — **Client.** The light/dark/system cycle button. One `useAdminTheme()` call (`lib/adminTheme.ts`); mounted in `AdminHeader.tsx` and the mobile bar in `AdminSidebar.tsx` . Renders nothing while `ADMIN_DARK_MODE_ENABLED` is `false` (dark mode is currently off).
- `AdminCharts.tsx` — **Client.** The admin's data-visualisation kit, deliberately separate from `AdminUI.tsx` (that file is controls/layout; this is the other half). No charting library — the only genuine time series in the admin (the weekly audit reports' stats, and a day-bucketed count of the audit log) is small enough that hand-rolled inline SVG covers it.
- `Sparkline` — a minimal SVG line, no axes or tooltip; the shape is the point, the number sits in text beside it.
- `TrendChip` — "▲12%"/"▼4%", reusing `MetricCard` 's existing up/down/muted tone language.
- `MiniBarRow` — a proportional bar breakdown (by section, by action), longest first.
- `audit/*` — **Client.** The three pieces of `/admin/audit` , split out because the page is two different tools sharing a filter state:
- `AuditAsk.tsx` — the ask box. The answer never appears alone: the entries the model read come back with it and are listed underneath, openable. The AI is a way *into* the record, not a replacement for reading it — when an answer looks wrong, the rows that produced it are right there. The answer panel itself is `.glass.admin-glass` (see "Dark mode" above) with a height/ opacity reveal, rather than the plain grey box it shipped with — the one surface in the admin deliberately given a fixed identity independent of the light/dark toggle.
- `AuditDetail.tsx` — one entry in full. The field-by-field before/after (long values get their own stacked panel rather than a table cell, which turns a paragraph into a one-word-per-line ribbon), the roles the actor held *at the time* rendered as badges, the request itself, and a raw-JSON escape hatch for anything the layout didn't anticipate. Two links out: everything else this person did, and the whole history of this one thing.
- `AuditReports.tsx` — the weekly reports, rendered from the same markdown the email sends, now with `MiniBarRow` s for each report's section/action breakdown and a `Sparkline` of the per-week change count across the whole run of reports — both come from `report.stats` , already fetched and previously discarded. The accordion's expand/collapse is a real height/opacity transition (`motion`) rather than only a rotating chevron.
- `analytics/*` — **Client.** The three tab bodies behind `/admin/analytics` , plus its charts:
- `ShortLinksPanel.tsx` — clicks on `redirects` . Metric row, a daily bar chart, a ranked "top links" table whose rows drill into a single-slug view (also reachable pre-filtered from `/admin/redirects` ' click count via `?target=`), and breakdowns by country/referrer/device/ `src_tag` /Connection

(`ip_category` — Private Relay is labelled and captioned distinctly from VPN/Tor/datacenter, see Database Schema §24b).

- `InPersonPanel.tsx` — `/nfc` taps and `/links` clicks, side by side, each its own metric-row-plus-chart-plus-top-list — the one view of what a service's seat backs and Next Steps cards actually got used for.
- `SitePanel.tsx` — the whole-site Vercel panel. Renders one of three states from `VercelAnalyticsResult`: a setup `EmptyState` (env vars named, a link to Vercel's docs), an `ErrorNote` (with a plan-limit hint when the reason is `"plan"`), or the real charts. Always ends with one line of copy: Vercel's numbers only include visitors who accepted analytics cookies, so they read lower than the click-log tabs, which don't depend on consent.
- `Charts.tsx` — `DayChart` (a bar-per-day SVG) and `BarRows` (proportional divs, not SVG — a ranked list is a table with a visual cue, and a div gets text truncation/wrapping/focus rings a plotted `<text>` element doesn't). No charting library: none exists in this project and the codebase avoids a dependency it can write itself (see `lib/cn.ts`).
- `redirects/RedirectQrModal.tsx` — **Client.** The QR action on `/admin/redirects`. Encodes `destinytees.uk/<slug>?s=qr` (hardcoded, not user-editable — see `lib/engagement.ts`'s closed `SRC_TAGS` set) via `qrcode.react`'s `QRCodeSVG`, with client-side SVG and PNG downloads produced from the rendered `<svg>` (PNG via an in-page `<canvas>`, white background painted in first so it doesn't print as a grey smear).

Course admin pages

The four course pages used to be four copies of the same ~630-line file, differing only in slug, accent colour and wording — `bible-course` and `cap-money` were 627 lines each and differed by 84. Adding CAP meant copying one wholesale, and a fix to the event list had to be applied four times with nothing in the code to say so. They are now four-line route files over `CourseAdminPage`, driven by `COURSE_ADMIN_PAGES` in `lib/courseEvents.ts` (2,620 lines across 4 files → 766 across 5).

```
// app/admin/cap-money/page.tsx — the whole file
import CourseAdminPage from "@/components/admin/CourseAdminPage";

export default function CapMoneyAdminPage() {
  return <CourseAdminPage page="cap-money" />;
}
```

To add a course: write the `alpha_events_type_check` migration, add a `COURSE_EVENT_META` entry (label, href, colour, hint, events label, banner message and link text), add a `COURSE_ADMIN_PAGES` entry (heading, blurb, accent, promote copy, and the types it owns), and create the four-line route file. `tests/unit/course-events.spec.ts` fails if a type has incomplete metadata, is owned by two pages, or is owned by none.

Notes worth knowing before changing it: - **A page can own several types.** `/admin/alpha` manages Alpha and Youth Alpha together, which is why `types` is a list; the create form only shows the Type selector when there is more than one, and each type gets its own banner card and event list. - **Accent colour is applied as inline `rgba()`**, not Tailwind opacity classes, because the accent is per-course data and cannot be a compile-time class name. `hexToRgb` reproduces the `ACCENT_RGB` constants the pages used to hardcode. - **Headings are stored verbatim, not derived** — the article does not follow a rule ("Promote Destiny 12:2", "Promote The Bible Course", "Promote the CAP Money Course"). - **The routes stayed put.** Keeping the four URLs as wrappers rather than moving to a `[type]` route means no redirects, no `ROUTE_RULES` change and no sidebar change. - `tests/admin-courses.spec.ts` covers the rendered pages but is skipped unless `ADMIN_EMAIL` / `ADMIN_PASSWORD` are set.

> Redirects and banner management (`/admin/redirects`, `/admin/banner`) are built inline in their `page.tsx` files rather than as separate reusable components — there is no standalone `RedirectManager.tsx` / `BannerManager.tsx` / `MediaUploader.tsx` / `PageEditor.tsx` / `AdminSermonManager.tsx`. Sermon hiding was removed entirely (migration `20260711_07_remove_sermon_hiding.sql`), and the earlier page-builder/ Studio editor (`PageEditor.tsx`'s likely origin) was removed by `20260711_06_remove_page_builder.sql`.

Content Rendering (`components/content/*`)

- `RichContent.tsx` — **Shared (server-first).** Renders admin-authored rich text and upgrades any embedded content blocks into real React components. Replaces the bare `dangerouslySetInnerHTML` previously used on `/[slug]`, `/jobs/[slug]`, `/training/.../[postSlug]`, `/shop/[slug]` and `/whats-on/[slug]`. Content with no blocks takes the exact previous code path, so adopting it everywhere was a no-op. Also emits merged schema.org JSON-LD contributed by blocks. See Content Blocks.

Content Blocks (`components/blocks/*`)

- **Shared (no `"use client"`).** The blocks themselves — FAQ, callout, quote, card grid, gallery, buttons — plus the registry, wire-format serialisation and design tokens. These server-render with zero client JS on public pages and the same modules render inside the editor. See Content Blocks.

Public Training Components (`components/training/*`)

The member-facing pieces of the `/training` resource library.

- `PasswordGate.tsx` — **Client.** Shown in place of a sub-group's content until the correct password is entered; on success the server sets a signed unlock cookie (via `/api/training/unlock`) and the route refreshes to render the real content server-side.
- `CompleteButton.tsx` — **Client.** "Mark complete" control on a post. When `min_read_seconds > 0` it drives the HMAC read timer (`/api/training/posts/[id]/timer`): it `start`s a token on mount, counts down, and only allows completion once the server `verify`s the minimum read time.
- `CompletablePostList.tsx` — **Client.** Post list with a per-post completion indicator, kept in sync with the hero progress bar via the shared completed set.
- `TrainingProgress.tsx` — **Client.** Completion progress bar in the sub-group hero — shows how many of the group's posts are done.
- `useTrainingProgress.ts` — Per-browser completion store in `localStorage` (no per-user accounts; trainees share a group password). Exposes `useCompletedSet()` and `toggleCompleted()`; starts empty on first render to avoid hydration mismatch, then fills in after mount and stays reactive across tabs via a custom event + the `storage` event.

Admin Content/Training/HR Components (`components/admin/{posts,training,hr}/*`)

- `posts/PostEditor.tsx` — Standalone page editor (uses `RichTextEditor`). Full-screen at **both** breakpoints; only the panel placement differs. Desktop gets the permanent Blocks and Settings sidebars; mobile edits the title in the header, puts the slug and published switch behind a "Page settings" sheet, and gives the rest of the screen to the editor. It was previously a `Modal` on mobile — the whole form inside a scrolling popup, with the editor capped at 420px and scrolling separately inside that, so the page content got about a third of the screen and a newly added block was immediately pushed out of sight. The desktop right sidebar has **Page** and **Block** tabs: Page holds the URL and `PageSettings`, Block the `BlockInspector`; selecting a block switches to Block and deselecting returns to Page (derived from the selection, not synced by an effect). Tracks unsaved changes (closing, Escape and tab close ask first), saves on Ctrl/Cmd+S, and has a **Preview** button that saves then opens a signed preview link. New posts start from a template (`lib/postTemplates.ts`).
- `posts/PageSettings.tsx` — Page-level settings shared by the desktop sidebar and the mobile sheet: header style picker (Plain / Image / Banner thumbnails), header image, subtitle, description with a 160-character counter, share image and the promo-rails toggle. Reuses the block inspector's field components (`ImageField` , `TextField` , …) so uploads go through the same `post-media` route.
- `training/PostEditor.tsx` — Training post editor (uses `RichTextEditor`). Still a `Modal` on mobile: its body is one field among many rather than the whole point of the screen, and it inherits the mobile block sheets and toolbar from `BlockTools` either way.
- `training/CategoryModal.tsx` , `SubgroupModal.tsx` , `FolderModal.tsx` , `IconPicker.tsx` — Training tree CRUD modals
- `hr/HrUI.tsx` — Staff directory, leave requests, documents (main HR dashboard shell)
- `hr/JobModal.tsx` — Create/edit job listing (uses `RichTextEditor`)
- `hr/modals.tsx` — Remaining HR CRUD modals (staff, leave, reviews, applications)

Post Hero (`components/posts/PostHero.tsx`)

Server. `PostHero` renders the page header for the `image` (full-bleed photo, title over a scrim) and `banner` (solid brand-orange band) styles; `PostPlainTitle` is the in-column title for `plain` . An `image` post with no image falls back to plain rather than an empty dark box.

Post Templates (`lib/postTemplates.ts`)

Starter layouts shown when creating a post: Blank, Event, Campaign, Info + FAQ. Each is ordinary body HTML plus page settings. Blocks are serialised from each block's own `defaults` via `encodeProps` , so a template can't drift from a block's schema — `tests/unit/post-templates.spec.ts` parses every template block against the wire format and its schema. The admin list also has a **Duplicate** action (a draft copy at `<slug>-copy`), an Edited column and thumbnails.

Post Promo Rails (`components/posts/*`)

Desktop-only internal advertising in the empty margins of a post page. Added 2026-07-26.

- `PostRails.tsx` — **Server.** Exports `EventsRail` (left) and `CoursesRail` (right). Builds serialisable `PromoCard[]` and hands them to the client rotator. Events come from `fetchChurchSuiteEvents` (filtered to future dates and sorted — unlike `EventsGrid` / `WhatsOnSection` , which show past events too); courses come from `lib/courses.ts` with the admin-featured one first via `getFeaturedCourseId` .
- `PromoRail.tsx` — **Client.** Mounts every card stacked and shows one at a time, advancing every 15s. Pauses on hover/focus and does not auto-rotate under `prefers-reduced-motion` (WCAG 2.2.2). Card = date, artwork, Anton title, clamped description, CTA.
- `lib/promo-palette.ts` — Pure colour maths: 4-bit histogram dominant colour, reduced to an `"H S%"` pair. Near-black/near-white histogram buckets are skipped so cards don't all take the colour of a white studio backdrop.
- `lib/promo-palette.server.ts` — Fetches and decodes remote artwork (JPEG/PNG only), memoised per image URL. Cards with no usable artwork get `DESTINY_PALETTE` (brand orange).
- `scripts/precompute-course-palettes.ts` — Regenerates `COURSE_PALETTES` in `lib/courses.ts` . Run after changing any course `card.image` : `npx tsx scripts/precompute-course-palettes.ts` .

Card treatment. The card is light on purpose: a near-white tint of the sampled hue, a hairline border, dark text, and the sampled colour at full strength only in the CTA pill. One `"H S%"` pair drives all three (`hsl(${accent} 97%)` / `88%` / `30%`), so the whole lightness ramp is tunable in one place. An earlier version filled the card with a dark gradient, which made two 300×600 slabs the heaviest thing on the page and pulled the eye off the article — if you are tempted to add weight back, that is the trade being made.

Why the rotation has no animation. Cross-fading double-exposed two opaque text-bearing cards (both titles legible at 50%), and fading the incoming card up from `opacity: 0` left it stranded invisible when a background tab throttled the transition. The swap is instant and nothing transitions opacity. Unannounced motion in the page margins also competes with the article. The hover lift stays — that one is user-driven. All cards stay mounted so their images load once up front; mounting only the active card meant every rotation inserted a fresh lazy `<img>` that flashed blank. Inactive cards are `aria-hidden` with `tabIndex={-1}` .

> **Why no `sharp` here.** The first version used `sharp().stats().dominant` . That traced ~20MB of libvips into the `/[slug]` function and pushed it to 256MB, over Vercel's 250MB uncompressed limit — the build passed and the *deploy* failed. `jpeg-js` + `pngjs` are ~800KB combined and produce identical output. Course artwork is WebP, which neither decodes, so those four palettes are precomputed into `lib/courses.ts` instead — they're static images, so a literal is cheaper and safer than any runtime decode. **Do not import `sharp` into anything reachable from a page component.** It is fine in the admin upload routes, which are their own functions.

Layout lives in `app/[slug]/page.tsx` : below 1600px the page is byte-identical to before; at 1600px+ it becomes `grid-cols-[300px_minmax(0,768px)_300px]` with `gap-16` . The maths is `300 + 64 + 768 + 64 + 300 = 1496` of content, `+64` of `lg:px-8` = the `max-w-[1560px]` cap, which is why the breakpoint sits at 1600. **Every pixel of gutter costs two**, so changing the gap means recomputing the cap and the breakpoint together. The article is first in the DOM and placed with `col-start-2` , so promo content never precedes it for crawlers or screen readers. **Do not add `self-start` / `h-fit` to the rail** `<aside>` — the grid's default stretch is what gives the sticky card its scroll range; hugging the content silently disables sticky.

Links Pages (`components/links/*`)

The public renderer for `/links` and `/links/<slug>`, also used — unchanged — as the editor's live preview.

- `LinkPageView.tsx` — **no** `"use client"`: a shared component, like the content blocks. It server-renders the page and the editor renders the very same component from unsaved state, so the preview is exactly the page. The theme arrives as `--lp-*` custom properties on the wrapper (`themeToCssVars`) plus `data-*` attributes for the discrete choices; `links.css` is the only place those turn into CSS. Groups consecutive plain link buttons so a "grid" theme can set them two-up.
- `links.css` — the whole stylesheet. Width rules are **container queries** on `.lp-main`, not media queries: in the editor the page sits in a 375px phone frame on a desktop screen and has to lay out like the phone. All motion is off under `prefers-reduced-motion`, background video included.
- `fonts.ts` — the extra theme typefaces (Inter, Space Grotesk, DM Serif Display, Caveat) via `next/font`, loaded only where a links page renders. Anton is deliberately not offered.
- Client islands, one per interactive block: `LinkButton.tsx` (click beacon, spotlight animation, ChurchSuite links in `ChurchSuiteModal`), `LinkEvents.tsx` (event cards; signups open in the modal, the rest link to `/whats-on/<slug>`), `LinkEmbed.tsx` (YouTube/Vimeo/Spotify/Apple Podcasts/Maps behind the media-cookie consent gate, ChurchSuite via `ChurchSuiteEmbed` ; inline or in a `ui/Modal`), `LinkForm.tsx` (collapsed by default, honeypot field, posts to `/api/links/submit`). All take `preview` and swallow clicks/submits in the editor.
- `SocialIcons.tsx` — brand marks from `simple-icons` (CC0; Material Symbols has no brand glyphs).
- `LinkPageRoute.tsx` — server-only glue for both routes: loads the page once per request (`React.cache`), builds metadata, and records the page view (`links_view`) via `after()` .

Links Page Editor (`components/admin/links/*`)

- `LinksEditor.tsx` — holds page + blocks in local state until Save, which PUTs everything in one request. Sends the `updated_at` it loaded; a 409 means someone else saved first. Wraps its fields in `ImageUploaderContext` so `ImageField` uploads to `/api/admin/links/upload` (event_admin can't reach the posts upload route).
- `BlockList.tsx` — the block stack. Reordering is **@dnd-kit**, not the native-drag `useListReorder` hook, because HTML5 drag-and-drop doesn't fire on touch screens. Keyboard sensor included (Space + arrows on the handle).
- `BlockFields.tsx` — per-type settings, built from `components/admin/blocks/fields/*` . The event block uses the shared `components/admin/EventPicker.tsx` (lifted out of `/admin/nfc`).
- `AppearanceTab.tsx` + `ColorField.tsx` — presets, then every theme value, grouped into collapsible sections (Theme, Background, Colours, Buttons, Header, Fonts, Social icons, Layout and motion) that show their current value while closed. Theme options beyond the basics: background pattern (dots/grid/lines + strength), button size, spacing, border width, custom corner radius, icons and arrows on/off, header alignment, heading size and uppercase, photo size and accent ring, social icon style and size, page width, and the site footer on/off. All default to the original look, so older saved themes render unchanged. Updates are functional (`setTheme(prev => …)`) so rapid edits don't overwrite each other. `ColorField` wraps `react-best-gradient-color-picker` (loaded on first open) and only commits values `ThemeSchema` would accept.
- `controls.tsx` — the editor's shared `Section` (collapsible, with a summary line), `Segmented` (radio-group picker for short choices) and `Slider` .
- Per-block overrides: a link can set its own button colour, text colour, text alignment and hide its icon ("Customise this button"); a heading can set its own colour. They ride on the same `--lp-*` custom properties as the theme, so every button style honours them.
- `ProfileTab.tsx` , `SettingsTab.tsx` (slug, publish, noindex, SEO, share links, QR PNG/SVG via `qrcode.react`), `SubmissionsTab.tsx` (responses, CSV, per-response delete), `AnalyticsTab.tsx` (reuses `DayChart` / `BarRows` from `components/admin/analytics/Charts.tsx`).
- `editorTypes.ts` — `previewBlocks()` turns unsaved blocks into what the live page would show now (inactive, out-of-schedule and invalid blocks dropped; events resolved from the picker feed). Share and QR links use `window.location.origin` , **not** destinytees.uk, which still points at the old site.

NFC Page (`components/nfc/*`)

The tiles on `/nfc` — the "digital back of seats" page an NFC tag or QR code on a seat opens.

- `NfcTileGrid.tsx` — the card grid. Cards are `<button>` s, not links: everything opens in place, because the page exists to be finished before the next song starts. Holds the open-tile state and the ref to the invoking card so focus can be restored on close. The `BADGE` lookup is what the card promises before you tap it — "Form", "Sign up", "Details". Every tap also fires `trackClick("nfc", tile.id, tile.title)` (`lib/track.ts`) before opening the modal — a fire-and-forget beacon to `POST /api/track` , so taps show up on `/admin/analytics` .
- `NfcTileModal.tsx` — the popup. Three modes, two layouts: `embed` frames a ChurchSuite form (`ChurchSuiteEmbed`) with the "more details" link hung off the bottom, so the popup answers the question *and* the full page stays one tap away; `event` is the same layout with no branch of its own, because `getNfcTiles()` resolves an event tile's signup URL into `embedUrl` server-side; `info` shows artwork + copy + an orange CTA, the `PopupShell` layout. This is where the dialog semantics were first written — `role="dialog"` , `aria-labelledby` , focus in on open, focus restored on close, and a Tab trap — because `/nfc` is the one page used cold by people who have never been to the site. The four hand-rolled modals that had none of them are now a single `ui/ChurchSuiteModal.tsx` , which carries the same treatment. Its entrance is a CSS keyframe (`.nfc-modal-*` in `globals.css`) rather than the visible-state-plus-double-rAF the others use: closing unmounts immediately, so the entrance was the only transition that ever ran.

Chrome suppression. `/nfc` is deliberately standalone. Four components carry the path check: `ChurchHeader.tsx` , `FooterGate.tsx` , `FloatingSmartSearch.tsx` , and both popups (`SitePopup.tsx` , `events/EventPopup.tsx`). The popups matter most — an announcement auto-opening on top of a tile popup, mid-service, is the one failure this page can't afford. The cookie banner and site banners stay: they're consent and legal, not chrome.

Connect Card (`components/connect-card/*`)

- `ConnectCardCTAs.tsx` — Call-to-action buttons
- The prayer/connection form itself is built inline in `app/connect-card/page.tsx` , not a separate component

Report a Bug (`components/report-bug/*`)

- `ReportBugLink.tsx` — a "Report a Bug" link rendered in `ChurchFooter.tsx`. Opens an accessible in-app modal (Escape-to-close, body-scroll lock) with a small form: **Full Name**, **Email**, and **How to reproduce**. On submit it captures the current `window.location.href` as `pageUrl` and calls the `submitBugReport` server action, showing inline loading / success / error states.
- `actions.ts` — `submitBugReport(formData)` server action. Validates name/email/steps, then uses `GITHUB_TOKEN` to open a GitHub Issue on `SquareMediaGroup/destinychurch` via the GitHub REST API (`POST /repos/.../issues`), titled `Bug Report: <name>` with the reporter, page URL, and reproduction steps in the body. Returns `{ success, error? }` ; a missing `GITHUB_TOKEN` yields a friendly server-misconfiguration error. Because it's public and spends our GitHub token, it's fenced with a hidden `website` honeypot (answered like a success), a 3-per-minute per-IP rate limit, and field length caps. **Requires the `GITHUB_TOKEN` env var** (see Configuration).

About Page (`components/about/*`)

- `AboutHero.tsx` — the About page hero band.
- `AboutMissionStatement.tsx` — the `#mission` mission-statement section (white background).
- `MagnifySection.tsx` — the church's guiding pillars, rendered from a local `pillars` list.
- `BeliefsSection.tsx` — a summary of core beliefs, linking through to `/beliefs` .
- `MeetPastorsSection.tsx` — the lead pastors introduction on a dark gradient panel.
- `TeamSection.tsx` — the staff/leadership grid, built from a local team list (name, role, photo, email); cards are off-white by default and pick up colour on hover.
- `AboutGovernanceNote.tsx` — a short governance/transparency note pointing to `/governance` .

Visit (`components/visit/*`)

- `VisitSlideshow.tsx` — a client, auto-advancing photo slideshow of church life used on the Plan-a-Visit page.

Alpha (`components/alpha/*`)

- `AlphaTopics.tsx` — a client, expandable list of the big questions Alpha explores (each an accordion of question + description), with `AnimateIn` reveals.

Home Page (`components/home/*`)

- `HomePageBody.tsx` — the whole homepage composition, extracted as a server component so the real page (`app/page.tsx`) and the variant-preview route (`app/home/page.tsx`) render identical markup with a different event-card treatment (`cardVariant`) rather than duplicating the layout. It resolves the latest sermon once — `getLatestVideo()` , falling back to `getLatestVideoFromRSS()` when the YouTube quota is exceeded — and renders, in order: `HomeOverscrollColor` , `HeroSection` , `MissionSection` , `LatestSermonSection` , `WhatsOnSection` , `EveryoneHasAPlaceSection` , `WorshipWithUsSection` , `GetInvolvedSection` .
- `HomeOverscrollColor.tsx` — a render-nothing client component that sets `document.documentElement.style.background` to Destiny orange (`#F58021`) on mount and restores it on unmount, so the overscroll/rubber-band area at the top of the homepage matches the hero rather than flashing white.
- `HeroSection.tsx` — main hero banner with video/image.
- `MissionSection.tsx` — centered "Our Mission" statement with the lead pastors' avatar, on a white background.
- `LatestSermonSection.tsx` — dark image banner (rounded-3xl card, blurred backdrop, left-to-right black gradient, orange + ghost buttons) showing the most recent sermon. Title and speaker come pre-parsed from `video` (see `lib/sermonTitle.ts`); the Watch button links to `/sermons/[id]` , or straight to YouTube when the quota is exceeded. Renders nothing when there is no video.
- `WhatsOnSection.tsx` — the What's On block, all inside the `max-w-7xl` container: header + "View Church Calendar", the event carousel (max 6 events, featured one pinned first), and the "View All" button. Falls back to three standing weekly gatherings when the ChurchSuite feed is empty or down.
- `EventsCarousel.tsx` — the horizontal scroll track with prev/next arrows (client). Its `pt-3 pb-10` is load-bearing: `overflow-x` clips the cross axis too, so without it the cards' hover lift and drop shadows are sliced off. The arrows use `top-3 bottom-10 my-auto` rather than `top-1/2` so that asymmetric padding doesn't push them below the card midline, and the track's negative margin tracks the container padding at `lg` (`-mx-8` , not a fixed `-mx-4`).
- `EveryoneHasAPlaceSection.tsx` — a grid of ministry entry points (Kids, Youth, Young Adults, Connect Groups) with photo, blurb, and link, each revealed with `AnimateIn` .
- `WorshipWithUsSection.tsx` — full-bleed "Worship With Us" banner (rounded-3xl, lazy-loaded blurred background photo, left-to-right black gradient) inviting visitors to a service.
- `GetInvolvedSection.tsx` — two-up cards ("Get Connected" → `/connect` , "Join a Team" → `/serve`) on a white background, each with photo, blurb, and CTA.

Shop (`components/shop/*`)

- `ShopProductGrid.tsx` — client wrapper around the `/shop` grid; derives a category filter chip row from the distinct `product.category` values present ("All" + one chip per category, alphabetical), filters the grid client-side on click, and shows an empty state ("Nothing here yet — Try a different category.") when a filter matches nothing. Renders no chips at all if every product shares one/no category.
- `ProductCard.tsx` — storefront product tile (editorial `.shop-card` hover wipe)
- `ProductGallery.tsx` — main image + thumbnail strip (client)
- `ProductBuyPanel.tsx` — colour swatches + size pills + quantity + add-to-basket (client)
- `CartButton.tsx` — header basket icon with live item-count badge (client)
- `CheckoutForm.tsx` — Stripe Express Checkout Element (Apple Pay / Google Pay / Link, shown only when a wallet is available) above the Payment Element card form; both confirm the same PaymentIntent (client)

- `ShopDiagnostics.tsx` — **TEMPORARY.** Mounted for every `/shop` route by `app/shop/layout.tsx` to chase an intermittent blank/white page in Chrome. Renders nothing. See "Shop blank-page diagnostic" below; delete this component, the layout and `/api/shop-diagnostics` once the cause is confirmed.

Shop blank-page diagnostic (temporary)

The bug: `/shop` and its sub-routes intermittently paint blank/white in Chrome. Ruled out by investigation — server errors (12/12 identical 200s), JS heap growth (flat ~15MB), image weight (largest source image 110KB), console errors (none) and a renderer crash (Chrome's Crashpad directory empty). Not reproducible on demand, hence instrumentation rather than a speculative fix.

Two candidate failure modes need different evidence, which is what the payload is shaped around:

Mode	What the report looks like
React/JS died	`dom.mainChildren === 0`, plus a `js-error` / `dom-blank` reason
Compositing failed	DOM completely intact, **no** error — the GPU simply never painted

The second mode is invisible to any automatic check, which is why a **manual hotkey (Ctrl/Cmd + Shift + 9)** exists: a `manual` report showing a healthy DOM and zero errors is itself the finding, because it eliminates the first mode.

Automatic tripwires: `window.onerror`, `unhandledrejection`, a 1×1 WebGL context listening for `webglcontextlost` (the only JS-visible signal of a GPU process reset), a `PerformanceObserver` for long tasks, and a 3s poll for an emptied `<main>`. Observation is deliberately passive — `PerformanceObserver` rather than a `requestAnimationFrame` loop — so the instrument does not perturb the compositing behaviour it is measuring.

`render.refractActive` is the key field: it records whether the SVG refraction `backdrop-filter: url(#glass-refract)` is live at that moment. It differs by route — the `/shop` index disables it via `html:has(.shop-page)` in `globals.css`, the sub-routes do not — so it discriminates the prime suspect automatically.

Per-route state: the report budget, event buffer and timers all reset on `pathname` change. The component lives in the shop layout and survives client-side navigation, so without that reset the budget would be spent once per browsing session and the instrument would fall silent exactly when the bug next appeared.

> **Apple Pay:** enabled automatically by `automatic_payment_methods` — no extra API params. The only prerequisite is a **registered payment-method domain**; Stripe hosts the Apple domain-association file, so there is no `/.well-known` file to deploy. Register the live domain(s) once via Stripe Dashboard → Settings → Payment methods → Apple Pay, or run `scripts/register-apple-pay-domain.mjs` with the live `STRIPE_SECRET_KEY` (e.g. `destinytees.uk` and `www.destinytees.uk`). Apple Pay renders on supported devices/browsers only (Safari on Apple hardware).

API Routes & Server Actions

Server Actions vs. API Routes

Server Actions (`app/*/actions.ts`): - Call from client components directly - Typed, type-safe - Auto-serialize arguments & return values - Good for: Form submission, data mutation - Example: `app/contact/actions.ts`

API Routes (`app/api/*/route.ts`): - HTTP endpoints - Called from fetch() or external integrations - Good for: Webhooks, public APIs, external tools - Example: `/api/chat` (Smart Search)

Key Server Actions

`app/contact/actions.ts`

```
"use server";

export async function submitContactForm(data: ContactFormData) {
  // Validate input
  const parsed = contactFormSchema.parse(data);

  // Rate limit: max 5 submissions per IP per hour
  const remaining = await rateLimit(request.ip, "contact-form", 5, 3600);
  if (remaining <= 0) throw new Error("Too many submissions. Try again later.");

  // Insert into database
  const supabase = createServiceClient();
  await supabase.from("contact_messages").insert({
    name: parsed.name,
    email: parsed.email,
    subject: parsed.subject,
    message: parsed.message
  });

  // Send notification email to admin
  await sendEmail({
    to: "hello@destinytees.uk",
    subject: `New contact form: ${parsed.subject}`,
    html: renderEmailTemplate("contactNotification", parsed)
  });

  return { success: true };
}
```

`app/hire/actions.ts`

Similar structure to contact form: - Validate form data - Rate limit - Insert to `hire_enquiries` table - Send email notification

`app/contact/actions.ts`, `app/hire/actions.ts` and `app/jobs/actions.ts` all share `lib/formEmail.ts` (`escapeHtml`, `escapeHtmlMultiline`, `isValidEmail`) for the HTML-escaping and email-format validation their notification emails need — add new form actions against that helper rather than redefining it, so a fix doesn't have to be repeated in three places.

`app/connect-card/actions.ts`

- Submit prayer request
- Submit connection card (first-time visitor form)
- Inserts to `contact_messages` with category prefix
- Sends email to prayer team

`app/jobs/actions.ts`

```
export async function applyForJob(jobId: string, formData: ApplicationData) {
  // Validate
  const parsed = applicationSchema.parse(formData);

  // Upload CV to storage
  const file = parsed.cv;
  const { path } = await uploadToStorage(file, `job-cvs/${jobId}`);

  // Insert application
  await supabase.from("job_applications").insert({
    job_id: jobId,
    name: parsed.name,
    email: parsed.email,
    phone: parsed.phone,
    cv_url: path,
    cover_letter: parsed.coverLetter
  });

  // Send confirmation email to applicant
  await sendEmail({
    to: parsed.email,
    subject: "We've received your application",
    html: renderEmailTemplate("applicationReceived", { jobTitle })
  });

  return { success: true };
}
```

`app/login/actions.ts`

```
// signInCore(formData)  — internal: IP → checkRateLimit
//                         → signInWithPassword → sb-remember cookie
// adminSignIn(prev, fd) — signInCore, then redirect(resolvePostLoginPath(...))
//                         → /admin for an admin, /portal for a linked staff
//                         member, else an "account not set up" error
// adminSignOut()        — signOut + delete sb-remember
```

`adminSignIn` sends admins to `/admin` and non-admin staff (a linked `hr_staff` row) to `/portal`; admin roles take priority, so someone who is both lands on `/admin` and can still reach `/portal` by URL.

Admin API Routes

`POST /api/admin/sermons/upload`

```
// FormData: title, speaker, notes, youtubeVideoId (all optional except title
// and the audio file itself) → createBuzzsproutEpisode() (lib/buzzsprout.server.ts)
// → 201 with the created episode.
//
// AUTHORIZATION: sermon_admin, via ROUTE_RULES.
//
// Publishes straight to Buzzsprout — no Supabase Storage write, nothing
// persisted in this app's own database (the "no DB" sermons architecture is
// unchanged). Audio MIME allowlist (mpeg/mp3/wav/x-m4a/mp4), 300MB cap.
// A youtubeVideoId, when given, is embedded on the Buzzsprout episode as an
// HTML-comment pairing hint (lib/podcast.ts's extractYouTubeIdHint reads it
// back out of the public RSS feed) — see the /sermons write-up above.
```

`POST /api/admin/sermons/review-speakers`

```
// Body: { force?: boolean } → reviewSermonSpeakers() (lib/speakerReview.server.ts)
// → { reviewed, changed[], skipped, errors } and an audit-log entry.
//
// AUTHORIZATION: sermon_admin, via ROUTE_RULES.
//
// Runs an AI pass over the sermon archive's speaker attribution, writing
// corrections to the speaker_overrides table. By default skips videos already
// reviewed (an override row exists) so a re-run only costs new sermons; force
// re-reviews everything. Node runtime, maxDuration = 300 — a full first pass is
// a few dozen batched OpenAI calls. Driven from the Speaker Review panel on
// /admin/sermons (components/admin/SpeakerReviewPanel.tsx).
```

`POST` / `DELETE /api/admin/sermons/speaker`

```
// POST   Body: { video_id, speaker } → upserts speaker_overrides
//        (reviewed_by: "admin"). speaker: "" is stored as null — a
//        *confirmed* "no individual speaker", same rule the AI path uses.
// DELETE Body: { video_id } → deletes the override row outright (the
//        "Revert to YouTube" action — falls back to lib/sermonTitle.ts's
//        regex-parsed speaker, distinct from POSTing an empty speaker).
//
// AUTHORIZATION: sermon_admin, via ROUTE_RULES.
//
// The manual, hand-typed counterpart to review-speakers above — both write
// the same speaker_overrides row. Driven from the manual speaker editor on
// /admin/sermons (components/admin/SpeakerEditor.tsx). recordAudit() on
// both verbs.
```

`GET` / `POST /api/admin/sermons/series`, `DELETE /api/admin/sermons/series/[id]`

```
// GET    → sermon_series rows merged with a live YouTube snippet per row.
// POST   Body: { input } → a pasted playlist URL or bare id. Extracts the
//        id from a URL's `list=` param, validates it resolves via
//        getPlaylistSnippet() (lib/youtube.ts), then inserts into
//        sermon_series. 409 on a duplicate playlist_id.
// DELETE (on /[id], the playlist id) → removes the row only — the YouTube
//        playlist itself is never touched.
//
// AUTHORIZATION: sermon_admin, via ROUTE_RULES.
//
// Manages the playlist ids that back the public Series filter on /sermons
// (see Database Schema §27, lib/sermonSeries.server.ts). Driven from
// components/admin/SeriesManager.tsx. recordAudit() on POST and DELETE.
```

Design tickets — `/api/admin/design/tickets/**`

```
// GET    /                          → the whole queue + a deliverable_count per row
// POST   /                          → a designer filing a ticket on someone's behalf
// GET    /[id]                      → one ticket with its thread and its files
// PATCH  /[id]                      → editable fields. NOT status — see below
// DELETE /[id]                      → the ticket, and any stored deliverable files
// POST   /[id]/status               → every designer-side move
// POST   /[id]/notes                → a thread note, internal or visible
// POST   /[id]/deliverables/upload   → image/PDF straight into Supabase Storage (<=50MB)
// POST   /[id]/deliverables/link     → record a Drive/Playbook video share link
// DELETE /[id]/deliverables/[did]    → the row and its backing file (if any)
// GET    /[id]/deliverables/[did]/download → 302 to a fresh signed URL (or the link)
//
// AUTHORIZATION: design_admin, via ROUTE_RULES. Handlers don't re-check —
// middleware already did, and every one uses the service client.
//
// STATUS IS NOT IN THE PATCH ALLOW-LIST. It moves only through
// applyTransition() in lib/designTickets.server.ts, called from /status here
// and from the requester's PATCH in /api/design-request/[token]. That is the
// single writer, so the workflow can't be sidestepped by a field update.
//
// applyTransition also enforces what the UI can only suggest:
//   - in_progress → delivered needs >=1 deliverable AT THE CURRENT REVISION.
//     A disabled button is a courtesy; "delivered" with nothing to download is
//     the one state that makes the requester's email a lie.
//   - the UPDATE is conditioned on the status we read (.eq("status", from)), so
//     two designers with the queue open in two tabs is a clean 409 for the
//     second, not a silent overwrite.
//
// UPLOADS split by kind (Playbook's upload endpoint sends no CORS headers, which
// silently blocked every direct browser upload, so the old prepare/complete +
// direct-PUT dance was retired):
//   - image/PDF (<=50MB): /deliverables/upload takes the bytes in ONE request and
//     writes them into the private design-deliverables Supabase Storage bucket.
//     The 50MB cap is why a single request is fine here.
//   - video (any size): NOT uploaded. /deliverables/link records a Drive or
//     Playbook share URL the designer pasted; the bytes never touch our server.
//   - legacy Playbook assets: still readable, but nothing writes new ones.
//
// DOWNLOADS mint a fresh URL per click and 302 to it, no-store — never persisted
// or emailed, because it expires (a Supabase signed URL in ~60s, a Playbook
// display URL in ~24h), so a cached one is a button that works until it silently
// doesn't. A 'supabase' row gets a signed URL from the bucket, a 'playbook' row
// getTemporaryDisplayUrl(), and a 'link' row redirects straight to the pasted URL.
// No Playbook permalink is ever requested — see lib/playbook.server.ts.
```

`GET|PATCH /api/admin/onboarding`

```
// The signed-in admin's own onboarding progress.
// GET    → { completed_steps: string[], gate_seen: boolean, dismissed: boolean }
// PATCH  → the same, after merging the body in
//
// AUTHORIZATION: in OPEN_PATHS (lib/adminRoles.ts) because every admin needs
// their own progress regardless of role. The row is keyed off auth_user_id
// taken from the caller's cookie session, never from the body, so there is no
// path here to read or write anybody else's.
//
// PATCH merges rather than replaces: completed_steps unions with what's
// already stored, so two tabs finishing different sections can't clobber each
// other. Pass reset: true to clear it (the "run it again" button).
```

HR checklist routes (`/api/admin/hr/checklist-templates/*` , `/api/admin/hr/checklists/*`)

```
// HR Admin only (hr_admin/super_admin, via /api/admin/hr/** route rules).
// Templates:
GET|POST         /api/admin/hr/checklist-templates       // list / create a reusable template
PATCH|DELETE     /api/admin/hr/checklist-templates/[id]  // rename / edit items / delete
// A staff member's live checklist (rows copied from a template on start):
GET|POST         /api/admin/hr/checklists                // ?staffId= — list / start from a template
PATCH|DELETE     /api/admin/hr/checklists/[id]           // tick an item / edit / remove
```

Audit log routes (`/api/admin/audit/*`)

```
GET  /api/admin/audit          // the log itself
// Query: ?q= &actor= &section= &action= &entity= &entity_id= &range= &before=
//   range: today | week | month | quarter | all   (default month)
// Returns: { entries, hasMore, nextBefore, facets }
//
// Paginated with a keyset (before=<id>), not an offset: the log only grows and
// is always read newest-first, so an offset would re-scan what it had already
// returned and could skip a row that landed mid-scroll.
//
// `facets` (first page only) counts actors/sections/actions over the range and
// search but NOT the chip filters — a chip has to keep showing what picking it
// would give you, the same way lib/useAdminList.ts counts.

POST /api/admin/audit/ask      // { question } → { answer, entries, degraded? }
// The plain-English half. Tool-calling over the log (lib/auditAI.ts):
// search_audit_log and count_audit_log run real queries and hand back bounded
// slices, so the model never sees the whole log and every claim is grounded in
// rows the reader can then open. Not streamed, unlike /api/chat: the answer and
// the entries behind it arrive together so the claim and its evidence can't be
// read apart. With no OPENAI_API_KEY (or on an API failure) it falls back to a
// plain keyword search, labelled as one, rather than refusing to answer.

GET  /api/admin/audit/reports  // the stored weekly reports, newest first
```

Analytics routes (`/api/admin/analytics/*`)

```
// Site Admin or Super Admin (ROUTE_RULES in lib/adminRoles.ts — landed
// together with the /admin/analytics nav entry, since the nav test asserts
// the two never drift apart). GET only, so tests/unit/audit-coverage.spec.ts
// has nothing to say about it — reads aren't audited in this codebase.

GET  /api/admin/analytics       // engagement_events, via one RPC round trip
// Query: ?range= (today|week|month|quarter|all, default month) &source=
// (redirect|nfc|links) &target=<slug|id|href> &bots=1 (include, default off)
// Calls engagement_rollup(); the whole response IS that function's jsonb
// return value: { totals, timeseries, bySource, byTarget, byCountry,
// byReferrer, byDevice, byBrowser, bySrcTag }. 503 with a "run the migration"
// message if the RPC doesn't exist yet (PostgREST PGRST202).

GET  /api/admin/analytics/site  // the "Whole site" tab's data
// Query: ?range= (same set as above; "all" falls back to a 24-month window,
// the longest any Vercel plan offers, and lets the plan itself decide via
// the "plan" error path). Wraps lib/vercelAnalytics.server.ts#fetchSitePanel.
// A SEPARATE route from the one above on purpose: a slow or failing call to
// Vercel's API must never hold up the click-log numbers on the other tabs.
```

`/api/song-requests` — public song requests

```
// GET  /api/song-requests[?device=] → { open, eventName, playing, requests[], cooldown }  (polled every 10s; queue is Spotify's own, cooldown
// GET  /api/song-requests/search?q= → { tracks[] }  explicit tracks already removed; empty while closed
// POST /api/song-requests          { trackId, deviceId, website } → { ok } | { error }
//      honeypot `website`; per-IP rate limit; 403 closed, 422 explicit, 409 already queued,
//      429 cooldown (one request per device per 5 min, returns `cooldown` seconds) / queue full / device limit, 503 Spotify unavailable or no
```

`/api/admin/song-requests/*` — song requests (Event Admin)

```
// GET    /api/admin/song-requests          → { configured, connection, device, settings, requests }
// PUT    /api/admin/song-requests          { open?, eventName?, maxQueue?, perDeviceLimit? } (audited)
// DELETE /api/admin/song-requests/[id]     hide from the public list (Spotify cannot dequeue) (audited)
// GET    /api/admin/song-requests/spotify/connect   → redirect to Spotify (state cookie)
// GET    /api/admin/song-requests/spotify/callback  → stores refresh token, back to the admin page
```

`POST /api/links/submit` — public form-block submissions

```
// Body: { blockId, values: { [fieldId]: string | boolean }, website }.
// `website` is a honeypot: filled in → quiet { ok: true }, nothing stored.
// The block must be an active form block on a published page; values are
// checked against that block's own field list (required, email format,
// length), and anything not in the list is dropped. Stores [{ id, label, value }]
// in field order in link_form_submissions, then — via after(), never failing the
// request — emails notifyEmail if the block has one (lib/emailCard.ts). The
// email's link uses SITE_ORIGIN (lib/appApi.ts), not destinytees.uk.
// Rate-limited per IP at the site-wide default.
```

`POST /api/track` — public beacon for `/nfc` and `/links`

```
// Unauthenticated, reachable by anyone — the pages it serves are public. Not
// under /api/admin, so middleware.ts doesn't guard it and it's outside
// audit-coverage.spec.ts's walk (it records visitors, not admins).
// Body (text/plain, matching sendBeacon's constraints — application/json is
// not CORS-safelisted for beacons): { source: "nfc"|"links", targetKey }.
// "redirect" is never accepted here — see lib/track.ts's BeaconSource.
// targetKey is checked against the real thing it claims to be (a live
// nfc_tiles id/PINNED_TILES fixture, an active link_blocks id on a published
// links page, or — for /links' database-down fallback — an href in
// lib/linksSteps.ts) before
// anything is written; the label always comes from that lookup, never the
// body. Rate-limited via lib/rateLimit.ts's checkRateLimit(ip, 600) — a much
// higher ceiling than the site-wide default of 15/min, because this endpoint
// is hit by a whole room on a church WiFi's single NAT IP, not one person.
// Always answers 204, whatever happened — a beacon has nobody to tell.
```

AUTHORIZATION: Super Admin only, by omission. `/api/admin/audit` has no `ROUTE_RULES` entry, and anything under `/api/admin` that isn't listed there is Super Admin only (fail closed). That is deliberate rather than an oversight: the log spans every section, so a rule granting any other role would let them read HR's activity, and a rule narrow enough to prevent that would be a second, drifting copy of the RBAC table.

`/api/admin/links/*` — links pages (Event Admin)

```
// GET    /api/admin/links                → { pages } with 30-day views30/clicks30
// POST   /api/admin/links                { slug, title, preset?, duplicateFrom? } → new draft page
// DELETE /api/admin/links?id=            (not the main page; blocks + responses cascade)
// GET    /api/admin/links/[id]           → { page, blocks (raw, so broken ones can be fixed), updatedAt }
// PUT    /api/admin/links/[id]           { page, blocks, updatedAt } → validateSave()
//        (lib/linkPages/save.server.ts: zod per block, pinned events re-resolved
//        against the live feed, main page can't move or unpublish) → link_page_save()
//        409 if updated_at moved since the editor loaded (someone else saved).
// GET    /api/admin/links/[id]/submissions[?format=csv]   (CSV cells formula-escaped)
// DELETE /api/admin/links/[id]/submissions?submission=    (audited without the person's details)
// GET    /api/admin/links/[id]/analytics?range=week|month|quarter
//        views (links_view) + clicks (links) for the page's blocks, bots excluded
// POST   /api/admin/links/upload         sharp → WebP ≤2000px into post-media, `links-` prefix
```

`GET /api/admin/search`

```
// Cross-section record search — the records half of the ⌘K palette.
// Query: ?q=<at least 2 characters>
// Returns: { hits: AdminSearchHit[] }  — up to 6 per section
//
// Sections searched: posts, redirects, products, orders, training categories,
// training posts, course dates, admin users.
//
// AUTHORIZATION: this path is in OPEN_PATHS (lib/adminRoles.ts) because every
// admin needs the palette — but it is NOT unguarded. The route re-reads the
// caller's roles from the service client and only queries the sections that
// role can open, so results can never include a section the caller would be
// bounced out of. Each section runs in parallel and a failing one is dropped
// rather than blanking the whole palette.
//
// HR is deliberately not searched. See "Admin navigation, search and keyboard
// shortcuts".
```

`GET /api/admin/me`

```
// The signed-in admin's own email, id and role flags, in one request.
// Returns: { email, id, roles }
//
// Shared by the sidebar, header and palette through lib/useAdminSession.ts,
// which caches the promise at module scope so mounting three consumers is
// still one request. /api/admin/me/roles remains for existing callers.
//
// Also in OPEN_PATHS: it only ever returns the caller's own details.
```

`GET|POST /api/admin/redirects`

```
// GET  — select("*") ordered by created_at desc.
// POST — body is { slug, target_url, label } (snake_case, matching the
// `redirects` columns — not { targetUrl, active } as previously documented
// here). slug is trimmed/lowercased; target_url is trimmed. No rebuild is
// triggered — redirects resolve at request time in app/[slug]/page.tsx, not
// via next.config.ts. Audited via recordAudit() on create.
// /api/admin/redirects/[id] (not shown) handles PATCH (toggle active,
// partial update) and DELETE, each audited the same way.
```

`GET /api/admin/events`

```
// Event picker feed for /admin/featured-event.
// Projects the ChurchSuite index to { slug, identifier, sequence, id, name,
// start, end, endsAt, image, category, location, sessionCount }.
// force-dynamic — admin must never see a stale list.
```

`GET|PUT /api/admin/featured-event`

```
// Read/write the featured_event singleton: target + hero overrides.
// PUT deletes the superseded image from `popup-images`, then
// revalidatePath("/whats-on") and revalidatePath("/").
// Deliberately never writes popup_* — see /api/admin/featured-event/popup.
```

`PUT /api/admin/featured-event/popup`

```
// Partial update of the popup_* columns only. A full-row upsert from the
// event-popup admin page would blank every hero override.
// Rejects popup_active when no event is featured, with a readable message.
// No revalidatePath — app/layout.tsx reads the popup with noStore().
```

`POST /api/admin/popup/upload`

```
// Upload image for pop-up (also used by the featured-event admin)
// FormData: { file: File, prefix?: "popup" | "featured-event" | "event-popup" | "nfc" }
// `prefix` is allowlisted rather than interpolated so it can't escape into a path.
// Logic:
// 1. Check auth
// 2. Upload to storage bucket
// 3. Return public URL or signed URL
// 4. Admin uses URL in pop-up form
```

`GET|POST|PUT|DELETE /api/admin/nfc`

```
// CRUD for the nfc_tiles rows behind /nfc. Auth is free: middleware.ts matches
// /api/admin/:path*, so an unauthenticated caller never reaches the handler.
//
// Writes go through one buildPayload() validator so POST and PUT can't drift:
//   - mode = "embed" requires an embed_url that passes isEmbeddable()
//     (hostname ends with churchsuite.com). Anything else sets X-Frame-Options
//     and would render as a blank white box, so it's rejected with a sentence
//     telling the admin to use a details tile instead.
//   - mode = "event" takes only an event_identifier from the client and
//     re-resolves it server-side via resolveEvent() → getEventIndex(). That's
//     where embed_url (the signup URL + #form_event_signup), event_slug,
//     event_name, event_sequence and event_ends_at are written. Three named
//     failures, each naming the way out: the event is gone from the calendar,
//     it takes no signups, or it books through a site that refuses framing.
//   - cta_link must start with "/" or "https://".
//   - Length limits mirror the DB checks so the admin gets a readable error
//     rather than a Postgres constraint string.
//
// PUT/DELETE remove the superseded image from `popup-images` only *after* the
// row write succeeds, so a failed write can't orphan the live artwork.
// No revalidatePath — getNfcTiles() reads with noStore().
```

`POST /api/admin/revalidate`

```
// Manually trigger ISR for a path
// Body: { path: string }
// Logic:
// 1. Check auth
// 2. Call revalidatePath(path) or revalidateTag(tag)
// 3. Return success
```

`GET /api/admin/posts/[id]/preview`

```
// Signed preview URL for a (possibly unpublished) post
// Returns: { url: "/<slug>?preview=<expiry>.<hmac>" } — valid for one hour
// See lib/postPreview.server.ts and the /[slug] catch-all
```

`GET /api/admin/posts/check-slug`

```
// Live slug-availability check for the post editor
// Query: ?slug=<candidate>&excludeId=<postId?>
// Logic:
// 1. Delegate to lib/posts-slug.ts → checkSlug() with the service client
// 2. Always returns 200 with { available, slug, reason? } so the editor
//    can show inline validation feedback while typing (excludeId lets an
//    existing post keep its own slug when editing)
```

Public API Routes

Design requests — `/api/design-request/[token]/**`

```
// GET   /                                → the requester's view of their ticket
// PATCH /                                → changes_requested | closed | cancelled
// GET   /deliverables/[did]/download     → 302 to a fresh signed URL (or the link)
// POST  /deliverables/[did]/confirm      → the requester's "I have this, you can
//                                          delete it" — stamps confirmed_at for the
//                                          purge cron. Idempotent (re-confirming
//                                          keeps the original timestamp, so it can't
//                                          push the deletion window back). 'link'
//                                          deliverables can't be confirmed (400).
//
// AUTHORIZATION: the share token, not a session — most requesters aren't staff
// and shouldn't need an account to collect a poster. Every route re-derives the
// ticket from the token and matches child rows against it, so a valid token for
// ticket A cannot reach ticket B's files. A bad, malformed or deleted token all
// return the same 404, so probing tells you nothing.
//
// The response is hand-built by requesterView() — deliberately not a select("*")
// passthrough. Internal notes, the assignee's email and the requester IP hash
// are dropped there, and adding a column to the table exposes nothing until
// someone edits that function.
//
// PATCH is rate-limited per IP and passes actor "requester" to canTransition,
// so posting {"to": "delivered"} on your own ticket is rejected. A change
// request must carry a note, and is capped at MAX_CHANGE_REQUESTS (5) — past
// that it isn't a design problem any more. Cancelling stays available.
```

`POST /api/chat`

```
// AI-powered conversational Smart Search (the FloatingSmartSearch widget).
// Body: { messages: { role: "user" | "assistant"; content: string }[] }
// Response: NDJSON stream of { type: "text" | "tool_call" | "tool_result" | "done" | "error", ... }

// Logic:
// 1. Rate limit: 20 requests per IP per minute (429 on excess)
// 2. Validate messages: user/assistant roles only (blocks injected system turns),
//    <=2000 chars each, last must be user <=300 chars (else 400)
// 3. getOpenAI() null-check → NDJSON fallback routing to /contact
// 4. Tool-calling loop (up to MAX_TOOL_ROUNDS=4 — enough for search → extract_page
//    → answer, plus headroom) on gpt-4.1-mini:
//    - System prompt: buildSmartSearchPrompt() (lib/siteKnowledge.ts) — church
//      facts + today's date + tool guidance
//    - tools: TOOL_DEFINITIONS (lib/smartSearch/tools.ts) — find_products,
//      get_weather, get_directions, search_web, extract_page
//    - A per-request ToolContext (createToolContext(), tracks seenUrls) is
//      threaded through every executeTool() call so extract_page can only
//      open URLs a search_web call in this same request actually returned
//    - Streams assistant text as `text` events; emits a `tool_call` event per
//      tool invoked (drives the client's "Searching the web…" status line),
//      runs tool calls, emits each result as a `tool_result` event, feeds
//      results back, repeats
// 5. Client (FloatingSmartSearch) accumulates `text` (parsed for PAGE/CTA/OPTION)
//    and renders each `tool_result` as a card (ResultCards.tsx)
```

`GET /api/events/[slug]/ics`

```
// Calendar download for an event series — the .ics link on every /whats-on/[slug]
// page and event card. Public and outside the /api/admin/* matcher: it serves only
// what the ChurchSuite feed already publishes, so it needs no auth.
// Logic:
// 1. getEventIndex() (lib/events.server.ts), then resolve `slug` via bySlug with the
//    same identifier fallback as the event page (a `-<identifier>` suffix on a
//    renamed/disambiguated URL still downloads instead of 404ing). 404 if unmatched.
// 2. buildIcs({ series, occurrence, baseUrl }) from @destiny/shared builds the VCALENDAR.
//    Without ?occurrence= the file contains every upcoming session, so subscribing to a
//    recurring event adds the whole run at once; ?occurrence=<id> narrows it to one.
// 3. Returns text/calendar as an attachment (filename `<slug>.ics`).
// revalidate = 300; Cache-Control public, s-maxage=300, stale-while-revalidate=600.
```

`GET /api/youtube/videos`

```
// Returns getAllVideos(50) — the sermon archive's video list (lib/youtube.ts).
// Fetched client/server-side by the /sermons page; no separate sync job or
// cache table — YouTube is queried directly on each call.
```

`GET /api/youtube/status`

```
// { quotaExceeded: boolean } — lib/youtube.ts isYouTubeQuotaExceeded().
// revalidate = 3600. Lets the client show a friendly fallback if the
// YouTube Data API daily quota has been used up.
```

`GET /api/youtube/thumbnail/[id]`

```
// Proxies a YouTube video thumbnail image (avoids hot-linking i.ytimg.com directly).
```

`GET /api/sermons/more` was removed in September 2026 — `SermonGrid.tsx` now receives the full archive (`getFullSermonArchive()`) from the server component and filters/reveals it entirely client-side, no further network calls.

`GET /api/youtube/live`

```
// Livestream status, polled client-side every 30s by LiveContext.
// Response: { live, videoId, title?, startedAt?, scheduledFor?,
//             serverTime?, checkedAt }
// dynamic = "force-dynamic"; Cache-Control: s-maxage=30, stale-while-revalidate=30
//
// ?debug=1 additionally returns `source` — which detection layer answered
// (channel-page | videos.list |
// confirmed-offline | no-signal | disabled | no-channel | error) — and sets
// no-store. That is the fastest way to work out why the banner is or isn't
// showing in production without a redeploy.

// Backed by lib/liveStatus.server.ts getLiveStatus() — see Libraries & Utilities.
```

`GET /api/cron/hr-review-reminders`

```
// Daily at 06:00 (vercel.json). Bearer CRON_SECRET — fails closed with 503 if
// unset (an open "send an email" endpoint is worse than not running).
//
// Finds hr_reviews with next_review_date within the next 14 days that haven't
// been reminded yet (reminder_sent_at IS NULL; no lower bound, so overdue-but-
// never-reminded reviews still surface), emails ONE digest to HR_NOTIFICATIONS_EMAIL
// via lib/hrEmail.ts (reviewer is free text, so there's no per-reviewer address),
// then stamps reminder_sent_at so the same review isn't re-sent each day it stays
// in the window. NOT under /api/admin, so middleware.ts doesn't guard it.
```

`GET /api/cron/analytics-anonymise`

```
// Daily at 03:00 (vercel.json). Bearer
// CRON_SECRET — fails closed with 503 if unset. Calls
// engagement_anonymise_ips(ANALYTICS_IP_RETENTION_DAYS ?? 90), which blanks
// `ip` on old engagement_events rows and returns how many it touched.
// visitor_hash and every other column survive — the row keeps counting, it
// just stops being personal data. See Database Schema §24.
```

`GET /api/cron/ip-reputation-refresh`

```
// Weekly, Monday 05:00 (vercel.json). Bearer CRON_SECRET — fails closed with
// 503 if unset. Fetches the four range lists behind ip_category (Tor, X4BNet
// VPN, X4BNet datacenter, Apple Private Relay — see Database Schema §24b),
// chunks each into ~10,000-row upserts through the service-role client, then
// deletes whatever the previous run for that source left behind (a
// batch-swap, not a truncate — the table is never briefly empty). Each
// source is independent: one failing (a GitHub outage, a changed URL) does
// not stop the other three refreshing. maxDuration = 300.
```

`GET /api/cron/design-deliverables-purge`

```
// Daily at 04:00 (vercel.json). Bearer CRON_SECRET — fails closed with 503 if
// unset (an open "delete confirmed deliverables" endpoint is worse than not
// running). NOT under /api/admin, so middleware.ts doesn't guard it.
//
// Sweeps design_ticket_deliverables where confirmed_at is at least 48h old,
// deletes the backing file (deleteDeliverableStorage handles the Supabase bucket
// object or a legacy Playbook asset; a 'link' row owns no bytes and is never
// swept because it can't be confirmed), then deletes the row. Because it runs
// once a day, the real-world floor is 48-72h, not exactly 48. Per-row try/catch:
// one file that won't delete is logged and skipped, not a whole failed run.
```

`GET /api/cron/destiny-one-sync`

```
// Daily at 03:30. Bearer CRON_SECRET (fails closed). OPTIONAL ChurchSuite part: only when
// configured, and only for members verified BY ChurchSuite (verification_source='churchsuite') —
// staff-verified members are never touched. Removed from ChurchSuite → `pending`, which pauses
// any group left with < 2 adults. An outage changes nothing. Then d1_reconcile_all() re-checks
// every live group (always runs). A failed member read or re-check emails D1_OPS_ALERT_RECIPIENT
// (sendOpsAlert, lib/destinyOne/opsAlertEmail.server.ts; job and error only, no member data).
```

`GET /api/cron/destiny-one-purge`

```
// Daily at 04:15. Bearer CRON_SECRET. d1_purge_expired(D1_MESSAGE_RETENTION_DAYS, default 365,
// floor 30) deletes messages, their attachment rows, erased members with no remaining messages,
// and closed reports / resolved events past the window; the route then removes the storage
// objects. A message under an open report is kept. ⚠️ 365 is a placeholder pending the
// safeguarding policy decision (scoping doc D6). A failed purge, or files it couldn't delete,
// emails D1_OPS_ALERT_RECIPIENT (sendOpsAlert).
```

Destiny One API (`/api/app/v1/one/*`)

The backend for the Expo app. Every route: `Authorization: Bearer <Supabase access token>` (no cookies, so `middleware.ts` doesn't see these), resolved by `lib/destinyOne/auth.server.ts`, which applies three gates — signed in → `active`, staff-verified member (`access_request_needed` / `not_verified` / `forbidden` otherwise, per `onboardingState`) → current notices accepted (`consent_required`). Responses use the app envelope with `Cache-Control: private, no-store`; errors are `{ error: { code, message } }` with a stable `code` and a message the app can show. Writes call the `d1_*` SQL functions, whose own errors (e.g. "A group needs at least 2 verified adults.") pass through as `rule_violation` (422). Typed client: `createDestinyOneClient` in `@destiny/shared`.

Route	Methods	Notes
`config`	GET	No sign-in, no token, and the only `/one` route the CDN may cache (`s-maxage=60`): `D1AppConfig` — `minBuild { ios, android }` , `forceUpdateMessage` , `maintenanceMessage` , `storeUrl` (env `D1_IOS_STORE_URL` , default `itms-beta://` = TestFlight; `D1_ANDROID_STORE_URL`). The app's forced-update gate (`appGate()` in `@destiny/shared` , `src/lib/appGate.ts`)
`auth/link`	POST	After every sign-in: accept an open invite for the email (`onboardMember`), return `D1Me` with `onboarding`
`auth/code`	POST	No sign-in: `{ email }` . Sends the email sign-in code only if `d1_sign_in_status` says this email can get in (member, open invite, or requests open), from `after()` , and always answers `{ sent: true }` , so neither the reply nor its timing reveals who is a member. The code comes from the Supabase admin API (`generateLink` , which doesn't email) and is sent through Resend, so sign-in doesn't hit Supabase's per-IP limit for the server's address or its capped built-in sender. Rate-limited per IP and per (hashed) email. Replaced `auth/check` (2026-09-28), which said "no account" outright
`me/access-request`	POST	`{ name, dateOfBirth?, note? }` — ask to join; a Destiny One Admin approves
`auth/churchsuite/start` → `callback` → `exchange`	GET, GET, POST	Sign in with ChurchSuite (below)
`me`	GET, PATCH, DELETE	PATCH `{ firstName, lastName }` changes my own name (5/min; at most 2 changes in any 30 days, else `rule_violation` with the date it reopens; Profile tab → Change name, `edit-name.tsx` , which shows a live preview of a message from you). `D1Me.nameChangesLeft` / `nextNameChangeAt` carry the allowance. DELETE = GDPR erasure (`{ "confirm": "DELETE" }`). `D1Me` includes `firstName` , `lastName` , `blocked` (people I've blocked), `avatarUrl` (a signed link) and `isStaff` (has Destiny One / Safeguarding / Super Admin access; used only to check an "Add admin account")
`me/avatar`	POST, DELETE	Profile picture (multipart `file` , 5 MB). Active members only; stored privately in `d1-avatars` . In the app, tapping your picture on Profile offers Choose photo / Remove picture
`me/email` → `me/email/confirm`	POST, POST	Change my own sign-in email (Profile → Email, `src/app/change-email.tsx`). `{ email }` emails a 6-digit code to the new address through Resend and returns a sealed `ticket` (`lib/destinyOne/emailChange.ts` : account + address + code hash, 15 minutes); `{ ticket, code }` then sets the auth email with the admin API (`email_confirm: true`) and emails a notice to the old address. "Already has an account" is only said after the code checks out, so it never reveals who has one. Doesn't use `auth.updateUser({ email })` : Supabase's project-wide "Change email" template is link-based (the portal uses it) and secure email change would also need a code from the old inbox. Active members only; rate-limited per account. The app refreshes its Supabase session afterwards
`me/password`	GET, POST	Set or change my password (Profile → Password, `src/app/set-password.tsx`). `GET` returns `{ hasPassword }` (`d1_has_password` , service role only, migration `20260929_04`). `POST { password, current? }` : if the account already has a password, `current` is required and checked server-side against Supabase with a throwaway client; then the password is set with the admin API. Accounts with no password yet are gated on the phone instead (Face ID / passcode via `confirmDeviceOwner` , refusing devices with no lock). Supabase weak/leaked refusals come back as `invalid` with message `password_rejected:<reasons>` . Rate-limited per account
`members/[id]/block`	POST, DELETE	Block / unblock someone; returns `D1Me` . Hides their messages and notifications for me only; logged for safeguarding
`me/consents`	POST	Current versions only (`REQUIRED_CONSENTS`)
`me/export`	GET	GDPR access: profile (incl. access-request note, declared and staff-set 18th birthday, how and when verified, a link to the profile picture), consents, memberships, own messages (incl. deleted), files sent (24-hour links), reactions, blocks, own reports, feedback sent
`me/push-tokens`	POST, DELETE	Expo tokens
`communities`	GET, POST	POST: any leader role. Each group's `lastMessage.preview` is the first line of its text; a poll or a shared event (no text) reads "Poll: …" / "Event: …" (`contentPreview` in `@destiny/shared` , filled in by `overview()` in `lib/destinyOne/chat.server.ts` with one extra query, no SQL change)
`communities/[id]`	GET	
`communities/[id]/members`	POST, DELETE	DELETE without `memberId` = leave
`communities/[id]/groups`	POST	Create a sub-group (any size; paused until ≥3 people, ≥2 adults)
`groups/[id]`	GET, PATCH	PATCH: rename/describe/archive (managers). A rename or new description sends `group_updated` on `d1-group:<id>` (`announceGroupUpdated`), so every member's chat list updates straight away
`groups/[id]/members`	POST, DELETE	Leaving never blocked
`groups/[id]/icon`	POST, DELETE	Group icon (multipart `file` , 5 MB). Any current member can change it, not just admins; not for Announcements or paused groups. Stored in `d1-avatars` as `d1_groups.icon_path` ; `iconUrl` on group summaries is a signed link. Sends `group_updated` like a rename. The app asks people to avoid the church logo
`groups/[id]/messages`	GET, POST	Only messages since you joined; POST takes `body` , an attachment, a `poll` draft, or an `event` ref (the event is re-fetched and snapshotted server-side) and pushes a notification via `after()` (group name, "Sender: first line", or

Route	Methods	Notes
		"Poll: ..." / "Event: ..."). A message in the DestinyAI chat, or with "@DestinyAI" in it, is then answered by DestinyAI (part 18)
`messages/[id]/vote`	POST	`{ optionIds }` — your full vote set on a poll (empty clears it). 60 a minute
`messages/[id]`	PATCH	`{ body }` — edit my own message's text (15 minutes, 10 times; `d1_edit_message`). Sealed with new search terms; `message_edited` goes to the group over the REST broadcast (it carries text). 20 a minute, no push
`messages/[id]/pin`	POST, DELETE	Pin or unpin (group managers; at most 3, a fourth unpins the oldest). 30 a minute
`messages/[id]/receipts`	GET	"Seen by": `{ read, notYet, hidden }` for the sender or a group manager, while their own read receipts are on (`d1_read_receipts`). 60 a minute
`messages/[id]/forward`	POST	`{ groupIds }` (1–5): a copy into other chats I can post in, marked Forwarded, file copied per group, re-sealed per group. Not polls. 20 a minute
`me/settings`	PATCH	`{ readReceipts? }` — account settings that change what others see. `D1Me.readReceipts`
`events`	GET	Upcoming ChurchSuite events for the Event picker in the attach sheet
`assistant`	GET	My one-to-one chat with DestinyAI as a `D1GroupSummary`, made on first use (`d1_assistant_group`). Send to it through `groups/[id]/messages` like any chat; DestinyAI answers every message (see part 18). The app pins it above the communities in Chats
`groups/[id]/invites`	POST	Leaders: `{ email, name, adult, note? }` — invite someone new; they become an access request for staff to approve, then join the group
`search/messages`	GET	`?q=` — search of your messages: groups you are in, since you joined, never deleted, never from people you've blocked; newest 30. Each word matches as a prefix, on the blind index (text is encrypted at rest, see "Message encryption"). `&groupId=` searches one group (search opened from a chat): same rules; not found if you aren't in it, nothing for an archived group
`groups/[id]/read`, `/mute`, `/archive`, `/attachments`	POST	Read marker, mute, archive/unarchive for the caller only (`{ archived }`), signed upload URL
`groups/[id]/attachments?ids=`	GET	Fresh signed links (1 hour) for cached attachments whose links expired: only files in this group, sent since you joined, not deleted, not from someone you've blocked. Up to 60 ids
`groups/[id]/media?before=&limit=`	GET	Group info → Photos and files: messages that carry a photo or file, paged like `messages` (60 a page, up to 100). Same visibility as the chat (since you joined, not deleted, not blocked senders), with fresh signed links
`groups/[id]/typing`	POST	"I'm typing": broadcasts `typing { groupId, memberId, name }` on the group topic over the REST broadcast (nothing stored). Only if you can post there. The app calls it at most every 4 s while the box has text; each one shows you as typing for 6 s. 30 a minute
`messages/[id]`	DELETE	Soft delete (content kept for review)
`messages/[id]/report`, `/reactions`	POST (+DELETE)	Report → safeguarding bell, and an email to every Safeguarding Admin (`lib/destinyOne/safeguardingEmail.server.ts`; no message content, names or group in it)
`directory`	GET	Leaders only; names + adult flag, never contact details
`feedback`	POST	`{ kind: "problem" | "idea", body, appVersion?, platform?, osVersion?, device?, errorId? }` — Profile → Report a problem / Send feedback. Saved to `d1_feedback` for Destiny One Admins (bell, no text in it). Skips the consent gate, like reporting. 5 a minute

Sign in with ChurchSuite. ChurchSuite's OAuth (authorisation code, scope `user`, `GET /account/users/current`) identifies ChurchSuite **users** — staff and leaders with a login — not every member, so members sign in with a Supabase email one-time code instead. `start` takes the app's redirect (`destinyone://…` only; `exp://` in development) and the **app's** PKCE challenge, and keeps our own PKCE verifier + state in a sealed HttpOnly cookie (`DESTINY_ONE_SECRET`, AES-256-GCM). `callback` exchanges the code, ensures a confirmed Supabase user for the ChurchSuite email, links it, mints a magic-link token hash — and redirects with a sealed 2-minute code, **not** the token. The app redeems that at `exchange` with its verifier, then `verifyOtp({ token_hash })`. An app that hijacks the `destinyone://` scheme on Android gets a code it can't use.

`callback` refuses (`error=use_email_sign_in`) when the email belongs to an account that holds more than a membership — an `admin_roles` row or an `hr_staff` link — unless that account's member record is already linked to the **same** ChurchSuite user id, and always when the member is linked to a *different* ChurchSuite user. The email on a ChurchSuite user is editable by ChurchSuite admins, so a matching email alone would let one sign in as a website admin (security audit 2026-10-09).

Destiny One admin API (`/api/admin/destiny-one/*`)

Two roles, split by path in `lib/adminRoles.ts` ROUTE_RULES (the safeguarding rule is listed first so the broader one can't swallow it) and re-checked in every route by `requireDestinyOneAdmin` / `requireSafeguardingAdmin` (`lib/destinyOne/admin.server.ts`). Every mutation calls `recordAudit` (section `destiny_one` or `safeguarding`).

Bearer tokens (D1 Admin desktop app). This is the one `/api/admin` area that also accepts a Supabase access token as `Authorization: Bearer …` when there's no cookie session: `middleware.ts` resolves the user from it and then runs the same role check and audit actor headers as for a cookie, and `requireRole` in `admin.server.ts` falls back to it the same way. Every other `/api/admin` path is cookie-only. Response shapes are typed in `@destiny/shared` (`destinyOne/adminTypes.ts`, re-exported by `lib/destinyOne/adminTypes.ts`) and the desktop app calls them through `createDestinyOneAdminClient` (`destinyOne/adminClient.ts`).

- `GET whoami` — open to any signed-in user (OPEN_PATHS): `{ destinyOneAdmin, safeguardingAdmin, transcriptReader, superAdmin }` for the caller, so the desktop app knows which sections to show.

Destiny One Admin (`destiny_one_admin`) — runs the app, never sees message content: - `GET overview` — counts for the landing page. - `GET members?status=` , `GET/PATCH/DELETE members/[id]` — name, age decision (`age: { adult, dateOfBirth? }` → `adultOnForDecision`), leader roles, suspend/reinstate, optional ChurchSuite reference link, GDPR erasure. - `POST members/[id]/approve` (`{ adult, dateOfBirth?, displayName?, communityIds? }`), `POST members/[id]/decline` (suspends, so they don't re-queue). - `GET/POST invites` (one or up to 100; per-invite errors returned), `PATCH invites/[id]` (resend restarts expiry / revoke). Invite email: `lib/destinyOne/inviteEmail.server.ts` — the invitee's name only, no community names. - `GET/POST communities` , `GET/PATCH communities/[id]` , `POST/PATCH/DELETE communities/[id]/members` , `POST communities/[id]/groups` , `GET/PATCH groups/[id]` (archive/restore), `POST/PATCH/DELETE groups/[id]/members` — all through the `d1_admin_*` SQL functions, so the rules hold. - `GET/PATCH settings` — access requests on/off, invite expiry; retention shown read-only. - `GET feedback` , `PATCH feedback/[id]` (`{ status: "new" | "done" }`) — app feedback. This is the one place these routes read something members wrote, and it's what they chose to send to staff. - `GET churchsuite?q=` — OPTIONAL lookup on the approval screen; 404 when ChurchSuite isn't configured.

Safeguarding Admin (`safeguarding_admin`) — `/api/admin/destiny-one/safeguarding/*` , the only place message content can be read: - `GET events?open=1` , `PATCH events/[id]` — the pause/report queue; mark handled. - `GET reports?status=` , `PATCH reports/[id]` — triage reports (resolution text redacted in the audit log). - `GET groups` — every group with counts, to pick one to review or pause. - `GET groups/[id]/transcript?reason=…&from=…&to=…` — **needs the Safeguarding Admin role itself; super admin alone isn't enough** (`requireTranscriptReader` , decided 2026-09-28). Full history incl. deleted messages and membership history. **Requires a reason** and defaults to the last 30 days; every read is written to the audit log (`action: "view"` , section `safeguarding`) with who, which group, window and reason. - `POST groups/[id]/freeze` — manual pause / lift (a manual pause isn't lifted by the automatic rule). - `POST messages/[id]/remove` `{ reason }` — take a (reported) message down for everyone via `d1_admin_delete_message` ; content kept for review; audited. - `POST members/[id]/suspend` `{ suspended, reason }` — suspend a sender (groups that drop below the rules pause as usual) or reinstate someone who is suspended (never a way round the approval queue); audited. Lets the safeguarding team act without the Destiny One Admin role. - `GET events?kind=block` — the log of members blocking / unblocking each other (the "Blocks" tab). - `POST/DELETE members/[id]/restrict` `{ days: 1 | 7 | 30, reason }` — timed posting restriction (can read, can't post or react, lifts itself) or lift it early; audited. `GET members` rows carry `postingRestrictedUntil` . - `POST reports/[id]/escalate` — escalate to the Designated Safeguarding Lead: sets `escalated_at` , moves an open report to `reviewing` , and emails every Safeguarding Admin (`emailSafeguardingAboutEscalation` , content-free like the report alert). A second call is a no-op. - `GET audit?before=` — the Destiny One slice of the audit log (sections `destiny_one` and `safeguarding` , summary and who/when only), 50 a page. The full log stays Super Admin only.

> There is no `/api/webhooks/vercel` or GitHub webhook route, and no `youtube-sync` cron/cache job — `app/api/webhooks/` currently only contains `stripe/` (see Shop API below). YouTube data is fetched live on each request, not synced to a cache table.

Staff Portal API (`/api/portal/*`)

```
// The staff self-service side of HR — a SEPARATE auth boundary from /admin.
// middleware.ts guards /api/portal (matcher covers it), but "signed in" is not
// enough: the caller must have a linked hr_staff row (lib/staffPortalAuth.ts,
// isLinkedToStaff / readPortalUser). Every handler re-derives identity via
// readPortalUser() — never trusting request state from middleware — and scopes
// every query to the returned staff.id, never to anything the client supplies.

GET  /api/portal/me            // the caller's own hr_staff profile
PATCH /api/portal/me           // change own email — auth.updateUser() on the caller's own session (Supabase sends a confirmation email); hr_st
POST /api/portal/me/password   // change own password — re-authenticates with currentPassword (signInWithPassword) before auth.updateUser(), so
POST /api/portal/me/avatar     // upload a profile picture to the `staff-avatars` bucket, replaces hr_staff.avatar_url
DELETE /api/portal/me/avatar   // remove the current profile picture
GET  /api/portal/leave         // own leave requests (newest first)
POST /api/portal/leave         // file own leave — staff_id + status forced server-side, never from the body
DELETE /api/portal/leave/[id]  // withdraw own request, pending only; 404 (not 403) on a mismatch, so IDs can't be probed
GET  /api/portal/documents     // own documents + org-wide ones (staff_id IS NULL)
GET  /api/portal/documents/[id] // 60s signed download URL from the hr-documents bucket; 404 on a foreign doc
GET  /api/portal/reviews       // own hr_reviews — id/review_date/type/reviewer/next_review_date only, never `summary`
GET  /api/portal/checklists    // own hr_checklist_items where kind = 'onboarding' (offboarding is HR/manager-only)
PATCH /api/portal/checklists/[id] // toggle is_done on one of your own onboarding items; 404 on a mismatch
GET  /api/portal/team          // directory of non-"left" hr_staff — name, role, department, email, avatar; never phone or notes
```

A leave request/decision here (and from `/admin/hr`) triggers a notification email via `lib/hrEmail.ts` .

`GET /api/health/smart-search` — self-healing health check (cron)

```
// Vercel Cron endpoint that keeps Smart Search from showing a broken chat when
// OpenAI is unreachable. Auth: Authorization: Bearer <CRON_SECRET> (Vercel Cron)
// or a manual ?secret=<CRON_SECRET> run; if CRON_SECRET is unset (local/dev) it
// runs open. maxDuration = 30.
//
//   1. Reads service_status via getSmartSearchStatus(). While healthy it only
//      actually pings once next_check_at is due (daily), so an hourly cron can
//      retry fast while DOWN but stays cheap while UP ({ skipped: true } otherwise).
//   2. Live-pings OpenAI (SMART_SEARCH_MODEL, max_tokens 1, 15s timeout, no retry).
//   3. On success  → enabled = true, next check in 24h, failures reset to 0.
//      On failure   → enabled = false, next check in 1h, failures++, reason stored.
//   4. Emails an alert (lib/smartSearchAlertEmail.ts, via Resend) ONLY on a state
//      change — healthy→down or down→recovered — so the inbox never gets hourly spam.
//
// app/layout.tsx reads isSmartSearchEnabled() to decide whether to mount
// FloatingSmartSearch, so a disabled service simply hides the feature site-wide.
```

Training API

```
// PUBLIC (outside the middleware matcher)
POST /api/training/unlock
//   Body: { subgroupId, password }. Verifies a sub-group password against the
//   hashed password_hash (server-side only; the hash is never sent to clients).
//   On success sets a scoped cookie so the gated sub-group's posts render.

POST /api/training/posts/[id]/timer
//   Enforces a training post's min_read_seconds "minimum read time" before the
//   member can mark it complete. Stateless — no DB writes; state rides an
//   HMAC-signed, base64url token so it survives across serverless instances.
//     action: "start"  → returns { token } signed over { postId, startedAt }.
//     action: "verify" → re-checks the token (timingSafeEqual) and postId, then
//                        compares elapsed time to minReadSeconds. Returns
//                        { success:true } once met, else 403 with { remaining }.
//   HMAC key precedence: TRAINING_TIMER_SECRET → HMAC of SUPABASE_SERVICE_ROLE_KEY
//   (never used raw) → a per-boot random key (last resort, instance-local).
//   Driven by components/training/CompleteButton.tsx on the training post page.
```

Shop API

```
// PUBLIC (outside the middleware matcher)
POST /api/store/checkout
//   Body: { items: [{ variantId, quantity }], customer: { name, email, phone?, notes? } }
//   Recomputes every price + stock from the DB (client prices never trusted),
//   creates a pending order + items, creates a Stripe PaymentIntent, returns
//   { clientSecret, orderNumber }. Rate-limited per IP.

POST /api/webhooks/stripe
//   Stripe signature-verified (STRIPE_WEBHOOK_SECRET), runtime = nodejs, raw body.
//   payment_intent.succeeded → order 'paid', decrement variant stock, Resend
//   confirmation emails (customer + church). Idempotent. payment_failed → 'cancelled'.

POST /api/store/checkout/bypass
//   TEST ONLY. 404 unless server env SHOP_TEST_BYPASS=1. Creates a real order and
//   finalises it WITHOUT Stripe (paid, stock decremented, emails) so the full flow
//   can be demoed without a payment. Restricted server-side to
//   @squaremediagroup.org / @destinytees.uk emails. When NEXT_PUBLIC_SHOP_TEST_BYPASS=1
//   and the customer enters a @destinytees.uk email, pressing "Continue to payment" on
//   the checkout page shows a popup asking whether to "Complete test order" (skips
//   Stripe) or "Continue to payment" (real payment) — there's no standing test button
//   visible to regular customers. Never set either flag in production — and since
//   the 2026-10-09 security audit (which found it set there) the route 404s and the
//   popup is hidden whenever VERCEL_ENV is "production", whatever the flags say.
//   The email is typed, not verified, so the domain check alone protects nothing.

// Shared order logic (pricing recompute, order creation, paid-finalisation) lives
// in lib/checkout.server.ts and is used by checkout, the webhook, and the bypass.

POST /api/shop-diagnostics
//   TEMPORARY — remove with components/shop/ShopDiagnostics.tsx once the /shop
//   blank-page bug is identified. Receives a client snapshot captured when /shop
//   renders blank and writes it to the Vercel runtime logs (console.error;
//   search for "shop-diagnostics"). It used to also file GitHub issues; that was
//   removed in the 2026-10-09 security audit because the Origin check is
//   forgeable and the dedupe key is caller-chosen, so anyone could open
//   unlimited issues with our token. Fenced: same-origin only (403), 32KB body
//   cap (413), 5 reports per IP per 10 min (throttled).

// ADMIN (gated by middleware, site_editor role)
GET|POST            /api/admin/store/products
GET|PUT|DELETE      /api/admin/store/products/[id]        // PUT reconciles variants
POST|DELETE         /api/admin/store/products/[id]/images // sharp → WebP → product-images bucket
GET                 /api/admin/store/orders
GET|PATCH           /api/admin/store/orders/[id]          // PATCH updates fulfilment status
GET|POST            /api/admin/shop-hero                  // list / create hero slides
PATCH|DELETE        /api/admin/shop-hero/[id]             // update (content/active/sort_order) / delete
POST                /api/admin/shop-hero/upload           // sharp → WebP → shop-hero-images bucket
```

Content Blocks

Admin-authored, configurable components that staff drop into a page from the **Blocks** sidebar in the editor — FAQ accordions, callouts, quotes, card grids, image galleries and button rows. The point is that a new FAQ or a new set of cards no longer needs a developer and a deploy.

Why it is built this way

A JSONB page builder (`builder_pages.document` , `studio_components`) was built here previously and removed in `supabase/migrations/20260711_06_remove_page_builder.sql` . This deliberately does **not** rebuild that.

Content stays exactly what it always was: **a single HTML string** in `posts.body` (and `jobs.description` , `training_posts.body` , `products.description`). There is no migration, no new table, and no second representation of a page that can drift out of sync with the first. A block is just a marker inside that string.

Wire format

A block serialises into the stored HTML as one **empty, top-level** div:

```
<div data-block="faq" data-block-version="1" data-props="{&quot;heading&quot;:&quot;FAQs&quot;,…}"></div>
```

All of the block's settings are JSON in `data-props` . `components/content/ RichContent.tsx` splits the stored HTML on these markers and renders the real component in their place.

That split is done with a regex rather than an HTML parser, which is only safe because of **two invariants**. Both are enforced, not assumed:

1. **The attribute value never contains `"` , `<` or `>` .** The browser's DOM serialiser escapes `&` , `"` and U+00A0 in attribute values — but *not* `<` and `>` . So `encodeProps` escapes those itself (plus U+2028/9), with a dev-mode invariant throw. The value provably matches `/^[^"<>]*$/` .
2. **A block can never nest.** Blocks belong to the ProseMirror group `dcblock` , and `Document` is extended to `content: "(block | dcblock)+"` . Since `blockquote` and `listItem` declare `content: "block+"` , a block inside one is structurally impossible — so a block div is always a self-contained top-level token and can never split surrounding markup into unbalanced halves.

`tests/unit/blocks-serialize.spec.ts` and `blocks-wire-format.spec.ts` guard both, and `RichContent` warns in development if the format ever drifts.

Why not `html-react-parser`

Five live routes render stored HTML. A parser would re-parse and *re-serialise* all of that existing content as React elements — `class` → `className`, style strings to objects, boolean attributes, whitespace — and every one of those is a chance to change output on pages nobody intended to touch. The split approach passes block-free content through the identical code path it used before, so adopting it everywhere was a provable no-op. It also avoids shipping ~15–20KB of parser to every public page.

The three layers

```
components/blocks/          SHARED — no "use client"
components/content/         RichContent — the public renderer
components/admin/blocks/    "use client" — TipTap nodes, sidebar, inspector
```

Block components carry no `"use client"` directive. That makes them RSC "shared" components: they server-render on the public page with **zero client JS**, and the identical module renders inside the editor's React node view. That is what makes the editor genuinely WYSIWYG rather than a placeholder preview — and it is why no block may import `AnimateIn`, `motion`, `next/image` or anything else client-only.

Adding a block

1. Create `components/blocks/<name>/{def.ts,<Name>Block.tsx}`
2. Import the def in `components/blocks/registry.ts` and add it to `BLOCK_LIST`

That is the whole checklist. One `BlockDefinition` drives the sidebar tile, the settings form, the TipTap node and the public renderer. **If adding a block ever requires touching the sidebar, the inspector, the TipTap plumbing or RichContent, the abstraction has leaked — fix that rather than special-casing.**

Things that will bite you:

- **Every schema key must have `.default()`.** Props are parsed as `{ ...defaults, ...stored }`, which is what lets content authored before a field existed keep working. Without the default, `safeParse` fails and the block silently resets to defaults, losing the author's content.
- **A `<select>` always yields a string.** A schema expecting `z.literal(2)` will fail to parse and reset the block. Model numeric choices as string enums (`z.enum(["2","3"])`).
- **Never rename a `name`.** It is the wire format, written into every page using the block.
- **Anything used as `href` or `src` must go through `safeUrl`.** React escapes props it renders as children but not values placed in those attributes.
- **Blocks render in four different column widths** (posts, training, jobs, shop). Size with `@container` queries and `cqi` units, never viewport breakpoints. `[data-dc-block]` carries `container-type: inline-size`.
- Block widths are `column` or `wide` only. There is deliberately no full-bleed: at ≥1600px `app/[slug]/page.tsx` places the article in an off-centre grid track beside the promo rails, so the usual `margin-inline: calc(50% - 50vw)` trick would be wrong there.

Key files

File	Role
`components/blocks/types.ts`	`BlockDefinition`, `FieldSchema`
`components/blocks/serialize.ts`	`encodeProps` / `decodeProps` / `unescapeAttr` / `BLOCK_RE`
`components/blocks/tokens.ts`	Shared design vocabulary — card shell, eyebrow, heading scale, tones, buttons, `safeUrl`
`components/blocks/registry.ts`	`BLOCK_LIST` — the single source of truth
`components/content/RichContent.tsx`	Public renderer + merged JSON-LD
`components/admin/blocks/createBlockNode.tsx`	Generic TipTap node factory
`components/admin/blocks/UnknownBlockNode.tsx`	Data-preserving fallback
`components/admin/blocks/BlockNodeView.tsx`	Editor chrome, drag grip, click shield
`components/admin/blocks/BlockPalette.tsx`	The block inserter — sidebar on desktop, searchable grid in the sheet
`components/admin/blocks/BlockInspector.tsx`	Schema-driven settings panel
`components/admin/blocks/BlockOutline.tsx`	Jump list of every block in the document
`components/admin/blocks/blockCommands.ts`	Move / duplicate / delete / add-paragraph, by position
`components/admin/blocks/BlockTools.tsx`	Sheets + the mobile selected-block toolbar; routes the inserter to the desktop drawer or the phone sheet by `useIsDesktop()`
`components/admin/blocks/BlockDrawer.tsx`	Desktop right-hand drawer wrapping the drag-and-drop `BlockPalette` (`layout="sidebar"`) for the scrolling product/job/training editors — stays open across inserts; mobile keeps the bottom sheet
`components/admin/Sheet.tsx`	The admin bottom sheet (grabber, detents, drag-to-dismiss)
`app/dev/blocks`	Development-only gallery; 404s in production

Mobile is a different interaction, not a narrower one

Desktop edits a block by hovering it, reading the chrome bar that appears and clicking a 24px control. None of those three steps exist on a touch screen, so below `lg` the blocks UI is replaced rather than reflowed — the same model every touch editor converged on:

- **Tap a block to select it**, and a toolbar docks to the bottom of the screen with that block's actions at 44px: move up, move down, its name, Settings, and a "more" action sheet holding duplicate / add-paragraph / delete. The hover-revealed chrome bar in `BlockNodeView` is `hidden lg:flex`; what mobile keeps is a standing label chip above each block, because with no hover there is otherwise nothing saying a block is one tappable object.
- **Settings open at the `half` detent**, so the block stays on screen above the sheet and visibly updates as you type. That live feedback is most of what makes the settings legible on a small screen.
- **With nothing selected**, the settings sheet lists the blocks in the page (`BlockOutline`) instead of saying "click a block" — the canvas is behind the sheet, so that was advice a thumb could not follow.
- **The inserter is a searchable two-column grid** in the sheet, and its copy does not mention dragging, which touch cannot do. On desktop, the scrolling product/job/training editors instead open the inserter as a right-hand drawer (`BlockDrawer`) that reuses the drag-and-drop `BlockPalette` sidebar and stays open across inserts, so several blocks can be dragged in a row — the full-page post editor already keeps that palette as a permanent sidebar.
- **Every inspector input is 16px on touch** (`fieldInputClass`). Below that, iOS Safari zooms the page on focus and does not zoom back out, which put the block being edited off-screen on the first tap of the first field.
- Repeater rows keep move up/down in the header and moved duplicate/remove into the open row as labelled buttons — four 24px icon buttons in a row put "remove" a thumb-width from "duplicate".
- `RichTextEditor`'s formatting toolbar is one horizontally scrolling row below `lg`. Wrapped, its nineteen controls took six rows — about a third of a phone screen — above an editor with no room left.

`components/admin/Sheet.tsx` is the shared surface underneath all of it: grabber, drag down to dismiss (full → half → closed) and up to expand, `auto` / `half` / `full` detents, safe-area inset, body-scroll lock, and it lifts above the keyboard via `useKeyboardInset`. Escape is handled in the **capture** phase: these sheets open on top of admin modals that close on Escape too, and the modal's document listener is registered first, so nothing in the bubble phase can stop it.

It portals to `document.body` at `z-[110]` (the toolbar at `z-[105]`). Portalled because the admin modal backdrops use `backdrop-filter`, which makes them a containing block for `fixed` descendants — the toolbar would anchor to the modal instead of the screen. The z-indexes sit above the admin modals (`z-50`) and the dev sandbox (`z-[100]`), and below `DialogProvider` (`z-[200]`) so the editor's link prompt still opens over a sheet rather than underneath one.

Embeds go through the cookie-consent gate

`MediaEmbed` and `ChurchSuiteEmbed` hold third-party iframes behind a consent placeholder until the visitor opts in to media cookies (`consent.media`). The Video and ChurchSuite form blocks wrap those components rather than emitting their own `<iframe>`, so admin-authored embeds behave like the rest of the site.

The old toolbar buttons did not: they wrote a bare `<iframe>` into the stored HTML, which loads YouTube or ChurchSuite — and their cookies — before the visitor has agreed to anything. **Content authored with those buttons still renders that way**; this only changes what new content does. Worth a pass over existing posts if that matters.

Two consequences:

- These two blocks are the only ones that ship client JavaScript, because the consent hook needs it. Everything else stays a shared component and renders with zero JS.
- The Custom embed block *can't* be gated — we have no idea what's in it. It sits in the sidebar's Advanced group precisely so it's the last thing reached for, and its help text points at the other two.

Non-obvious things that will waste your time

- **`UnknownBlockNode` is not optional.** ProseMirror drops elements its schema doesn't recognise. If a block is removed from the registry, or a deploy is rolled back, opening an affected page would quietly discard the block and the author's next save would write that loss to the database permanently. This node round-trips the marker verbatim (it does **not** re-encode `data-props`, since there's no schema to re-encode against) and shows a placeholder.
- **The drag grip must not be a `<button>`.** TipTap's `NodeView.stopEvent` returns early for `INPUT/BUTTON/SELECT/TEXTAREA` targets *before* the bookkeeping that records a drag started from `[data-drag-handle]`. A `<button data-drag-handle>` is silently undraggable. It is a `<span role="button">`. The other chrome buttons *are* real buttons, because that same early return is what they want.
- **Never pass a custom `stopEvent` to `ReactNodeViewRenderer`** — it short-circuits the whole default, including that drag bookkeeping.
- **`renderHTML` is hand-written, not `mergeAttributes`.** That pins attribute order so `BLOCK_RE` cannot drift. "Tidying" it back would make every block silently vanish from every public page.
- **Do not add `value` to the `[editor]` effect in `RichTextEditor.tsx`.** The parent re-renders with a new `value` on every keystroke, so a `value` dep is an infinite loop — and even a guarded version calls `setContent`, which rebuilds the document, destroys every node view, drops the selection and wipes undo history mid-edit.
- **Blocks are never in the rich-text toolbar.** That toolbar formats the current text selection; blocks are page structure. On desktop the post editor gives them a persistent sidebar and the modal editors get `BlockTools` sheets; on mobile they are sheets plus a docked toolbar for the selected block. All three are outside the formatting strip.
- The inspector debounces writes ~200ms and derives its draft during render rather than syncing via an effect, so a typed word is one undo step and an external Cmd+Z is reflected.

Testing

`npm run test:unit` (or `npx playwright test --project=unit`) runs the unit specs in plain Node — no browser, no dev server, no second test runner. They cover the block wire format against an adversarial corpus (script tags, quotes, HTML entities, U+2028/9, angle brackets, 200-item arrays), block registry integrity (every field name exists in `defaults`; every schema parses `{}`), attribute-order drift, sermon/podcast pairing, course-registry integrity (`course-events.spec.ts` — every type has complete metadata, no type is owned by two admin pages or none), livestream page parsing (`live-detection.spec.ts` — a live broadcast is detected even when recommendation shelves carry `isLiveNow: false` first, a finished broadcast isn't, and a channel-home redirect is a definitive offline rather than an escalation), Sunday service times across both DST boundaries (`service-times.spec.ts`), admin navigation integrity (`admin-nav.spec.ts` — every registry entry's declared role actually grants its own path, so drift between `lib/adminNav.ts` and `ROUTE_RULES` fails the suite instead of shipping a link that bounces; plus role filtering, longest-prefix path resolution, and that no breadcrumb dead-ends), and CSV quoting for the orders export (`csv.spec.ts`).

Browser specs run with `npm run test:e2e`. Those needing an admin session (`admin-blocks`, `admin-courses`) skip themselves unless `ADMIN_EMAIL` and `ADMIN_PASSWORD` are set; credentials come from the environment only and live in the gitignored `CLAUDE.local.md`.

CI. `.github/workflows/ci.yml` runs `typecheck`, `lint` and `test:unit` on every pull request and every push to `main`. It needs no secrets — the unit project never touches Supabase, a browser or a dev server — which is what lets it run on every PR, including ones from forks. Keep it that way: a unit spec that needs credentials belongs in the e2e projects instead. `next build` is deliberately not in CI because page-data collection needs live Supabase env vars; Vercel's preview build covers that. The browser suite stays in `playwright.yml`, manual-trigger only, because it needs a running site to point at. Three more jobs, also secret-free: **Destiny One app** (`npm ci`, typecheck and `expo lint` inside `apps/destiny-one`, which is its own npm project), **D1 Admin desktop app** (`npm ci` and `npm run build` — typecheck plus the Vite bundle — inside `apps/d1-admin`; the Rust shell is only built on a Mac) and **Database rules** (`scripts/test-sql.sh` against the PostgreSQL that ships on the Ubuntu runner, so the safeguarding SQL checks run on every PR).

`tsconfig.json` and `eslint.config.mjs` both exclude `.claude/`: agent worktrees under `.claude/worktrees/` are gitignored full copies of the app, and without the exclusion a local `npm run typecheck` or `lint` checks every file twice and reports the stale copy's errors as yours.

Libraries & Utilities

`lib/spotify.server.ts` and `lib/songRequests.server.ts`

Event song requests. `spotify.server.ts` wraps the Spotify Web API: an app token (Client Credentials) for search and track lookup, and the church account's refresh token for adding to the live queue, reading what is playing and listing devices. Explicit tracks are dropped from search, and a track with no `explicit` flag counts as explicit. `songRequests.server.ts` holds the request rules: open/closed, queue and per-device limits, a fresh Spotify lookup (the browser's flag is never trusted), duplicate refusal via a unique partial index, then queue-add (the row is removed again if Spotify refuses). Covered by `tests/unit/song-requests.spec.ts`.

`lib/adminNav.ts`

The one list of everywhere you can go inside `/admin`: `href`, `label`, icon, required role, group, a one-line description and search keywords.

Before it existed the same navigation was written out three times — in `AdminSidebar` (with roles), as a title map in `AdminHeader` (without), and as hand-written cards on the dashboard. The dashboard's copy had already drifted: it listed Redirects and the five course pages and nothing else, so Posts, Training, Store, Announcements and Users were reachable only via the sidebar. The ⌘K palette would have made it a fourth copy.

Exports `visibleGroups` / `visibleItems` / `visibleQuickActions` (role-filtered), `itemForPath` (longest-prefix match, so `/admin/store/products/123` resolves to Products rather than Store), `titleForPath` and `breadcrumbsFor`.

`isActive(item, pathname)` is the shared "is this the page we're on" rule — prefix matching that stops at a segment boundary, so `/admin/store` doesn't light up for `/admin/store-something`. It lives here rather than in the sidebar because the tab bar needs the identical rule and two copies would drift.

`tabsFor(roles)` and `activeTabKey(tabs, pathname)` back the mobile tab bar. `tabsFor` is a *projection* of `visibleGroups`, not a second navigation: one tab per ungrouped item, one tab per labelled group, and a group holding a single visible item collapsed to a leaf tab (matching how the sidebar renders the same case). Add a section to `ADMIN_GROUPS` and it appears in the tab bar too, correctly role-gated, with no second list to remember. `activeTabKey` resolves a pathname to the tab owning the deepest matching entry, so `/admin/store/orders/123` lands on Store rather than Dashboard.

Two things to keep in mind when editing it:

- **The `role` on each entry must match `ROUTE_RULES` in `lib/adminRoles.ts`.** This file decides what a user is *shown*; that one decides what they can reach. Showing a link that bounces to `?forbidden=1` is worse than not showing it. `tests/unit/admin-nav.spec.ts` asserts every declared role actually grants its own path, so drift fails the suite rather than shipping.
- **`unlisted: true`** keeps an entry out of the sidebar, dashboard and palette while leaving it reachable and resolvable for breadcrumbs — for a section that's built but not ready to launch. HR used this until it launched 2026-08-25; nothing uses it currently.

`ADMIN_QUICK_ACTIONS` holds things you *do* rather than places you go ("New post", "Clear the cache"). Those hrefs use `?new=1`, which the list pages read to open their editor straight away.

`lib/adminTheme.ts`

The admin's light/dark/system theme store (dark mode currently switched off via `ADMIN_DARK_MODE_ENABLED`) — see "Dark mode — `/admin` only" under Styling for the full mechanism (`@custom-variant dark`, why the `.dark` class lives on `/admin/layout.tsx`'s own wrapper rather than `document.documentElement`, and why two overlays deliberately opt out of it entirely). `useAdminTheme()` follows the exact `useSyncExternalStore` shape `AdminSidebar.tsx`'s remembered-dropdown-state store already established, under the `dc-admin-theme` localStorage key — a per-browser preference, like every other admin UI preference in this codebase, not a database column.

`lib/adminOnboarding.ts` / `lib/onboardingProgress.ts` / `lib/demoMode.ts`

Role-based onboarding for `/admin`. Same idea as `lib/adminNav.ts`: one registry, everything derives from it.

`lib/adminOnboarding.ts` holds every tour, keyed by access level. A tour is sections, a section is steps, and a step names a `route`, a `data-tour` anchor, a title and a body. `toursFor(roles)` returns the tours a person is owed — someone with three roles is taught three tours back to back — and `checklistFor(roles)` prefixes them with the shared Orientation section.

`super_admin` deliberately has no tour: it can reach everything, so "the super admin tour" would be every tour at once. Super admins preview a single role from `/admin/onboarding` instead.

Progress is counted in **sections**, not steps. "Add a product" is the unit a person recognises, and step ids churn every time a tour is reworded. `tests/unit/admin-onboarding.spec.ts` asserts every step's route actually passes `hasAccess()` for its own role — the same drift guard `admin-nav.spec.ts` applies to the sidebar, and for the same reason: a tour that bounces someone to `?forbidden=1` is worse than no tour.

`lib/onboardingProgress.ts` is the client half — the module-scope cached promise pattern from `lib/useAdminSession.ts`, so the welcome gate, the checklist and the running tour share one request. **It fails open**: if `/api/admin/onboarding` can't be read it resolves to "gate already seen, checklist already dismissed". A welcome modal that gates the dashboard must never be able to lock someone out because a table is missing.

`lib/demoMode.ts` is the guarantee that nothing done inside a tour is real. Teaching "add a product" must not add a product, and building a parallel set of fake admin screens would have drifted from the real ones within a month — so the tour drives the real pages and this cuts the wire.

It works because every write in `/admin` goes through `window.fetch` to `/api/admin/*`: there are no server actions under `app/admin` except the two logged-out password pages, no client-side Supabase writes, and no `XMLHttpRequest` anywhere. So swapping `window.fetch` is a complete boundary, not a best-effort one.

- **GET** passes through to the real API, then overlays this session's phantom mutations onto the JSON — which is what makes a "saved" product appear in the list a moment later. Two shapes cover everything: an array of `{ id }`, and a singleton object (banner, popup, featured event — see `SINGLETONS`).
- **POST/PUT/PATCH/DELETE** are never sent. They're recorded in memory and answered with a synthesised `Response` in the shape the real route returns.
- `*/upload` is answered with an object URL for the file the user picked, so the preview looks right and nothing reaches storage.
- A GET for a `demo-` id is answered from memory, so the editor the tour navigates into after "create" doesn't 404 on a row that was never real.

Two independent layers. Entering demo mode also sets a `dc-demo-mode` cookie, and `middleware.ts` rejects every non-GET to `/api/admin/*` while it is present. The interceptor means the request never gets that far; the cookie is what catches it if the interceptor is ever bypassed. The cookie is cleared on exit, on `beforeunload`, and again on every admin mount where no tour is running — a tab closed mid-tour must not leave an admin unable to save.

`lib/useAdminList.ts`

The hook behind every searchable admin list: fuzzy search, status filters with live counts, optional sorting, and all three mirrored into the query string.

Before it, none of the admin lists had search at all — finding one redirect among a hundred, or one order in a year, meant Cmd-F over whatever the browser had rendered.

- **Fuzzy, not `includes()`.** Uses Fuse.js (already a dependency for Smart Search) at `threshold: 0.4`, `ignoreLocation: true`. Typo tolerance matters here — "aplha" finds Alpha, "obrien" finds O'Brien — and matches are usually mid-string, in a long product name.
- **Client-side on purpose.** Every one of these endpoints already returns its whole collection in one request, so filtering in the browser is instant and needs no new API surface. If a collection outgrows that (orders, most likely), move it behind `/api/admin/search`.
- **Counts are taken against the searched set, not the filtered one**, so the number on each chip tells you what clicking it would show.
- `useRowSelection` is separate, so lists that don't need bulk actions pay nothing. It filters dead ids on read rather than pruning them in an effect — deleting a selected row would otherwise leave a phantom in the count. The `useRowSelection` + `BulkBar` pattern (first used on `app/admin/posts/page.tsx`) now also drives bulk actions on the HR staff, applications and jobs pages, the training categories and sub-groups pages, and the store products and orders pages — each reusing its existing per-row `[id]/route.ts` via `Promise.allSettled` rather than any new bulk API endpoint.

Sorting and drag-reorder don't mix. On the training pages a row's position *is* the public ordering, so dragging inside a filtered view would write a meaningless `sort_order`. Those pages hide the drag handles while a search or filter is active and say so.

`lib/useAdminSession.ts` / `lib/adminRecents.ts` / `lib/csv.ts`

`useAdminSession` also carries the **role preview**: a super admin can ask to see the admin as one of the other access levels, to check what a new Store Admin will actually be handed. The override is applied inside the hook rather than in each consumer, so the sidebar, the header, the dashboard grid and the ⌘K palette all narrow with no changes of their own. `roles` becomes the effective flags and `actualRoles` keeps the truth.

It is presentation only, and the preview bar says so. Middleware still sees a real super admin and will still open anything they type into the address bar — this narrows what the interface *offers*, which is the question being asked. It must never be treated as a security boundary.

- `useAdminSession` — the signed-in admin's email and roles from `/api/admin/me`, with the promise cached at module scope. The sidebar, header and palette all need it; without sharing, mounting the palette would have added a second and third identical request per navigation.
- `adminRecents` — the last six admin paths you opened, in `localStorage`. Feeds the dashboard's "Jump back in" strip and the palette's empty state. **Paths only, never record contents**, so nothing sensitive is left behind on a shared office machine.
- `csv.ts` — `toCsv` + `downloadCsv` for the orders export. RFC 4180 quoting (doubled quotes, CRLF rows) and a UTF-8 BOM, without which Excel on Windows mangles the £ in every price column.

Admin helpers (`lib/adminUpload.ts` , `lib/useAdminLoader.ts` , `lib/useKeyboardInset.ts`)

- `lib/adminUpload.ts` — `uploadPostImage(file)` posts to `/api/admin/posts/upload` (auto-rotate from EXIF, resize to max 1600px, re-encode to WebP q82, `post-media` bucket). The one implementation, shared by `RichTextEditor` 's toolbar and the block inspector's image field. `UPLOAD_ACCEPT` and `UPLOAD_MAX_BYTES` are **re-exported from** `lib/ai/media-types.ts` , the route's own constants, not restated: the hand-kept copies had already drifted — the accept attribute omitted `image/jpg` , which the server allowed, so the file dialog blocked a file the API would have taken.
- `lib/useAdminLoader.ts` — the loading/error lifecycle for an admin page that fetches on mount and re-fetches after mutations. The fetcher only fetches and sets its own state; clearing the spinner and surfacing the failure are the hook's job, on every path. Every HR page had grown its own copy and they disagreed on failure: half used `try/finally` , which cleared the spinner but left the page silently empty; the other half awaited a bare `Promise.all` , so one non-OK response (an expired admin session returning HTML, say) made `res.json()` throw, `setLoading(false)` never ran, and the page spun forever. `fetchAdminJson` / `fetchAdminArray` check `res.ok` first so the hook gets an error worth logging. Cancels in flight on unmount.
- `lib/useKeyboardInset.ts` — how many pixels the on-screen keyboard covers at the bottom of the window, from `visualViewport` . `position: fixed` resolves against the *layout* viewport, which iOS Safari does not shrink for the keyboard, so bottom-anchored sheets and toolbars would otherwise sit underneath it. Thresholded at 120px and rounded: URL-bar collapse and pinch-zoom also move the visual viewport, and an unrounded value hands `useSyncExternalStore` a new snapshot on every scroll frame.

Render-timing hooks (`lib/useHydrated.ts` , `lib/useMediaQuery.ts` , `lib/useIsDesktop.ts` , `lib/useScrollLock.ts`)

All of these exist because the obvious `useState` + effect version is a synchronous setState inside an effect — a cascading render, and one the React lint rules reject.

- `lib/useHydrated.ts` — false on the server and the hydrating render, true after. Guards `createPortal(…, document.body)` . `useSyncExternalStore` with a `false` server snapshot and a `true` client snapshot, so React swaps them at hydration with nothing to subscribe to. This absorbed `useIsClient.ts` , which was the same hook written twice, and the fourteen hand-rolled `const [mounted, setMounted] = useState(false)` pairs scattered across the site.
- `lib/useMediaQuery.ts` — any CSS media query, read synchronously and kept subscribed. Matters most where a component *branches its tree* on the result: reading it in an effect mounts the false branch first and then swaps to a completely different tree, unmounting everything inside it.
- `lib/useIsDesktop.ts` — the ≥1024px breakpoint, over `useMediaQuery` . The admin editors branch their whole layout on this, and the effect version mounted the mobile tree first and then swapped, silently destroying and recreating the TipTap instance (and its undo history) on every open.
- `lib/useScrollLock.ts` — holds `document.body` still while active. Body scroll is a single global that a dozen components want to own at once, and the hand-rolled copies fell into two traps: most set `overflow = ""` on the way out instead of restoring what was there, and none of them counted. `Sheet` nests inside `Modal` and both locked — on a shared unmount, child cleanups run first, so `Sheet` restored `""` and `Modal` then restored the `"hidden"` it had captured *from* `Sheet` , leaving the page permanently unscrollable. Counting the locks fixes both: the first saves the real previous value, only the last restores it.

`lib/accessibilityPrefs.ts`

The single owner of the Glass FX / reduced-motion preferences. They are read in three places that each used to parse the stored JSON themselves — the blocking script in `app/layout.tsx` (which must stay inline so the dataset attributes land before first paint), `AccessibilityContext` , and `PerformanceGate` .

It is an external store rather than component state because the value exists before React does: the inline script has already applied it to `<html>` by the time hydration starts. `useSyncExternalStore` lets the provider read it during the first client render while still handing SSR the defaults, so the hydrating markup matches the server's.

`readStoredPrefs()` returns **null** when the visitor has never stored a choice, and that distinction is the point: `PerformanceGate` treats "nothing stored" as permission to apply its own device heuristics. The old provider wrote the defaults to `localStorage` on every mount, so `destiny-a11y` came to mean "a page was loaded" rather than "the visitor chose a setting". Preferences are now persisted only when a setter actually runs.

`lib/embedLoading.ts`

The stage timings behind `ui/EmbedLoadingOverlay` (see Components), kept apart from the component so the boundaries are plain data and can be unit-tested without a browser — `tests/unit/embed-loading.spec.ts` .

`stageIndexAt(elapsed)` picks the line to show, and `msUntilNextStage(elapsed)` says how long until the next one. Both derive from elapsed milliseconds rather than counting ticks, because browsers clamp timers in a backgrounded tab: a counter would walk through every stage it slept through, one per wake-up, whereas this lands straight on the right line. `msUntilNextStage` returns `null` on the last stage, which is how the component knows to stop scheduling.

`lib/serviceTimes.ts`

Sunday service times in the church's own timezone. The site is served from UTC infrastructure and read on phones set to anything, but "11am" always means 11am in Stockton-on-Tees — so this resolves London wall-clock times to real instants rather than trusting the runtime's local zone. `Intl` only: no date library, and no hardcoded BST/GMT switchover dates to go stale.

- `nextSundayService(from?)` — the start of the next Sunday 11am service. Returns *today's* service right up to the 12:30 finish, so someone watching the stream isn't told the next one is a week away. Steps forward from London noon rather than from `from` , because adding raw 24h multiples across

a DST boundary can shunt the result onto the Saturday.

- `formatServiceDay(date)` — "Sunday 14 September" in London's calendar.
- `formatCountdown(target, from?)` — "2 days 4 hours" → "18 minutes"; null once the target has passed.

Covered by `tests/unit/service-times.spec.ts`, which pins both DST boundaries and the mid-service behaviour.

`lib/iconNames.ts` / `scripts/sync-icon-names.mjs` — icon font subset

The Material Symbols Rounded font is ~410 KB in full. `app/layout.tsx` requests it from Google Fonts with `&icon_names=…` built from `ICON_NAMES`, which brings it down to ~40 KB (PageSpeed flagged the full font as the page's largest and render-blocking resource).

- **Generated, never hand-edited.** `scripts/sync-icon-names.mjs` intersects every quoted string literal and bare JSX text under `app/`, `components/`, `lib/`, `contexts/`, `utils/` and `packages/shared/src` with the real Material Symbols names in `scripts/material-symbols-names.txt`, then writes the sorted list (Google requires alphabetical order). It deliberately over-collects — a stray `"home"` string adds the `home` glyph for ~100 bytes — so dynamic icons (`{item.icon}` fed from config arrays, ternaries) are caught without parsing the code.
- **Runs automatically** as `predev` and `prebuild`, so a new `<Icon name="…">` just works. By hand: `npm run icons:sync`. Refresh Google's name list (new icons released): `npm run icons:sync -- --refresh`.
- **Admin-entered icons can't escape the list.** `IconField` (block editor + links blocks) searches `ICON_NAMES` and has no free-text entry; the NFC tile and training `IconPicker` lists are literals in code, so they're collected automatically.
- **Gotcha:** if an icon only ever exists in the database (not in any picker or code), it would drop out of the subset. Every current DB value (`nfc_tiles.icon`, `training_categories.icon`, `link_blocks.data.icon`) is in a picker list.

`lib/alphaSession.ts`

`getNextAlphaSession(startDate, frequency, customIntervalDays?, now?)` — the next session of a course (Alpha, 12:2, Bible course, CAP) as a local-midnight `Date`. Works on calendar days: a date-only `start_date` ("2026-09-30") is parsed as that day, not UTC midnight (which is the previous day west of UTC), and "today" is London's date. Before this, the site banner showed US visitors the wrong weekday and threw React hydration error #418. Covered by `tests/unit/alpha-session.spec.ts`, which runs each case in four timezones.

`lib/maps.ts`

Opening the address in whatever map app the visitor's device has. The address itself is **not** defined here — `lib/churchInfo.ts` owns it and this derives from it. What lives here is the part `churchInfo` has no opinion on: which map *app* to send someone to.

`churchInfo`'s `MAPS_URL` and `DIRECTIONS_URL` are both Google, which is the right default and the wrong answer on Apple hardware, where Google Maps often isn't installed and a `google.com` link strands the visitor in a browser instead of the map app they actually use. So:

- `googleMapsUrl(query?)` — `https://www.google.com/maps/search/?api=1&query=…`. The cross-platform default: a plain https link, so it works on desktop, and Android/iOS hand it to the Google Maps app when installed.
- `appleMapsUrl(query?)` — `https://maps.apple.com/?q=…`. The right answer on Apple platforms. Degrades to a web map elsewhere, so it's never a dead end.
- `isApplePlatform()` — browser-only user-agent check. Also tests `navigator.maxTouchPoints > 1`, because iPadOS reports itself as "Macintosh".
- `deviceMapsUrl(query?)` — Apple URL on Apple devices, Google everywhere else (including the server, where `navigator` is undefined).

`DESTINY_CENTRE_MAP_QUERY` is what every builder here defaults to: `` `${ADDRESS.street}, ${ADDRESS.locality}, ${ADDRESS.postcode}` `` — the postal address, **without** `ADDRESS.venue`. That omission is deliberate and is the whole point of the module. Hand a map app "Destiny Centre" and it searches for a place by that name and can land anywhere; hand it a street number and it geocodes to the building. The UI still prints the venue name — see `components/MapsLink.tsx` under Components, which also covers why the accessible name has to be built from the visible text rather than from the query.

Because the query is built from `churchInfo`'s `ADDRESS` rather than typed out, editing the address in one place moves the map pin too.

`lib/churchInfo.ts`

The address, phone, email and Sunday schedule, in one typed place — display strings (`SCHEDULE.mainServiceStart` = `"11:00am"`) alongside 24-hour equivalents (`SCHEDULE.iso.mainServiceStart` = `"11:00"`) for JSON-LD. Also `ADDRESS_ONE_LINE`, `ADDRESS_LINES`, `MAPS_URL`, `DIRECTIONS_URL`, `PARKING_NOTE`, `BUS_NOTE`, `VISIT_FACTS`, `ACCESSIBILITY`.

Before this file, the same handful of facts were retyped independently in the JSON-LD in `app/layout.tsx`, `/visit`, `/contact`, `/help`, and the `CHURCH_FACTS` prose block in `lib/siteKnowledge.ts` (Smart Search's source of truth) — and they had drifted: the JSON-LD `Organization` schema listed a different support email than every other page on the site.

Deliberately scoped to "when, where, what to expect" — it does **not** absorb the charity/company registration numbers, leadership names or mission statement out of `CHURCH_FACTS`. Those have a different canonical source (`/governance`, backed by the Charity Commission / Companies House registers), so they're still edited directly in `siteKnowledge.ts`. `CHURCH_FACTS` quotes `churchInfo` via template literals for the facts it does own; `tests/unit/church-info.spec.ts` pins the internal consistency (schedule in chronological order, the phone formats agreeing, `CHURCH_FACTS` genuinely containing the same strings rather than a hand-typed copy that happens to match today) and — deliberately — asserts the support email equals `admin@destinytees.uk`, the exact drift this file exists to prevent.

Consumed by `app/layout.tsx`'s JSON-LD, `app/contact/page.tsx`, `app/help/page.tsx`'s two schedule/location FAQ answers, `components/ChurchFooter.tsx`, `components/home/ServiceTimesBar.tsx`, and `app/visit/page.tsx`.

`lib/useCourseEvents.ts`

Client-side polling for `/api/alpha-events`, factored out of Alpha, The Bible Course, Destiny 12:2 and CAP Money — each page previously carried its own copy of the same fetch/filter/loading-state effect, differing only in which event `type` it kept.

`useCourseEvents(matchesType)` fetches once on mount, keeps rows where `matchesType(event) && event.active !== false`, and returns `{ events, loading }`. A page passes its own filter, e.g. `(e) => e.type === "cap"`, or — Alpha's one real variation, since it runs Alpha and Youth Alpha through the same feed — `(e) => e.type === "alpha" || e.type === "youth_alpha"`.

`summarizeCourseSession(event)` wraps `lib/alphaSession.ts`'s `getNextAlphaSession` into the `subtitle` string every course's `ChurchSuiteModal` shows ("Starting 12 January 2026" / "Next session 19 January 2026"), so that formatting lives in one place instead of four.

Paired with `components/courses/CourseEventCard.tsx` for the "when and where" card each of the four pages renders per event — see *Course pages* under Components for why the rest of each page stayed separate rather than becoming one shared template.

`lib/useFocusTrap.ts`

Focus trap + Escape + focus-restore for a dialog that can't hand its markup to `components/ui/Modal.tsx` wholesale — `Modal.tsx` owns its panel's mount lifecycle and unmounts the instant `open` goes false, with no room for an exit transition. `components/give/TextToGiveCTA.tsx`'s panel needs its own 350ms scale/fade-out, so it keeps its own markup and takes just the trap logic via this hook. `Modal.tsx` carries the same logic inline; not deduplicated onto this hook since it isn't broken, only *also* correct.

Shop (`lib/shop*.ts`, `lib/stripe.ts`, `lib/cart-store.ts`)

- `lib/shop.ts` — client-safe types (`Product`, `ProductVariant`, `Order`, `CartItem`), `formatPrice(pennies)`, `variantPrice`, `fromPrice`, `totalStock`. Prices are integer pennies (GBP).
- `lib/shop.server.ts` (`server-only`) — public read fetchers: `getPublishedProducts()`, `getProductBySlug()`, `getAllProductsAdmin()` (via `createServiceClient()`).
- `lib/stripe.ts` (`server-only`) — `getStripe()` singleton from `STRIPE_SECRET_KEY`.
- `lib/cart-store.ts` — `useCart` zustand store persisted to `localStorage` (`destiny-cart`), plus `cartCount` / `cartSubtotal` helpers.

`lib/youtube.ts`

`CHANNEL_HANDLE` / **`CHANNEL_VANITY`** / **`CHANNEL_URL`** — the church's YouTube channel, in one place. The channel answers on **two different names** and they are not interchangeable:

Form	Value	URL
`@` handle	`DestinyOnlineChurch`	`youtube.com/@DestinyOnlineChurch`
Custom URL	`destinychurchteesvalley`	`youtube.com/destinychurchteesvalley`

`CHANNEL_URL` is the custom-URL form and is what every public link uses (sermons platform chips, `WatchOnYouTubeBand`, `LatestSermonSection`, the org schema's `sameAs`). Mixing the two produced `@DestinyChurchTeesValley` — neither name, and a dead link — which is what the org schema and the `/help` FAQ both used to point at.

Deliberately plain constants rather than env reads: `lib/youtube.ts` is reachable from client components, and a non-`NEXT_PUBLIC_` variable is `undefined` in the browser bundle, which would hydrate a different `href` than the server rendered. Live detection alone may override them server-side via `YOUTUBE_CHANNEL_HANDLE` / `YOUTUBE_CHANNEL_VANITY`.

`getUploadedVideos(pageToken?)` — the /sermons archive grid's data source. Pages the channel's uploads playlist (`"UU" + CHANNEL_ID.slice(2)`) via `playlistItems.list` (1 unit/50 videos) rather than `search.list` (~100 units/call), then filters to items with `liveStreamingDetails` so only actual broadcasts show up. See the `/sermons` write-up above for the full quota story. `getAllVideos()` below still exists and is still `search.list`-based — `/api/youtube/videos` and the mobile app's sermons endpoint still use it; nothing else should.

`getLatestVideo()` (redefined September 2026) delegates to `getUploadedVideos()` and returns its first result, rather than its own independent `search.list?eventType=completed` query — the two could disagree, which meant the featured card on `/sermons` could show a different video than the top of its own archive grid.

`getFullSermonArchive()` (added September 2026) pages `getUploadedVideos()` to exhaustion (capped at 1000 videos defensively) so the archive's speaker/ month filters and the sitewide Smart Search chat's `find_sermons` tool can search or facet across every sermon rather than one loaded page. Every `YTVideo` also now carries `speaker: string | null`, parsed from the title by `lib/sermonTitle.ts`'s `parseYouTubeTitle()` in a single shared `toYTVideo()` constructor used by every function on this page that builds one.

`getPlaylistVideoIds(playlistId)` / **`getGuestSpeakerVideoIds()`** (added September 2026) — returns the `Set` of video ids in a playlist, fetching only `part=contentDetails` (skips the snippet fields `getPlaylistVideos()` pulls, since only membership matters). `getGuestSpeakerVideoIds()` wraps it for `GUEST_SPEAKERS_PLAYLIST_ID`, backing the archive grid's "Guest speakers only" filter. Fails open to an empty set so a YouTube outage hides the filter rather than breaking the page.

`getPlaylistSnippet(playlistId)` (added September 2026) — a playlist's own title/description/thumbnail via `playlists.list` (`part=snippet`). Backs the Series feature: it's both how `lib/sermonSeries.server.ts` resolves a configured playlist's display name, and the existence check the admin "add series" API uses before inserting a row — a `null` return means "not found or private" either way.

```
// Wrapper around YouTube Data API v3
export async function getAllVideos(maxResults = 50): Promise<YTVideo[]> {
  const youtube = google.youtube({
    version: "v3",
    auth: process.env.YOUTUBE_API_KEY
  });

  // Search for videos in our channel
  const { data } = await youtube.search.list({
    part: "snippet",
    channelId: process.env.YOUTUBE_CHANNEL_ID,
    maxResults,
    order: "date",     // Newest first
    type: "video"
  });

  // Transform to YTVideo type with metadata
  return data.items.map((item) => ({
    id: item.id.videoId,
    title: item.snippet.title,
    description: item.snippet.description,
    thumbnail: item.snippet.thumbnails.high.url,
    publishedAt: item.snippet.publishedAt,
    // ... extract speaker, series from description/title
  }));
}

export async function getLatestVideo(): Promise<YTVideo | null> {
  const videos = await getAllVideos(1);
  return videos[0] ?? null;
}

// Livestream detection — the *real broadcast* half. Zero-quota on the happy
// path; escalates only when scraping is inconclusive. Callers want
// getLiveStatus() from lib/liveStatus.server.ts, which stamps serverTime on it.
export async function getYouTubeLiveStatus(): Promise<LiveStatus> { /* ... */ }
```

How `getYouTubeLiveStatus()` decides, in order:

1. **Scrape the `/live` URL**, trying all three forms of the same channel in order: `youtube.com/channel/{id}/live`, `youtube.com/@DestinyOnlineChurch/live`, `youtube.com/destinychurchteesvalley/live`. The handle and custom URL are the safety net for a missing or stale `YOUTUBE_CHANNEL_ID`, which previously made detection fail silently and permanently. A definitive verdict from any one form breaks the loop — the later forms are only tried when the earlier one came back blocked or unreadable, so the offline path stays at exactly one fetch.
2. **Parse properly.** `parseLivePage()` brace-matches the `ytInitialPlayerResponse` blob out of the HTML and reads `videoDetails.isLive` / `liveBroadcastDetails.isLiveNow` from *that object*. The old code regex-matched `"isLiveNow":(true|false)` anywhere in the page — and YouTube emits `ytInitialData`, full of recommendation shelves each carrying their own `isLiveNow`, **before** the player response. The first match was routinely an unrelated video's flag, so real broadcasts were scored offline. It also never distinguished `isLiveContent` ("was a broadcast at some point", true of every past Sunday) from `isLive` ("streaming right now").
3. **A clean "no" costs nothing.** A watch page with an `endTimestamp`, or a redirect to the channel home, returns `offline` outright — that is the path taken every minute of every day the church isn't streaming, and it must never spend quota. All four channel-home canonical forms (`/channel/UC…`, `/@handle`, `/c/name`, bare custom URL) are recognised; treating any of them as unreadable would escalate to an API call on every check.
4. **Inconclusive results escalate** — consent wall, bot check, or an unrecognised shell. First the free second opinion: `embed/live_stream?channel={id}`, a few KB rather than a few MB and not consent-gated. Then the RSS feed's recent uploads (also free). Then **one** `videos.list` call over the collected candidate ids (1 unit) for YouTube's own `liveBroadcastContent === "live"` verdict.
5. **Never `search?eventType=live`** — 100 units a call would burn the entire 10,000/day quota inside an hour of polling, taking the sermon pages down with it.

Caching. The scrape uses `cache: "no-store"` and `getYouTubeLiveStatus()` memoises the result in-process instead (30s while live, 60s otherwise, with concurrent callers sharing one detection pass). A watch page is several MB — past the 2MB ceiling on Next's fetch data cache — so the previous `next: { revalidate: 60 }` was caching nothing and re-scraping YouTube on every render of every page, which is both slow and enough traffic to earn a bot check. On a detection failure the last known answer is kept rather than yanking a running stream off the page.

Fails closed. Any unexpected state, or `LIVE_DISABLED=1`, returns `{ live: false }`.

Why Supabase Storage isn't used for videos: - YouTube handles CDN/caching automatically - Bandwidth costs for video are lower on YouTube - YouTube playlist/channel management features - Comments, engagement metrics built-in

---

`lib/speakerOverrides.server.ts` — speaker-corrected YouTube reads

Thin wrappers around `lib/youtube.ts` (`getFullSermonArchive`, `getUploadedVideos`, `getVideo`, `getLatestVideo`, `getAllVideos`) that overlay the `speaker_overrides` table onto each returned video. Every video-serving path in the app reads through **this** module, not `lib/youtube.ts` directly, so a stored speaker correction takes effect everywhere at once and `lib/youtube.ts` stays a pure YouTube client with no Supabase dependency. Only overridden ids are replaced — a video with no override row keeps its regex-parsed speaker untouched. See the `speaker_overrides` table for the null-vs-no-row distinction.

`getSpeakerOverrides(ids)` (added September 2026) — the admin-only sibling of the internal `fetchOverrides()` above: returns the raw override rows (`speaker` *and* `reviewed_by`, not just the resolved speaker value) so `components/admin/SpeakerEditor.tsx` can tell an AI-set correction apart from a

manually-set one and know whether its Revert button has anything to delete. Public read paths keep using the plain wrapper functions above; this is only for the admin editor's own state.

`lib/sermonSeries.server.ts` — playlist-backed sermon series

The `sermon_series` (Database Schema §27) read side, mirroring `lib/speakerOverrides.server.ts`'s shape: the DB only ever holds a `playlist_id` pointer, everything else is resolved live against YouTube.

```
export async function getSermonSeriesRows(): Promise<SermonSeriesRow[]>   // raw DB rows, no YouTube calls
export async function getSermonSeriesList(): Promise<SermonSeries[]>      // DB rows + live title/description/videoIds
```

`getSermonSeriesRows()` is what the admin `SeriesManager` list reads, so a playlist that's gone private or been deleted still shows up (as "Unresolved") and can be removed. `getSermonSeriesList()` is what `app/sermons/page.tsx` reads for the public Series filter — it resolves each row against `getPlaylistSnippet()` and `getPlaylistVideoIds()` in parallel and silently drops any playlist that fails to resolve, the same fail-open posture as `getGuestSpeakerVideoIds()`, so one bad playlist can't take the whole page down.

`lib/speakerReview.server.ts` — AI speaker review

Reviews the sermon archive's speaker attribution with the LLM (`SMART_SEARCH_MODEL`) and persists corrections to `speaker_overrides` . Triggered from `/admin/sermons` via `POST /api/admin/sermons/review-speakers` .

```
export async function reviewSermonSpeakers(opts?: { force?: boolean }): Promise<SpeakerReviewResult>
```

- Fetches the full archive, then by default **skips** videos that already have an override row, so a re-run only costs newly published sermons. `force: true` reviews everything again.
- Batches the remaining videos (20 per LLM call) and runs a few batches concurrently (`CONCURRENCY = 3`) to stay wide of OpenAI rate limits without one giant sequential queue. Each call gets the title, the regex-parsed guess, and a 300-char description excerpt, and returns strict JSON.
- The system prompt keeps a good guess, corrects obvious garbage (a date, `"FULL SERVICE"`), returns `null` for genuinely speaker-less videos, and prefers the existing guess over inventing a name when unsure.
- Returns `{ reviewed, changed[], skipped, errors }` ; `changed` only lists videos whose speaker actually moved (compared via `normalizeSpeakerName`). The route records the run in the audit log. Node runtime, `maxDuration = 300` — a full first pass is a few dozen LLM calls.

`lib/liveStatus.server.ts` — the composed "are we live?"

The one function the site should ask. A thin wrapper over `getYouTubeLiveStatus()` that stamps `serverTime` on the answer. The banner, the homepage status line and the mobile app's BFF all read it.

```
export async function getLiveStatus(): Promise<LiveStatus>
```

(Before the on-site `/live` page and chat were retired this also layered a "simulated" broadcast underneath the real one; that is gone.)

`lib/smartSearch.ts`

Parses the LLM response and validates navigation:

```
export interface SmartSearchResult {
  answer: string;              // Main prose answer
  page: string | null;         // Valid page URL (or null)
  ctaLabel: string | null;     // Button text
  options?: string[];          // For clarifying questions
}

export function parseAnswer(raw: string): SmartSearchResult {
  // Extract OPTION: lines (clarifying question choices)
  const options = Array.from(
    raw.matchAll(/^\s*(?:[-*•]\s*)?OPTION:\s*(.+?)\s*$/gim)
  )
    .map((m) => m[1])
    .slice(0, 4);  // Max 4 options

  // Strip OPTION/PAGE/CTA lines from prose
  const prose = raw
    .replace(/^\s*OPTION:\s*.+$/gim, "")
    .replace(/^\s*PAGE:\s*\S+/gim, "")
    .replace(/^\s*CTA:\s*.+$/gim, "")
    .trim();

  // If options present, this is a clarifying question
  if (options.length >= 2) {
    return { answer: prose, page: null, ctaLabel: null, options };
  }

  // Extract PAGE and CTA from end of response
  const pageMatch = raw.match(/^\s*PAGE:\s*(\S+)\s*$/im);
  const ctaMatch = raw.match(/^\s*CTA:\s*(.+?)\s*$/im);

  let page: string | null = null;
  if (pageMatch) {
    const candidate = pageMatch[1].replace(/[.,)"']+$/, "");
    // Validate against allowlist (prevents hallucinated links)
    if (ALLOWED_PAGES.has(candidate)) page = candidate;
  }

  let ctaLabel: string | null = null;
  if (page && ctaMatch) {
    ctaLabel = ctaMatch[1].trim();
  }

  return { answer: prose, page, ctaLabel };
}

export const FALLBACK_ANSWERS = {
  tooShort: "Ask me anything about Destiny...",
  unavailable: "I can't reach the assistant right now...",
  empty: "I'm not totally sure on that one..."
};

export function cooldownAnswer(): SmartSearchResult {
  // User hit rate limit
  return {
    answer: "You've made a lot of searches in a short time...",
    page: "/contact",
    ctaLabel: "Contact Us"
  };
}
```

Key Security:

1. **Allowlist validation** — Page URLs must be in `PAGE_INTENTS`; any hallucinated links are dropped
2. **Rate limiting** — Max 5 searches per IP per minute
3. **Fallback answers** — If service unavailable, still show a friendly response (never "error")

`lib/siteKnowledge.ts`

Single source of truth for the AI assistant. Its system prompt also includes:

- **Charity/company identity guardrail:** the church's registered charity number (1119951) and company number (06261423), plus an explicit warning that other unrelated organisations share the "Destiny Church" name (notably a Scottish charity, SC017898) — the model is told to confirm any record it reads matches before quoting a figure from it.
- **Widened, tool-first web search guidance:** the model is told to prefer calling `search_web` / `extract_page` over guessing or deflecting for real-world or financial questions about the church (UK charities publish their accounts), reserving the `/contact` fallback for genuinely unpublished internals (staff pay, member data) or requests with no connection to Destiny at all.

```
export const PAGE_INTENTS = [
  { href: "/give", cta: "Give Now", intent: "giving, donations, bank details..." },
  { href: "/visit", cta: "Plan Your Visit", intent: "visiting, first time..." },
  { href: "/sermons", cta: "Watch Sermons", intent: "sermons, messages..." },
  { href: "/shop", cta: "Browse Merch", intent: "shop, apparel, merch, buy" },
  { href: "/help", cta: "Help Centre", intent: "help, FAQ, questions" },
  // ... 17 more pages (kids, youth, Alpha, serve, connect, missions, etc.)
];

export const ALLOWED_PAGES = new Set(PAGE_INTENTS.map(p => p.href));

// System prompt sent to OpenAI (truncated here):
const SEARCH_INSTRUCTIONS = `
You are a friendly assistant for Destiny Church Tees Valley.

HOW TO REPLY:
- Write 2–5 conversational sentences
- Never restart the question or use full church name
- Append PAGE: and CTA: tags ONLY if relevant:
  PAGE: /the-path
  CTA: Short Action Label

GROUNDING (MOST IMPORTANT):
- Only state facts in the KNOWLEDGE section below
- Do NOT guess, invent, or assign roles unless listed
- If you don't know, warmly say so and point to /contact

CLARIFYING QUESTIONS:
- If a request is ambiguous, ask ONE short question with 2–4 OPTION: choices
- Do NOT ask follow-ups for simple factual questions

KNOWLEDGE:
... (church-specific facts: pastor names, service times, livestream, shop, mission, etc.) ...
`;
```

Why this pattern?

1. **Prevents hallucination** — Model is grounded in actual church knowledge
2. **Limits links** — Only allows pages we've defined
3. **Consistent tone** — Friendly, warm, not corporate
4. **Scalable** — Easy to update facts without retraining

`lib/linkPages/*` — links pages

```
// types.ts — client-safe. One zod schema per block type (BLOCK_DATA_SCHEMAS),
//   LinkBlockSchema (discriminated union + schedule check), LinkPageSchema,
//   SLUG_RE, MAIN_SLUG ('main' = /links), BLOCK_META, defaultBlockData(), blockLabel().
// theme.ts — client-safe. ThemeSchema: every field defaults (the defaults ARE
//   Destiny Light, so '{}' is a finished theme) and every field `.catch()`es, so
//   one bad value falls back alone instead of failing the page. THEME_PRESETS
//   (7), FONT_OPTIONS allowlist, themeToCssVars(), readableOn() (hover text: white, as on /help;
//   near-black only on a very light accent). Themes never carry raw CSS:
//   colours/gradients are regex-checked and url( is refused.
// urls.ts — safeHref() (http(s)/mailto/tel/site path; no javascript:, no //host,
//   no bare #), safeMediaUrl() (https or site path), toEmbed() — an allowlist:
//   YouTube (nocookie), Vimeo (dnt), Spotify, Apple Podcasts, Google Maps
//   *embed* URLs, ChurchSuite.
// socials.ts — SOCIAL_PLATFORMS (simple-icons paths + Material Symbols for
//   email/phone/website), normaliseSocialUrl().
// paths.ts — isLinksPagePath(), for the site-chrome gates.
// linkPages.server.ts — getLinkPage(slug) (noStore; schedule on the server
//   clock; event blocks resolved, finished/missing single events dropped),
//   fallbackMainPage(), listIndexableLinkPages() (sitemap), toPage()/toBlock().
// save.server.ts — validateSave(), the admin PUT's whole check.
```

`lib/eventTargets.server.ts`

Pointing something at a ChurchSuite event, shared by `/nfc` event tiles and links page event blocks so the two can't drift: `findSeries()` (identifier, then the more durable sequence), `seriesEndsAt()`, `framableSignupUrl()` (anchored to the form, or null), and `resolveEventTarget(identifier, sequence, { requireSignup })` for saves — `/nfc` requires a framable signup (a tile *is* the form), a links page event card doesn't.

`lib/nfcTiles.ts` / `lib/nfcTiles.server.ts`

The tile list behind `/nfc`, deliberately split in two.

```
// lib/nfcTiles.ts — client-safe: types, fixtures, pure helpers
export type NfcTileMode = "embed" | "info" | "event";
export const PINNED_TILES: NfcTile[];              // Connect Card, Giving — always first
export const SIGNUP_ANCHOR: string;                // "#form_event_signup"
export const NFC_TILE_COLUMNS: string;             // the explicit select list
export function isEmbeddable(url: string): boolean; // https, churchsuite.com or a subdomain of it

// lib/nfcTiles.server.ts — `import "server-only"`
export async function getNfcTiles(): Promise<NfcTile[]>;
```

Why the split: `/admin/nfc` is a client component and imports `PINNED_TILES`. Event resolution needs `getEventIndex()` from `lib/events.server.ts`, which is `server-only` — putting the two in one file would break the admin bundle. Keeping them apart also stops the service client being reachable from a client bundle at all.

`getNfcTiles()` is `noStore()` + service client, returns `[...PINNED_TILES, ...activeRows]`, and **catches everything** — a Supabase outage returns the two fixtures rather than a blank page, which matters because the page's whole audience is holding a phone in a service right now.

`mapRow()` re-checks `isEmbeddable()` on read, not just on write: a row that somehow holds a non-framable URL is demoted to `info` mode so the tile shows copy and a link instead of a dead white iframe. `isEmbeddable` is the same rule as `components/events/EventSignupButton.tsx` — only our own ChurchSuite subdomain permits framing.

Event tiles cost a feed fetch only when a row actually has `mode = "event"`. For each one, `resolveEventTile()`: - drops the tile when `event_ends_at` has passed (a *server-side* clock read — the same reason `EventsGrid` and `EventCard` are clock-free is why this can't move to the client); - looks the series up by identifier, then by `event_sequence` — ChurchSuite reissues occurrence identifiers when a series is edited, so the sequence is the more durable key; - on a hit, refreshes the signup URL, the slug and `event_ends_at` from the feed, and fills a blank subtitle with `formatKicker()` so the tile face carries the live date; - on a miss — feed down (`fetchChurchSuiteEvents` returns `[]` on any error) or the event pulled from ChurchSuite — keeps the stored snapshots, which is why the signup URL is snapshotted into `embed_url` at write time.

`lib/pageContent.ts`

Fetch dynamic content from the `page_content` table:

```
export async function getPageContent(page: string, key: string): Promise<string> {
  const supabase = createServiceClient();
  const { data } = await supabase
    .from("page_content")
    .select("value")
    .eq("page", page)
    .eq("key", key)
    .maybeSingle();
  return data?.value ?? "";
}

// Used by:
export async function getGivingDetails() {
  return Promise.all([
    getPageContent("give", "account_name"),
    getPageContent("give", "sort_code"),
    getPageContent("give", "account_number"),
    getPageContent("give", "text_keyword"),
    getPageContent("give", "text_number")
  ]);
}

export async function getServiceInfo() {
  return Promise.all([
    getPageContent("general", "service_day"),
    getPageContent("general", "service_time"),
    getPageContent("general", "service_address")
  ]);
}
```

Why key-value store?

1. **Editable by non-technical users** — No code changes needed
2. **Centralized** — All dynamic content in one table
3. **Cacheable** — Rarely changes; can be cached aggressively
4. **Typed access** — TypeScript ensures correct key names

> **Note:** Role logic lives in `lib/adminRoles.ts` (not `lib/roles.ts`) — five independent per-user booleans stored in the `admin_roles` table, enforced by `middleware.ts` on every `/admin/*` and `/api/admin/*` request. See Authentication & Authorization below for the route → role mapping.

`lib/smartSearch/tools.ts` — Smart Search tools

```
export interface ToolContext { seenUrls: Set<string> }
export function createToolContext(): ToolContext   // one per /api/chat request

export async function executeTool(name: string, rawArgs: string, ctx: ToolContext): Promise<ToolResult>
```

- `find_products`, `get_weather`, `get_directions` — unchanged from prior behavior (see Components → `FloatingSmartSearch.tsx`).
- `search_web` — Tavily `/search` with `search_depth: "advanced"` (curated chunks, not raw page tops) and the API key sent as an `Authorization: Bearer` header (not in the body). Snippets truncated to `SNIPPET_CHARS` (1200 chars). Every result URL is recorded into `ctx.seenUrls`.
- `extract_page` — Tavily `/extract`, `extract_depth: "advanced"`. **Only** opens a URL already present in `ctx.seenUrls` for that request — this is the sole guard against the model being prompted (by a visitor or by web content) into fetching an arbitrary URL. Content truncated to `EXTRACT_CHARS` (8000 chars).
- Deliberately does **not** request Tavily's own `include_answer` summary: it was observed conflating Destiny Church Tees Valley with an unrelated Scottish charity of a similar name and reporting a wrong income figure. The model instead reads raw snippets/page content and is told (via `lib/siteKnowledge.ts`) to cross-check the charity/company number before quoting any figure.

`lib/rateLimit.ts`

Per-IP sliding-window rate limiter with escalating cooldowns, **in-memory** (a `Map`, not a database table) — adequate for this site's traffic, per-instance on serverless. Callers namespace keys (e.g. `contact:${ip}`) so endpoints don't trip each other's limits:

```
const WINDOW_MS = 60_000;      // 1-minute sliding window
const MAX_PER_WINDOW = 15;     // requests allowed per window
const COOLDOWN_MS = [5, 10, 20, 40, 60].map((m) => m * 60_000); // escalating cooldowns per offense

const rlStore = new Map<string, { timestamps: number[]; cooldownUntil: number; offenses: number }>();

export function clientIp(source: { headers: Headers } | Headers): string { /* ... */ }
// checkRateLimit(key) reads/writes rlStore, returns { limited, retryAfterMs }
```

// Used by: // Form submissions (contact, hire, apply for job) // `/api/chat` (Smart Search) — separate simpler per-IP counter, 20 req/min // Login attempts (`lib/loginRateLimit.ts`, a related but distinct limiter)

`lib/training.ts`

Training course management:

```
export interface TrainingCategory {
  id: string;
  name: string;
  description: string;
  icon: string;
  order: number;
}

export interface TrainingModule {
  id: string;
  categoryId: string;
  title: string;
  description: string;
  content: string;  // Markdown or HTML
  duration: number; // Minutes
  videoUrl?: string;
  completed: boolean;
}

export async function getTrainingCategories(): Promise<TrainingCategory[]> {
  const supabase = createServiceClient();
  const { data } = await supabase
    .from("training_categories")
    .select("*")
    .order("order");
  return data ?? [];
}

export async function getUserCourseProgress(userId: string) {
  // Fetch completed modules for this user
  // Used to show progress bars and completion badges
}
```

`lib/events.ts` & `lib/events.server.ts`

`lib/events.ts` is client-safe: the `EventCardVariant` union, `parseCardVariant` for the preview routes' `?card=` param, and `RESERVED_EVENT_SLUGS` (currently `{"new"}`, because `app/whats-on/new/` statically shadows `[slug]`).

`lib/events.server.ts` is `server-only` and holds everything touching the feed or Supabase: - `getEventIndex()` — fetch + `buildEventIndex`, `revalidate: 300` - `getFeaturedEvent()` — merges admin overrides over live feed data, applies the promote window and auto-expiry, and handles the feed-outage snapshot fallback - `getActiveEventPopup()` — the same row projected for the popup; returning non-null is what suppresses the generic site popup

`lib/audit.ts` / `lib/audit.server.ts` / `lib/auditAI.ts` / `lib/auditEmail.ts`

The audit log, split four ways along the lines that actually matter:

- **`lib/audit.ts`** — the vocabulary, client-safe. `AUDIT_ACTIONS` and `AUDIT_SECTIONS` are closed sets so the filter chips can be built from them and a typo can't invent a section nothing will ever match. Also the diffing (`diffRecords` only considers keys the caller actually sent, so a three-field PATCH produces a three-field diff and `updated_at` never counts as a change), the redaction (`sanitiseValue`), and the presentation helpers the page and the detail panel share.

Times are church time, everywhere. `SITE_TIME_ZONE` is `Europe/London`, and `siteDateTime` / `siteDate` / `siteDayKey` / `siteDayBounds` all format and resolve against it. Rows are stored in UTC and Vercel runs in UTC, so anything formatted without naming a zone reads an hour early from late March to late October — which is exactly wrong for a record of *when* things happened. The zone is pinned to the church rather than the reader's device so the AI's answer, the list, the detail panel and the weekly email all say the same time as each other. The detail view names the zone on screen (`GMT+1`) because ambiguity there is the whole problem. `siteDayBounds` exists so a "today" filter covers today *here*: compared against UTC midnight, through BST it would quietly include the last hour of yesterday evening.

Nothing reaches a screen as a column name. `fieldLabel` (columns) and `humanise` (values) map the schema to words — `is_published` → "Published", `base_price_pennies` → "Price", `one_off` → "One-off", `super_admin` → "Super Admin", the role labels coming from the one `ROLE_LABELS` map in `lib/adminRoles.ts` that the users page shares. The two maps are separate on purpose: `body` is the column "Content", but "body" as a *value* is just the word. `formatValue` tidies a value only when it is plainly an identifier (lowercase words joined by underscores), so emails, URLs, slugs and prose are never mangled. - **`lib/audit.server.ts`** — `recordAudit()`, the only writer. Three rules it exists to enforce: it never throws (an audit write failing must not fail the thing the admin was doing), it never stores a secret, and it costs one insert — the actor comes from headers `middleware.ts` already set (`AUDIT_ACTOR_HEADERS`), so logging a change doesn't add an auth round trip to every save. `readForAudit()` is the "read it before you change it" helper that gives entries their before-state and their human label. - **`lib/auditAI.ts`** — the tools and prompt shared by the ask box and the weekly report. The model never sees the whole log: `search_audit_log` and `count_audit_log` run real queries and return bounded slices, so answers are grounded in rows and a year of history doesn't need to fit in a context window. **Timestamps reach the model already formatted** in church time, and the prompt tells it to quote them verbatim. It shipped once being handed raw UTC and asked to "write it naturally", and spent BST answering an hour early; a model can't get a conversion wrong that it is never asked to do. Bare `YYYY-MM-DD` filter arguments are resolved through `siteDayBounds` for the same reason. - **`lib/auditEmail.ts`** — the weekly report email. Same brand as `lib/hrEmail.ts` (orange rule, rounded card) but a different shape inside — a stat strip over a written report rather than labelled rows — so the two are deliberately not forced into one template. Includes a deliberately tiny markdown renderer: the output has to be inline-styled, table-safe HTML, which a general-purpose renderer doesn't emit.

> **Adding a section that writes to the database.** Call `recordAudit()` from the handler, with a `summary` written as a sentence naming the thing (`Added the product "Faith Hoodie" to the store`) — that sentence is what the search matches and the only thing the AI reads. Write it the way you'd say it: run enum values through `humanise()` and dates through a formatter, so a summary never reads `(one_off, 2026-09-14)`. `tests/unit/audit-coverage.spec.ts` fails the build if a mutating route under `/api/admin` neither logs nor carries an `audit-exempt: <why>` comment, because a log with holes in it is worse than no log: it teaches people to trust an answer that is silently missing the change they were looking for.

`lib/engagement.ts` / `lib/engagement.server.ts` / `lib/botDetect.ts` / `lib/track.ts` / `lib/vercelAnalytics.server.ts` / `lib/linksSteps.ts` / `lib/useEngagementRollup.ts`

The click-analytics feature behind `/admin/analytics` — the writers and shared vocabulary around the `engagement_events` table (see Database Schema §24), plus the whole-site traffic panel and the client hook the page's three tabs share. Modelled closely on the audit log, split the same way.

- **`lib/engagement.ts`** — the vocabulary, client-safe (touches neither the database nor `next/headers`). Closed sets so the page's filters can be chips: `ENGAGEMENT_SOURCES` (`redirect` / `nfc` / `links` / `links_view`, each with its own noun — "click" vs "tap" — icon and blurb), `SRC_TAGS` (`qr` / `nfc` / `print` / `social`, the `?s=` values that separate "scanned the flyer" from "clicked the post"), and `IP_CATEGORIES` (`apple_private_relay` / `vpn` / `tor` / `datacenter` — see Database Schema §24b; `ipCategoryLabel()` names Private Relay distinctly from "VPN" rather than lumping default-on iPhone privacy in with something that reads as suspicious). Re-exports `AUDIT_RANGES` as `ENGAGEMENT_RANGES` so the range chips match the audit log's exactly. Also the wire-format types (`EngagementRollup`, `EngagementBucket`, `TrackBeacon`) and the display helpers the page shares — `countryName`, `regionName` (UK regions read "England", not "ENG"), `referrerName` ("facebook.com" → "Facebook"), `compactNumber` (1200 → "1.2k").
- **`lib/engagement.server.ts`** — `server-only`. `recordEngagement()`, the only writer, with the same first rule as `recordAudit()`: **it never throws** — a failed analytics write must never break the redirect the visitor was actually following. `readRequestContext(headers)` reads everything off the request headers (Vercel geo, referrer, UA, IP, `visitor_hash`, prefetch detection) and returns a plain serialisable object, because the caller invokes `recordEngagement()` from inside `after()` where `headers()` throws — so the headers must be read during render and handed in. **Prefetches are dropped** (Next prefetches every `<Link>` in view; counting them would make the most linked-to slug top the chart for no reason). `visitor_hash` is warned about, never silently defaulted, when `ANALYTICS_HASH_SALT` is unset — an unsalted sha256 of an IPv4 is brute-forceable in seconds. `readCampaignParams()` pulls `?s=` and UTM params off a resolved `searchParams`.
- **`lib/botDetect.ts`** — the single most load-bearing piece: telling a person from a link-preview crawler. Dependency-free (a page of regexes, not a megabyte UA-parser) with the same instinct as `lib/cn.ts`. `isBot()` matches ~50 patterns — messaging-app preview fetchers first (WhatsApp, facebookexternalhit, Slack, Telegram…), then search/AI crawlers, monitors, and CLI agents, with generic catch-alls last; a missing UA counts as a bot. `detectDevice()` / `detectOs()` / `detectBrowser()` read a device off the UA, ordered around the fact that every browser lies (Chrome's UA contains "Safari", iPadOS reports as macOS, in-app Facebook/Instagram browsers claim to be both).

- **`lib/track.ts`** — client-side. `trackClick(source, targetKey, targetLabel)` for the two surfaces where the click happens in the page rather than as a server redirect (`/nfc`, `/links`). Uses `navigator.sendBeacon` (falling back to `fetch({ keepalive: true })`) so the report survives the navigation that fires it, and sends `text/plain` because `application/json` is not CORS-safelisted for beacons. **`redirect` is deliberately absent from `BeaconSource`** — a security boundary, not an oversight: shortlink numbers decide print spend, so they may only ever be written server-side, never from a browser anyone could `curl`. Stores nothing on the device — no cookie, no visitor id.
- **`lib/vercelAnalytics.server.ts`** — `server-only`. `fetchSitePanel(since, until)` wraps the Vercel Web Analytics REST API (public since May 2026) for the page's "Whole site" tab. Deliberately **not Google Analytics**: `@vercel/analytics` is already installed and already consent-gated (`AnalyticsGate.tsx`), so this reads data already being collected rather than standing up a second tracker. Returns a discriminated union — `{ ok: true, data }` or `{ ok: false, reason: "not-configured" | "auth" | "plan" | "error", message }` — because an empty chart could mean "nobody visited" or "the Vercel plan's reporting window doesn't reach back that far", and those need to read differently on screen. The five dimensions (`day`, `route`, `country`, `deviceType`, `referrerHostname`) are fetched with `Promise.allSettled`, so one dimension failing doesn't blank the rest; only the totals call failing takes down the whole panel. Cached via Next's fetch cache at `revalidate: 300`.
- **`lib/linksSteps.ts`** — client-safe. `LINKS_STEPS`, the six Next Steps `/links` used to hardcode. No longer what `/links` renders (that's `link_pages` now); kept as `fallbackMainPage()`'s content for when the database can't answer, and still accepted by `POST /api/track` as `links` targets because that fallback reports clicks against their hrefs. `links` clicks are otherwise keyed by `link_blocks.id`, and `links_view` rows (page views, keyed by `link_pages.id`) are written server-side by `components/links/LinkPageRoute.tsx`.
- **`lib/useEngagementRollup.ts`** — client hook. One fetch/loading/error effect behind every tab that reads `engagement_events` (`ShortLinksPanel` / `InPersonPanel`), rather than three copies of it. The fetch runs inside an async IIFE — including the first `setLoading(true)` — so every `setState` call happens inside a callback rather than synchronously in the effect body; a bare synchronous `setLoading(true)` at the top of the effect trips `eslint-plugin-react-hooks`'s `set-state-in-effect` rule on a component small enough for it to fully analyse (`app/admin/audit/page.tsx` uses the same shape but is large enough that the rule's analysis appears to give up on it — not a pattern worth copying).

`lib/governance.server.ts`

`server-only`. The single place the site talks to either regulator, powering `/governance`.

- `getGovernanceData()` — the only public entry point. Fetches Companies House and the Charity Commission concurrently and returns registration details, officers, filings, trustees, five-year financial history and annual-return submissions, plus a `sources` flag marking each regulator `"live"` or `"fallback"`.
- Exports `CHARITY_NUMBER` (1119951), `COMPANY_NUMBER` (06261423), the two public register URLs, and the `formatRegisterDate` / `formatCurrency` helpers the section components share.

Two rules shape the implementation:

1. **Never look up by name.** Several unrelated organisations are also called "Destiny Church" — notably Scottish charity SC017898, "Destiny Church Trust". Every request is a direct lookup by the hardcoded number, and each response is checked to confirm the number returned matches the number requested. A mismatch is discarded and the section falls back, so a wrong-entity record can never reach the page. The same warning lives in `lib/siteKnowledge.ts` and `lib/smartSearch/tools.ts`.
2. **Never throw.** Same contract as `lib/events.server.ts`: a missing key, timeout, non-OK status or malformed payload resolves to `null`, and the page renders a stored snapshot instead of 500-ing. The two regulators degrade independently — a Companies House outage leaves charity data live.

Auth differs per regulator: Companies House uses HTTP Basic with the API key as the username and a blank password; the Charity Commission uses an `Ocp-Apim-Subscription-Key` header. Charity Commission responses are read through a case- and underscore-insensitive `field()` helper because their payload casing has shifted between endpoint revisions and the full schema sits behind the developer-portal login — a casing change upstream degrades one field rather than the whole section.

Caching is weekly (`revalidate: 604800` on every fetch, matching the page), which keeps both keys far inside their rate limits (Companies House allows 600 requests per 5 minutes).

`lib/jobs.ts` & `lib/hr.ts`

Similar patterns for job listings and HR management. `lib/hr.ts` also carries the shared HR types (`Staff`, `LeaveRequest`, …), the `dayCount()` / `fullName()` / `formatDate()` helpers and the `LEAVE_TYPE_LABELS` / `REVIEW_TYPE_LABELS` label maps used by both the admin UI and the notification emails.

`lib/staffPortalAuth.ts`

`server-only`. The authorisation boundary for the staff self-service portal (`/portal`, `/api/portal/*`) — deliberately separate from the admin RBAC in `lib/adminRoles.ts`. A portal user's identity is "does this auth user have a linked `hr_staff` row", **not** an access-level boolean, and must never become one: conflating it with `admin_roles` would let "can see my own payslip" leak into "can manage HR".

- `isLinkedToStaff(supabase, authUserId)` — cheap boolean for `middleware.ts`.
- `readPortalUser()` — returns `{ userId, staff }` or `null`; every `/api/portal/*` handler calls it (never trusting middleware) and scopes every query to `staff.id`, never to a client-supplied id.
- `resolvePostLoginPath(supabase, userId)` — used by `adminSignIn` to route an admin to `/admin`, a linked-but-non-admin staff member to `/portal`, else `null`.

`lib/hrEmail.ts`

`server-only`. HR notification emails via Resend — leave requested, leave decision, and the review-reminder digest — over one shared badge/heading/rows/CTA card template (generalised from the job-application email in `app/jobs/actions.ts`). Sends to `HR_NOTIFICATIONS_EMAIL`

(falling back to the recruitment inbox). Called from `/api/portal/leave`, the admin leave routes, and `/api/cron/hr-review-reminders`.

`lib/designTickets.ts` / `lib/designTickets.server.ts`

The design request queue. `designTickets.ts` is **client-safe** — both admin pages are `"use client"` and the requester's tracker renders the same labels — so it holds only the vocabulary and the rules: status/category/priority types, `DESIGN_STATUS_LABELS` / `_TONE` / `_BLURB` (the last being what the *requester* is told a status means, as opposed to the shorthand people who live in the queue read), `ticketRef()` → `"DT-0007"`, and `canTransition(from, to, actor)`.

`canTransition` is the whole workflow, and it is checked twice: once by the UI to decide which buttons exist, once by `applyTransition` to decide whether to honour a POST. Only the second is a guarantee. `tests/unit/design-tickets.spec.ts` pins the matrix — in particular that a requester can never reach `delivered`, which would send a "your files are ready" email for a ticket with no files.

`designTickets.server.ts` is `server-only` and holds:

- **`applyTransition()`** — the single writer of `design_tickets.status`. It re-reads the row rather than trusting a status the caller passed, checks `canTransition`, applies the side effects (assignee on claim, `revision + 1` on a change request, `delivered_at` / `closed_at`), writes the event row, and conditions the UPDATE on the status it read so a second designer clicking in another tab gets a clean 409 instead of silently winning. It also enforces the two rules the UI can only suggest: no delivering without a file at the current revision, and no change request without a note or past `MAX_CHANGE_REQUESTS`.
- **`resolveRequesterIdentity()`** — priority, verification and the staff link, derived from the session cookie and an `hr_staff` lookup, **never** from the submitted form. Fast-track means a linked `hr_staff` row (this codebase's own definition of "staff", read rather than folded into `admin_roles`), or failing that any admin role — which covers a designer with a login and no HR record, whose requests would otherwise queue behind the public. A signed-out staff address is soft-linked by email so `/portal` stays complete, but is *not* fast-tracked and *not* blocked: a domain proves nothing, so refusing it would buy no safety while turning away a real request. When a signed-in user has a linked staff row, their stored email is overwritten with the staff record's — otherwise someone could aim their own tracking link at another inbox.
- **`requesterView()`** — the hand-built response for both requester surfaces. Deliberately not a `select("*")` passthrough: the defence against leaking the next column added to the table is that this function must be edited for a field to become visible.
- **`DESIGN_DELIVERABLES_BUCKET`** (`"design-deliverables"`) and **`deleteDeliverableStorage()`** — the storage constant shared by the upload and download routes, and the one helper that knows how to free a deliverable's bytes by `storage_kind`: remove the object from the Supabase bucket for a `'supabase'` row, delete the asset for a legacy `'playbook'` row, and no-op a `'link'` row (nothing of ours to delete). Used by the delete route and the purge cron so the cleanup logic lives in exactly one place.

`lib/playbook.server.ts`

The Playbook (dev.playbook.com) DAM client. It used to be the delivery mechanism for design deliverables, but Playbook's upload endpoint sends no CORS headers, which silently blocked every direct browser upload — so uploads moved to Supabase Storage (image/PDF) or a pasted share link (video), and this module now only mints signed **download** URLs for the legacy Playbook-backed rows that predate the switch (`getTemporaryDisplayUrl`). `lib/directUpload.ts`, the browser-side resumable-upload half, was deleted with the change: nothing PUTs straight to Playbook any more.

The file was itself deleted with the `/media` gallery in `e7d7687` and restored here, trimmed: the board-picker reads (`listBoards`, `listBoardAssets`, `searchAssets`) have no counterpart in this feature, and **`requestPermalink` / `ensurePermalink` were left out on purpose.**

That omission is load-bearing. Playbook offers two URL lifetimes, and only one suits this feature: `display_url` is signed and expires in ~24h, while `permalink` is permanent but **plan-capped (Pro = 1,000 for this org)**. A design deliverable is only ever fetched by someone following a fresh link from their own ticket, so it never needs a permanent URL, and spending a finite permalink on one would be waste. `tests/unit/design-access.spec.ts` fails the build if any route in the module reaches for one, and asserts this file exports no way to.

The legacy `getOrCreateBoard("Design Tickets")` board still exists for the rows minted before the switch, cached per-ticket in `playbook_board_token`. No new Playbook assets are created, so this is history rather than an active write path.

`lib/emailCard.ts` / `lib/designEmail.ts`

`server-only`. `emailCard.ts` is the one transactional card — orange rule, badge, heading, intro, label/value rows, optional CTA — extracted verbatim from `lib/hrEmail.ts` when the design queue became the fifth trigger on top of it.

`designEmail.ts` holds the design senders: request received (the only place a share token is ever emailed), the new-request alert, claimed, delivered, changes requested, closed and cancelled.

Team notifications are addressed by role, not by config. `designRecipients()` reads every `admin_roles` row holding **both** `design_admin` and `super_admin` on each send — the two together, deliberately, not either alone. A design admin who isn't also trusted with everything else doesn't get the mail; a super admin who hasn't specifically been given the design role doesn't either. Granting both on `/admin/users` is all it takes to put someone in the loop — a second, manual step to also add their address is exactly the step that doesn't get done, and the failure is silent. `DESIGN_NOTIFICATIONS_EMAIL` survives only as a fallback for when nobody qualifies, so an empty result degrades to "someone still gets it" rather than "requests quietly stop being announced".

A change request also reaches the assignee even if they've since lost either role — they still want to hear that the thing they were working on has come back — which is why recipients go through `uniqueEmails()` (`lib/designTickets.ts`). It deduplicates case-insensitively and drops blanks: the assignee is usually already on the team list, `admin_roles.email` is nullable, and an empty recipient makes Resend reject the whole send. Two rules hold throughout: every send is fire-and-forget in a `try/catch` at the call site, so a Resend outage can never turn into a failed claim or a rejected delivery; and **a deliverable's URL is never in an email** — signed URLs expire, so a link mailed on Tuesday is broken by Thursday and reads as lost work. Emails link to the tracking page, which mints a fresh URL per click.

D1 Admin — the Mac app (`apps/d1-admin`)

Destiny One for leaders and admins on macOS, built from the design canvas "The One App — Mac". Tauri 2 (Rust shell, `src-tauri/`) around a React + Vite UI (`src/`). Its own npm project, like the Expo app; imports `@destiny/shared` through a `file:` link.

Who can use it. After sign-in it calls `GET /api/app/v1/one/me` and `GET /api/admin/destiny-one/whoami` . Leaders (any `D1LeaderRole`) and admins get in; anyone else sees "use the phone app". The sidebar's Admin section is filtered by role: Destiny One Admin → Access requests, People, Invites, Communities, Feedback, App settings; Safeguarding Admin → Reports, Paused groups, Audit log. Leaders get chat and can manage groups they lead. Every route still checks for itself.

How it talks to the backend. - Chat: `createDestinyOneClient` (`/api/app/v1/one/*`), exactly as the phone app. - Admin: `createDestinyOneAdminClient` (`/api/admin/destiny-one/*`) with the Supabase token as a bearer (see "Destiny One admin API"). - Both go through Tauri's HTTP plugin (`src/lib/native.ts` `apiFetch`), so the website needs no CORS and the token can only reach hosts in `src-tauri/capabilities/default.json` . - Supabase is used only for auth and Realtime; the session is stored in the macOS Keychain (`keychain_*` commands in `src-tauri/src/lib.rs`).

Native pieces (`src-tauri/src/lib.rs`): Touch ID / Mac password on launch (`touch_id.rs` , LocalAuthentication, setting "Require Touch ID"); a menu bar item with a count and a popover window (`menubar.html` → `src/menubar/`); closing the window hides it so notifications keep coming; open at login (autostart plugin, starts hidden). The popover holds no session: the main window sends it state and acts for it over Tauri events (`src/lib/menubarTypes.ts` , `src/lib/live.ts`).

Notifications (`src/lib/live.ts`) only while the app or its menu bar item is running — no APNs. New messages come over Realtime (same hub as the phone app, `src/lib/realtime.ts`); reports and access requests refresh on a timer (1 and 2 minutes). Banners respect mute, "Show message previews" and "Pause notifications for 1 hour".

Keyboard and menus (`src/lib/shortcuts.ts` , `src/components/Floating.tsx` , `ContextMenu.tsx` , `EmojiPicker.tsx`). One window-level key listener holds the shortcuts (Cmd-K search, Cmd-N new group, Cmd-1 to 9 open the Nth chat in sidebar order via `orderedGroups` , Option-Up/Down previous or next chat, Cmd-I group info, Cmd-comma settings, Cmd-slash lists them); the list shown in the shortcuts sheet is the same `SHORTCUTS` array, so the two can't drift. Right-clicking a message opens `ContextMenu` (quick reactions, Reply, Copy text, Open file, Report, Delete). `Floating` is the shared popover: it opens at a point, clamps itself inside the window, and closes on Escape, outside click, page scroll or blur. The emoji picker is a curated list in `src/lib/emoji.ts` (no dependency) and serves both reactions and the message box; reactions accept any emoji (the API only limits length to 16 characters).

Composing (`src/features/chat/Conversation.tsx` , `Sheets.tsx`). The message box takes up to 10 files at once (dropped, pasted or picked): each goes as its own message, the text and reply with the first, and sent files leave the tray one by one so a failure keeps only what's left. Typing `@` suggests the other members of the group (`mentionQuery` / `mentionSuggestions` from `@destiny/shared`); on send, `findMentions` turns the names into ids for the API, and `mentionSegments` draws them bold. The plus button offers files or a poll (`validatePoll` rules, sent with `poll` on `api.send`); polls in the list are tappable and update at once (`castVote` , undone if the server refuses). "Forward" in the right-click menu opens a sheet to pick up to 5 chats (`api.forward` ; polls and deleted messages can't be forwarded).

DestinyAI (`src/lib/assistantStream.ts` , `features/chat/AssistantStreaming.tsx`). The assistant chat belongs to no community, so it comes from `api.assistant()` (`useAssistant`) and sits at the top of the sidebar; the realtime hub joins its channel with the other groups. While it answers, the server broadcasts `assistant_status` (what it's doing) and `assistant_delta` (the text so far); `applyEvent` feeds the in-memory stream and the chat draws it as a bubble (dots and a label, then the words with a blinking bar). The finished `message` event from `DESTINY_AI_ID` clears it. Nothing is stored. The chat is text-only (no files, polls or mentions) and has no group info pane.

Building. `npm run tauri:dev` / `npm run tauri:build` on a Mac (Rust + Xcode command line tools). The build is unsigned for now; `tauri.conf.json` has the signing fields ready for when the Apple Developer account exists. See `apps/d1-admin/README.md` .

Destiny One app — look and feel (`apps/destiny-one/src`)

Polished to Apple's Human Interface Guidelines. The pieces, and why they exist:

- **Personal appearance** (Profile → Appearance, `src/app/appearance.tsx`). Each person picks the colour of the messages *they send* and a conversation wallpaper (a pattern, a stock photo, or their own photo). Kept on the phone only (`src/state/appearance.ts` , AsyncStorage, `d1.appearance.v1` : `wallpaper` , `dim` , `blur` , `customFile`), so it changes nothing for anyone else. `useTheme()` (`src/theme/tokens.ts`) folds the choice in as `t.send` , `t.onSend` , `t.onSendCard` and `t.wall` , so bubbles, the send button, poll bars and the message-actions preview all follow it.
- **The choices are a curated list, not a colour picker** (`src/theme/appearance.ts` , pure data, no React Native). A free picker can't promise legibility. Every send colour is chosen so its text is at least 4.5:1 and the bubble is at least 3:1 against the page and every wallpaper, in light and dark; every wallpaper keeps body text at 7:1 and timestamps at 4.5:1. `tests/unit/destiny-one-appearance.spec.ts` checks every combination, so adding a colour or wallpaper that fails turns the tests red. The default light send colour is a deeper orange (`#BF5200` , white text) because the brand orange (`#F58021`) is only 2.6:1 against white. Dark mode also uses white text, on a deep orange (`#C25400` , 4.6:1); the Contours and Waves dark line colours were darkened a touch so that bubble still clears 3:1 against every wallpaper.
- **Wallpapers** (`src/components/Wallpaper.tsx`) are drawn in code with `react-native-svg` (gradient, hills, dots, contours, sunburst, waves), not shipped as photos: sharp at any size, near-zero weight, and each has light and dark colours. Applied to conversations only; list and settings screens stay plain so text is always on a known background.
- **Photo wallpapers** are never uploaded and never touch Supabase. Two sources: ten stock photos shipped in `assets/wallpapers/` (Unsplash licence, free to use, no credit required; 900×1800 crops with a 240×480 `-thumb` each so the picker doesn't decode ten full-size images; about 3.3 MB added to the app; wired up in `src/theme/photoWallpapers.ts` , ids are `photo:<key>`), and the person's own photo (`src/lib/customWallpaper.ts`): picked with `expo-image-picker` , re-encoded with `expo-image-manipulator` (shorter side 1200 px, JPEG 0.75, which also strips GPS and other metadata), then copied into the app's documents folder with `expo-file-system` as `wallpaper-<timestamp>.jpg` . Only that file *name* is remembered, because iOS can move the app's folder between updates; the path is rebuilt from `Paths.document` each time, and a missing file falls back to plain. Nothing to moderate, so none of the Apple 1.2 user-generated-content duties apply (sharing a wallpaper with other members would change that). A photo can't be contrast-checked like a gradient, so it is made safe two ways. The **Dim** slider (0 to 90%, labelled **Fade** in light mode) lays the page colour over the photo, so in dark mode it darkens and in light mode it lightens; the **Blur** slider (`Image.blurRadius` , up to 24) softens detail. And

everything that would sit bare on a photo (times, sender names, the day and "New messages" dividers, "This message was deleted", the empty state) goes on a chip in the page colour at 90% opacity (`photoChip()` in `MessageBubble.tsx` , `PHOTO_CHIP_ALPHA`). The unit tests check that chip against pure black and pure white behind it in both modes (7:1 body, 4.5:1 muted). Fade/Dim and Blur are five-step tap pickers (`src/components/PresetPicker.tsx` : Low, Low-Med, Med, High-Med, High; `DIM_PRESETS` / `BLUR_PRESETS` in `theme/appearance.ts`), not sliders: a dragged slider fought the swipe-back gesture. Stored values stay numeric; an older saved value shows as its nearest step (`nearestPreset`). Screen readers get a radio group. `Backdrop` (`Wallpaper.tsx`) draws whichever wallpaper is active and is used by the chat and the live preview.

- **Three looks: Light, Dark, True dark** (Settings > Appearance > Theme; `ThemeMode` is `system | light | dark | black`). **Dark** is a warm tint of the brand orange (`#1A110A` page, `#26190F` cards). **True dark** (`black`) is near-black and neutral (`#0B0B0C` page, `#151517` cards, `#1C1C1F` / `#28282C` fills), deliberately not pure `#000` (harsh against white text); surfaces step up in small even lightness steps. True dark is a dark scheme to the phone (`applyMode`), so System still maps a dark phone to warm Dark. Page colours live in `PAGE_BG` ; each wallpaper has `light` / `dark` / `black` tones and send colours reuse their `dark` tone for `black` , all covered by the contrast tests. The splash screen stays warm (it can't know the mode).
- **Bubbles** (`MessageBubble.tsx`) join in runs (the corner facing the sender flattens between neighbours) and the last one in a run gets a small curved tail. The time shows once per run.
- **Gestures** (`src/components/Swipe.tsx`): swipe a message left to reply (`SwipeToReply` , left as in Telegram so it can never clash with swipe-back); swipe a chat row left for Read and Mute (`SwipeActions`). Both track the finger 1:1, rubber-band at the edge, and spring on from the release velocity; `SwipeActions` snaps using where the flick was heading (`project()`), and only one row is open at a time. Plain `PanResponder` + `Animated` , no extra native module. Long-press still opens the actions sheet, which springs up from the message's side.
- **Chat peek** (`(tabs)/chats.tsx` `ChatRow`): press and hold a chat in the list to preview it, as in WhatsApp. It uses expo-router's native `Link.Preview` / `Link.Menu` (a UIKit context menu) with Mark as read / Mute / Group info underneath; tapping the preview opens the chat. The conversation screen checks `useIsPreview()` and renders read-only in the peek: no read receipt, no `setOpenGroup` , no composer or header buttons, no receive haptic, no notification prompt.
- **Haptics** (`src/lib/haptics.ts` , `expo-haptics`), always tied to an action: tab and filter changes, primary buttons, each code digit, sending, a message arriving while the chat is open, reacting, poll votes, a gesture reaching its threshold, opening the message menu, pull to refresh, a warning when a destructive confirm dialog opens, success when a form goes through, and an error with every new `FormError` . Never on scroll.
- **Emoji** (`src/components/Emoji.tsx`): renders an emoji at 1.4x and shrinks it with a static transform, so ancestor scale animations (press-grow, spring overshoot) do not stretch the emoji bitmap and blur it. Use it for any emoji inside an animated view.
- **Motion** (`src/components/Motion.tsx`): shared spring primitives on RN `Animated` with the native driver (no Reanimated, so no new binary). `PressableScale` shrinks under the finger and springs back (`holdMs` makes a long-press target squeeze over the delay, used by bubbles); `Appear` springs content in on mount (`once` keys stop recycled list cells from re-entering); `Pop` bounces on a value change (unread badges, reaction counts, code digits); `Shake` for a "no" (form errors, a wrong code); `animateLayout()` for filter changes. Bubbles that arrive while the chat is open spring in (sent ones rise from the composer, received ones from the sender's side); poll bars spring to their new share; confirm dialogs spring in. All of it drops the movement under Reduce Motion.
- **Text size**: chrome text (tab labels, chips, badges, reactions) caps its scaling with `maxFontSizeMultiplier` and controls use `minHeight` , so large Dynamic Type sizes grow the layout instead of clipping it. Body text scales freely.
- **Pressed states** respond on touch-down with a small spring scale (`PressableScale`), not just an opacity change.

`lib/destinyOne/*` — Destiny One backend

Part 2 additions: `onboarding.ts` (pure: `onboardingState` , `ONBOARDING_MESSAGES` , `adultOnForDecision` — turns an admin's adult/under-18 choice + optional DOB into `adult_on` , and refuses a mismatch), `settings.server.ts` (`getSettings` , `retentionDays`), `adminData.server.ts` (admin reads: members with email, groups with counts, community/group people, invites), `adminTypes.ts` + `adminClient.ts` (client-safe shapes and `adminSend` for the admin pages), `inviteEmail.server.ts` . `identity.server.ts` now exports `onboardMember` , `submitAccessRequest` and a ChurchSuite-only `resyncMember` .

- `http.ts` — `oneJson` / `oneError` / `OneError` , `oneRoute` (turns a thrown `OneError` into the error response), `fromDbError` (maps the SQL functions' SQLSTATEs: `P0001` → 422 `rule_violation` , `42501` → 403, `P0002` → 404; anything else is logged and reported generically), `readBody` (zod), `limit` (per-member `lib/rateLimit.ts` wrapper).
- `auth.server.ts` — `authenticate` (Bearer → Supabase user), `requireMember` (the three gates), `toMe` .
- **Message encryption** — `sealing.ts` (pure, unit-tested in `tests/unit/destiny-one-sealing.spec.ts`) and `crypto.server.ts` (loads the keys, knows which fields are sealed). Message bodies, poll questions and option labels, and report reasons are sealed by the API before they reach Supabase and opened on the way out, so a database dump or a backup / PITR snapshot on its own shows only ciphertext. **Not end-to-end encryption**: the API holds the key, which is what keeps safeguarding review (transcripts, reports, takedowns), search and push previews working. Since 2026-10-06 the keys live in **Supabase Vault**, not Vercel: anyone with full SQL access or the service key can fetch them (`d1_message_keyring()`), as can code running in the API, so they can read everything. Push previews still pass through Expo and Apple in plain text. Attachments are not encrypted yet (private bucket + signed links, as before).
- Format: `d1e:<keyId>:<iv>.<ciphertext+tag>` (base64url), AES-256-GCM, fresh 12-byte IV, AAD `"<msg|report>:<groupId>"` , so a value only opens in its own group and as its own kind of thing. `open` passes unsealed values through (only rows from before the backfill); one that won't open is logged and shown as "This message couldn't be decrypted." rather than failing the whole page.
- Keys: three Vault secrets, `d1_msg_keys` (`v1:<base64 32 bytes>` , comma-separated during a rotation), `d1_msg_key_current` , `d1_search_key` (HMAC only). They were generated inside the database by migration `20261006_00_destiny_one_keys_in_vault.sql` , so they've never been on a laptop. The API reads them through `d1_message_keyring()` (service role only): `loadMessageKeyring()` once per server, refreshed every 10 minutes, awaited by `oneRoute` and the safeguarding routes before anything is sealed or opened. Every environment shares them automatically, because they share one Supabase project. The `D1_MSG_KEYS` / `D1_MSG_KEY_CURRENT` / `D1_SEARCH_KEY` env variables are only a fallback when Vault has no keys (local dev against another database). **Losing the keys loses every message**: Vault is backed up with the project, but consider an offline copy too. Rotation: append `,v2:<key>` to `d1_msg_keys` and set `d1_msg_key_current` to `v2` (`vault.update_secret`); servers pick it up within 10 minutes. Old values keep opening with `v1` until it is removed, which needs a re-seal pass first.

- Search: a blind index. Each word's prefixes (1–24 characters) become an HMAC under a per-group key derived from `D1_SEARCH_KEY`, stored in `d1_message_terms`; a query hashes each typed word the same way, for each group searched, and a message must have them all. Prefix matching survives ("pra" finds "prayer"); English stemming and stop words from the old tsvector do not. Bodies only, as before.
- Live delivery: `broadcastNewMessage` in `chat.server.ts` sends the opened message on `d1-group:<id>` through Realtime's REST broadcast (private), which does not store it.
- One-off backfill for rows from before: `scripts/destiny-one/encrypt-messages.ts` (`--dry-run` first), then migration `20261004_02`.
- `churchsuite.ts` (pure, unit-tested) — `toPerson`, the **data-minimisation allow-list**: a ChurchSuite record becomes `{ kind, id, displayName, email, adultOn, status }` and nothing else — no phone, address, medical notes or DOB ever leave it. Children-module records are always minors. `pickByEmail` (exact, active, and *ambiguous* on a shared family email → stays pending). PKCE, `seal` / `unseal` (AES-256-GCM with expiry), `isAllowedAppRedirect`.
- `churchsuite.server.ts` — ChurchSuite API v2: client-credentials token (cached), contact/child lookups, auth-code exchange, current user. Throws `ChurchSuiteUnavailable` on outage so callers treat it as "no change".
- `emailChange.ts` / `emailChange.server.ts` — changing your own sign-in email: the sealed code ticket (pure, unit-tested in `tests/unit/destiny-one-email-change.spec.ts`; sealed with `DESTINY_ONE_SECRET`, falling back to the Supabase secret key) and the send / confirm steps behind `me/email`.
- `identity.server.ts` — `onboardMember` (invite → active; optional ChurchSuite sign-in → verified; else `pending`), `submitAccessRequest`, `resyncMember` (ChurchSuite-verified members only; outages never downgrade).
- `chat.server.ts` — reads shaped into `@destiny/shared` types: community list, group detail (adult flags only for managers), message pages with signed attachment URLs; deleted messages returned without body.
- `push.server.ts` — Expo push: title = group name, body = `pushPreviewText()` ("Sender: first line", 100 chars, from `@destiny/shared`), `data.groupId`; prunes dead tokens. Previews pass through Expo/APNs/FCM — see docs/destiny-one-gdpr.md §3.
- `schemas.ts` — zod request schemas (limits mirror the SQL CHECKs). `admin.server.ts` — `requireSafeguardingAdmin`. `signin.server.ts` — ChurchSuite hand-off constants.
- `packages/shared/src/destinyOne/policy.ts` — the rules as pure functions (`adultOnFromDateOfBirth`, `canCreateGroup`, `checkComposition`, `canPost`, `REQUIRED_CONSENTS`, …) for early refusals in the API and button-hiding in the app. The database is the authority; this is a copy.

Authentication & Authorization

Supabase Auth Flow

1. **User signs up/in:** - Email + password sent to Supabase - Returns JWT token - Token stored in secure HTTP-only cookie (via `@supabase/ssr`)
2. **On every request:** - Next.js middleware checks for valid token - If valid, attached to request context - If expired, refresh token is used automatically
3. **On protected routes:** - Server component checks if user exists - Redirects to login if not authenticated
4. **Server actions & API:** - Get user ID from token - Query database with service role (not user role) - Apply RLS policies based on table settings

"Keep me signed in"

The login form (`app/login/LoginClient.tsx`) has a "Keep me signed in" checkbox (`name="remember"`, checked by default). `app/login/actions.ts`'s `adminSignIn` reads it and:

- Passes `{ remember }` into `utils/supabase/server.ts`'s `createClient`, which strips `maxAge` / `expires` from the Supabase auth cookies it writes when `remember` is `false`, turning them into browser-session cookies.
- Sets a small side cookie, `sb-remember` (`utils/supabase/sessionCookie.ts`), recording the choice — `"1"` with a 400-day `maxAge` when remembered, `"0"` as a session cookie when not.

This side cookie exists because `@supabase/ssr` unconditionally re-applies its own 400-day default `maxAge` every time it refreshes the auth cookies — which happens on every `/admin/*` request via `utils/supabase/middleware.ts`. Without it, a "don't remember me" session would silently become persistent the moment middleware refreshed the token. Middleware reads `sb-remember` on each request and re-strips `maxAge` on any cookies it sets when the value is `"0"`. A missing `sb-remember` cookie (e.g. sessions created before this feature) defaults to remembered, so existing sessions aren't unexpectedly downgraded.

`adminSignOut` deletes the `sb-remember` cookie alongside signing out of Supabase.

Authorization Layers

Access levels live in `lib/adminRoles.ts` + the `admin_roles` table — eleven independent per-user booleans (`training_admin`, `event_admin`, `store_admin`, `site_admin`, `hr_admin`, `design_admin`, `sermon_admin`, `safeguarding_admin`, `destiny_one_admin`, `super_admin`; see admin_roles). Auth *and* role enforcement both happen centrally in `middleware.ts`, not in `app/admin/layout.tsx` (which is a client component purely responsible for the sidebar/header shell; it does not check auth or roles itself).

Route → role mapping (`ROUTE_RULES` in `lib/adminRoles.ts`; `super_admin` always passes and isn't repeated per rule; anything under `/admin` or `/api/admin` that isn't listed is Super Admin only — fail closed):

Role	Admin pages	API routes
`training_admin`	`/admin/training/**`	`/api/admin/training/**`
`event_admin`	Courses (`alpha` , `recovery` , `bible-course` , `cap-money` , `featured-course`) + Announcements except Banner (`popup` , `featured-event` , `event-popup` , `nfc`)	`/api/admin/{alpha-events,events,featured-course,featured-event,popup,nfc}`
`store_admin`	`/admin/store/**`	`/api/admin/{store,shop-hero}/**`
`site_admin`	`/admin/posts` , `/admin/redirects` , `/admin/analytics`	`/api/admin/{posts,redirects,analytics}/**`
`hr_admin`	`/admin/hr/**` (staff, leave, jobs, applications, documents, reviews, checklists)	`/api/admin/hr/**`
`design_admin`	`/admin/design/**` (the design ticket queue)	`/api/admin/design/**`
`super_admin`	Everything, plus Banner, Clear Cache, `/admin/users` and `/admin/audit`	`/api/admin/{banner,revalidate,users,audit}/**`

`OPEN_PATHS` — reachable by any authenticated admin regardless of role: `/admin` (the dashboard), `/api/admin/logout` , `/api/admin/me` , `/api/admin/me/roles` and `/api/admin/search` .

The first four are trivially safe: a signed-in admin's own identity and roles tell them nothing they don't already have. **`/api/admin/search` is the one that needs care.** It is open because every admin needs the ⌘K palette, but it is not unguarded — the route re-reads the caller's roles from the service client and only queries the sections that role can open, so a Store Admin's results contain orders and products and nothing else. If you add a section to that route, gate its query on a role the same way. HR is deliberately not searched at all — its records are the most sensitive in the admin (staff records, leave reasons, applicant CVs), so they don't belong in a fuzzy search box regardless of role.

`/admin/users` (Super Admin only, `app/api/admin/users/**`) is where roles are assigned — a tickbox per role per user — and where a user's email can be changed (`PATCH /api/admin/users/[id]` updates the auth login and `admin_roles.email` together, confirmed immediately, and audits the old and new address; passwords are untouched). `GET /api/admin/me` (and the older `/me/roles`) lets the sidebar, header and palette know which sections to show; that's a UI convenience only, not an authorization boundary. What a user is *shown* comes from `lib/adminNav.ts` , which must stay in step with the table above — `tests/unit/admin-nav.spec.ts` enforces it.

One thing `getRoles` will not tell you it got wrong. It spells its column list out by hand rather than `select *` , so adding an access level is a deliberate act — but the cost is that forgetting that one line makes the new role read as `false` for everyone, everywhere, with no error and nothing in the UI to explain why the section never appears. `tests/unit/design-access.spec.ts` asserts every `ADMIN_ROLES` entry appears inside that `.select(` string; it is the only guard there is.

A third authorization mode: share tokens. Alongside admin RBAC and portal staff-linkage (`lib/staffPortalAuth.ts`), the design queue authorises a requester by a 128-bit `share_token` in the URL — `/design-request/<token>` and `/api/design-request/<token>/**` . There is no session involved, because most requesters aren't staff and shouldn't need an account to collect a poster. What makes that safe is scope, not secrecy:

- The token resolves to exactly one ticket. Every nested route re-derives the ticket from the token and matches child rows against it, so a valid token for ticket A cannot download a deliverable belonging to ticket B.
- `requesterView()` in `lib/designTickets.server.ts` hand-builds the response. It is deliberately not a `select("*")` passthrough — the defence against leaking the next column someone adds is that the function has to be edited for a field to become visible. Internal notes, the assignee's email and the requester IP hash are all dropped there.
- The two moves a requester can make are enforced by `canTransition(..., "requester")` server-side, so posting `{"to": "delivered"}` to their own ticket is rejected.

> **Adding an access level.** Add it to the `AdminRole` union, `ADMIN_ROLES` and `NO_ROLES` in `lib/adminRoles.ts` , give it `ROUTE_RULES` entries (without them the new section is Super Admin only), add labels in `app/admin/users/page.tsx` and a nav entry in `lib/adminNav.ts` (which now drives the sidebar, the header breadcrumbs, the dashboard grid and the ⌘K palette) — and add the column to the **hand-written `.select(...)` string in `getRoles()`** , plus the matching line in the object it returns. That last one is the trap: it isn't `*` , so a role missing from it silently reads as `false` everywhere and the feature dies quietly.

The audit log — the record of what everyone did with all of the above

Every change made through `/api/admin` is written to `audit_log` by `recordAudit()` (`lib/audit.server.ts`), along with sign-in and sign-out, which happen outside that prefix. `middleware.ts` forwards the actor it has already resolved on the request headers (`AUDIT_ACTOR_HEADERS`), so an entry knows who made it without a second auth round trip per save.

What it records: changes — creates, edits, deletions, uploads, approvals, moderation, cache clears, role grants, sign-ins (including failed ones on a real address). Each entry carries a plain-English sentence, the field-by-field diff, and the roles the actor held at the time.

What it doesn't: reads. Nobody is logged for *opening* a page, running a ⌘K search or asking the audit assistant a question — the admin doesn't log reads anywhere, and logging questions about the log would feed it its own traffic. Onboarding-tour progress is exempt for the same reason (it fires several times a minute and touches nothing). Free-text HR values — leave reasons, one-to-one notes, staff notes — are recorded as *changed* but their contents are redacted, the same instinct that keeps HR out of the ⌘K search.

Reading it is Super Admin only, by omission from `ROUTE_RULES` rather than by a rule: the log spans every section, so any narrower grant would leak one team's activity to another. See audit_log / audit_reports.

The staff portal — a second auth boundary

`/portal` and `/api/portal/*` (staff self-service — own profile, leave, documents) are **not** part of the `admin_roles` RBAC above. `middleware.ts` matches them but applies a different test: the caller must be signed in **and** have a linked `hr_staff` row (`lib/staffPortalAuth.ts` → `isLinkedToStaff`), checked independently of `hasAccess()` . A signed-in admin with no staff record cannot reach the portal, and a staff member with no admin role cannot reach `/admin` — the two boundaries are deliberately orthogonal. Every `/api/portal/*` handler re-derives identity with `readPortalUser()` and scopes queries to that staff id, so the portal never trusts request state it can't verify itself.

Layer 1: Middleware (`middleware.ts`)

```
// Matcher: "/admin/:path*", "/api/admin/:path*", "/portal/:path*", "/api/portal/:path*"
//          (+ a "/lite" redirect helper, unrelated to auth)
export async function middleware(request: NextRequest) {
  const { supabase, supabaseResponse } = createClient(request);
  const { data: { user } } = await supabase.auth.getUser();

  if (pathname === "/api/admin/logout") return supabaseResponse;      // always reachable
  if (pathname === "/admin/login") return NextResponse.redirect(new URL("/admin", request.url)); // stale bookmark

  // Public admin auth pages must stay reachable while logged out
  if (["/admin/forgot-password", "/admin/reset-password"].some((p) => pathname.startsWith(p))) {
    return supabaseResponse;
  }

  if (!user && pathname.startsWith("/api/admin")) {
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }
  if (!user) return NextResponse.redirect(new URL("/login", request.url));

  // Access levels — see lib/adminRoles.ts for the route → role mapping.
  const roles = await getRoles(createServiceClient(), user.id);
  if (!hasAccess(roles, pathname)) {
    if (pathname.startsWith("/api/admin")) {
      return NextResponse.json({ error: "Forbidden" }, { status: 403 });
    }
    return NextResponse.redirect(new URL("/admin?forbidden=1", request.url));
  }

  return supabaseResponse;
}
```

Layer 2: Database (RLS)

```
-- Example: Only service role can read HR staff
CREATE POLICY "service only" ON hr_staff USING (false);
```

API routes use the service role key for reads/writes (RLS is bypassed), relying on the middleware gate above rather than per-table role checks. `admin_roles` itself follows the same deny-all pattern — read only via `createServiceClient()` .

`SECURITY DEFINER` **functions need explicit grants.** Postgres gives `PUBLIC` EXECUTE on every new function, and PostgREST exposes everything in `public` at `/rest/v1/rpc/<name>` . A definer function runs as its owner and bypasses RLS, so unless you revoke that grant, anyone with the anon key can call it. The pattern:

```
-- Server-only (called through createServiceClient()):
revoke all on function public.my_fn(...) from public, anon, authenticated;
grant execute on function public.my_fn(...) to service_role;
```

A helper that an RLS policy calls runs as the querying role, so that role needs EXECUTE. Put it in the non-exposed `private` schema instead of `public` . Trigger functions don't need the caller to hold EXECUTE. Run `get_advisors` (security) after any migration that adds a function: lints 0028/0029 flag definer functions that anon or authenticated can reach. Current example: `decrement_variant_stock` .

Why two layers? - **Defense in depth** — the middleware gate is the single source of truth for "is this visitor allowed into this `/admin` section"; RLS (deny-all + service role) means even a bug in the app layer can't expose sensitive tables to the anon key. - **Fail closed** — a new admin page that isn't added to `ROUTE_RULES` is Super Admin only by default, rather than accidentally open to everyone.

Layer 0: abuse limits on the two public surfaces (ahead of both layers above)

The admin login form and the Smart Search chat API are the two public surfaces most worth abusing — password-guessing on one, OpenAI/Tavily spend on the other. Neither runs a bot challenge (an earlier Cloudflare Turnstile gate was removed); each leans on cheaper, server-side limits instead:

- `/login` (`LoginClient.tsx` + `app/login/actions.ts`) is protected by a per-IP login rate limit (`lib/loginRateLimit.ts`) checked inside `signInCore()` before any `signInWithPassword` call — repeated failures from one IP are throttled with a "try again in N minutes" message, and successful sign-ins reset the counter.
- **Smart Search** (`FloatingSmartSearch.tsx` → `POST /api/chat`) is protected by a per-IP rate limit (20 requests/minute → 429) and strict message-shape validation: only `user` / `assistant` roles are accepted (blocks injected `system` turns), each message is capped at 2000 chars, and the final message must be a user turn ≤300 chars. The per-request `ToolContext` also confines `extract_page` to URLs a prior `search_web` call in the same request actually returned.

Destiny One member identity (`/api/app/v1/one`)

Separate from admin auth: app members are Supabase Auth users with **no admin_roles row** and **no cookies** — the Expo app keeps its session in the iOS Keychain / Android Keystore (`expo-secure-store`) and sends a Bearer token. Two sign-in methods, neither with a phone number: email one-time code (Supabase OTP) and Sign in with ChurchSuite (optional, for staff/leaders).

Staff verify people, not ChurchSuite. After sign-in, `onboardMember` (`identity.server.ts`): existing member → as is; an open invite for the email → active with the invite's name, age, roles and communities; Sign in with ChurchSuite (if configured) → verified from the linked contact; anyone else → `pending` . `D1Me.onboarding` tells the app which screen to show (`request_needed` → the access request form, `POST /me/access-request` ; `request_submitted` ; `invite_only` when `d1_settings.allow_access_requests` is off; `suspended` ; `active`), with `onboardingMessage` copy served from the server (`lib/destinyOne/onboarding.ts`). A Destiny One Admin approves requests as adult or under 18 at `/admin/destiny-one/requests` . Members must also accept the current privacy / terms / chat-review notices. Leader roles (`admin` , `cg_leader` , `senior_leader`) are set by a Destiny One Admin. Supabase's phone auth provider should be **disabled**; the database refuses to activate an account with a phone number regardless.

Session Management

- **Session storage:** Secure HTTP-only cookie (managed by `@supabase/ssr`)
- **Token refresh:** Automatic when expired (hooks in Providers.tsx)
- **Logout:** Clear cookie, redirect to home
- **CSRF protection:** Next.js built-in (automatic for POST requests)

Configuration

Song requests (Spotify)

`SPOTIFY_CLIENT_ID` and `SPOTIFY_CLIENT_SECRET` (from a Spotify developer app). Optional `SPOTIFY_REDIRECT_URI` (defaults to `<origin>/api/admin/song-requests/spotify/callback` ; register it in the Spotify app) and `SONG_REQUEST_SALT` (salts the stored device/IP hashes). The church Premium account authorises once from `/admin/song-requests` ; its refresh token lives in `song_request_spotify` (service-only).

`next.config.ts`

```
const nextConfig: NextConfig = {
  // Server actions default to a 1MB body. The jobs form takes a CV up to 4MB
  // (app/jobs/actions.ts MAX_CV_BYTES) — Vercel itself caps a function request
  // at 4.5MB, so a larger limit here would never be reachable.
  experimental: {
    serverActions: { bodySizeLimit: "4.5mb" },
  },

  // Include ffmpeg binary for serverless functions (if needed)
  outputFileTracingIncludes: {
    "*": ["node_modules/ffmpeg-static/ffmpeg"],
  },

  // Permanent redirects (handled at edge)
  async redirects() {
    return [
      { source: "/prayer-request", destination: "/connect-card", permanent: true },
      { source: "/prayer-requests", destination: "/connect-card", permanent: true },
      // Retired 2026-08-06; /governance now carries its financial content
      { source: "/annual-report-2025", destination: "/governance", permanent: true }
    ];
  },

  // Security headers
  async headers() {
    return [
      {
        source: "/(.*)",
        headers: [
          {
            key: "Permissions-Policy",
            value: "camera=(), microphone=(), geolocation=()"  // Disable dangerous features
          }
        ]
      },
      // Long-lived cache for static assets (immutable = only update if filename changes)
      {
        source: "/img/:path*",
        headers: [
          { key: "Cache-Control", value: "public, max-age=31536000, immutable" }
        ]
      }
    ];
  },

  // Image optimization
  images: {
    minimumCacheTTL: 2592000, // 30 days
    // Allow images from these domains
    remotePatterns: [
      { protocol: "https", hostname: "i.ytimg.com" },      // YouTube thumbnails
      { protocol: "https", hostname: "storage.buzzsprout.com" },  // Podcast images
      { protocol: "https", hostname: "cdn.churchsuite.com" },     // ChurchSuite events
      { protocol: "https", hostname: "lwmnrbglbtbyypzcenzf.supabase.co" },  // Our Supabase bucket
      // ... more patterns
    ]
  }
};
```

Styling: Tailwind v4, no `tailwind.config.ts`

This project uses **Tailwind CSS v4**, which has no JavaScript config file. Everything lives in `app/globals.css`:

```
@import "tailwindcss";          /* establishes @layer theme, base, components, utilities */

:root {
  --background: #ffffff;
  --foreground: #363f48;
  --surface: #ffffff;
  --surface-muted: #f5f7fa;
  --surface-overlay: rgba(255, 255, 255, 0.92);
  --border-subtle: rgba(0, 0, 0, 0.05);
  /* Motion literals live here, not in @theme, so both the utilities below AND
     hand-written animation CSS elsewhere in this file can reference them —
     see "Radius, elevation, hairline, motion" below. */
  --motion-ease-standard: cubic-bezier(0.4, 0, 0.2, 1);
  --motion-ease-expo: cubic-bezier(0.16, 1, 0.3, 1);
  --motion-ease-overshoot: cubic-bezier(0.34, 1.56, 0.64, 1);
  --motion-dur-fast: 200ms;
  --motion-dur-base: 300ms;
  --motion-dur-slow: 500ms;
}

@theme inline {
  --color-destiny-orange: #f58021;
  --color-destiny-orange-dark: #d96d10;
  --color-destiny-red: #fd0000;
  --color-destiny-blue: #0857ba;
  --color-destiny-green: #028002;
  --color-destiny-purple: #8106b1;
  --color-destiny-grey: #363f48;
  --color-destiny-white: #ffffff;
  --color-destiny-black: #000000;
  --color-destiny-brown: #2c1a0e;
  --color-destiny-brown-light: #3d2b1a;
  --color-muted: #5b6570;   /* 5.9:1 on white — see "Accessible text" below */
  --color-subtle: #6b7580;  /* 4.7:1 on white */
  --color-surface: var(--surface);
  --color-surface-muted: var(--surface-muted);
  --color-hairline: rgb(0 0 0 / 0.07);
  --color-divider: rgb(0 0 0 / 0.12);
  --font-sans: var(--font-roboto), system-ui, -apple-system, sans-serif;
  --font-heading: Arial, "Helvetica Neue", sans-serif;
  --font-admin-heading: var(--font-anton), Arial, "Helvetica Neue", sans-serif;
}
```

(Trimmed for readability — the real file also carries 50–900 shade ramps per hue, the `success` / `warning` / `danger` / `info` semantic aliases, and `--radius-card` / `--radius-panel` / `--radius-shell` and `--shadow-card` / `--shadow-card-hover`, all covered in their own subsections below.)

Tokens declared in `@theme inline` become utilities automatically, so `--color-destiny-orange` gives you `text-destiny-orange`, `bg-destiny-orange`, `border-destiny-orange` and so on. Add a colour or font by adding a variable here — there is no config file to edit.

Fonts are loaded by `next/font` in `app/layout.tsx` — Roboto (body), Anton (display/hero), Playfair Display (serif accent), Poppins (loaded site-wide; `/twelvetwo` is its only current caller) — and exposed as `--font-roboto` / `--font-anton` / `--font-playfair` / `--font-poppins`.

Colour ramps and semantic tokens

Each brand hue above also has a full 50–900 shade ramp (`--color-destiny-orange-50` … `-900`, and the same for red/blue/green/purple/grey), generated by mixing the flat base colour toward white (tints, 50–400) and toward black (shades, 600–900) so lightness stays monotonic regardless of how light or dark a given brand hue already is (green's base is much darker than orange's, and both still ramp correctly). **The existing flat token is always identical to that colour's `-500` step** — nothing that already used `bg-destiny-orange` etc. changed colour when the ramps were added.

On top of the ramps sit semantic aliases — `--color-success` / `-bg` / `-fg`, `--color-warning`, `--color-danger`, `--color-info` (each `-bg` / `-fg` pair) — so "this is a good outcome" is one concept (`bg-success/10 text-success`) reused everywhere instead of every call site hand-picking which green to use. `Badge`'s tone map and `MetricCard`'s chip tone (`components/admin/AdminUI.tsx`, `app/admin/page.tsx`) both read from these rather than a brand colour directly.

Accessible text — `text-muted` / `text-subtle`

The site's idiom for secondary text used to be an opacity modifier on the brand grey — `text-destiny-grey/60`, `/50`, `/40` — which reads as "quieter" but doesn't survive a contrast check: `#363f48` is 10.5:1 on white, but at 60% opacity that drops to 3.4:1 and at 40% to 2.1:1, both below the WCAG AA floor of 4.5:1 for normal-size text. `/40` was also the *single most common* secondary-text value in the codebase before this was fixed.

Two solid-colour steps replace it, each measured on white and pinned by `tests/unit/contrast.spec.ts`:

Token	Hex	Ratio on white	Use
`text-muted`	`#5B6570`	5.9:1	Secondary body copy, card descriptions
`text-subtle`	`#6B7580`	4.7:1	Metadata, captions, eyebrow labels — the quietest allowed to carry meaning

On dark surfaces the equivalents are alphas rather than solid colours, because the backdrop varies: `text-on-dark-muted` (`rgb(255 255 255 / .82)`, 7.8:1 over `--color-destiny-grey`) and `text-on-dark-subtle` (`/.72`, 6.4:1). Over **photography** neither is sufficient by itself — the fix there is a stronger scrim, not a lighter text colour, and is applied by hand per hero.

A codemod moved 602 occurrences across 152 files onto these two tokens. **Not** touched: `/admin` and `/portal` (each light class there commonly pairs with a `dark:` sibling, which needs checking against both themes as its own pass) and any `material-symbols-rounded` icon span (decorative; darkening a large empty-state glyph to the AA floor would be a visual regression, not a fix).

Radius, elevation, hairline, motion

Four more scales exist for the same reason as the two above: each replaced a handful of near-identical values that had drifted apart from being independently invented per call site.

Token(s)	Values	Replaces
`rounded-card` / `rounded-panel` / `rounded-shell`	20px / 24px / 32px	`rounded-2xl` , `rounded-3xl` , `rounded-[20px]` , `rounded-[24px]` all meaning "a card"
`shadow-card` / `shadow-card-hover`	the two-layer contact + soft shadow	The literal `shadow-[0_1px_2px_rgba(16,24,40,.04),0_8px_24px_-8px_rgba(16,24,40,.10)]` , pasted verbatim into 16 files
`border-hairline` / `border-divider`	`rgb(0 0 0 / .07)` / `rgb(0 0 0 / .12)`	Six notations in use for a hairline: `border-black/5` , `/6` , `/8` , `/10` , `/15` , `/[0.07]`
`ease-standard` / `ease-expo` / `ease-overshoot` (utilities); `--motion-dur-fast/base/slow` (`:root` only — Tailwind has no duration namespace to hang a utility on)	see `:root` literals above	`cubic-bezier(0.16,1,0.3,1)` was written two different ways (with and without spaces) across 23 sites, alongside two other curves and five ad-hoc durations

`components/ui/Card.tsx` and `components/ui/Section.tsx` / `Container.tsx` (see Components, above) are the primitives built on these; adopting one usually means adopting the others.

Dark mode — `/admin` only

> **Currently switched off (September 2026).** Destiny One's admin pages were never checked in dark and it read as broken rather than dark, so `ADMIN_DARK_MODE_ENABLED` in `lib/adminTheme.ts` is `false` : `useAdminTheme()` always resolves to light (whatever is stored or the OS prefers) and `AdminThemeToggle` renders nothing. Everything below still describes how it works — the `dark:` classes are left in place, dormant — so re-enabling it is flipping that constant once each admin page has been checked in dark.

Tailwind v4's default `dark:` variant follows `prefers-color-scheme` , which would flip the whole *site* dark the moment a visitor's OS is set to dark — wrong for a public church site whose brand is deliberately light. `globals.css` redefines the variant instead:

```
@custom-variant dark (&:where(.dark, .dark *));
```

so `dark:` utilities do nothing anywhere a `.dark` class doesn't exist. The only place that class is ever applied is the `/admin` shell's own wrapper div (`app/admin/layout.tsx`), driven by `lib/adminTheme.ts` 's `useAdminTheme()` hook — so the public site is structurally unaffected; there is no code path that can put `.dark` on anything outside `/admin` .

`useAdminTheme()` stores the choice (`"light" | "dark" | "system"`) under the `dc-admin-theme` localStorage key, following the exact `useSyncExternalStore` pattern already used by the sidebar's remembered-dropdown-state store (`AdminSidebar.tsx`) — a stable server snapshot avoids a hydration mismatch, and a same-window custom event keeps every reader in step. It returns a `themeClass` string (`"dark"` or `""`) for the caller to render directly, **not** an imperative `document.documentElement.classList` mutation: that element is shared with the whole page, and toggling it directly would leave `.dark` stuck on `<html>` with no natural cleanup on client-side navigation away from `/admin` . `AdminThemeToggle.tsx` is the 3-way (light/dark/system) control, mounted in `AdminHeader.tsx` and the mobile bar in `AdminSidebar.tsx` .

`components/admin/AdminUI.tsx` 's own header comment documents the `dark:` mapping every token in that file follows (`bg-white` → `dark:bg-destiny-grey-800` , `text-destiny-grey` → `dark:text-white` , etc.) — since every one of the ~30 admin pages imports its shared strings (`cardClass` , `primaryBtn` , `Modal` , …) rather than writing its own classes, this one file's dark-mode support propagates to all of them with no per-page changes.

The mobile tab bar is glass too, but the *other* way round: `.admin-tabbar` (`app/globals.css`) is a theme-aware tint — frosted white over the light admin, frosted grey over the dark one — because unlike the surfaces below it is persistent chrome that sits on screen the whole time, not an overlay announcing itself. It deliberately omits `glass-refract` : the displacement filter is already dropped below 768px on GPU cost, and that bar only ever renders below 768px.

Two surfaces deliberately don't react to the toggle at all. `AuditAsk` 's answer panel and the ⌘K command palette (`AdminCommandPalette.tsx`) use the public site's `.glass` / `.admin-glass` classes instead — the same fixed, near-black frosted treatment the onboarding tour already uses, chosen because both are overlays (a one-off AI answer, a page-dimming search modal), not persistent chrome, and a fixed identity reads the same recognisable way regardless of the admin's own theme. This also sidesteps a real wiring problem: `AdminCommandProvider` (and `DialogProvider` , mounted site-wide in `components/Providers.tsx`) render their modals as siblings of the whole app tree, not descendants of `/admin/layout.tsx` 's `.dark` -carrying wrapper — a `dark:` class there would never match its ancestor selector. `DialogProvider` 's own dialog *does* need to react to the theme (it's a plain confirm/prompt, not a distinct "AI" moment, and is also used by `/portal` , which has no theme toggle and must always stay light), so it carries its own `.dark` class computed from `useAdminTheme()` directly rather than relying on inheriting one.

Cascade layers — read this before adding global CSS

`@import "tailwindcss"` establishes `@layer theme, base, components, utilities` . **Unlayered CSS beats every layered rule regardless of specificity**, so a global style written outside a layer will silently outrank every Tailwind utility, and no amount of specificity in the utility will win.

Two rule sets in `globals.css` were unlayered and caused exactly that:

- `h1, h2, h3 { font-family: var(--font-heading) }` beat every `font-*` utility, so a utility could never change a heading's font. Now in `@layer base`.
- The `.rte-content` prose rules beat every utility used inside rich-text content, which made content blocks impossible to style. Now in `@layer components`.

When adding global CSS, put it in the right layer. Base element defaults go in `@layer base`; reusable classes go in `@layer components`. Leave it unlayered only when you genuinely intend it to beat everything.

Note that `@layer base` still loses to utilities, which is why blocks that want a non-Arial heading set `style={{ fontFamily: FONT_ROBOTO }}` — an inline style beats layered CSS entirely. See `components/blocks/tokens.ts`.

`tsconfig.json`

- `strict: true` — Strict type checking
- `jsx: "react-jsx"` — React 17+ JSX transform
- `@/*` — Path alias for imports (e.g., `@/components/Button`)

`.env.local` Template

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...
SUPABASE_SERVICE_ROLE_KEY=eyJ...

# YouTube
YOUTUBE_API_KEY=AIza...
YOUTUBE_CHANNEL_ID=UCxx...
YOUTUBE_CHANNEL_HANDLE=DestinyOnlineChurch       # optional; overrides the CHANNEL_HANDLE constant (the @ name) for live detection
YOUTUBE_CHANNEL_VANITY=destinychurchteesvalley   # optional; overrides the CHANNEL_VANITY constant (the custom URL)
LIVE_DISABLED=                               # set to 1 to force the live banner off air

# Buzzsprout (sermon podcast audio)
BUZZSPROUT_API_TOKEN=<token from the Buzzsprout account's My Account page>   # /admin/sermons upload flow only — reading the feed needs no key
BUZZSPROUT_PODCAST_ID=268765                                                  # the podcast id in both the RSS feed URL and the write API's bas

# Email
RESEND_API_KEY=re_...
PAGE_AUDIT_FROM=Destiny AI <noreply@support.squaremediagroup.org>  # from-address for system alert emails
SMART_SEARCH_ALERT_RECIPIENT=malachi@squaremediagroup.org          # Smart Search down/recovered alerts
HR_NOTIFICATIONS_EMAIL=techteam@destinytees.uk                     # optional; inbox for HR leave/decision + review-reminder emails (lib/hrEmai
DESIGN_NOTIFICATIONS_EMAIL=techteam@destinytees.uk                 # optional FALLBACK only; design notifications go to admins holding BOTH des
DESIGN_IP_SALT=<random string>                                     # optional but recommended; salts the sha256 of a requester's IP. Without it

# Playbook (dev.playbook.com) — the DAM that USED to hold design ticket
# deliverables. New image/PDF deliverables now go to Supabase Storage and video
# is a pasted Drive/Playbook link, so this is only needed to serve legacy
# Playbook-backed deliverable downloads. Nothing else uses it.
PLAYBOOK_TOKEN=<bearer token, write scope>
PLAYBOOK_ORG_SLUG=dctv                                             # optional; defaults to dctv (the Pro workspace). The account also has an ur

# OpenAI (for Smart Search chat — gpt-4.1-mini, tool-calling)
OPENAI_API_KEY=sk-...

# Regulator APIs for /governance (optional — page falls back to a stored snapshot without them)
COMPANIES_HOUSE_API_KEY=                         # HTTP Basic, key as username + blank password
CHARITY_COMMISSION_API_KEY=                      # sent as the Ocp-Apim-Subscription-Key header

# Smart Search tools (optional — each degrades gracefully without its key)
NEXT_PUBLIC_GOOGLE_MAPS_EMBED_API_KEY=AIza...   # get_directions embed
TAVILY_API_KEY=tvly-...                          # search_web + extract_page

# Stripe (shop checkout)
STRIPE_SECRET_KEY=sk_live_...
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_live_...
STRIPE_WEBHOOK_SECRET=whsec_...
SHOP_TEST_BYPASS=            # leave unset in production — enables /api/store/checkout/bypass

# GitHub (CI/CD + "Report a Bug" footer form — used by components/report-bug/actions.ts
# to open issues on SquareMediaGroup/destinychurch via the GitHub REST API)
GITHUB_TOKEN=ghp_...

# Cron bearer token — gates the jobs declared in vercel.json:
#   /api/cron/analytics-anonymise    (daily 03:00) — anonymises engagement_events IPs
#   /api/cron/hr-review-reminders    (daily 06:00) — HR review digest to HR_NOTIFICATIONS_EMAIL
#   /api/cron/audit-weekly-report    (Sun 20:00)   — AI admin-activity report to every Super
#                                                    Admin, then the audit-log retention purge
#   /api/cron/ip-reputation-refresh  (Mon 05:00)   — refreshes the VPN/Tor/datacenter/Private
#                                                    Relay range lists behind ip_category
#   /api/cron/design-deliverables-purge (daily 04:00) — deletes design deliverables the
#                                                    requester confirmed >=48h ago
# and the self-healing Smart Search health check (GET /api/health/smart-search).
CRON_SECRET=            # every /api/cron job FAILS CLOSED (503) if unset; the health check runs unauthenticated (local/dev)

# Audit log (both optional)
#   AUDIT_REPORT_EMAIL   — comma-separated override for who gets the weekly report.
#                          Unset, it goes to every Super Admin in admin_roles.
#   AUDIT_RETENTION_DAYS — how long entries are kept (default 365). Purged at the
#                          end of the weekly cron, once the week has been reported.
AUDIT_REPORT_EMAIL=
AUDIT_RETENTION_DAYS=365

# Click analytics — /admin/analytics, engagement_events (see Database Schema §24)
#   ANALYTICS_HASH_SALT          — salt for visitor_hash. Without it, a sha256 of
#                                  an IPv4 is brute-forceable, so hashes are NOT
#                                  pseudonymous; recordEngagement() warns loudly.
#   ANALYTICS_IP_RETENTION_DAYS  — days before engagement_anonymise_ips() nulls a
#                                  row's raw IP in place (default 90). visitor_hash
#                                  survives, so uniques stay correct.
ANALYTICS_HASH_SALT=
ANALYTICS_IP_RETENTION_DAYS=90

# Whole-site traffic on /admin/analytics — lib/vercelAnalytics.server.ts. All
# optional: omitting VERCEL_API_TOKEN or VERCEL_ANALYTICS_PROJECT_ID shows a
# setup card rather than an empty chart, never silently. Not Google Analytics —
```

```
# @vercel/analytics is already installed and consent-gated (AnalyticsGate.tsx),
# so this reads data already being collected.
#   VERCEL_API_TOKEN            – a Vercel account token (Account Settings → Tokens)
#   VERCEL_ANALYTICS_PROJECT_ID – the Vercel project id
#   VERCEL_ANALYTICS_TEAM_ID    – only if the project belongs to a team
VERCEL_API_TOKEN=
VERCEL_ANALYTICS_PROJECT_ID=
VERCEL_ANALYTICS_TEAM_ID=

# Destiny One (the Expo messaging app's backend, /api/app/v1/one)
#   ChurchSuite is OPTIONAL for Destiny One (staff verify people via invites/approvals). Leave these
#   unset to run without it; set them to enable Sign in with ChurchSuite and the approval lookup.
#   CHURCHSUITE_CLIENT_ID / _SECRET        – OAuth app, client credentials; scopes addressbook.read
#                                            children.read (override with CHURCHSUITE_SCOPES)
#   CHURCHSUITE_OAUTH_CLIENT_ID / _SECRET  – "Sign in with ChurchSuite" app (auth code + PKCE, scope
#                                            user). Falls back to the pair above.
#   DESTINY_ONE_SECRET                     – 32+ random chars; seals the OAuth state cookie and the
#                                            one-time sign-in hand-off code
#   D1_MESSAGE_RETENTION_DAYS              – default 365 (placeholder pending safeguarding sign-off)
#   EXPO_ACCESS_TOKEN                      – optional; only if Expo push security is enabled
#   D1_APP_STORE_URL / D1_PLAY_STORE_URL   – optional; store links in the invite email once published
#   D1_OPS_ALERT_RECIPIENT                 – optional; who is emailed when the nightly purge or sync fails
#                                            (lib/destinyOne/opsAlertEmail.server.ts). Falls back to
#                                            SMART_SEARCH_ALERT_RECIPIENT
CHURCHSUITE_CLIENT_ID=
CHURCHSUITE_CLIENT_SECRET=
CHURCHSUITE_OAUTH_CLIENT_ID=
CHURCHSUITE_OAUTH_CLIENT_SECRET=
DESTINY_ONE_SECRET=
D1_MESSAGE_RETENTION_DAYS=365
#   D1_MSG_KEYS / D1_MSG_KEY_CURRENT / D1_SEARCH_KEY – NOT NEEDED: the message encryption keys live in
#                                          Supabase Vault (lib/destinyOne/crypto.server.ts). Only a
#                                          fallback for local dev against a database without them.
D1_MSG_KEYS=
D1_MSG_KEY_CURRENT=
D1_SEARCH_KEY=
EXPO_ACCESS_TOKEN=
D1_APP_STORE_URL=
D1_PLAY_STORE_URL=
D1_OPS_ALERT_RECIPIENT=

# Feature flags (also toggleable via the `service_status` DB table, e.g. 'smart_search')
ENABLE_SMART_SEARCH=true
```

Deployment & Performance

Deployment Target: Vercel

Why Vercel? - **Next.js first-party** — Built by Vercel team - **Edge functions** — Faster redirects, rate limiting - **ISR (Incremental Static Regeneration)** — Cache pages, revalidate on-demand - **CDN** — Automatic global distribution - **Analytics** — Built-in performance monitoring

Caching Strategy

Content	Strategy	TTL	Invalidation
Static assets (`/img` , `/fonts`)	Immutable	1 year	Filename change
Home page	ISR	1 hour	`revalidatePath("/")`
Sermon archive	ISR	4 hours	`revalidatePath("/sermons")`
Dynamic pages (`/[slug]`)	ISR	24 hours	`revalidatePath("/[slug]")`
Redirects	Edge	Infinite	Deploy
API responses	None	-	Fresh on every request
Images	Browser cache	30 days	`next/image` optimization

Performance Optimizations

1. **Image optimization:** - `next/image` component auto-resizes - WebP format for modern browsers - Lazy loading by default
2. **Code splitting:** - Dynamic imports for heavy components - Suspense boundaries for loading states
3. **Database queries:** - No N+1 queries (use `select()` joins) - Cache at Supabase level (pg_cache extension) - Use materialized views for complex queries
4. **Edge caching:** - Long TTLs on assets - Geography-aware CDN (UK origin)
5. **Monitoring:** - Vercel Analytics — tracks Web Vitals - Vercel Speed Insights — performance timeline - Custom events logged to analytics

---

Key Design Decisions & Rationale

Decision 1: Service Role Key for Backend

Why: Instead of client tokens accessing database directly:

- **Security** — Client can't be trusted; always use server role for writes
- **RLS consistency** — Service role bypasses RLS; we apply auth in app code
- **Error messages** — Can give user-friendly feedback (vs. RLS "you don't have permission")
- **Audit logging** — API routes log who did what; database doesn't

Decision 2: Supabase for Database

Why: Not a custom Node/Express API:

- **PostgreSQL power** — JSON, full-text search, PostGIS for geo (future)
- **RLS built-in** — Security at database layer
- **Real-time** — Subscriptions for live updates (not currently used)
- **Managed** — No DevOps overhead

Decision 3: Server Components Over Client

Why: Use React server components wherever possible:

- **SEO** — Content rendered server-side, searchable
- **Security** — Secrets stay on server (not in browser)
- **Performance** — Less JavaScript shipped to client
- **Simplicity** — No useState/useEffect for data fetching

Decision 4: AI Knowledge Base Over Retrieval

Why: Single `siteKnowledge.ts` instead of RAG:

- **Reliability** — Model grounded in actual facts; no hallucination risk
- **Control** — Easy to edit facts without retraining
- **Cost** — Fewer API calls (no embedding search)
- **Speed** — Lower latency (fewer round-trips)

Decision 5: TipTap for Rich Text

Why: Not Slate, Draft.js, or Lexical:

- **Smaller bundle** — ~50KB gzipped (vs. 150KB+)
- **Extensions** — Underline, highlight, image, YouTube, placeholder
- **Prosemirror foundation** — Battle-tested
- **No config** — Works out of box

File Manifest

Public Pages

- `app/page.tsx` — Home (hero, latest sermon, CTAs)
- `app/sermons/page.tsx` — Sermons (video-first featured message, video archive grid)
- `app/sermons/[id]/page.tsx` — Sermon detail (video, next steps)
- `app/about/page.tsx` — About church
- `app/beliefs/page.tsx` — Statement of faith
- `app/kids/page.tsx` — Kids ministry
- `app/youth/page.tsx` — Youth ministry
- `app/young-adults/page.tsx` — Young adults
- `app/missions/page.tsx` — Missions partners
- `app/serve/page.tsx` — Volunteer opportunities
- `app/connect/page.tsx` — Connect groups
- `app/give/page.tsx` — Giving & donations
- `app/visit/page.tsx` — Plan a visit
- `app/new-here/page.tsx` — First-time visitor guide
- `app/hire/page.tsx` — Venue hire enquiry
- `app/contact/page.tsx` — Contact form
- `app/connect-card/page.tsx` — Prayer requests
- `app/alpha/page.tsx` — Alpha course

- `app/whats-on/page.tsx` — Events (ChurchSuite embed)
- `app/jobs/page.tsx` — Job listings
- `app/jobs/[slug]/page.tsx` — Job detail
- `app/shop/page.tsx` — Store (product grid)
- `app/shop/[slug]/page.tsx` — Product detail (variants, gallery)
- `app/shop/cart/page.tsx` — Shopping cart (Zustand + localStorage)
- `app/shop/checkout/page.tsx` — Stripe Payment Element checkout
- `app/shop/checkout/success/page.tsx` — Order confirmation
- `app/baptism/page.tsx` — Baptism information & registration
- `app/child-dedication/page.tsx` — Child dedication requests
- `app/safeguarding/page.tsx` — Safeguarding & child protection
- `app/governance/page.tsx` — Charity/company registration, trustees, finances & filings (live from both regulators)
- `app/help/page.tsx` — Help centre & FAQ
- `app/training/page.tsx` — /training resource library
- `app/volunteer/page.tsx`, `app/links/page.tsx`, `app/twelvetwo/page.tsx`, `app/dckids/page.tsx` — Volunteer sign-up, next-steps links, recovery info, kids camp campaign
- `app/accessibility/page.tsx`, `app/privacy/page.tsx`, `app/terms/page.tsx`, `app/data-gdpr/page.tsx` — Preferences & legal pages
- `app/[slug]/page.tsx` — Dynamic catchall (posts table)

Admin Pages

- `app/login/page.tsx` — Staff sign-in (there is no `app/admin/login`; that path is a stale-bookmark redirect to `/admin`)
- `app/admin/page.tsx` — Admin home (dashboard)
- `app/admin/destiny-one/**` — Destiny One: overview, access requests, invites, app members, communities & groups (Destiny One Admin), safeguarding (Safeguarding Admin), app settings
- `app/admin/banner/page.tsx` — Banner management
- `app/admin/popup/page.tsx` — Pop-up management
- `app/admin/redirects/page.tsx` — Redirect management
- `app/admin/cache/page.tsx` — Cache invalidation
- `app/admin/alpha/page.tsx`, `app/admin/bible-course/page.tsx`, `app/admin/cap-money/page.tsx`, `app/admin/recovery/page.tsx` — Course event management; four-line wrappers over `components/admin/CourseAdminPage.tsx`
- `app/admin/featured-course/page.tsx` — What's On featured course picker
- `app/admin/store/page.tsx` — Store management
- `app/admin/store/products/new/page.tsx` — Create product
- `app/admin/store/products/[id]/page.tsx` — Edit product (variants, stock)
- `app/admin/store/orders/page.tsx` — Orders list
- `app/admin/store/orders/[id]/page.tsx` — Order detail (fulfillment)
- `app/admin/store/hero/page.tsx` — Shop hero slides (add/edit/reorder rotating hero)
- `app/admin/sermons/page.tsx` — Publish sermon audio to Buzzsprout; manage playlist-backed series; AI or manual speaker correction (Sermon Admin)
- `app/admin/users/page.tsx` — Admin logins and access-level roles (Super Admin only)
- `app/admin/audit/page.tsx` — Audit log: ask it in plain English, or search/filter and read the field-by-field detail. Second tab holds the weekly AI reports (Super Admin only)
- `app/admin/analytics/page.tsx` — Click analytics: Short links, In person (`/nfc` + `/links`), Whole site (Vercel Web Analytics)
- `app/admin/posts/page.tsx` — Standalone pages; search, status filter, sort, bulk publish/delete
- `app/admin/training/page.tsx` (+ `[categoryId]`, `[categoryId]/[subgroupId]`) — Training categories → sub-groups → posts

Component Directory

- **~120 components** organized by feature:
- Global: Header, Footer, Providers, CookieBanner, Analytics, FloatingSmartSearch
- Admin: Sidebar, AdminHeader, AdminUI (the shared kit — PageHeader, Badge, Modal, EmptyState, SearchInput, FilterChips, ListToolbar, SortHeader, skeletons, Toggle, BulkBar; every token dark-mode aware, see "Dark mode" under Styling), AdminThemeToggle (light/dark/system control), AdminCharts (Sparkline, TrendChip, MiniBarRow — hand-rolled data viz, no charting library), AdminCommandPalette (⌘K + global shortcuts; its own fixed dark-glass surface, independent of the admin theme toggle), AdminTabBar (mobile bottom tab bar — glass, role-derived, view-transitioned), RichTextEditor (shared editor), Sheet (mobile bottom sheet), blocks/ (palette, inspector, outline, tools), posts/training/hr editors, audit/ (AuditAsk, AuditDetail, AuditReports)
  - `components/admin/hr/HrUI.tsx` is now a re-export shim over `AdminUI`. The kit used to live there, under a feature folder, with its page header named `HrHeader` — which Posts and Training both imported, reading as if those pages were part of HR. `HrHeader` remains as an alias for `PageHeader`; new code should import from `@/components/admin/AdminUI`.
- Ministry-specific: Kids, Youth, Young Adults, Alpha
- Governance: registration cards, trustees/directors, financial history, filings, source note
- Shop: ProductCard, ShopProductGrid, ShopHero, cart, checkout

- Forms: ConnectCard, ContactForm, HireForm
- Content: Training, Jobs, HR

Libraries (`lib/`)

- **~40 utility files** including:
- Supabase clients (admin, browser)
- API wrappers (YouTube, OpenAI, Resend, Stripe, Companies House + Charity Commission)
- Domain logic (sermons, jobs, HR, training, shop)
- Email templates
- Rate limiting (in-memory), auth is handled by `middleware.ts` (no `roles.ts`)
- Feature flags (`serviceStatus.ts`)
- Admin shell: `adminNav.ts` (the one navigation registry), `useAdminList.ts` (list search/filter/sort), `useAdminSession.ts` , `adminRecents.ts` , `csv.ts` , `adminTheme.ts` (light/dark/system theme store)
- Audit log: `audit.ts` (vocabulary + diffing + redaction), `audit.server.ts` (`recordAudit()` , the only writer), `auditAI.ts` (tools + prompt), `auditEmail.ts` (the weekly report email)
- Notification center: `notify.server.ts` (`recordNotification()` , the one writer — writes the row then Broadcasts it, never throws), `useNotifications.ts` (the client bell hook — fetch + per-role Realtime subscription)
- Click analytics (`/admin/analytics`): `engagement.ts` (vocabulary + wire format), `engagement.server.ts` (`recordEngagement()`), `botDetect.ts` (crawler/device/OS/browser detection), `track.ts` (client `sendBeacon`), `vercelAnalytics.server.ts` (Vercel Web Analytics API wrapper), `linksSteps.ts` (the `/links` database-down fallback, shared with the beacon validator), `useEngagementRollup.ts` (the fetch hook the page's tabs share)

API Routes (`app/api/`)

- **Admin endpoints:** Banners, redirects, pop-ups, cache revalidation, posts, training, alpha-events, featured-course, HR, store management, plus `me` (signed-in identity + roles), `search` (role-filtered cross-section record search behind the ⌘K palette), `notifications` (the notification bell's feed: `GET` for the role-filtered list with the caller's read state folded in, `/[id]/read` and `/read-all` to mark read — every admin may call it and it narrows the rows itself, like `search`), `audit` (the audit log: list, `ask` for the plain-English answer, `reports` for the weekly ones — Super Admin only) and `analytics` (`engagement_rollup` for Short links/In person, `analytics/site` for the Vercel panel — Site Admin or Super Admin)
- **Cron endpoints (`/api/cron/*` , `CRON_SECRET` -gated, scheduled in `vercel.json`):** `analytics-anonymise` (daily, nulls old `engagement_events` IPs), `hr-review-reminders` (daily), `audit-weekly-report` (Sunday evening — writes and emails the week's admin-activity report, then runs the audit-log retention purge and `purge_old_notifications()`), `ip-reputation-refresh` (weekly — refreshes the VPN/Tor/ datacenter/Private-Relay range lists behind `engagement_events.ip_category`), `design-deliverables-purge` (daily — deletes design deliverables the requester confirmed at least 48h ago)
- **Public endpoints:** `/api/chat` (Smart Search tool-calling chat, per-IP rate-limited), YouTube (videos/status/thumbnail/live), Alpha info, training unlock + read timer (`/api/training/posts/[id]/timer`), `/api/track` (click-analytics beacon for `/nfc` and `/links`)
- **Store endpoints:** Stripe checkout + Payment Element, order management
- **Webhooks:** Stripe only (`/api/webhooks/stripe`) — no GitHub or Vercel deployment webhook route
- **App BFF (`/api/app/v1/*`):** Backend-for-frontend routes serving the native iOS app. Because the Swift client cannot import `@destiny/shared` , every one of these routes fully normalises its payload server-side (absolute URLs, timezone-qualified timestamps, sanitised HTML, real booleans) and wraps it in a versioned envelope (`{ status: "ok"|"degraded", generatedAt, data, notice }`) via `lib/appApi.ts` + `lib/appSerializers.ts` . Absolute URLs are stamped from `SITE_ORIGIN` , which resolves in order `NEXT_PUBLIC_SITE_URL` → `VERCEL_PROJECT_PRODUCTION_URL` → the Vercel literal (`https://destinychurch.vercel.app`), so emitted thumbnail/event/ `.ics` links always point at the origin that serves this app rather than the church's `destinytees.uk` nginx site. Seven endpoints:
- `config` — remote config for the More tab (ChurchSuite form links, service times, venue, giving URL, `minSupportedBuild` kill switch); `revalidate = 300` .
- `home` — the whole Home screen in one request (featured event, next service, live status, latest sermon, latest podcast episode) to avoid five cold-launch round trips.
- `events` / `events/[slug]` — the events feed, built with `buildEventIndex` so app and web share the same slugs; the full payload ships in the list response so the detail screen needs no network.
- `sermons` — YouTube sermon videos; a quota trip degrades (with a "Watch on YouTube" link) rather than errors.
- `podcast` — the DCTV Buzzsprout feed (direct-MP3 `audioUrl` for AVPlayer range streaming).
- `live` — live-stream status, polled every 30s, mirroring `contexts/LiveContext.tsx` 's offline grace.
- A degraded response overrides the caller's cache down to `s-maxage=30` so a ChurchSuite blip isn't pinned in the edge cache for the full TTL. Dev-only `?simulate=degraded|empty|error` exercises the app's non-happy states (hard-disabled in production). The legacy `/api/app/events` route is removed.

Mobile App (Native SwiftUI iOS)

The Expo/React Native scaffold was removed in favour of a **native SwiftUI app** (commits `3f68680` onward). The app lives in `mobile/` and is a pure renderer over the `/api/app/v1/*` BFF — it does no feed normalisation of its own, because the BFF already does all of it. - **Project & build.** Swift 6 with `SWIFT_STRICT_CONCURRENCY: complete` , built against the iOS 26 SDK (which opts standard components into Liquid Glass) with an iOS 18 deployment target. The Xcode project is **generated, not committed**: `mobile/project.yml` is the XcodeGen source of truth (sources are globbed, so adding a Swift file needs no project edit), and `mobile/Makefile` wraps the `xcodegen generate` + `xcodebuild` flow (`make project` / `make build` / `make test` / `make launch`). Bundle id `uk.destinytees.app` , portrait-only, iPhone + iPad. `DC_API_BASE` is set per-configuration (`http://localhost:3000` in Debug, `https://destinychurch.vercel.app` in Release). The Release base points at the Vercel origin that actually serves this Next.js app, **not** the church's

public `destinytees.uk` domain (currently an nginx site that 404s every `/api` route); it moves back to the public domain only once that domain is pointed at Vercel. - **Structure** (`mobile/DestinyChurch/`): - `App/` — `DestinyChurchApp` entry point and `RootTabView` , the five-tab shell (**Home / Sermons / Events / Give / More**). Live is deliberately *not* a tab — it surfaces as a Home hero card, a Sermons badge, and a More row instead. - `Features/` — one folder per tab (`Home` , `Sermons` , `Events` , `More` incl. `GiveView`), each with its SwiftUI views and an `@Observable` model. - `Networking/` — `APIClient` (the single network entry point; `Endpoint` enum is the only place a path string appears), `Envelope` (decodes the BFF's status/notice wrapper), `APIError` , `LoadState` . - `Models/` — `Codable` mirrors of the BFF payloads (`AppConfig` , `EventSeries` , `Sermon` , `PodcastShow`). - `Design/` — `Brand` (colours), `Typography` , `GlassSurface` , `StateContainer` (loading/empty/error scaffolding), `ComingSoonView` . - `Web/` — `BrowserView` , an in-app browser for ChurchSuite forms opened from the More tab. - `Resources/` — `Info.plist` , `config-fallback.json` (a baked-in copy of `/api/app/v1/config` so the first launch has content before the network returns), and the generated `Assets.xcassets` . - **Generated assets.** `scripts/generate-ios-assets.mjs` (`npm run gen:ios-assets`) rasterises the app icon and launch logo from the brand SVGs in `public/` and emits colorsets from `packages/shared/src/design/tokens.ts` , so the app's colours can never drift from the website's. - **Remote config.** The More tab, giving link, service times and venue all render from `/api/app/v1/config` , so changing a ChurchSuite form slug is a server-side JSON edit, not an App Store release; `minSupportedBuild` in that payload is a kill switch for a bad build. - **`packages/shared/`** (`@destiny/shared`) is an npm workspace of framework-agnostic types/logic shared by the web app and the App BFF (the native app consumes it only indirectly, via the BFF's JSON). It ships raw TypeScript (Next transpiles it via `transpilePackages: ["@destiny/shared"]`). Modules: - `src/churchsuite/events.ts` — `ChurchSuiteEvent` , `deduplicateEvents` , `fetchChurchSuiteEvents` , `churchSuiteEventUrl` , `eventSignupUrl` , `stripEmoji` , `eventDescriptionText` , and `eventImage` . Two feed shapes to be careful with: `images` is an empty **array** when an event has no artwork, and the `thumb/sm/md/lg` entries are *objects* whose URL sits on `.url` (only the `original_*` keys are bare strings) — always go through `eventImage` , which guards both. - `src/churchsuite/dates.ts` — `parseFeedDate` and friends. The feed sends `"2026-07-30 00:00:00"` (a space, not `T`), which Safari parses as Invalid Date, so **every** read of a feed timestamp goes through `parseFeedDate` and that fix lives in one place. `isUpcoming` compares `datetime_end` , so a multiday event already in progress stays visible. - `src/churchsuite/series.ts` — `buildEventIndex` , the single source of truth for event slugs (grid, cards, detail pages, `generateStaticParams` , sitemap and the admin picker all read it). Groups occurrences by `sequence ?? id` — ChurchSuite's `sequence` is the series key, so the seven "Destiny 12:2" rows collapse to one entry. Slugs are assigned in date order, so the soonest event wins a contested name and later claimants take an identifier suffix. - `src/churchsuite/sanitize.ts` — `sanitizeEventHtml` . An allowlist rewriter that drops every attribute except a validated `href` , because the live feed carries `class="branded"` and `style="color:#f9c100"` (invisible on white) written against ChurchSuite's own stylesheet. Dependency-free on purpose: `isomorphic-dompurify` drags jsdom into the server bundle, which is the trap commit `2185023` had to unwind at the 250MB function limit. - `src/churchsuite/ics.ts` — `buildIcs` . Emits local wall-clock times with `TZID=Europe/London` plus a `VTIMEZONE` block rather than converting to UTC, so DST is the calendar client's problem. - `src/design/tokens.ts` — canonical DC brand palette/typography (pillar colours, accent, gradient), matching `app/globals.css` . - **`docs/mobile-app-scope.md`** (+ printable `docs/mobile-app-scope.pdf`) remains the scoping document — BFF architecture, a self-hosted Matrix homeserver for group chat with safeguarding constraints (adult/minor boundary via ChurchSuite DOB data), ChurchSuite API integration, and payments/sermon-feed reuse. It now also includes **Appendix A (ChurchSuite API v2 technical reference)** and **Appendix B (Apple Human Interface Guidelines considerations)**. Phase 1 (the native SwiftUI tab shell over the `/api/app/v1` BFF) is now built. **Chat has since moved off Matrix** — see Destiny One below.

Destiny One (Expo, iOS + Android) — `apps/destiny-one/`

The members' messaging app — "WhatsApp Communities" with department sub-groups, under the safeguarding rules in the `d1_*` schema (§29). **Separate from the Swift app in `mobile/`** , which remains the content app (sermons/events/give). Chat was scoped on a self-hosted Matrix homeserver; it was built on Supabase instead (Postgres + triggers + Realtime Broadcast) because it needed shipping urgently, needs no new server to run and patch, and the safeguarding rules sit in the same database as the data rather than in a separate Synapse module. - **Status: every screen built on the complete backend.** Built from the Claude Design prototype (`DestinyOne.dc.html` , project "Destiny One") and `docs/destiny-one-ui-spec.md` . Chosen variants: **1B "Cards and filters" chat list** (All / Unread / Announcements chips, one rounded card per community with "See all", count badges) and **1F "Avatars" conversation** (bubbles; the sender's avatar beside the last message of a run and their name above the first). Not yet run on a device: there's no Xcode on the dev Mac, so it has only been checked with `expo export` and a web preview on mock data. - **Routes (`src/app/`):** `index` (launch gate → `routeFor(me)`), `welcome` (A1), `email` (A2), `code` (A3), `request` (A5), `waiting` (A6–A8, copy from `me.onboardingMessage`), `notices` (A9), `(tabs)/{chats,find,profile}` with a floating glass tab bar (Chats / Search / Profile; there's no Groups tab, since Chats already shows every community and "See all" opens `community/[id]`), `group/[id]` (B3 conversation), `group/[id]/info` (B6; leaders: rules panel, make admin / remove), `group/[id]/media` (Photos and files: a photo grid that opens the viewer and a file list, newest first, older pages on scroll; cached as `["media", groupId]` , kept current by the same Realtime events as the chat), `group/[id]/edit` (C4), `community/[id]` (B2), `new-group` (C1, modal), `add-people` (C2; `?groupId` adds to a group, `?communityId` picks for New group), `notifications` (D2 + per-group mute), `search` (full-screen search opened from a chat; the Search tab `(tabs)/find` uses the same `SearchView` component), `report` + `report-sent` (B5), `feedback` (D5, Profile → Report a problem / Send feedback, `?kind=problem|idea` ; also reached by shaking the phone, see below), `chat-safety` , `help` (Profile → Help: short answers grouped by topic, one open at a time, with Report a problem underneath), `whats-new` (Profile → Support, or tap the version once: release notes by version, `RELEASES` in `src/lib/releases.ts`), `release` (the release splash, a full-screen modal shown once per release: every X.X release, and an X.X.X patch only if its `RELEASES` entry has `splash: true` ; last-seen kept in AsyncStorage `d1.releaseSplash.seen.v1` , new members are marked seen when they accept the notices; logic in `src/lib/releaseSplash.ts`), `beta` (hidden: tap the version at the bottom of Profile 10 times within 2s gaps; "Become a beta tester?" modal that sends a request to the feedback inbox so an admin adds the person's email to the nightly TestFlight group; asked-state kept on the phone in AsyncStorage `d1.betaRequested.v1`), `viewer` , `message-info` , `forward` , `share` , `events` , `send-event` (v0.8, see below), `delete-account` (D3, type DELETE), `accounts` (account switcher, a fit-to-content form sheet), `add-account` (Profile → Add account: Add child / Add admin account, a form sheet), `password` (password sign-in) and `set-password` (Profile → Password). - **Profile tab** (`(tabs)/profile.tsx`) is grouped like iOS Settings: the name card (tap the picture to change it, the name to open `edit-name`), then headed sections Account (email, password, add/switch account), Preferences (notifications, appearance), Privacy and safety (blocked people, chat safety, privacy notice, terms), Support (help, report a problem, send feedback) and Your data. Download my data saves a dated `.json` file to the cache folder and opens the share sheet on it (iOS); Android's share sheet can't take a file URL, so it still shares the text. - **Notification actions** (`src/lib/notificationActions.ts`): message pushes carry `categoryId: "message"` , so they offer **Reply** (a text field; needs the phone unlocked, `isAuthenticationRequired`) and **Mark as read**, neither of which opens the app. The push's `data` has `groupId` , `messageId` and `memberId` (which account it's for). A notification for another account on the phone uses that account's own API client (`apiFor`). iOS delivers the response to the listener in `_layout.tsx` ; Android to a background task (`expo-task-manager` , `registerTaskAsync`). A reply that fails posts a local "Your reply wasn't sent" notification rather than vanishing. - **Share into Destiny One** (`share.tsx` ,

`+native-intent.ts`): `expo-sharing` adds an iOS share extension (target `expo-sharing-extension`, app group `group.uk.destinytees.one`; EAS provisions both) and Android share intents. Text, one link, up to 5 photos or one PDF. The extension opens the app on an `expo-sharing://` link, which `+native-intent.ts` sends to `/share`: a preview of what's coming in, then every chat you can post in (active, and Announcements only where you're an admin). Pick one and it's sent, then that chat opens. Photos go through `cleanImage` like any other; anything else is refused with a reason. Signed out: "Sign in first". - **Upcoming events** (`events.tsx`, `send-event.tsx`, `src/lib/events.ts`): the Search tab's empty state shows "Coming up" (the next 3 church events) with See all → `events` (the whole calendar, searchable, pull to refresh). Tapping an event offers Share to a chat (`send-event`: pick a chat, the server snapshots the event as for the composer's Event button, then that chat opens) or View details (its What's On page; `D1EventSummary.webUrl`). Cached as `["events"]`, the one query that goes stale on a timer (30 minutes), because the calendar isn't on Realtime. `EventRow` is shared with the composer's event picker. - **Typing** (`src/state/typing.ts`): a memory-only store fed by the `typing` event (never me, never someone I've blocked). Each event shows that person for 6 seconds; their message arriving clears them. The chat header's subtitle swaps the member count for "Leah is typing..." / "Leah and Sam are typing..." / "3 people are typing...". `typingPing` throttles the API call to once every 4 seconds per group. - **Photo viewer** (`viewer.tsx`, a transparent full-screen modal): tapping a photo in a chat opens it on black. Pinch or double-tap to zoom (the native scroll view's zoom), swipe sideways through every photo the chat has cached, swipe down to close, tap to hide the bars. Save to Photos asks for add-only access on iOS (`expo-media-library`, `savePhotosPermission`; it can't read the library); Android opens the share sheet instead, so the app never needs Google Play's photo-access permission (`granularPermissions: []`, and `READ_MEDIA_VISUAL_USER_SELECTED` / `WRITE_EXTERNAL_STORAGE` are in `blockedPermissions`). Share downloads the file to the cache folder first (`src/lib/media.ts`), since attachment links are private and short-lived. The message menu has Share for any photo or file. PDFs still open in the in-app browser. - **Leaked passwords.** With Supabase's leaked password protection on (Auth settings, Pro plan), a new password found in a known data breach is refused with `AuthWeakPasswordError` (reason `pwned`). `setPassword` (`src/lib/auth.ts`) turns that into `PasswordRejectedError`, and `set-password` shows a card saying the password has been leaked, clears the field and asks for a different one. Signing in with an existing password that has since leaked still works, but Supabase flags it (`weakPassword`), so `password` offers "Change password", which opens `set-password?leaked=1` with the same card. The wording is `passwordRejection()` in `@destiny/shared` (unit-tested), shared with the website's `/admin/reset-password`. - **State:** `src/state/session.tsx` (auth session, `me`, the communities list, the Realtime hub, catch-up, `routeFor`, `errorMessage`); `src/state/picker.ts` (Add people selection for New group). Small external stores hand things between screens: `pollDraft.ts` and `eventPick.ts` (a poll or event composed on its own screen, sent by the chat that opened it), `jump.ts` (a search result asking its chat to show one message) and `notificationTap.ts` (a tapped notification waiting for the app to be ready). Each is addressed to one group id, because more than one chat can be mounted at once (chat A → info → search → chat B) and each would otherwise act on it. - **Drafts** (`src/state/drafts.ts`): leave a chat half-way through a message and the text is still in the box when you come back, and the chat list shows "Draft: ..." on that chat. Memory only (nothing typed is written to the phone), keyed by member and group, forgotten on sign-out. The composer saves on every keystroke silently; the list hears about it once, when the chat closes. `src/lib/useConversation.ts` owns a chat: paging, optimistic sends with "Not sent. Tap to retry.", uploads, reactions, deletes, read receipts. - **Data cache — instant screens, database-driven updates.** All app data lives in one TanStack Query cache (`src/lib/queryClient.ts`), saved to AsyncStorage (one file per account, `d1.cache.v2:<member id>`; see Accounts below) and restored at launch, so the app opens straight onto the last known chat list and any chat opened before renders on its first frame. `staleTime: Infinity`: **nothing is re-fetched because a screen mounted or a tab was pressed.** Keys and hooks are in `src/lib/queries.ts` (`["me"]`, `["communities"]`, `["community", id]`, `["group", id]`, `["messages", groupId]`; directory searches are memory-only). Data changes reach the app in three ways: 1. **Realtime events** — `src/lib/realtime.ts` runs one hub that joins `d1-member:<me>` and `d1-group:<id>` for *every* group in the chat list (not just the open chat). `applyEvent()` patches the cache straight from the payload: a new message updates that chat's messages, the row's last message, and its unread count (not for my own messages or the chat on screen); deletes, reactions and pause state patch in place; member changes, joins/leaves and `group_updated` (rename, description, icon) mark the affected entries stale so they re-fetch in the background. No database change was needed: everything goes through `d1_emit`. 2. **Catch-up** — Broadcast doesn't replay, so the whole cache is marked stale (on-screen entries re-fetch quietly, others when next opened) on cold start, on returning after more than 30s in the background, and when the member channel re-joins after a socket drop. A re-fetched message page is **merged** by id; if it doesn't overlap the cached messages the old copy is replaced, so a long absence can't leave a silent gap. 3. **The app's own writes** update the cache from the server's reply or optimistically (sends, reactions, leave group/community, mute, edit, new group). Nothing waits on a chat-list refresh. Rows prefetch on touch-down (`prefetchGroup`). Only the newest 60 messages per chat are saved to disk, never unsent ones. Sign-out, and a different member signing in, wipe that account's cache (`clearCache`). `gcTime` is `Infinity` on purpose: a finite 30 days overflows `setTimeout`'s 24.8-day limit and fires at once, dropping every chat not on screen. - **Accounts: several on one phone, quick switching** (`src/lib/accounts.ts`, `accounts` route). **Double-press the Profile tab** to hop to the most recently used other account (two accounts simply toggle), with a "Switched to X profile" banner (`SwitchBanner`, picture included, 2.5 s, Reduce Motion fades instead of sliding); hold the Profile tab, or Profile → Switch account, for the full list. Each account has its own "slot": its own Supabase client and session under its own Keychain key (`d1.auth.<slot>`; slot `0` keeps supabase-js's default key, so single-account installs carry straight over). Switching changes the active slot and never signs anyone out or moves tokens between clients. Only the active slot auto-refreshes and holds Realtime channels. Rules worth knowing: - **Order of a switch** (`runSwitch` in `session.tsx`): save the cache now → activate the other slot → cancel in-flight queries and empty memory → restore that account's file. Anything fetched after activation runs as the new account; requests from the old one are cancelled rather than landing in the new cache. - **Cache writes are keyed by the data's owner**, not "whoever is active": `serialize` prefixes the `me` id and the storage wrapper writes to that account's file. The persister throttles, so a write queued just before a switch can land after it; this keeps it in the right file. Data with no `me` is never saved. The old `d1.cache.v1` is handed to slot `0` once. - **Add account** (Profile, under the name) offers **Add child** and **Add admin account**. Both are ordinary sign-ins to accounts that already exist, in "add mode": the current account stays active until the new one has passed `auth/link` and `checkAddedAccount` (child needs `isAdult` false; admin needs `D1Me.isStaff`, computed server-side from `admin_roles`). A wrong kind is dropped from this phone only (no global sign-out) and the current account is untouched. The kind is a label on the saved account (`Account.kind`), never a permission. - **Send as** (`Composer` hold-Send, `SendAsMenu`, `useConversation.sendAs`): holding Send (350 ms, with a haptic) offers your other signed-in accounts that are in this group, and the message is posted with that account's own token (`accounts.apiFor(slot)`), so authorship, membership, freezes and reports are exactly as if it sent from its own phone. **Never from or as a child account** (`canSendAs` / `sendAsCandidates`, which also treat any under-18 account as a child, whatever its label); the hold is only wired when another account exists and the active one is not a child, so a slow tap on Send still just sends. - **Pure rules** live in `packages/shared/src/destinyOne/accountRules.ts` (double-press window 300 ms, owner-check grace 60 s, next account, add-account check, send-as, password rules, the single sign-in failure message), pinned by `tests/unit/destiny-one-accounts.spec.ts`. - **Face ID / passcode before switching** (`confirmOwner`, `expo-local-authentication`), so a child handed an unlocked family phone can't walk into a parent's chats. Skipped on a device with no passcode. Adding an account needs its email code as normal. - **Push follows the active account.** After a switch the device's token is re-registered (`movePushToActiveAccount`, never prompts); the server keeps one owner per token, so the account you left stops getting notifications. - **Add account** signs into a pending slot through `signInClient()` / `signInApi`, so the current account stays active and untouched until the code checks out. Signing

into an account already on the phone replaces its old slot (locally; a server sign-out would end both). - **Sign out** (Profile, after a confirmation) ends the active account everywhere (as before) and switches to the next account on the phone, or shows Welcome if none. If the server can't be reached the phone still forgets the account (cache, drafts, saved session). Holding an account in the switcher signs that one out. A session revoked elsewhere is forgotten on its next use. Up to 5 accounts. - **UI kit:** `src/theme/tokens.ts` (the prototype's light/dark tokens), `src/components/ui.tsx` (the rotating orange **beam** border on primary buttons and focused fields — a spinning linear gradient in a clipped frame, since RN has no conic-gradient — plus buttons, fields, cards, dialogs, and pulsing loading skeletons — `SkeletonGroup`, `Bone`, `SkeletonRows` — used by Chats, Community, Group info and Edit group while data loads), `Icon.tsx` (the design's line icons via `react-native-svg`), `MessageBubble.tsx`, `MessageMenu.tsx` (press-and-hold menu drawn in React Native, WhatsApp-style: the message lifts over a dimmed chat, a bar of six quick reactions plus "+" for a full emoji grid sits above it, and a card of Reply / Copy / Report / Block / Delete sits below, both solid sheets in the app's colours rather than Liquid Glass; placed by the pure `lib/menuLayout.ts` so it always fits on screen, tested in `tests/unit/destiny-one-message-menu.spec.ts`. It replaced the SwiftUI context menu, whose hosted bubbles reported the wrong height and made messages and reactions overlap), `Composer.tsx` (its camera button opens the in-app camera, `app/camera.tsx` built on `expo-camera`, with the photo handed back through `lib/camera.ts`; it replaced the system camera screen, which closed and left the app unusable; needs a native build), `NotificationPrompt.tsx` (A10, asked once on first group open), `SafetyNotice.tsx`. - **Search** (`search` route): groups from the cached list, plus messages via `GET /search/messages` (`d1_search_messages`: groups you're in, since you joined, never deleted; since part 10 a blind index of hashed word prefixes, because message text is encrypted at rest). Migrations `20260927_03_destiny_one_message_search.sql`, `20261004_01_destiny_one_message_encryption.sql`. Opened from a chat (its search button, or the Search tile in Group info) it searches that chat (`?groupId=`), with an "All chats" chip to widen it. Tapping a message opens its chat scrolled to that message, highlighted for a moment: the chat loads older pages until it's there (up to 10 pages of 40), and a message from the chat the search was opened from goes back to that chat (`router.dismissTo`) rather than opening a second copy. - **Chats tab badge:** the number of chats with unread messages (muted ones don't count), on the native tab bar (`NativeTabs.Trigger.Badge`). - **Notifications:** a tap is noted (`notificationTap.ts`) and the chat opened by `OpenFromNotification` in the root layout once someone is signed in, has accepted the notices and is past the launch screen, so on a cold start it lands on top of the chat list. Each notification is handled once (the launch tap can arrive both as the last response and through the listener) and the last response is cleared; a tap for the chat already on screen does nothing. While notifications are allowed, the device token is re-registered once per signed-in session (`movePushToActiveAccount`, never prompts), so a failed first registration or a rotated token heals itself. The A10 prompt is skipped when notifications are already on or the system can no longer ask. - **Blocking** (Apple guideline 1.2): long-press a message → "Block {name}" (confirm), which calls `api.block`, then `hideSender()` in `src/lib/queries.ts` drops their messages from every cached chat; `applyEvent` ignores live messages from anyone in `me.blocked`. Settings → "Blocked people" (`blocked` route) unblocks, and `showSendersAgain()` resets the message caches so their messages come back in place. - **Photos lose their hidden details before upload.** `src/lib/cleanImage.ts` re-encodes every image (camera, photo library, images picked as files, and the profile picture) with `expo-image-manipulator`, which drops EXIF metadata including GPS location; HEIC becomes JPEG, PNG stays PNG, and the longest side is capped at 2048 px. PDFs are sent as they are. - **Expired attachment links.** Signed links last an hour but cached chats last up to 30 days. `useConversation` reads each link's expiry from its token (`signedUrlNeedsRefresh` in `@destiny/shared`, unit-tested) and swaps in fresh links in one `api.attachmentUrls` request when the chat opens or changes; tapping a file checks again first (`attachmentUrl`). - **Account changes mid-session.** `src/lib/api.ts` wraps the shared client: any call that fails with `forbidden`, `not_verified`, `access_request_needed` or `consent_required` asks the session to re-check `me`. `AccessGuard` (`src/state/session.tsx`, mounted in the root layout) then replaces any in-app screen with `routeFor(me)` (waiting, notices, …), and a member who is no longer active has their chats, groups and messages removed from the device cache. - **Shake to report a problem.** `useShakeToReportListener` (`src/lib/useShakeToReport.ts`, mounted in the root layout) reads the accelerometer (`expo-sensors`, 10 times a second) while the app is open and an active member is signed in. `createShakeDetector` (`src/lib/shake.ts`, pure and unit-tested in `tests/unit/destiny-one-shake.spec.ts`) needs three jolts over 1.8 g within a second, so a knock or a drop doesn't count, then waits 3 seconds. A shake asks first ("Report a problem?" / Not now / Turn off shake to report), and never on the feedback screen itself. On by default; the switch is on the feedback screen and is kept on the phone (`src/state/shakeToReport.ts`). Nothing about movement is stored or sent. `app.json` gives iOS a motion purpose string anyway, as App Review can ask for one when the sensors library is linked. - **Crash reporting:** Sentry (`@sentry/react-native`, EU region), set up in `src/lib/sentry.ts` and imported first in the root layout. Off unless `EXPO_PUBLIC_SENTRY_DSN` is set at build time. It sends as little as possible: the member's internal id only (`setReportingMember`, no name, email or IP), no screenshots, screen recordings or performance tracing, and only network, navigation and app-lifecycle breadcrumbs (console output and taps, which could carry chat text, are dropped; route params and query strings are stripped). "Report a problem" attaches the last error's id (`lastErrorId`) so staff can match the two. Source maps upload during EAS builds when `SENTRY_AUTH_TOKEN`, `SENTRY_ORG` and `SENTRY_PROJECT` are set; `SENTRY_ALLOW_FAILURE=true` in `eas.json` keeps builds working until they are. - **Lint:** `eslint.config.js` (`eslint-config-expo`). The React Compiler rules (`react-hooks/refs`, `set-state-in-effect`, `preserve-manual-memoization`) are errors. The only exceptions are the two PanResponder handlers in `Swipe.tsx`, whose refs are read in touch handlers rather than during render (marked with a reasoned `eslint-disable-next-line`). `import/namespace` is off: `tsc` checks namespace members, and the plugin misses SDK 58's compiled enums (`Notifications.AndroidImportance`). `npx expo lint` caches in `.expo/cache/eslint`; delete it if results look stale. - **React Native 0.87+ strict types:** refs use instance types — `useRef<TextInputInstance>`, `useRef<ViewInstance>` (exported from `react-native`), not `useRef<TextInput>` / `useRef<View>`. `ListEmptyComponent` / `ListFooterComponent` take `undefined`, not `null`, for "nothing". - **Differences from the prototype:** Settings adds Download my data and Delete my account (safeguarding policy + UK GDPR access and erasure). Emoji reactions are allowed as member content (confirmed 2026-09-27). - **Sign-in:** the email screen calls `api.requestCode` (`POST /auth/code`); the server sends a code only to someone who can get in and answers the same either way, and the code screen says what to do if none arrives. "Sign in with ChurchSuite" is hidden (the flow in `lib/auth.ts` is built but not switched on; App Review tends to reject "coming soon" placeholders). - **Minimum age 13** (decided 2026-09-28): `MIN_AGE` / `isUnderMinimumAge` in `@destiny/shared`. Staff approvals, age edits and invites refuse a date of birth under 13 (`adultOnForDecision`), so do access requests (`submitAccessRequest`) and the app's request form. No parent or carer step. - **Staff invites create the member up front** (`d1_invite_create_member`, auth_user_id null, active, in the invite's communities), so staff can put them in groups on the website before they open the app; first sign-in links the login (`d1_accept_invite`), folding in any earlier pending row. Revoking before sign-in erases that member. They count toward the 2-adults rule from the moment they're invited. - **Tab bar:** the highlight slides between tabs on a spring, the new icon bounces, scenes cross-fade; Reduce Motion turns the slide and bounce off. Double-pressing Profile switches account; holding it opens the account switcher. - **Invite by email (leaders)** — `invite` route, opened from Add people for a group. `POST /groups/[id]/invites` creates a `needs_approval` invite: on sign-in the person becomes an access request pre-filled "Invited by X to Group (leader says: adult)", and staff approval in `/admin/destiny-one/requests` confirms their age and joins them to the group (`request_group_ids`). Migration `20260927_02_destiny_one_leader_invites.sql`. - Expo SDK 58 (React Native 0.88), Expo Router (`src/app/`), TypeScript. Its own npm project with its own lockfile — deliberately **not** a root workspace so Vercel never installs React Native. It imports `@destiny/shared` via `"file:../../packages/shared"`; `metro.config.js` watches that folder. Excluded from the root `tsconfig.json` and `eslint.config.mjs`. - `app.json`: name "Destiny One", scheme `destinyone`, bundle/package id `uk.destinytees.one`, Android `blockedPermissions`

strips phone-state/SMS/contacts/location permissions any dependency might add. - `src/lib/` : `config.ts` (EXPO_PUBLIC_* — see `.env.example`), `secureStorage.ts` (Supabase session in Keychain/Keystore, chunked for Android's size limit), `supabase.ts` (auth + Realtime only — never data; one client per account), `accounts.ts` (the accounts on this phone, switching, Face ID check), `api.ts` (the shared typed client), `auth.ts` (email OTP; password sign-in and `setPassword` ; ChurchSuite via `expo-web-browser` auth session + app-side PKCE), `realtime.ts` (the app-wide hub over private `d1-group:*` / `d1-member:*` channels), `queryClient.ts` + `queries.ts` (the saved data cache, see above), `push.ts` (ask contextually, never on launch). - `src/components/GlassSurface.tsx` — Liquid Glass (`expo-glass-effect` `GlassView`) on iOS 26+, a translucent solid fallback on Android / older iOS. The one surface primitive for app chrome. - Checks: `npm run typecheck` , `npx expo lint` , `npx expo-doctor` , `npx expo export --platform ios --platform android` . CI runs typecheck and lint (the "Destiny One app" job). - GDPR notes (data map, processors, retention, erasure, review audit): `docs/destiny-one-gdpr.md` . - Staff guide (approvals, reports, review, takedown, suspension, blocks, settings, out of hours): `docs/destiny-one-staff-guide.md` . Draft privacy-notice, terms and safeguarding-policy sections for sign-off (not live): `docs/content/destiny-one-notices-draft.md` .

Live Caption (macOS app) — `apps/live-caption/`

A standalone **macOS app** (not part of the website deploy) that captions live audio in real time for Destiny's AVL setup (ATEM, ProPresenter, Dante, NDI). It captures from a Core Audio device or an NDI network source, transcribes locally with a Metal-accelerated whisper.cpp model, and shows the caption on a connected display and/or publishes it as a live NDI source. **Audio never leaves the machine** — transcription is entirely local. - **Project & build.** SwiftUI, Swift 6, built against the macOS 26 SDK (for the real Liquid Glass `glassEffect()` caption plate). Like the iOS app the Xcode project is **generated, not committed**: `project.yml` is the XcodeGen source of truth and the `Makefile` wraps the build (`make whisper-build` → `make ndi-setup` → `make build` / `make run` / `make test`). `make build` fails with a clear error if either vendor step is missing. - **Vendored dependencies.** whisper.cpp is a git submodule at `Vendor/whisper.cpp` (pinned to a release tag, MIT). The **NDI SDK is proprietary and license-gated** — its headers/libraries are *not* committed (see `Vendor/NDI/README.md`); only the compiled app's embedded `libndi.dylib` is redistributed, which the NDI license permits. Whisper model files are multi-GB and fetched via `Scripts/fetch-whisper-model.sh` , not committed. - **Structure** (`LiveCaption/`): - `Audio/` — an `AudioSource` abstraction with Core Audio and NDI inputs, plus the 16 kHz mono `AudioResampler` both converge on. - `Transcription/` — the `WhisperEngine` wrapper, a sliding-window `StreamingTranscriber` , the `VocabularyPrompt` (decoding bias toward names/places; the 66 books of the Bible by default), and the `ModelCatalog` . - `Captioning/` — `CaptionStabilizer` (holds words back until they've survived the configurable *stability delay* unchanged, so re-transcription churn doesn't show on screen) and the `CaptionStore` every display reads from. - `Output/` — `NDIOutputPublisher` and its `CaptionFrameRenderer` , which emit BGRA frames with a **transparent background** (white glyphs, black outline) so a switcher can key captions straight over program video. - `UI/` — the control panel, caption display, full-screen `CaptionWindowManager` , and settings; a menu-bar control so the operator can start/stop/toggle without leaving ProPresenter. - `App/` — `LiveCaptionApp` entry point and `AppState` ; `Settings/Preferences.swift` ; `About/` . - `LiveCaptionTests/` — unit tests for the resampler, stabiliser, frame renderer, NDI publisher, and vocabulary prompt. - `Scripts/BenchmarkRTF/` — a standalone real-time-factor/latency harness; the documented exit criterion is a model sustaining RTF ≤ 0.5–0.6 on the target **Apple M2 Max** with no latency growth across a full service. `large-v3-turbo` is the starting model, `medium` the fallback.

Talent Show Buzzers — `apps/talent-buzzer/`

A standalone **Node app** (not part of the website deploy) for the Destiny talent show. Four Android tablets are red X buzzers and one is a dedicated golden buzzer. A press fires an Avolites lighting cue, shows an X (or a gold takeover with confetti) on the LED screen, and plays a buzzer/fanfare from the LED screen machine. **LAN only** — no internet, Supabase or accounts, so it survives the building connection dropping. - **One app, two roles** (`app.js` , launched by `start.bat` on both machines): a local-only setup page (`setup/setup.html` , `127.0.0.1:8090` , Host-header checked) asks whether this is the LED screen or Avolites machine, saves it to gitignored `machine.json` , and forks `hub/server.js` or `agent/avolites-agent.js` as a child process (restarted on crash; status flows back over IPC). The hub announces itself by UDP broadcast on port 8099 (`lib/discovery.js` , no key in the packet) so the agent finds it without typing IPs. - **Hub** (`hub/server.js` , runs on the LED machine): plain `http` static server + `ws` WebSocket at `/ws` . Clients say `hello` with a role (`tablet` + seat, `display` , `control` , `agent`). Show rules live in `lib/state.js` (pure, unit tested): each X latches until Reset, golden fires once until Reset, per-buzzer debounce, Lock ignores all presses. When the last judge buzzes the hub also broadcasts an `all-x` event (display strobes red, optional `cues.allX` playback). Only `control` clients can reset/lock/trigger/test. Optional shared `key` gates every WebSocket for audience-accessible Wi-Fi. - **Pages** (`hub/public/`): `tablet.html` (fires on `pointerdown` for latency, auto-reconnect, seat picker when no `?seat=`), `display.html` (sharp extruded 3D SVG X per judge with names underneath, all-out red strobe, gold takeover, Web Audio playback of `sounds/buzzer.mp3` / `golden.mp3` with synthesised fallbacks; a one-time click arms audio because of browser autoplay rules; `?bg=transparent` for keying), `control.html` (presence per seat, audio-armed state, lighting status + log, Reset/Lock, per-seat Trigger backup, test cue/sound buttons). - **Lighting**: `lib/titan.js` calls the Titan Web API (port 4430; URL templates in config, `{userNumber}` filled in; retries once). Default `agent` mode runs `agent/avolites-agent.js` on the Titan PC, which connects *out* to the hub so the lighting PC needs no inbound firewall rule; `direct` mode has the hub call a console's IP itself (hardware consoles). Reset can `KillPlayback` the X/golden cues (`releaseOnReset`). - `tools/fake-titan.js` stands in for the console when rehearsing; `npm test` runs the state tests. Setup, tablet kiosk tips, show-day checklist: `apps/talent-buzzer/README.md` .

- `apps/.gitkeep` keeps the `apps/` directory (for future standalone companion apps) tracked. See `apps/live-caption/README.md` for the full setup and operating guide.

Database Migrations

- **52 migration files** defining schema for:
- URL redirects, hidden videos (removed), site content (banners, pop-ups, page_content)
- Event management (Alpha course, Bible Course, CAP Money, Recovery, featured course, featured event)
- HR management (staff, leave, documents, reviews)
- Job listings & applications
- Feature flags, staff login audit
- Training resource library, standalone posts
- Shop (products, variants, orders, hero slides) and RLS/security hardening passes
- NFC tiles (the `/nfc` "digital back of seats" page, incl. event mode) and admin roles
- The admin audit log (`audit_log`) and its weekly AI reports (`audit_reports`)

- Click analytics (`engagement_events`) — storage layer for shortlink / nfc / links engagement, with `security definer` rollup and IP-anonymise functions
- Design tickets (`design_tickets` , `design_ticket_deliverables` , `design_ticket_events`) and the `design_admin` access level
- The `sermon_admin` access level (`/admin/sermons`) and `speaker_overrides` (AI/human speaker corrections for the sermon archive)

Summary

Destiny Church Tees Valley is a **professional, full-stack Next.js application** that combines:

1. **Public-facing website** — Sermons, ministries, contact, events, livestream
2. **Member engagement** — Prayer requests, volunteering, training, small groups
3. **Admin dashboard** — Content management, HR tools, store management, design request queue
4. **Ecommerce** — Online store with Stripe payments, product variants, order management
5. **Intelligent features** — Smart Search tool-calling chat assistant, live banner management

Architecture: Server-driven React with Supabase PostgreSQL, deployed to Vercel with ISR caching. All sensitive operations use service-role API proxies with explicit auth checks. Code is type-safe (TypeScript), styled with Tailwind CSS, and tested with Playwright E2E.

Key Differentiators: - **No vendor lock-in** — Open-source stack (Next.js, React, Tailwind, PostgreSQL) - **Security first** — RLS, rate limiting, CSRF protection, signed URLs - **AI-ready** — Grounded knowledge base (`siteKnowledge.ts`) powering Smart Search's tool-calling chat (products, weather, directions, web search) - **Accessible** — Semantic HTML, ARIA labels, keyboard nav - **Mobile-first** — Responsive design, tested on real devices - **Performant** — ISR caching, image optimization, edge functions

This repository serves as a **reusable platform** for churches nationwide, licensed through Square Media Group.

Document Version: 1.0.5
Created: June 18, 2026
For: Destiny Church Tees Valley
By: Square Media Group (Malachi malachi@squaremediagroup.org)